tclutils

Pure-Tcl Engines, 0.69.0

2026-10-27

Inhaltsverzeichnis

1 tclutils verstehen 11
1.1 Einführung 11
1.2 Abhängigkeiten 11
1.3 Rolle 11
1.4 Abgrenzung 11
1.5 Aufbau 12
1.6 Merkliste 12
2 Installation 12
2.1 Ein Verzeichnis eintragen 12
2.2 setup.tcl 12
2.3 Welche Datei geladen wird 13
2.4 Merkliste 13
3 Umbrella und Module 13
3.1 Was der Umbrella lädt 13
3.2 Datei, Doku, Test 14
3.3 common 15
3.4 Fehlercodes 15
3.5 Merkliste 15
4 Text und Dateien 16
4.1 Anfang und Ende 16
4.2 Suchen: tugrep 16
4.3 Sortieren und Doppelte 17
4.4 Felder und Ersetzen 18
4.5 Dateityp und Suche im Baum 19
4.6 ZIP 20
4.7 Merkliste 20
5 Weitere Zeilenfilter 21
5.1 Zeilen nummerieren: tunl 21
5.2 Umbrechen: tufold und tufmt 22
5.3 Tabulatoren: tuexpand 23
5.4 Spalten: tucolumn 24
5.5 Tabellen: tutable 25
5.6 Umkehren und Mischen: tutac, turev, tushuf 26
5.7 Zahlenfolgen: tuseq 27
5.8 Zeichen ersetzen: tutr 28
5.9 Zählen: tuwc 28
5.10 Seiten: tupr 29
6 Zusammenführen, Aufteilen, Abläufe 30
6.1 Spalten nebeneinander: tupaste 30
6.2 Über einen Schlüssel verbinden: tujoin 31
6.3 Zwei Mengen vergleichen: tucomm 32
6.4 An Mustern aufteilen: tucsplit 34
6.5 Nach Größe aufteilen: tusplit 35
6.6 In mehrere Ziele schreiben: tutee 36
6.7 Listen stapelweise abarbeiten: tuxargs 37

6.8 Abhängigkeiten ordnen: tutsort 38
6.9 Felder und Regeln: tuawk 38
6.9.1 Regeln 39
6.9.2 Felder und Variablen 39
6.9.3 Unterschiede zu awk 40
6.9.4 Beispiele 40
6.10 Ungefähr vergleichen: tufuzzy 41
6.11 Ungefähr suchen: tuagrep 42
7 Zeichen, Zahlen, Datum und Pfade 44
7.1 tustr 44
7.1.1 Sichtbare Zeichen 45
7.2 tupath 45
7.3 tudate 46
7.4 tunum 47
7.5 Merkliste 48
8 Datenformate 48
8.1 CSV: tucsv 49
8.2 JSON: tujson 50
8.3 INI: tuini 51
8.4 XML: tuxml 52
8.5 SQLite: tusqlite 53
8.6 LDIF: tuldif 54
8.7 Lesezeichen: tubookmark 55
8.8 Was hier keine eigene Liste hat 57
8.9 Merkliste 57
9 TDBC und MaxMind 57
9.1 TDBC: tutdbc 57
9.2 MaxMind: tummdb 58
9.3 Merkliste 58
10 Kodierung, Hash und Vergleich 59
10.1 Base64: tubase64 59
10.2 Base32: tubase32 59
10.3 CRC: tucrc 60
10.4 Digests: tuhash 61
10.5 xxHash32: tuxxhash 61
10.6 Kodierung wechseln: tuiconv 62
10.7 UUID: tuuuid 63
10.8 Query-Strings: tuurl 63
10.9 Verschachtelte Dicts: tudict 64
10.10 Zeilenvergleich: tudiff 65
10.11 Diff anwenden: tupatch 65
10.12 Merkliste 66
11 Binärdaten und Dateiinformationen 67
11.1 Hexdump: tuhexdump 67
11.2 Oktal, dezimal, Zeichen: tuod 68
11.3 Lesbare Zeichenketten: tustrings 69
11.4 Lesen, suchen, patchen: tuhexedit 70

11.5 Byteweise vergleichen: tucmp 72
11.6 Größen: tusize 73
11.7 Metadaten: tustat 74
11.8 Zeichentabellen: tucode 75
11.9 Merkliste 78
12 Notizen und Kalender 78
12.1 Markdown: tumd 78
12.2 iCalendar: tuical 79
12.3 Wiederholungen: turrule 79
12.4 vCard: tuvcard 81
12.5 Notizbaum: tunotes 82
12.6 Notizen in SQLite: tunotesdb 82
12.7 Monatskalender: tucal 84
12.8 Feiertage: tuholiday 86
12.9 Kalenderansichten: tical 86
12.10 Merkliste 87
13 Text aus PDF-Dateien 87
13.1 Befehle 87
13.2 Fehler 88
13.3 Was ankommt 88
13.4 Was es nicht tut 89
13.5 Kommandozeile 89
13.6 Merkliste 89
14 ODF und PDF-Struktur 90
14.1 PDF: tupdf 90
14.2 ODF: tuodf 91
14.3 Merkliste 91
15 Diagramme 91
15.1 Vom Text zur Datei: tuflow 93
15.2 Modell: tudiagram 94
15.3 Flussdiagramm: tuflow parse 96
15.4 Zustandsdiagramm: tustate 99
15.5 Klassendiagramm: tuclass 100
15.6 ER-Diagramm: tuer 101
15.7 Anforderungen: turequirement 102
15.8 Mindmap: tumindmap 103
15.9 C4: tuc4 104
15.10 Blöcke: tublock 105
15.11 Git-Verlauf: tugit 105
15.12 Architektur: tuarchitecture 106
15.13 Sequenzdiagramm: tusequence 107
15.14 Gantt: tugantt 109
15.15 Kreisdiagramm: tupie 111
15.16 XY-Diagramm: tuxychart 113
15.17 Quadrant: tuquadrant 114
15.18 Journey: tujourney 116
15.19 Zeitleiste: tutimeline 117

15.20 Sankey: tusankey 118
15.21 Radar: turadar 119
15.22 Treemap: tutreemap 120
15.23 Bitfelder: tupacket 121
15.24 Kanban: tukanban 122
15.25 PNG und Schrift 123
15.26 Merkliste 123
16 Bilder ohne Tk: PNG, SVG, Farben 124
16.1 Farben: tucolor 125
16.2 PNG kodieren und dekodieren: tupng 126
16.3 Zeichnen: tupngdraw 128
16.4 Tabelle als Bild: tutablepng 132
16.5 Zeichentabelle als Bild: tucodepng 134
16.6 Monatsblatt: tumonthpng 135
16.7 Einheitliche Größe: tupngpad 138
16.8 SVG: tusvg 139
16.8.1 Zeichenfläche 140
16.8.2 Dokument als Dict 141
16.8.3 Symbole 142
16.9 Bildformat erkennen: tuimage 142
16.10 Windows-Symbole: tuico 143
16.11 Ähnliche Bilder: tudhash 145
17 Provider 146
17.1 Eine API, mehrere Backends 146
17.2 Caps pro Schema 147
17.2.1 Eintrag 147
17.2.2 Schreiben, das die caps verbieten 148
17.3 Apps darüber 148
17.4 Zusammenfassung 148
18 Bausteine für Anwendungen 148
18.1 tulog 148
18.2 tuevent 150
18.3 turegistry 151
18.4 tuvalidate 151
18.5 tulist 153
18.6 tumath 154
18.7 tunumany 155
18.8 tuappinfo 156
18.9 tusysinfo 157
18.10 tuedid 159
18.11 tudeploy 160
18.11.1 roots und addModulePaths 161
18.11.2 require 161
18.11.3 resourceDirs 162
18.11.4 sourceModule 162
18.12 tuexe 163
18.13 tuopen 164

18.14 tuzipfs 166
18.14.1 Archiv lesen 167
18.14.2 Programm bauen 167
18.15 tuterm 169
18.16 tupagespec 170
18.17 tulayout 171
18.17.1 Seite und Maßstab 171
18.17.2 Blöcke 172
19 Netz und HTTPS 174
19.1 tuhttps 174
19.2 tls 1.x und tls 2.0 174
19.3 Was socketCmd zurückgibt 174
19.4 tufetch 175
19.5 tusparql 176
19.6 Zusammenfassung 177
20 tusettings 177
20.1 Zweck 177
20.2 API (Kurz) 178
20.3 Verzeichnisse pro Plattform 178
20.4 Recent-Liste 179
20.5 Grenzen und Fehler 179
20.6 Zusammenfassung 180
21 Tasks mit tutask 180
21.1 Zweck 180
21.2 Der Vertrag: Starter 180
21.3 Tasks 181
21.4 Warten: await und sleep 181
21.5 Abbrechen mit Aufräumen 182
21.6 Operationen 182
21.7 Gruppen 183
21.8 Thread-Pool 183
21.9 Sandbox 184
21.10 Lecks und vergessene Fehler 184
21.11 In Tk-Programmen 185
21.12 Fehlercodes 185
21.13 Zusammenfassung 186
22 Tests und tutestrun 186
22.1 Suite 186
22.2 tutestrun 187
22.3 Herkunft prüfen: tupkgfinder 0.2 188
22.4 Prüfregeln: tulint 188
22.5 Windows 189
22.6 Zusammenfassung 189
23 Grenzen 189
23.1 Windows-Hinweise 190
23.2 Zusammenfassung 191
24 Fallen 191

24.1 Behoben in 0.66.0 193
24.2 Zusammenfassung 193
25 Module (Orientierung) 193
25.1 Kategorien 193
25.2 Neu in 0.69.0 194
25.3 Neu in 0.68.0 198
25.4 Neu in 0.67.0 201
25.5 Neu in 0.66.0 202
25.6 Neu in 0.65.0 202
25.7 Neu und geändert in 0.64.0 202
25.8 Zusammenfassung 202
26 Glossar 202
26.1 Zusammenfassung 204
27 Fehlermeldungen 204
27.1 Zusammenfassung 209
28 API-Katalog: Tclutils-Module 209
28.1 common 210
28.2 tuagrep 210
28.3 tuappinfo 210
28.4 tuarchitecture 210
28.5 tuawk 210
28.6 tubase32 210
28.7 tubase64 211
28.8 tubin 211
28.9 tublock 212
28.10 tubookmark 212
28.11 tuc4 212
28.12 tucal 212
28.13 tucat 213
28.14 tuclass 213
28.15 tucmp 213
28.16 tucode 213
28.17 tucodepng 213
28.18 tucolor 213
28.19 tucolumn 214
28.20 tucomm 214
28.21 tucrc 214
28.22 tucsplit 214
28.23 tucsv 214
28.24 tucut 215
28.25 tudate 215
28.26 tudav 215
28.27 tudeploy 215
28.28 tudhash 216
28.29 tudiagram 216
28.30 tudict 216
28.31 tudiff 216

28.32 tuedid 217
28.33 tuer 217
28.34 tuevent 217
28.35 tuexe 217
28.36 tuexpand 217
28.37 tufetch 217
28.38 tufile 217
28.39 tufind 218
28.40 tuflow 218
28.41 tufmt 218
28.42 tufold 218
28.43 tufuzzy 218
28.44 tugantt 218
28.45 tugit 219
28.46 tugrep 219
28.47 tuhash 219
28.48 tuhead 219
28.49 tuhexdump 219
28.50 tuhexedit 219
28.51 tuholiday 220
28.52 tuhttps 220
28.53 tuical 220
28.54 tuico 220
28.55 tuiconv 221
28.56 tuimage 221
28.57 tuini 221
28.58 tujoin 221
28.59 tujourney 221
28.60 tujson 222
28.61 tukanban 222
28.62 tulayout 222
28.63 tuldif 222
28.64 tulint 223
28.65 tulist 223
28.66 tulog 223
28.67 tumath 223
28.68 tumd 224
28.69 tumindmap 224
28.70 tummdb 224
28.71 tumonthpng 224
28.72 tunl 224
28.73 tunotes 224
28.74 tunotesdb 225
28.75 tunum 225
28.76 tunumany 225
28.77 tunumfmt 225
28.78 tuod 225

28.79 tuodf 226
28.80 tuopen 226
28.81 tupacket 226
28.82 tupagespec 226
28.83 tupaste 226
28.84 tupatch 226
28.85 tupath 227
28.86 tupdf 227
28.87 tupdftext 227
28.88 tupie 227
28.89 tupkgfinder 227
28.90 tupng 228
28.91 tupngdraw 228
28.92 tupngpad 228
28.93 tupostgrest 228
28.94 tupr 228
28.95 tuprovider 228
28.96 tuquadrant 229
28.97 turadar 229
28.98 turegistry 229
28.99 turequirement 229
28.100 turev 229
28.101 turrule 229
28.102 tusankey 230
28.103 tused 230
28.104 tuseq 230
28.105 tusequence 230
28.106 tusettings 230
28.107 tushuf 231
28.108 tusize 231
28.109 tusort 231
28.110 tusparql 231
28.111 tusplit 231
28.112 tusqlite 231
28.113 tustat 232
28.114 tustate 232
28.115 tustr 232
28.116 tustrings 232
28.117 tusvg 232
28.118 tusysinfo 232
28.119 tutable 232
28.120 tutablepng 232
28.121 tutac 233
28.122 tutail 233
28.123 tutask 233
28.124 tutdbc 233
28.125 tutee 233

28.126 tuterm 233
28.127 tutestrun 234
28.128 tutimeline 234
28.129 tutr 234
28.130 tutreemap 234
28.131 tutsort 234
28.132 tuuniq 235
28.133 tuurl 235
28.134 tuuuid 235
28.135 tuvalidate 235
28.136 tuvcard 236
28.137 tuwc 236
28.138 tuxargs 236
28.139 tuxml 236
28.140 tuxxhash 236
28.141 tuxychart 236
28.142 tuzip 237
28.143 tuzipfs 237
29 Praxis: CSV, Datenstrukturen und sichere Fehlerbehandlung 237
29.1 CSV-Zeilen verarbeiten 237
29.2 Fehlerfälle gezielt behandeln 238
29.3 Dictionaries mit tudict 238
29.4 Dateipfade und Portabilität 238
30 Lizenz 239

1 tclutils verstehen

1.1 Einführung

tclutils ist eine Sammlung kleiner Pure-Tcl-Module ohne Tk: Textfilter, Datenformate, Provider, HTTPS-Politik, Einstellungen, Hintergrundaufgaben (`tutask`) und ein Test-Runner. Der Kern braucht nur Tcl selbst (`zlib` für ZIP gehört zu Tcl). Die Module verlangen `package require Tcl 8.6-` und laufen damit unter Tcl **8.6** und **9.x**.

Dieses Handbuch beschreibt tclutils **0.69.0**. Es richtet sich an Server-, CLI- und CI-Programme ohne Tk und an Anwendungen, die die Engines unter tkutils und ctrlutils nutzen.

1.2 Abhängigkeiten

Einige optionale Module brauchen mehr, und zwar erst beim Aufruf:

Modul	braucht
tufetch	tls oder ein curl/wget-Binary
tuhttps, tudav, tupostgrest	tls für HTTPS
tuprovider::ftp	tcllib-Paket ftp (erst bei open ftp …)
tuprovider::sftp	OpenSSH-sftp, eigenes package require
tusqlite / tutdbc	ein vom Aufrufer geöffnetes sqlite3-bzw. TDBC-Handle
tutask (nur pool)	Paket Thread (erst beim ersten pool)

`package require tclutils` lädt auch ohne tls und Thread. tcllib-`ftp` verlangt `tuprovider::ftp` erst bei `open ftp …`, nicht beim Laden des Umbrella.

1.3 Rolle

Stück	Rolle
Präfix	tu* / Namespace tclutils::…
Schicht	Model — Logik ohne GUI
Geschwister	tkutils (View), ctrlutils (Controller)
Laden	package require tclutils oder Einzelmodul

```
tclutils = Model · tkutils = View · ctrlutils = Controller
```

Die GUI-Schicht und die Controller beschreiben die Bücher *tkutils* und *Utils-Stack*. Dieses Buch behandelt Engines, Pfade, Provider, Einstellungen, Tasks und Tests.

1.4 Abgrenzung

- Keine Abschrift aller `docs/*.md`; die Detail-API steht dort, im tclutils-Baum.

- Kein Ersatz für *Pakete und Module* (`.tm`-Mechanik allgemein).
- Kein Dateimanager: Dateiblick und Explorer beschreibt *Utils-Stack*.

1.5 Aufbau

Kapitel 2 und 3 behandeln Installation und Module. Danach folgen Text und Dateien, Zeichen und Pfade, Datenformate, TDBC und MaxMind, Kodierung und Hash, Notizen und Kalender, PDF-Text, ODF und PDF-Struktur, Diagramme, Provider, Netz und HTTPS, Einstellungen, `tutask`, die Tests und die Grenzen. Die Anhänge enthalten Fallen (A), die Modulliste (B), das Glossar (C), Fehlermeldungen (D), den API-Katalog (E) und Praxisbeispiele (F).

1.6 Merkliste

- Pure Tcl, kein Tk, Version 0.69.0.
- Kern ohne Fremdpakete; tls, tcllib-`ftp`, curl/wget, Thread nur für optionale Module bzw. den Pool von `tutask`.
- Pairing laut CHANGELOG von tkutils 0.49.0: tclutils 0.69.0 + tkutils 0.49.0 + ctrlutils 0.5. tkutils 0.49.0 braucht mindestens tclutils 0.64.0, den CSV-Export von `tkutablelist` erst mit 0.69.0.

Nächstes Kapitel: Installation

2 Installation

tclutils ist ein Baum von Tcl-Modulen unter `lib/tm/`. Tcl lädt eine `.tm`-Datei nur, wenn ihr Verzeichnis auf dem Modulpfad steht. `lappend auto_path` reicht dafür nicht.

2.1 Ein Verzeichnis eintragen

```
::tcl::tm::path add /pfad/zu/tclutils-0.69.0/lib/tm
package require tclutils                  ;# -> 0.69.0
package require tclutils::tucsv           ;# ein Modul, der Umbrella
    ist nicht nötig
```

`package require` liefert die Paketversion. Fehlt das Paket, bricht es mit `can't find package tclutils::gibt-es-nicht` ab. Ein Verzeichnis, das es nicht gibt, trägt `setup.tcl` nicht ein. Der Fehler kommt erst beim `package require`.

2.2 setup.tcl

`tools/setup.tcl` liegt wortgleich in tclutils und in tkutils. Einmal sourcen trägt die vorhandenen `lib/tm` beider Bibliotheken ein:

```
source /pfad/zu/tclutils/tools/setup.tcl
package require tclutils::tubin
```

Die Kandidaten stehen in `tools/setup.tcl` in dieser Reihenfolge. Nur Verzeichnisse, die es gibt und die noch nicht auf dem Modulpfad stehen, werden eingetragen. Eingetragen wird von hinten nach vorn, damit der erste vorhandene Kandidat vorn liegt und gewinnt:

#	Kandidat
1	TCLUTILS_TM bzw. TKUTILS_TM, wenn die Variable gesetzt und nicht leer ist
2	lib/tm des Baums, aus dem setup.tcl kommt
3	neben der Tcl-Bibliothek: [file dirname [info library]]/<paket>/lib/tm
4	/usr/local/share/tcltk/<paket>/lib/tm
5	$XDG_DATA_HOME/tcltk/<paket>/lib/tm, sonst ~/.local/share/tcltk/<paket>/lib/tm
6	Geschwisterordner <paket>/lib/tm neben dem Baum
7	<paket>-*/lib/tm, die höchste Version, die ein lib/tm hat

Ist `TCLUTILS_TM` gesetzt, kommt `package ifneeded tclutils 0.69.0` aus diesem Verzeichnis, auch wenn der Quellbaum daneben liegt. Sobald Installation und Baum auseinanderlaufen, entscheidet die Variable, nicht die Nähe zum Skript.

Details der Suche: `tools/setup.md`. Was `.tm` allgemein ist: Buch *Pakete und Module*.

2.3 Welche Datei geladen wird

```
export TCLUTILS_TM=/pfad/zu/tclutils/lib/tm
tclsh8.6 tests/all.tcl
```

Der Runner schreibt ins Banner, welche Umbrella-Datei `tclutils` auflöst. Er sucht den Pfad, er lädt ihn an dieser Stelle nicht.

```
Modules resolve to:
  tclutils   0.69.0   …/tclutils/lib/tm/tclutils-0.69.0.tm
```

Liegt dieselbe Version noch einmal weiter hinten auf dem Modulpfad, gewinnt die vordere. Welche Kopien es gibt, zeigt `tupkgfinder` im Kapitel Tests und tutestrun.

2.4 Merkliste

- Modulpfad: `tcl::tm::path add …/lib/tm`, oder einmal `setup.tcl`.
- `TCLUTILS_TM` steht vor dem Baum, aus dem `setup.tcl` kommt.
- `package require tclutils` lädt den Umbrella. Ein einzelnes Modul geht ohne ihn.
- Fehlt das Paket, sagt das erst `package require`.

Nächstes Kapitel: Umbrella und Module

3 Umbrella und Module

3.1 Was der Umbrella lädt

```
package require tclutils    ;# -> 0.69.0
```

Die Datei ist `lib/tm/tclutils-0.69.0.tm`. Sie fordert 121 Module mit `package require tclutils::…`, darunter `common` 0.2, `tutask` 0.2, `tusettings` 0.1, `tupdftext` 0.1,

`tufetch` 0.4, `tuhttps` 0.1 und `tuflow` 0.4. `tuflow` fordert beim Laden `tudiagram`. Danach sind 122 Pakete `tclutils::*` geladen.

24 Moduldateien im Baum bleiben außen, bis sie jemand einzeln fordert:

Gruppe	Pakete
nur mit eigenem package require	tutestrun, tutdbc, tuprovider::sftp
Diagramm, tuflow lädt sie beim ersten Abruf der betreffenden Art	tustate, turequirement, tuer, tuclass, tumindmap, tuc4, tublock, tugit, tuarchitecture, tupie, tuxychart, tuquadrant, tujourney, tutimeline, tusankey, tugantt, tusequence, turadar, tutreemap, tupacket, tukanban

`tudiagram` steht nicht in der Liste der 121, ist nach dem Umbrella aber da, weil `tuflow` es sofort lädt. Die übrigen Diagramm-Module nicht. 25 Dateien stehen nicht in der Anforderungsliste des Umbrella.

```
::tcl::tm::path add /pfad/zu/tclutils-0.69.0/lib/tm
package require tclutils                        ;# -> 0.69.0
package present tclutils::tutask                ;# -> 0.2
package present tclutils::tusettings            ;# -> 0.1
package present tclutils::tupdftext             ;# -> 0.1
catch {package present tclutils::tutestrun}     ;# -> 1
```

`catch` liefert 1, weil `tutestrun` nicht geladen ist. Das Paket gibt es im Baum. `package require tclutils::tutestrun` lädt es ohne den Umbrella.

In 0.69.0 kamen `tucsv` 0.2 (Excel-Profile, BOM, Kodierung) und `tusqlite` 0.2 (Parameter-Dict, `update`, `delete`), Kapitel Datenformate. In 0.68.0 haben 76 Module Korrekturen und eine neue Version bekommen; die Liste steht in Anhang B. Neu in 0.67.0 ist `tupdftext` 0.1, Kapitel Text aus PDF-Dateien. In 0.66.0 wurde `tutask` 0.2, Kapitel Tasks mit tutask. In 0.65.0 kam `tutask` 0.1 in den Umbrella. Aus 0.64.0 bleiben `tusettings` 0.1, `tunum` 0.4 und `tupkgfinder` 0.2; `tustr` und `tudate` stehen seit 0.69.0 auf 0.3.

FTP liegt im Umbrella (`tuprovider::ftp` 0.2) und fordert tcllib-`ftp` erst bei `open`. SFTP nicht. TDBC nicht.

3.2 Datei, Doku, Test

Stück	Beispiel
Paket	tclutils::tujson
Datei	lib/tm/tclutils/tujson-0.1.tm
Befehle	Namensraum ::tclutils::tujson
Doku	docs/tujson.md
Test	tests/tujson.test

Stück	Beispiel
man	man/mann/tujson.n

Die Version im Dateinamen ist die des Moduls, nicht die des Umbrella. `tujson-0.1.tm` bleibt 0.1, während der Umbrella auf 0.69.0 steht.

```
tclsh tools/check-modules.tcl -require test,doc,man
```

146 Module, keines ohne Test, Doc oder man-Seite.

3.3 common

`tclutils::common` 0.2 ist die gemeinsame Hilfe, kein Modul für die Anwendung. Darüber laufen die Optionsfehler der anderen Module.

Befehl	Wirkung
parseOptions defaults args	unbekannte Option wird {TCLUTILS COMMON OPTION name}
ensureBoolean wert option	kein Wahrheitswert wird {TCLUTILS COMMON BOOLEAN …}
ensurePositiveInteger wert was	{TCLUTILS COMMON INTEGER …}
ensureOneOf wert liste was	Wert nicht in der Liste wird {TCLUTILS COMMON VALUE …}
readFile, writeFile	Text, Kanal schließt auch beim Fehler
readBinaryFile	Bytes, Kanal iso8859-1, damit Tcl 9 ohne die alte Kodierung binary auskommt

```
::tclutils::tudate::iso 0 -nope 1
```

Das wirft `{TCLUTILS COMMON OPTION -nope}` und nennt die bekannten Optionen, hier `-time`.

3.4 Fehlercodes

Der eigene Fehler eines Moduls hat die Form `{TCLUTILS <MODUL> <GRUND> …}`. `try`/`trap` vergleicht von vorn. Wer `{TCLUTILS TUFETCH}` fängt, bekommt jeden Fehler von `tufetch`. Die Liste der Codes, die dieses Buch ausprobiert hat, steht im Anhang Fehlermeldungen.

3.5 Merkliste

- `package require tclutils` lädt 122 Pakete, nicht die 146 Dateien im Baum.
- `tutestrun`, `tutdbc` und `tuprovider::sftp` bleiben außen.
- Die Diagramm-Arten außer `tudiagram` lädt `tuflow` erst beim Abruf. Befehle: Kapitel Diagramme.
- Unbekannte Optionen kommen als `{TCLUTILS COMMON OPTION …}`.

Nächstes Kapitel: Text und Dateien

4 Text und Dateien

Die Textfilter spiegeln Unix-Werkzeuge als Tcl-API. Rein kommt ein String oder eine Liste, heraus kommt ein String oder eine Liste. Die Filter selbst starten kein Programm. Das Mapping auf die Unix-Namen steht in `docs/guide/coreutils-mapping.md`. Dieses Kapitel beschreibt die Befehle, mit denen ein Ablauf aus Filtern, Dateityp und ZIP gebaut wird. Es führt nicht jedes Filtermodul aus dem Mapping auf.

Gemeinsames Muster, wo das Modul es hat: `text` arbeitet auf einem String, `file` auf einem Pfad, `files` auf einer Pfadliste.

4.1 Anfang und Ende

Befehl	Wirkung
tucat::text, file, files	Text unverändert, oder mit Zeilennummern
tuhead::text, file	die ersten n Zeilen; Vorgabe 10
tutail::text, file	die letzten n Zeilen; Vorgabe 10

`tucat` nummeriert mit `-number 1` jede Zeile, auch eine leere, als sechs Stellen, Tabulator, Text (`format %6d`). `-nonblank 1` nummeriert nur Zeilen mit Inhalt. Wie bei `cat -n` ist der Rest hinter dem letzten `\n` keine Zeile und bekommt keine Nummer, und `files` nummeriert die Dateien hintereinander in einem Lauf. Eine Datei ohne `\n` am Ende läuft in die nächste über. Eine unbekannte Option wirft `{TCLUTILS COMMON OPTION -name}` (seit 0.2; bis 0.1 bekam die leere letzte Zeile eine Nummer, jede Datei begann bei 1, und unbekannte Optionen fielen still weg).

```
package require tclutils::tucat
package require tclutils::tuhead
package require tclutils::tutail
set text "alpha\nbeta\nbeta\nalpha"
::tclutils::tuhead::text $text 2          ;# -> alpha\nbeta\n
::tclutils::tutail::text $text 1          ;# -> alpha
::tclutils::tucat::text "alpha\nbeta" -number 1
                                          ;# -> "     1\talpha\n
    2\tbeta"
```

Wie bei `head -n` und `tail -n` behält das Ergebnis die Zeilenenden: Jede Zeile endet mit `\n`, außer der letzten Zeile der Eingabe, wenn sie keins hat. `text` und `file` liefern dasselbe; `file` liest UTF-8, andere Kodierungen mit `-encoding`. Bis 0.1 war `tutail::text "eins\nJörg\n" 1` leer, und `file` ließ alle Zeilenenden weg. Zum Ausgeben deshalb `puts -nonewline`. `tuhead` und `tutail` haben kein Folgen von Dateien. Eine negative Anzahl wirft `{TCLUTILS TUHEAD COUNT}` bzw. `{TCLUTILS TUTAIL COUNT}`.

4.2 Suchen: tugrep

```
::tclutils::tugrep::search text muster ?option wert ...?
::tclutils::tugrep::match  zeile muster ?option wert ...?
::tclutils::tugrep::file   pfad muster ?option wert ...?
::tclutils::tugrep::files  pfade muster ?option wert ...?
```

Das Muster ist ein regulärer Ausdruck. `-fixed 1` sucht die Zeichenkette literal.

Option	Vorgabe	Wirkung
-nocase	0	ohne Groß/Klein
-linenumbers	0	jede Zeile als {nummer text}, Nummer ab 1
-invert	0	die Zeilen ohne Treffer
-fixed	0	keine Regex
-word	0	ganzes Wort, wie grep -w
-wholeline	0	die ganze Zeile, wie grep -x
-max	0	nach N Treffern aufhören; 0 heißt ohne Grenze
-count	0	nur die Anzahl
-fileswithmatches	0	nur die Pfade mit Treffer
-filenames	0	Treffer mit Dateinamen davor

```
package require tclutils::tugrep
set text "alpha\nbeta\nERROR eins\nerror zwei\nERRORCODE"
::tclutils::tugrep::search $text ERROR
                                          ;# -> {ERROR eins} ERRORCODE
::tclutils::tugrep::search $text error -nocase 1 -linenumbers 1
                                          ;# -> {3 {ERROR eins}} {4
    {error zwei}} {5 ERRORCODE}
::tclutils::tugrep::search "ERROR\nERRORCODE" ERROR -word 1
                                          ;# -> ERROR
::tclutils::tugrep::search "a\na\na" a -max 1
                                          ;# -> a
::tclutils::tugrep::search $text error -nocase 1 -count 1
                                          ;# -> 3
```

`-word` trifft `ERROR` und nicht `ERRORCODE`. `-max` zählt Zeilen, nicht Treffer in einer Zeile. Der Rest hinter einem abschließenden `\n` ist keine Zeile: `.*` trifft in `"eins\nJörg\n"` zwei Zeilen (seit 0.2; bis 0.1.3 drei, und `file` strich leere Zeilen am Dateiende).

4.3 Sortieren und Doppelte

`tusort` sortiert Zeilen oder eine Liste. `tuuniq` entfernt nur benachbarte Doppelte, wie Unix `uniq`. Wer alle Doppelten will, sortiert zuerst.

Option tusort	Wirkung
-numeric, -integer, -real	Zahlenvergleich
-dictionary	wie lsort -dictionary
-nocase	ohne Groß/Klein
-reverse	absteigend
-unique	Doppelte beim Sortieren weg

`tuuniq` kennt `-nocase`. `adjacentCount` liefert `{anzahl wert}` je benachbarter Gruppe. `count` zählt über die ganze Liste.

```
package require tclutils::tusort
package require tclutils::tuuniq
::tclutils::tusort::text "10\n2\n10" -numeric 1 -unique 1
                                          ;# -> 2\n10
::tclutils::tuuniq::uniqList {a b b a}    ;# -> a b a
::tclutils::tuuniq::uniqList [::tclutils::tusort::sortList {a b b a}]
                                          ;# -> a b
::tclutils::tuuniq::adjacentCount {a a b} ;# -> {2 a} {1 b}
```

4.4 Felder und Ersetzen

`tucut` schneidet Felder aus einer Zeile. Das ist kein CSV: Anführungszeichen und Kommas im Feld gehören zu `tucsv`, Kapitel Datenformate. Feldnummern zählen ab 1. Bereiche schreiben sich `2-4`. `-chars` schneidet Zeichen, ebenfalls ab 1, auch offen als `4-` oder `-3`.

```
package require tclutils::tucut
::tclutils::tucut::field "a;b;c" ";" 2         ;# -> b
::tclutils::tucut::fields "a;b;c;d" ";" {1 3}  ;# -> a;c
::tclutils::tucut::fields "a;b;c;d" ";" 2-3    ;# -> b;c
```

`text` und `file` nehmen `-delimiter`, `-fields` und `-chars`. Wie bei `tusort::text` und `tuuniq::text` endet das Ergebnis mit `\n` genau dann, wenn die Eingabe so endet; bis tclutils 0.68.0 fiel es weg.

`tused` ersetzt und löscht zeilenweise. Ohne `-all` gilt der erste Treffer der Zeile. `-fixed 1` sucht literal, sonst als Regex. `-nocase 1` ignoriert die Schreibung. `delete` wirft die passenden Zeilen weg. `process` nimmt eine Regeliste (`s`, `d`).

```
package require tclutils::tused
::tclutils::tused::replace "foo foo" foo bar         ;# -> bar foo
::tclutils::tused::replace "foo foo" foo bar -all 1  ;# -> bar bar
::tclutils::tused::delete "# c\nzeile" {^#}          ;# -> zeile
```

Adressen wie bei sed (`3`, `$`, `/regex/`, Bereich, Verneinung mit `!`, Schritt `erste~schritt`) hängen an der Regel in `process` und `script`. Die Formen stehen in `docs/tused.md`.

4.5 Dateityp und Suche im Baum

`tufile` liest nur den Kopf der Datei und vergleicht ihn mit einer Signaturtabelle. Kein libmagic, keine Masken.

Befehl	Ergebnis
detect pfad	Dict des genauesten Treffers, oder leer
type pfad	Kurzname (pdf, png, zip) oder ""
mime pfad	MIME-Typ oder ""
describe pfad	Satz, oder data
checkExtension pfad	Dict mit status: ok, mismatch, noext, unknown, unknownext

```
package require tclutils::tufile
::tclutils::tufile::type /pfad/seite.pdf    ;# -> pdf
::tclutils::tufile::mime /pfad/seite.pdf    ;# -> application/pdf
```

An einer Seite aus pdf4tcl ist der Typ `pdf`. Laut `docs/tufile.md` teilen sich DOCX, XLSX und PPTX nur die ZIP-Signatur und kommen als `zip`; ODT, ODS und ODP erkennt das Modul am `mimetype`-Eintrag ab Byte 30.

`tufind` läuft den Baum in Tcl.

Befehl	Was zurückkommt
files wurzel muster ?optionen?	Dateien
directories wurzel muster	Verzeichnisse
all wurzel muster	beides
walk wurzel muster script	ruft das Script je Treffer

Option	Vorgabe	Wirkung
-recursive	1	in Unterverzeichnisse
-maxdepth	−1	−1 heißt ohne Grenze
-mindepth	0	flacher wird verworfen
-hidden	0	Punkt-Dateien mitnehmen
-tails	0	nur der Dateiname, nicht der volle Pfad
-type		file, dir, any
-size		N, +N, -N, dazu k, m, g
-mtime		Alter in Tagen: +N älter, -N neuer

```
package require tclutils::tufind
::tclutils::tufind::files $dir *.pdf -tails 1 -maxdepth 1
```

Mit `-tails 1` und `-maxdepth 1` sind das die Namen direkt in `$dir`, keine Pfade aus Unterverzeichnissen.

4.6 ZIP

`tuzip` liest und schreibt ZIP mit gespeicherten und mit deflate-Einträgen. Das reicht für normale Archive und für ODF, das ein ZIP ist. Binärdateien öffnet es binär.

Befehl	Wirkung
names archiv	die Mitgliedsnamen
entries archiv	Dicts je Mitglied
readMember archiv name	der Inhalt
extract archiv name zieldatei	schreibt eine Datei
create archiv dateien ?-base dir? ?-compress 1?	neues Archiv
createMembers archiv member	aus dem Speicher; je Dict name, content, method

```
package require tclutils::tuzip
::tclutils::tuzip::create $archiv [list $datei] -base $dir -compress 1
::tclutils::tuzip::names $archiv                 ;# -> notiz.txt
::tclutils::tuzip::readMember $archiv notiz.txt  ;# -> hallo zip
```

`-base` schneidet das Verzeichnis vom Namen ab. Die Reihenfolge in `createMembers` bleibt; ein ODF legt `mimetype` so als ersten, ungespeicherten Eintrag.

Namen schreibt `tuzip` seit 0.2 als UTF-8; ein Name mit Umlaut bekommt das UTF-8-Flag, und `unzip`, Python und Windows lesen ihn richtig. Bis 0.1 standen Namen in Latin-1 ohne Flag, andere Programme lasen `Jörg` als `J÷rg`. Beim Lesen gilt `-encoding auto`: mit Flag oder gültigem UTF-8 UTF-8, sonst CP437, wie es das ZIP-Format vorsieht. Für Archive aus tuzip 0.1 mit Umlauten `-encoding iso8859-1` angeben. Der Provider über derselben Datei steht im Kapitel Provider.

Pfade ohne Dateisystem, sichtbare Zeichen und Beträge: Zeichen, Zahlen, Datum und Pfade.

Einige Module haben ein Programm unter `bin/`. Die Liste steht in `docs/guide/cli.md`. Dieses Buch führt sie nicht einzeln auf.

4.7 Merkliste

- `text`, `file`, `files`: String rein, String oder Liste heraus.
- `tuuniq` entfernt nur Nachbarn. Vorher `tusort`, wenn alle Doppelten weg sollen.
- `tucut` ist kein CSV.
- `tugrep -word` trifft das Wort, nicht den Anfang eines längeren Wortes.
- `tuzip` liest den Inhalt, der Provider entscheidet, ob Schreiben erlaubt ist.

Nächstes Kapitel: Weitere Zeilenfilter

5 Weitere Zeilenfilter

Dieses Kapitel setzt Text und Dateien fort. Die Module hier nummerieren, umbrechen, richten aus, kehren um, zählen und erzeugen Zeilen. Jedes hat ein Unix-Vorbild aus den coreutils oder util-linux; die Abweichungen stehen beim jeweiligen Modul. Kein Modul startet ein Programm.

Alle Optionen gehen über denselben Parser. Eine unbekannte Option wirft `{TCLUTILS COMMON OPTION -name}` und nennt in der Meldung die bekannten. Eine Ganzzahl unter 1, wo eine positive verlangt ist, wirft `{TCLUTILS COMMON INTEGER -name}`, ein Nicht-Wahrheitswert `{TCLUTILS COMMON BOOLEAN -name}`. Diese drei Codes stehen unten nicht bei jedem Modul noch einmal.

`file` liest die Datei mit der Systemkodierung und wandelt `\r\n` in `\n`. Das Ergebnis endet mit `\n` genau dann, wenn die Eingabe so endet:

Abschließendes \n	Module
wie in der Eingabe	tunl, tutac, turev, tuexpand, tutr, tufold, tufmt, tucolumn, tushuf
eigene Regel	tupr (Seiten, siehe dort)

Bis tclutils 0.68.0 ließen `tufold`, `tufmt`, `tucolumn` und `tushuf` das abschließende `\n` weg.

5.1 Zeilen nummerieren: tunl

tunl nummeriert Zeilen wie Unix `nl`.

```
::tclutils::tunl::text text ?option wert ...?
::tclutils::tunl::file pfad ?option wert ...?
```

Option	Vorgabe	Wirkung
-style	nonempty	all jede Zeile, nonempty nur Zeilen mit Inhalt, none keine
-start	1	erste Nummer
-increment	1	Schritt
-width	6	Stellen der Nummer, rechtsbündig
-separator	\t	zwischen Nummer und Zeile

Eine Zeile ohne Nummer beginnt mit `-width` Leerzeichen und dem Trenner. Unix `nl` setzt bei einer leeren Zeile nur Leerzeichen, ohne Tabulator. Ein abschließendes `\n` bleibt und erzeugt keine eigene Zeile. Ein leerer Text ergibt `""`.

`-start` und `-increment` sind Ganzzahlen, auch negative und beliebig große. `-width` ist eine Ganzzahl ab 1.

Fehler	-errorcode
-style außer all, nonempty, none	{TCLUTILS TUNL STYLE}
-start keine Ganzzahl	{TCLUTILS TUNL START}

Fehler	-errorcode
-increment keine Ganzzahl	{TCLUTILS TUNL INCREMENT}
-width keine Ganzzahl ab 1	{TCLUTILS TUNL WIDTH}

```
package require tclutils::tunl
::tclutils::tunl::text "a\n\nb" -width 2 -separator " "
                                          ;# -> " 1 a\n   \n 2 b"
::tclutils::tunl::text "a\n\nb" -style all -width 2 -separator " "
                                          ;# -> " 1 a\n 2 \n 3 b"
::tclutils::tunl::text "a\nb" -start 10 -increment 5 \
    -width 2 -separator ": "              ;# -> 10: a\n15: b
```

`tucat -number 1` aus Text und Dateien nummeriert dagegen fest mit sechs Stellen und Tabulator.

5.2 Umbrechen: tufold und tufmt

tufold bricht jede Zeile für sich, wie Unix `fold`. tufmt fasst Absätze zusammen und bricht sie neu, wie Unix `fmt`.

```
::tclutils::tufold::foldLine zeile ?-width n? ?-words 0|1?
::tclutils::tufold::text     text  ?-width n? ?-words 0|1?
::tclutils::tufold::file     pfad  ?-width n? ?-words 0|1?
::tclutils::tufmt::reflow     text ?-width n?
::tclutils::tufmt::reflowFile pfad ?-width n?
```

Option	Vorgabe	Wirkung
tufold -width	80	höchstens so viele Zeichen je Zeile
tufold -words	0	1 bricht an der letzten Leerstelle vor der Grenze
tufmt -width	75	Zielbreite der Absätze

`foldLine` liefert eine Liste der Teilzeilen, `text` und `file` einen String. Die Breite zählt Zeichen, nicht Bildschirmspalten: Ein Tabulator ist ein Zeichen. Ohne `-words` schneidet `tufold` hart nach `-width` Zeichen; eine Leerstelle am Schnitt bleibt am Ende der Zeile. Mit `-words 1` bricht `tufold` an der letzten Leerstelle bis Spalte `-width` plus eins. Alle Leerzeichen am Umbruch fallen weg, am Ende der Zeile wie am Anfang der Folgezeile. Unix `fold -s` lässt sie stehen: Aus `"ab cd"` mit Breite 3 macht es `"ab "`, `" "` und `"cd"`, `tufold` dagegen `ab` und `cd`. Ein Wort, das länger ist als `-width`, schneidet auch `-words 1` hart. Leere Zeilen bleiben leer.

`tufmt` trennt Absätze an Leerzeilen; mehrere Leerzeilen werden zu einer. Im Absatz fallen alle Zeilenwechsel und mehrfachen Leerzeichen weg, dann füllt das Modul die Zeilen Wort für Wort bis zur Breite. Der Einzug der ersten Absatzzeile gilt für alle Zeilen des Absatzes und zählt zur Breite; ein Tabulator im Einzug zählt als ein Zeichen. Ein Wort, das länger ist als die Breite, steht allein und ungekürzt in seiner Zeile. `\r` entfernt das Modul vorher.

```
package require tclutils::tufold
package require tclutils::tufmt
::tclutils::tufold::text "abcdefghij" -width 4
                                          ;# -> abcd\nefgh\nij
::tclutils::tufold::foldLine "eins zwei drei vier" -width 9 -words 1
                                          ;# -> {eins zwei} {drei vier}
::tclutils::tufold::foldLine "ab   cd" -width 3 -words 1
                                          ;# -> ab cd
::tclutils::tufmt::reflow "a  b\nc d e" -width 3
                                          ;# -> a b\nc d\ne
::tclutils::tufmt::reflow "  a b c d\n\n\nx" -width 5
                                          ;# -> "  a b\n  c d\n\nx"
```

5.3 Tabulatoren: tuexpand

tuexpand wandelt Tabulatoren in Leerzeichen und zurück, wie Unix `expand` und `unexpand`.

```
::tclutils::tuexpand::expand   text ?-tabs n?
::tclutils::tuexpand::unexpand text ?-tabs n? ?-all 0|1?
::tclutils::tuexpand::file     pfad ?-mode m? ?-tabs n? ?-all 0|1?
```

Option	Vorgabe	Wirkung
-tabs	8	Abstand der Tabstopps
-all	0	unexpand: 0 nur führende Leerzeichen, 1 alle Folgen
-mode	expand	nur file: welche Richtung

`expand` füllt jeden Tabulator bis zum nächsten Tabstopp auf. Ein abschließendes `\n` bleibt.

`unexpand` wandelt ohne `-all` nur den Leerraum am Zeilenanfang um, Leerzeichen wie Tabulatoren; der Rest der Zeile bleibt, wie er ist, auch Tabulatoren im Inneren. `-all 1` wandelt jede Folge von Leerraum um. In einer Folge aus mindestens zwei Zeichen, und in jeder Folge am Zeilenanfang, wird jeder erreichte Tabstopp zum Tabulator; was nach dem letzten Tabstopp übrig ist, bleibt Leerzeichen. Ein einzelnes Leerzeichen im Inneren der Zeile bleibt ein Leerzeichen, auch direkt vor einem Tabstopp. Leerzeichen direkt vor einem Tabulator gehen in ihm auf. So verhält sich auch Unix `unexpand` (`-a` für `-all 1`).

`file` wirft bei unbekanntem `-mode {TCLUTILS TUEXPAND MODE}`, bei fehlendem Wert `{TCLUTILS TUEXPAND OPTION}`. `expand` und `unexpand` kennen `-mode` nicht. `-all` gilt nur für `unexpand`; `expand` und `file -mode expand` werfen dafür `{TCLUTILS COMMON OPTION -all}`.

```
package require tclutils::tuexpand
::tclutils::tuexpand::expand "ab\tc" -tabs 4  ;# -> ab  c
::tclutils::tuexpand::unexpand "        x  y" ;# -> \tx  y
::tclutils::tuexpand::unexpand "ab  cd      e" -tabs 4 -all 1
                                             ;# -> ab\tcd\t\te
::tclutils::tuexpand::unexpand "a\tb"     ;# -> a\tb
::tclutils::tuexpand::unexpand "abcdefg          x" -all 1
                                             ;# -> abcdefg\t\t x
```

5.4 Spalten: tucolumn

tucolumn richtet Spalten aus, wie `column` aus util-linux. Es gibt zwei Arten: `table` für Zeilen mit Feldern (`column -t`), `fill` für eine Wortliste, die auf mehrere Spalten verteilt wird.

```
::tclutils::tucolumn::table text  ?option wert ...?
::tclutils::tucolumn::text  text  ?option wert ...?
::tclutils::tucolumn::fill  liste ?-width n? ?-gap n?
::tclutils::tucolumn::file  pfad  ?-mode table|fill? ?option wert ...?
```

Option	Vorgabe	Wirkung
-separator	""	Feldtrenner der Eingabe; leer heißt Folgen von Leerraum
-output	zwei Leerzeichen	Trenner der Ausgabe
-right	0	1 setzt alle Zellen rechtsbündig
-width	80	fill: Zielbreite der Zeile
-gap	2	fill: Leerzeichen zwischen Spalten, mindestens 1
-mode	table	nur file

`text` ist dasselbe wie `table`. Die Spaltenbreite ist die längste Zelle der Spalte, in Zeichen. Die letzte Spalte jeder Zeile füllt `table` nicht auf, so entsteht kein Leerraum am Zeilenende. Zeilen dürfen verschieden viele Felder haben. Ein Trenner aus mehreren Zeichen gilt als Ganzes. Leere Zeilen bleiben als leere Zeilen stehen; `column -t` lässt sie ohne `-e` weg.

`fill` füllt spaltenweise: erst die erste Spalte von oben nach unten, dann die nächste. Passt nicht einmal eine Spalte in `-width`, ist es eine. `file -mode fill` zerlegt die Datei an Leerraum in Wörter. Eine leere Liste ergibt `""`.

`file` wirft bei unbekanntem `-mode {TCLUTILS TUCOLUMN MODE}`, bei fehlendem Wert `{TCLUTILS TUCOLUMN OPTION}`.

```
package require tclutils::tucolumn
::tclutils::tucolumn::table "Name Alter\nBob 7"
                                          ;# -> Name  Alter\nBob   7
::tclutils::tucolumn::table "Name;Alter\nBob;7" -separator ";" \
    -output " | "                         ;# -> Name | Alter\nBob  | 7
::tclutils::tucolumn::fill {a b c d e} -width 7
                                          ;# -> "a  c  e\nb  d"
```

5.5 Tabellen: tutable

tutable setzt eine Tabelle aus Kopfzeile und Zeilen als Text, entweder als Markdown-Tabelle oder mit ASCII-Rahmen. Ein Unix-Vorbild gibt es nicht; für das Ausrichten fertiger Textzeilen ist `tucolumn` da.

```
::tclutils::tutable::render koepfe zeilen ?option wert ...?
```

Option	Vorgabe	Wirkung
-style	markdown	markdown (Pipe-Tabelle) oder box (+, -, |)
-align	leer	je Spalte l, r oder c; fehlende Einträge gelten als l

Die Rückgabe ist ein String ohne abschließendes `\n`. Die Zahl der Spalten bestimmt `koepfe`: Fehlende Zellen einer Zeile bleiben leer. Die Breite zählt Zeichen. Die Trennzeile im Markdown trägt die Ausrichtung (`:--`, `--:`,`:-:`); auch `l` bekommt den Doppelpunkt links. Im Markdown ist jede Spalte mindestens drei Zeichen breit, so breit wie`:--`; Zellen und Trennzeile stehen bündig. Im Rahmen ist eine Spalte so breit wie ihre längste Zelle.

Fehler	-errorcode
-style außer markdown und box	{TCLUTILS TUTABLE OPT}
Ausrichtung außer l, r und c	{TCLUTILS TUTABLE OPT}
Zeile mit mehr Zellen als koepfe	{TCLUTILS TUTABLE COLUMNS}
koepfe leer	{TCLUTILS TUTABLE HEADERS}

```
package require tclutils::tutable
puts [::tclutils::tutable::render {Name Alter} {{Alice 30} {Bob 7}}]
puts [::tclutils::tutable::render {Artikel Menge} \
    {{Äpfel 12} {Birnen 3}} -align {l r} -style box]
```

```
| Name  | Alter |
| :---- | :---- |
| Alice | 30    |
| Bob   | 7     |
+---------+-------+
| Artikel | Menge |
+---------+-------+
| Äpfel   |    12 |
| Birnen  |     3 |
+---------+-------+
```

5.6 Umkehren und Mischen: tutac, turev, tushuf

tutac kehrt die Reihenfolge der Zeilen um (Unix `tac`), turev die Zeichen in jeder Zeile (Unix `rev`), tushuf mischt die Zeilen (Unix `shuf`).

Befehl	Wirkung
tutac::text text, file pfad	Zeilen rückwärts; abschließendes \n bleibt
tutac::lines liste	die Liste rückwärts, wie lreverse
turev::line zeile	die Zeichen rückwärts, wie string reverse
turev::text text, file pfad	jede Zeile für sich; abschließendes \n bleibt
tushuf::lines liste ?opt?	gemischte Liste
tushuf::text text ?opt?, file pfad ?opt?	gemischte Zeilen; abschließendes \n bleibt

`tutac` behandelt die letzte Zeile ohne `\n` wie jede andere: `"a\nb"` wird zu `"b\na"`. Unix `tac` liefert hier `ba` und ein `\n`. `turev` kehrt nach Codepunkten um; zusammengesetzte Zeichen aus Basis und Akzent trennt es.

Option tushuf	Vorgabe	Wirkung
-seed	aus der Uhr	Ganzzahl; gleicher Wert, gleiche Reihenfolge
-count	`-1`	höchstens so viele Zeilen; negativ heißt alle, 0 keine

`tushuf` nutzt einen eigenen Zufallsgenerator, nicht `rand()`. Mit demselben `-seed` ist die Reihenfolge auf jeder Plattform und unter Tcl 8.6 und 9.0 gleich. Der Wert darf beliebig groß sein, auch unter Tcl 8.6 über 32 Bit hinaus; der Generator nutzt seine unteren 31 Bit. Ein `-seed`, der keine Ganzzahl ist, wirft `{TCLUTILS TUSHUF SEED}`, ein solcher `-count {TCLUTILS TUSHUF COUNT}`.

```
package require tclutils::tutac
package require tclutils::turev
package require tclutils::tushuf
::tclutils::tutac::text "a\nb\nc\n"       ;# -> c\nb\na\n
::tclutils::turev::text "abc\nxyz"        ;# -> cba\nzyx
```

```
::tclutils::tushuf::lines {a b c d e} -seed 7
                                          ;# -> b e d a c
::tclutils::tushuf::lines {a b c d e} -seed 7 -count 2
                                          ;# -> b e
```

5.7 Zahlenfolgen: tuseq

tuseq erzeugt Zahlenfolgen wie Unix `seq`.

```
::tclutils::tuseq::range ?erste? ?schritt? letzte ?option wert ...?
::tclutils::tuseq::text  ?erste? ?schritt? letzte ?option wert ...?
```

Ein Wert heißt `letzte` (ab 1, Schritt 1), zwei heißen `erste letzte`, drei `erste schritt letzte`. Optionen dürfen vor, zwischen oder hinter den Werten stehen: Ein Argument, das mit `-` beginnt und keine Zahl ist, leitet eine Option ein. Negative Zahlen gelten als Wert. `range` liefert eine Liste, `text` einen String.

Option	Vorgabe	Wirkung
-format	leer	format-Muster je Zahl, wie seq -f
-equalwidth	0	mit Nullen auf gleiche Breite, wie seq -w
-separator	\n	nur text: Trenner, wie seq -s

Sind alle drei Werte Ganzzahlen, rechnet das Modul ganzzahlig, in beliebiger Größe, auch unter Tcl 8.6 über 32 Bit hinaus. Sonst rechnet es mit Gleitkomma und gibt die Zahlen mit `%g` aus; `1.0` wird so zu `1`. Ein Schritt in die falsche Richtung ergibt eine leere Liste. Ein `-format` für Ganzzahlen (%d) scheitert an Gleitkommawerten mit `{TCL VALUE INTEGER}`. `-equalwidth` setzt das Minuszeichen vor die Nullen. `-separator` gilt nur für `text`; `range` wirft dafür `{TCLUTILS COMMON OPTION -separator}`.

Fehler	-errorcode
null oder mehr als drei Werte	{TCLUTILS TUSEQ ARGS}
Wert ist keine Zahl	{TCLUTILS TUSEQ VALUE}
Schritt 0	{TCLUTILS TUSEQ INCREMENT}
Option ohne Wert	{TCLUTILS TUSEQ OPTION}

```
package require tclutils::tuseq
::tclutils::tuseq::range 5                ;# -> 1 2 3 4 5
::tclutils::tuseq::range 5 -1 1           ;# -> 5 4 3 2 1
::tclutils::tuseq::range 1 0.5 2          ;# -> 1 1.5 2
::tclutils::tuseq::range 8 11 -equalwidth 1
                                          ;# -> 08 09 10 11
::tclutils::tuseq::range -2 1 -equalwidth 1
                                          ;# -> -2 -1 00 01
::tclutils::tuseq::range 1 3 -format %03d ;# -> 001 002 003
::tclutils::tuseq::text -separator , 3    ;# -> 1,2,3
```

```
::tclutils::tuseq::text 3 -separator ,    ;# -> 1,2,3
::tclutils::tuseq::text 1 2 7 -separator ,
                                          ;# -> 1,3,5,7
::tclutils::tuseq::range 5000000000 5000000001
                                          ;# -> 5000000000 5000000001
```

5.8 Zeichen ersetzen: tutr

tutr ersetzt, löscht und verdichtet Zeichen, wie Unix `tr`.

```
::tclutils::tutr::translate text menge1 menge2
::tclutils::tutr::delete    text menge1
::tclutils::tutr::squeeze   text menge1
::tclutils::tutr::file      pfad menge1 menge2 ?option wert ...?
```

Eine Menge besteht aus einzelnen Zeichen und Bereichen `a-z`. Ein `-` am Anfang oder Ende steht für sich selbst. Ein absteigender Bereich wie `c-a` wirft `{TCLUTILS TUTR RANGE}`. `translate` ersetzt das n-te Zeichen von `menge1` durch das n-te von `menge2`; ist `menge2` kürzer, gilt ihr letztes Zeichen für den Rest. Steht ein Zeichen in `menge1` mehrfach, gilt seine letzte Zuordnung. Alle Ersetzungen geschehen in einem Durchgang, `ab` nach `ba` vertauscht also. Ein leeres `menge2` wirft `{TCLUTILS TUTR SET2}`, außer auch `menge1` ist leer. `delete` entfernt die Zeichen der Menge. `squeeze` verdichtet aufeinanderfolgende gleiche Zeichen der Menge zu einem.

Option file	Vorgabe	Wirkung
-delete	0	1 löscht menge1 statt zu ersetzen
-squeeze	0	1 verdichtet danach die Zeichen aus menge2

`file` verlangt `menge2` auch bei `-delete 1`; sie wird dann nur für `-squeeze` gebraucht und darf sonst `""` sein. Mit leerem `menge2`, `-squeeze 1` und ohne `-delete` verdichtet `file` nur die Zeichen aus `menge1`, wie `tr -s menge1`. Ohne `-delete` und `-squeeze` wirft ein leeres `menge2 {TCLUTILS TUTR SET2}`.

Abweichungen von Unix `tr`: keine Klassen wie `[:alpha:]` und keine eigenen Escapes; `\n` oder `\t` schreibt Tcl selbst in den String.

```
package require tclutils::tutr
::tclutils::tutr::translate "hallo welt" a-z A-Z  ;# -> HALLO WELT
::tclutils::tutr::translate "abcde" a-e xy        ;# -> xyyyy
::tclutils::tutr::delete "a1b2c3" 0-9             ;# -> abc
::tclutils::tutr::squeeze "aaabccc  d" "ac "      ;# -> abc d
::tclutils::tutr::translate "abcabc" aa xy         ;# -> ybcybc
```

5.9 Zählen: tuwc

tuwc zählt Zeilen, Wörter, Zeichen und Bytes, wie Unix `wc`.

```
::tclutils::tuwc::text daten
::tclutils::tuwc::file pfad
```

Die Rückgabe ist ein Dict mit `lines`, `words`, `chars`, `bytes`; `file` fügt `file` mit dem Pfad hinzu.

Schlüssel	Bedeutung
lines	Zahl der \n, plus eins, wenn der Text nicht auf \n endet
words	Folgen ohne Leerraum
chars	Zeichen (Codepunkte)
bytes	text: Länge in UTF-8; file: Größe der Datei in Bytes

`lines` weicht von `wc -l` ab: Eine letzte Zeile ohne `\n` zählt mit. Ein leerer Text hat 0 Zeilen, `"\n"` eine. `file` liest mit der Systemkodierung und macht aus `\r\n` ein `\n`; `lines`, `words` und `chars` zählen diesen Text. `bytes` ist bei `file` die Dateigröße, wie bei `wc -c`.

```
package require tclutils::tuwc
set d [::tclutils::tuwc::text "eins zwei\ndrei"]
dict get $d lines                          ;# -> 2
dict get $d words                          ;# -> 3
set d [::tclutils::tuwc::text "Grüße"]
dict get $d chars                          ;# -> 5
dict get $d bytes                          ;# -> 7
```

5.10 Seiten: tupr

tupr teilt Text in Seiten mit Kopfzeile, wie Unix `pr` ohne Spaltensatz.

```
::tclutils::tupr::text text ?option wert ...?
::tclutils::tupr::file pfad ?option wert ...?
```

Option	Vorgabe	Wirkung
-length	66	Zeilen je Seite, mit Kopf und Fuß
-header	""; bei file der Dateiname	Titel im Kopf
-width	72	Breite der Kopfzeile
-number	0	1 nummeriert die Textzeilen (%5d, Tabulator)
-date	jetzt, %Y-%m-%d %H:%M	beliebiger String für den Kopf

Jede Seite hat fünf Kopfzeilen: zwei leere, die Infozeile, zwei leere. Die Infozeile hat links Datum und Titel, mit drei Leerzeichen getrennt, und rechtsbündig `Page n`; ist sie zu lang, steht mindestens ein Leerzeichen vor `Page`. Danach folgen `-length` minus 10 Textzeilen, auf volle Seitenlänge mit leeren Zeilen aufgefüllt, und fünf leere Fußzeilen. Die Seiten stehen ohne Seitenvorschub hintereinander, das Ergebnis endet ohne `\n`. `-width` gilt nur für den Kopf; lange Textzeilen kürzt das Modul nicht. `-number 1` zählt über alle Seiten. Ein leerer Text ergibt `""`.

`-length` von 10 oder weniger wirft `{TCLUTILS TUPR LENGTH}`.

```
package require tclutils::tupr
set seiten [::tclutils::tupr::text "a\nb\nc" -length 12 \
    -header Liste -date Montag -width 24]
set z [split $seiten \n]
llength $z                      ;# -> 24
lindex $z 2                     ;# -> Montag   Liste    Page 1
lrange $z 5 6                   ;# -> a b
lindex $z 14                    ;# -> Montag   Liste    Page 2
```

Nächstes Kapitel: Zusammenführen, Aufteilen, Abläufe

6 Zusammenführen, Aufteilen, Abläufe

Dieses Kapitel setzt Text und Dateien fort. Es behandelt die Module, die mehrere Zeilenströme verbinden, Text auf Dateien verteilen, Abläufe ordnen und Text ungefähr durchsuchen. Jedes Modul hat ein Unix-Vorbild oder lehnt sich an eines an. Keins startet ein Programm; alles läuft in Tcl.

Gruppe	Module	Unix-Vorbild
Zeilenströme verbinden	tupaste, tujoin, tucomm	paste, join, comm
Aufteilen	tucsplit, tusplit	csplit, split
Pipeline-Hilfen	tutee, tuxargs, tutsort	tee, xargs, tsort
Felder und Regeln	tuawk	awk
Ungefähre Suche	tufuzzy, tuagrep	agrep

Gemeinsam für die Textvarianten: Ein Text, der auf `\n` endet, ergibt keine leere letzte Zeile. Optionen stehen paarweise als `-option wert`. Wahrheitswerte nehmen alles, was `string is boolean` kennt.

6.1 Spalten nebeneinander: tupaste

tupaste setzt Zeile *n* jeder Eingabe zu einer Ausgabezeile zusammen, wie Unix `paste`.

```
::tclutils::tupaste::lines lineLists ?-delimiter d?
::tclutils::tupaste::texts textList  ?-delimiter d?
::tclutils::tupaste::files paths     ?-delimiter d?
```

`lines` nimmt eine Liste von Zeilenlisten, `texts` eine Liste von Texten, `files` eine Liste von Pfaden. Alle drei liefern einen String, Zeilen durch `\n` getrennt. `lines` endet ohne `\n`; `texts` und `files` enden mit `\n`, wenn eine der Eingaben so endet (bis tclutils 0.68.0 nie).

Option	Vorgabe	Wirkung
-delimiter	\t	steht zwischen den Spalten; auch mehrere Zeichen oder leer

Ist eine Eingabe kürzer, steht an ihrer Stelle ein leeres Feld. Die Zahl der Ausgabezeilen richtet sich nach der längsten Eingabe.

Abweichungen von `paste`: `-delimiter` gilt als Ganzes für jede Lücke; eine Liste von Trennzeichen, die reihum wechselt (`paste -d ',;'`), gibt es nicht. Ebenso fehlt `paste -s` (eine Eingabe als eine Zeile). Eine unbekannte Option und eine Option ohne Wert melden `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tupaste
::tclutils::tupaste::texts [list "a\nb\nc\n" "1\n2\n"] -delimiter ";"
                                          ;# -> a;1\nb;2\nc;\n
::tclutils::tupaste::lines {{a b} {1 2}} -delimiter " | "
                                          ;# -> a | 1\nb | 2
::tclutils::tupaste::files {namen.txt orte.txt} -delimiter ": "
    ;# -> Ada: London\nLinus: Helsinki
```

Im letzten Aufruf enthält `namen.txt` die Zeilen `Ada` und `Linus`, `orte.txt` die Zeilen `London` und `Helsinki`.

6.2 Über einen Schlüssel verbinden: tujoin

tujoin verbindet zwei getrennte Textquellen über ein Schlüsselfeld, angelehnt an Unix `join`.

```
::tclutils::tujoin::lines leftLines rightLines ?option wert ...?
::tclutils::tujoin::texts leftText  rightText  ?option wert ...?
::tclutils::tujoin::files leftPath  rightPath  ?option wert ...?
```

Option	Vorgabe	Wirkung
-delimiter	\t	Feldtrenner der Eingabe; mehrere Zeichen sind ein literaler Trenner
-joiner	wie -delimiter	Feldtrenner der Ausgabe
-leftfield	1	Schlüsselfeld links, ab 1
-rightfield	1	Schlüsselfeld rechts, ab 1
-header	0	erste Zeile beider Seiten ist Kopfzeile
-outer	none	none, left, right, full
-anti	none	none, left, right, both

Jede Ausgabezeile besteht aus dem Schlüssel, den übrigen Feldern links und den übrigen Feldern rechts. Die Rückgabe ist ein String mit `\n` zwischen den Zeilen. `texts` und `files` hängen ein `\n` an, wenn eine der Eingaben so endet (bis tclutils 0.68.0 nie); `lines` nicht.

- `-outer none` ist der innere Join: nur Zeilen mit Partner.
- `-outer left`, `right`, `full` nehmen Zeilen ohne Partner der genannten Seite mit. Die fehlende Seite wird mit leeren Feldern aufgefüllt, so viele, wie die breiteste Zeile dieser Seite Felder hat.
- `-anti left`, `right`, `both` liefern nur Zeilen ohne Partner: Schlüssel und Felder der eigenen Seite, ohne Auffüllen.

- Mit `-header 1` steht die verbundene Kopfzeile vorn. Bei `-anti` entfällt sie.

Abweichungen von `join`: Die Eingaben müssen nicht sortiert sein; die rechte Seite wird als Index gelesen. Die Ausgabe folgt der Reihenfolge links; Zeilen ohne Partner von rechts folgen danach in ihrer Reihenfolge. Doppelte Schlüssel ergeben wie bei `join` jede Kombination. Vorgabetrenner ist der Tabulator, nicht Leerraum. `-outer` entspricht `join -a`, füllt aber auf; `-anti` entspricht `join -v`.

Fehler	Meldung	-errorcode
unbekannte Option	unknown option "-x"	TCLUTILS COMMON OPTION -x
Option ohne Wert	missing value for option "-x"	TCLUTILS COMMON OPTION -x
Feldnummer kleiner 1	-leftfield must be a positive integer	TCLUTILS COMMON INTEGER -leftfield
-header kein Wahrheitswert	-header requires a boolean value	TCLUTILS COMMON BOOLEAN -header
unbekannter Wert	unknown -outer: x	TCLUTILS COMMON VALUE -outer
-outer und -anti zugleich	cannot combine -outer and -anti	TCLUTILS TUJOIN COMBINE
leerer -delimiter	delimiter must not be empty	TCLUTILS COMMON DELIMITER

Die Meldung zu einem unbekannten Wert nennt in einer zweiten Zeile die erlaubten Werte, etwa `Known: none left right full`; bei `-anti` heißt sie `unknown -anti: x` mit dem `-errorcode TCLUTILS COMMON VALUE -anti`.

```
package require tclutils::tujoin
set L "1;Ada\n2;Linus\n3;Grace\n"
set R "1;Berlin\n2;Helsinki\n4;Paris\n"
::tclutils::tujoin::texts $L $R -delimiter ";"
    ;# -> 1;Ada;Berlin\n2;Linus;Helsinki\n
::tclutils::tujoin::texts $L $R -delimiter ";" -outer full
    ;# -> 1;Ada;Berlin\n2;Linus;Helsinki\n3;Grace;\n4;;Paris\n
::tclutils::tujoin::texts $L $R -delimiter ";" -anti both
                                          ;# -> 3;Grace\n4;Paris\n
::tclutils::tujoin::texts $L $R -delimiter ";" -joiner ,
    ;# -> 1,Ada,Berlin\n2,Linus,Helsinki\n
::tclutils::tujoin::texts "id;name\n1;Ada" "ort;id\nBerlin;1" \
    -delimiter ";" -header 1 -rightfield 2
    ;# -> id;name;ort\n1;Ada;Berlin
```

Für CSV mit Anführungszeichen ist `tujoin` nicht gebaut; das Zerlegen übernimmt `tucsv`, Kapitel Datenformate.

6.3 Zwei Mengen vergleichen: tucomm

tucomm vergleicht zwei sortierte Zeilenlisten wie Unix `comm`.

```
::tclutils::tucomm::compareLines aLines bLines ?-sort b? ?-nocase b?
::tclutils::tucomm::text  aText bText ?-sort b? ?-nocase b?
::tclutils::tucomm::files aFile bFile ?-sort b? ?-nocase b?
::tclutils::tucomm::formatColumns comparison ?-separator s?
```

Option	Vorgabe	Wirkung
-sort	0	beide Eingaben vorher sortieren, in der Ordnung des Vergleichs
-nocase	0	Vergleich ohne Groß/Klein
-separator	\t	nur formatColumns: Einrückung je Spalte

Rückgabe der drei Vergleichsbefehle ist ein Dict mit den Schlüsseln `onlyA` (nur links), `onlyB` (nur rechts) und `both`. Bei `both` steht die Zeile aus der linken Eingabe. Doppelte Zeilen zählen einzeln, wie bei `comm`.

`formatColumns` macht daraus Text im Stil von `comm`: `onlyA` ohne Einrückung, `onlyB` mit einem Trenner davor, `both` mit zwei. Anders als `comm` stehen die Gruppen nacheinander, nicht in einer gemeinsamen sortierten Folge.

Der Vergleich ist ein Mischlauf mit `string compare`, mit `-nocase 1` über die klein geschriebene Zeile. Das ist die Ordnung nach Zeichencodes, wie bei `comm` und `sort` mit dem C-Locale. `-sort 1` sortiert beide Eingaben in genau dieser Ordnung (`lsort` ohne Option, mit `-nocase 1` nach der klein geschriebenen Zeile). Gleiche Zeilen landen damit immer unter `both`, auch bei Ziffern (`a9`, `a10`) und gemischter Schreibung. Unsortierte Eingaben ohne `-sort 1` ergeben ein falsches Ergebnis ohne Fehler. Wer selbst sortiert, nimmt `lsort` ohne Option; `lsort -dictionary` ordnet `a9` vor `a10` und passt nicht zum Mischlauf.

Fehler: unbekannte Option `TCLUTILS COMMON OPTION -name`, ein Wahrheitswert, der keiner ist, `TCLUTILS COMMON BOOLEAN -sort` bzw. `-nocase`.

```
package require tclutils::tucomm
set r [::tclutils::tucomm::text "apfel\nbirne\nkiwi\n" \
    "birne\nkiwi\nmango\n"]
dict get $r onlyA                          ;# -> apfel
dict get $r both                           ;# -> birne kiwi
::tclutils::tucomm::formatColumns $r -separator |
    ;# -> apfel\n|mango\n||birne\n||kiwi
::tclutils::tucomm::compareLines [lsort {a10 a9}] {a10}
    ;# -> onlyA a9 onlyB {} both a10
::tclutils::tucomm::compareLines {a9 a10} {a10} -sort 1
    ;# -> onlyA a9 onlyB {} both a10
::tclutils::tucomm::compareLines {a B} {B} -sort 1
    ;# -> onlyA a onlyB {} both B
```

Die beiden letzten Aufrufe zeigen die Fälle aus dem Absatz oben: `a10` und `B` stehen in beiden Eingaben und erscheinen unter `both`. Ein zeilenweiser Unterschied mit Reihenfolge ist Sache von `tudiff`, Kapitel Kodierung, Hash und Vergleich.

6.4 An Mustern aufteilen: tucsplit

tucsplit schreibt einen Text in mehrere Dateien und beginnt bei jeder Zeile, die zu einem Muster passt, eine neue Datei. Vorbild ist Unix `csplit`.

```
::tclutils::tucsplit::splitText text pattern ?option wert ...?
::tclutils::tucsplit::file      path pattern ?option wert ...?
```

Option	Vorgabe	Wirkung
-regexp	1	Muster als regulärer Ausdruck; 0 heißt string match
-keepmatch	1	die passende Zeile beginnt den neuen Teil; 0 lässt sie weg
-empty	0	leere Teile mitschreiben
-prefix	xx	Anfang des Dateinamens
-suffix	leer	Ende des Dateinamens, etwa .md
-digits	2	Stellen der laufenden Nummer, ab 0
-outdir	.	Zielverzeichnis; fehlt es, wird es angelegt
-encoding	utf-8	Kodierung der Teile; file liest auch darin

Rückgabe ist die Liste der geschriebenen Pfade, mindestens einer: Ein Text ohne Treffer und ein leerer Text ergeben je eine Datei. Vor dem ersten Treffer steht ohne `-empty 1` kein leerer Teil. Jede Zeile behält ihr Zeilenende, die Teile hintereinander ergeben wieder die Eingabe (seit 0.2; bis 0.1 endeten die Teile ohne `\n`). `file` liest die Bytes in `-encoding` und behält CRLF; das Muster sieht die Zeile ohne Zeilenende. Vorhandene Dateien gleichen Namens werden überschrieben.

Abweichungen von `csplit`: Ein einziges Muster gilt für alle Zeilen; es gibt keine Liste von Mustern, keine Zeilennummern als Grenze und keine Wiederholungsangabe `{*}`. Namen und Zählung (`xx00`, `xx01`) sind wie bei `csplit`.

Fehler	-errorcode
-outdir ist eine Datei	TCLUTILS TUCSPLIT OUTDIR
unbekannte Kodierung	TCLUTILS TUCSPLIT ENCODING name
unbekannte Option	TCLUTILS COMMON OPTION -name
-digits kleiner 1	TCLUTILS COMMON INTEGER -digits
kein Wahrheitswert	TCLUTILS COMMON BOOLEAN -name

```
package require tclutils::tucsplit
set dir [file join [pwd] kapitel]
set md "Vorwort\n# Eins\neins\n# Zwei\nzwei\n"
set teile [::tclutils::tucsplit::splitText $md {^# } -outdir $dir]
lmap p $teile {file tail $p}              ;# -> xx00 xx01 xx02
set teile [::tclutils::tucsplit::splitText $md {^# } -outdir $dir \
```

```
    -prefix kap -suffix .md -keepmatch 0]
lmap p $teile {file tail $p}
    ;# -> kap00.md kap01.md kap02.md
```

Im ersten Lauf enthält `xx00` die Zeile `Vorwort\n`, `xx01` den Text `# Eins\neins\n`, `xx02` den Text `# Zwei\nzwei\n`. Im zweiten Lauf fehlen die Überschriften: `kap01.md` enthält nur `eins\n`.

6.5 Nach Größe aufteilen: tusplit

tusplit teilt Text nach Zeilenzahl oder Daten nach Bytezahl, wie Unix `split`.

```
::tclutils::tusplit::splitLines text ?option wert ...?
::tclutils::tusplit::splitBytes data ?option wert ...?
::tclutils::tusplit::file  path ?-lines n | -bytes n? ?option wert ...?
::tclutils::tusplit::lines path n ?option wert ...?
::tclutils::tusplit::bytes path n ?option wert ...?
```

Option	Vorgabe	Wirkung
-lines	1000	Zeilen je Teil (splitLines, file)
-bytes	1024	Bytes je Teil (splitBytes); bei file schaltet die Option in den Bytemodus
-prefix	x	Anfang des Dateinamens
-suffix	leer	Ende des Dateinamens
-digits	2	Stellen der laufenden Nummer, ab 0
-outdir	.	Zielverzeichnis; fehlt es, wird es angelegt
-encoding	utf-8	nur splitLines: Kodierung der Teile

`file` teilt in beiden Modi die Bytes der Datei: CRLF und die Kodierung bleiben, wie sie sind (seit 0.3). Ohne `-lines` und `-bytes` gilt der Zeilenmodus mit 1000 Zeilen. `lines` und `bytes` sind Kurzformen von `file … -lines n` und `file … -bytes n`. Rückgabe ist die Liste der Pfade; eine leere Eingabe ergibt eine leere Datei `x00`.

`splitBytes` erwartet eine Bytefolge, also Zeichen bis `\xFF`. Text mit anderen Zeichen vorher mit `encoding convertto utf-8` umwandeln. Ein Zeichen über `\xFF` ist ein Fehler `TCLUTILS TUSPLIT BYTES`, unter Tcl 8.6 und 9.0 gleich; die Prüfung läuft, bevor eine Datei entsteht.

Abweichungen von `split`: Die Nummer ist dezimal (`x00`, `x01`, wie `split -d`), nicht `xaa`, `xab`. Wie bei `split` behält jede Zeile ihr Zeilenende; die Teile hintereinander ergeben wieder die Eingabe. Bis 0.2 endete jeder Teil ohne `\n`, und zusammengesetzt klebte die letzte Zeile eines Teils an der ersten des nächsten. Eine Aufteilung in eine feste Zahl von Teilen (`split -n`) gibt es nicht.

Fehler	-errorcode
-lines und -bytes zugleich bei file	TCLUTILS TUSPLIT MODE

Fehler	-errorcode
-outdir ist eine Datei	TCLUTILS TUSPLIT OUTDIR
Zeichen über \xFF bei splitBytes	TCLUTILS TUSPLIT BYTES
unbekannte Kodierung bei splitLines	TCLUTILS TUSPLIT ENCODING name
Anzahl oder -digits kleiner 1	TCLUTILS COMMON INTEGER -name
unbekannte Option, Option ohne Wert	TCLUTILS COMMON OPTION -name

```
package require tclutils::tusplit
set dir [file join [pwd] teile]
set teile [::tclutils::tusplit::splitLines "1\n2\n3\n4\n5\n" \
    -lines 2 -outdir $dir]
lmap p $teile {file tail $p}              ;# -> x00 x01 x02
set teile [::tclutils::tusplit::splitBytes abcdefg -bytes 3 \
    -outdir $dir -prefix teil -suffix .bin]
lmap p $teile {file tail $p}
    ;# -> teil00.bin teil01.bin teil02.bin
```

`x00` enthält `1\n2\n`, `x02` nur `5\n`. Die drei Binärteile enthalten `abc`, `def` und `g`.

6.6 In mehrere Ziele schreiben: tutee

tutee schreibt dieselben Daten in mehrere Dateien und auf Wunsch nach `stdout`, wie Unix `tee`.

```
::tclutils::tutee::write data paths ?option wert ...?
::tclutils::tutee::writeFile inputFile paths ?-append b? ?-stdout b?
::tclutils::tutee::copyChannel inChan outChans ?option wert ...?
```

Option	Vorgabe	Befehle	Wirkung
-append	0	write, writeFile	anhängen statt überschreiben
-stdout	0	alle	zusätzlich nach stdout
-nonewline	0	write	kein \n am Ende
-chunksize	8192	copyChannel	Zeichen je Lesevorgang

- `write` schreibt `data` mit `puts`, also mit `\n` am Ende, außer mit `-nonewline 1`; anders als das Kommando `tee`, das nichts anhängt. Rückgabe ist `data`.
- `writeFile` kopiert die Bytes von `inputFile` unverändert, CR und Kodierung eingeschlossen. Rückgabe ist die Zahl der Bytes. `-nonewline` gibt es hier nicht. Bis 0.2 las `writeFile` die Datei als Text und schrieb LF in der Systemkodierung.
- `copyChannel` liest `inChan` bis zum Ende und schreibt jeden Block in alle Kanäle aus `outChans`. Rückgabe ist die Zahl der kopierten Zeichen; bei binär geöffneten Kanälen sind das Bytes. Die Kanäle öffnet und schließt der Aufrufer.

`write` öffnet alle Ziele, bevor es schreibt, und leert sie erst, wenn alle offen sind. Schlägt das Öffnen eines Ziels fehl, kommt der Fehler von `open` (`POSIX ENOENT …`). Die schon geöffneten

Kanäle sind dann wieder geschlossen, vorhandene Dateien behalten ihren Inhalt, und Dateien, die erst dieser Aufruf angelegt hat, sind wieder entfernt. `writeFile` verhält sich ebenso.

Fehler der Optionen: `TCLUTILS COMMON BOOLEAN -name`, `TCLUTILS COMMON INTEGER -chunksize`, `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tutee
set dir [file join [pwd] tee]
file mkdir $dir
set a [file join $dir a.txt]
set b [file join $dir b.txt]
::tclutils::tutee::write hallo [list $a $b]
                                          ;# -> hallo
::tclutils::tutee::write noch [list $a] -append 1 -nonewline 1
                                          ;# -> noch
set in  [open $a rb]
set out [open [file join $dir kopie.txt] wb]
::tclutils::tutee::copyChannel $in [list $out]
                                          ;# -> 10
close $in; close $out
```

Danach enthält `a.txt` den Text `hallo\nnoch`, `b.txt` den Text `hallo\n`, `kopie.txt` dasselbe wie `a.txt`.

6.7 Listen stapelweise abarbeiten: tuxargs

tuxargs teilt eine Liste in Stapel und ruft für jeden Stapel einen Tcl-Befehl auf. Vorbild ist Unix `xargs`; statt eines Programms läuft ein Befehlspräfix.

```
::tclutils::tuxargs::batches items ?-n n? ?-skipempty b?
::tclutils::tuxargs::apply   items commandPrefix ?option wert ...?
::tclutils::tuxargs::command items commandPrefix ?option wert ...?
```

Option	Vorgabe	Befehle	Wirkung
-n	100	alle	höchstens so viele Elemente je Stapel
-skipempty	1	alle	leere Elemente vorher entfernen
-collect	1	apply, command	Ergebnisse sammeln; 0 liefert leer

`batches` liefert die Liste der Stapel. `apply` ruft je Stapel `{*}$commandPrefix {*}$stapel` auf und liefert mit `-collect 1` die Liste der Ergebnisse, ein Element je Stapel. `command` ist derselbe Befehl unter anderem Namen. Ein Fehler im aufgerufenen Befehl bricht den Lauf ab und kommt unverändert beim Aufrufer an.

Abweichungen von `xargs`: Die Eingabe ist eine Tcl-Liste, kein Text, der an Leerraum zerlegt wird; Anführungszeichen und Leerzeichen in Elementen sind damit kein Thema. Es gibt keine Grenze nach Zeichenzahl und keine parallele Ausführung (`xargs -P`). Die Vorgabe `-n 100` ist fest, nicht von der Länge der Kommandozeile abhängig.

Fehler: `-n` kleiner 1 ist `TCLUTILS COMMON INTEGER -n`; dazu `TCLUTILS COMMON BOOLEAN -name` und `TCLUTILS COMMON OPTION -name`.

```
package require tclutils::tuxargs
::tclutils::tuxargs::batches {a b c d e} -n 2
                                          ;# -> {a b} {c d} e
::tclutils::tuxargs::batches {a {} b} -n 2
                                          ;# -> {a b}
::tclutils::tuxargs::apply {1 2 3 4 5} ::tcl::mathop::+ -n 2
                                          ;# -> 3 7 5
::tclutils::tuxargs::apply {a b c} {list x} -n 2
                                          ;# -> {x a b} {x c}
::tclutils::tuxargs::apply $pfade {file delete} -n 50 -collect 0
```

Der letzte Aufruf löscht die Dateien aus `$pfade` in Stapeln zu 50.

6.8 Abhängigkeiten ordnen: tutsort

tutsort sortiert topologisch, wie Unix `tsort`: Jedes Paar `u v` bedeutet "u kommt vor v".

```
::tclutils::tutsort::pairs tokens
::tclutils::tutsort::text  text
::tclutils::tutsort::file  path
```

`pairs` nimmt eine flache Liste und liefert eine Liste. `text` und `file` zerlegen an Leerraum, wie `tsort`, und liefern die Knoten mit `\n` getrennt. Ein Paar `u u` meldet einen Knoten ohne Bedingung an. Die Reihenfolge ist stabil: Von den Knoten, die frei werden, kommt der zuerst gelesene zuerst. Die Ausgabe von GNU `tsort` ist bei gleicher Eingabe anders geordnet, aber ebenso gültig.

Fehler	Meldung	-errorcode
ungerade Zahl von Wörtern	input contains an odd number of tokens	TCLUTILS TUTSORT ODD
Zyklus	input contains a cycle involving: …	TCLUTILS TUTSORT CYCLE

Bei einem Zyklus bricht `tutsort` ab; GNU `tsort` gibt eine Warnung aus und liefert trotzdem eine Folge. Die Meldung nennt nur die Knoten, die auf einem Zyklus liegen, in der Reihenfolge des ersten Auftretens. Knoten, die nur hinter einem Zyklus liegen, fehlen darin.

```
package require tclutils::tutsort
::tclutils::tutsort::pairs {hemd jacke hose guertel hose schuhe}
    ;# -> hemd hose jacke guertel schuhe
::tclutils::tutsort::text "a b\nb c\nx x"
                                          ;# -> a\nx\nb\nc
::tclutils::tutsort::pairs {a b b a b c}
    ;# Fehler: input contains a cycle involving: a b
```

6.9 Felder und Regeln: tuawk

tuawk bringt das Modell von awk nach Tcl: Jede Zeile ist ein Datensatz, wird in Felder zerlegt, und eine Liste von Regeln aus Muster und Aktion läuft darüber. Es ist kein Interpreter der

awk-Sprache. Muster sind Tcl-Ausdrücke, Aktionen sind Tcl-Skripte.

```
::tclutils::tuawk::run  text rules ?-fs s? ?-ofs s? ?-ors s?
::tclutils::tuawk::file path rules ?-fs s? ?-ofs s? ?-ors s?
```

Option	Vorgabe	Wirkung
-fs	leer	Feldtrenner; leer heißt Folgen von Leerraum
-ofs	ein Leerzeichen	steht zwischen den Argumenten von emit
-ors	\n	steht nach jeder Ausgabe von emit

Rückgabe ist die gesammelte Ausgabe aller `emit`-Aufrufe als String. `tuawk` schreibt nichts nach `stdout`. `file` liest die Datei als Text und ruft `run`.

6.9.1 Regeln

`rules` ist eine flache Liste von Paaren `muster aktion`. Die Regeln laufen für jeden Datensatz in ihrer Reihenfolge; jede passende Regel führt ihre Aktion aus.

Muster	Bedeutung
BEGIN	Aktion läuft einmal vor dem ersten Datensatz
END	Aktion läuft einmal nach dem letzten Datensatz
leer ({})	passt immer
/regex/	regulärer Ausdruck gegen die ganze Zeile
sonst	Tcl-Ausdruck für expr, etwa {$2 > 3} oder {$1 eq "x"}

Eine leere Aktion bedeutet `emit $0`, also die Zeile ausgeben.

6.9.2 Felder und Variablen

Name	Inhalt
$0	die ganze Zeile
$1 …	die Felder, ab 1
$NF	Zahl der Felder dieser Zeile; vor dem ersten Datensatz 0
$NR	Nummer der Zeile, ab 1; vor dem ersten Datensatz 0
emit arg …	Ausgabe: Argumente mit -ofs verbunden, -ors danach

Felder sind Variablen und lassen sich in einer Aktion ändern (`set 2 …`); `$0` ändert sich dabei nicht.

Feldtrenner `-fs`: Leer zerlegt wie awk an Folgen von Leerraum und ignoriert Leerraum am Rand. Ein einzelnes Zeichen zerlegt mit `split`; zwei Trenner hintereinander ergeben ein leeres Feld. Mehrere Zeichen gelten als regulärer Ausdruck.

Alle Regeln eines Aufrufs laufen in einem eigenen Rahmen, wie der Rumpf einer Prozedur, in einem Namensraum, der nach dem Lauf gelöscht wird. `set name …` legt immer eine Variable dieses Rahmens an, nie eine globale Variable gleichen Namens; das gilt unter Tcl 8.6 und 9.0

gleich. Variablen aus Aktionen gelten über alle Datensätze bis `END`, wie globale Variablen in awk. Die Aktionen haben den vollen Tcl-Befehlssatz; Regeln aus fremder Quelle sind deshalb wie Code zu behandeln.

In einer Aktion überspringt `break` die übrigen Regeln für den aktuellen Datensatz, wie `next` in awk; `continue` geht zur nächsten Regel.

6.9.3 Unterschiede zu awk

- Ein Feld jenseits von `NF` ist ein Fehler, keine leere Zeichenkette: `can't read "3": no such variable`, `-errorcode TCL LOOKUP VARNAME 3`. Wer mit kürzeren Zeilen rechnet, prüft vorher `$NF`.
- Eine unbelegte Variable ist ebenfalls ein Fehler, nicht 0. Startwerte gehören in `BEGIN`.
- `-fs " "` zerlegt an jedem einzelnen Leerzeichen; die awk-Bedeutung "Leerraum" hat nur der leere Trenner.
- Kein `printf`, kein `getline`, kein eigener Datensatztrenner. Für Formatierung dient `format` im Argument von `emit`.

Fehler	**-errorcode**
unbekannte Option, Option ohne Wert	TCLUTILS TUAWK OPTION
ungerade Zahl von Elementen in rules	TCLUTILS TUAWK RULES
Feld jenseits von NF	TCL LOOKUP VARNAME n

Fehler in Mustern und Aktionen kommen unverändert beim Aufrufer an.

6.9.4 Beispiele

```
package require tclutils::tuawk
set daten "Ada 3\nLinus 5\nGrace 4\n"
::tclutils::tuawk::run $daten {{$2 > 3} {emit $1}}
                                          ;# -> Linus\nGrace\n
::tclutils::tuawk::run $daten {/^L/ {}}  ;# -> Linus 5\n
::tclutils::tuawk::run $daten {{} {emit $NR $NF $1}} -ofs ";"
    ;# -> 1;2;Ada\n2;2;Linus\n3;2;Grace\n
set regeln {
    BEGIN {set s 0}
    {}    {set s [expr {$s + $2}]}
    END   {emit "Summe: $s"}
}
::tclutils::tuawk::run $daten $regeln    ;# -> Summe: 12\n
```

Gruppieren mit einem Dict, Trenner und Ausgabe umstellen:

```
set log "INFO start\nERROR disk\nINFO ende\n"
::tclutils::tuawk::run $log {
    BEGIN {set n {}}
    {}    {dict incr n $1}
    END   {dict for {k v} $n {emit $k $v}}
}                                         ;# -> INFO 2\nERROR 1\n
::tclutils::tuawk::run $log {/ERROR/ {emit $NR $2}}
```

```
                                          ;# -> 2 disk\n
::tclutils::tuawk::run "Ada;3\nLinus;5" {{} {emit $2 $1}} \
    -fs ";" -ofs ,                        ;# -> 3,Ada\n5,Linus\n
::tclutils::tuawk::run "k=v; x=y" {{} {emit $2}} -fs {;\s*}
                                          ;# -> x=y\n
::tclutils::tuawk::run "a b" {{} {emit [format %-4s| $1]}}
                                          ;# -> a   |\n
```

Für ein vollständiges awk-Programm bleibt `exec awk`.

6.10 Ungefähr vergleichen: tufuzzy

tufuzzy stellt die Grundlagen der ungefähren Suche bereit. Das Maß ist der Levenshtein-Abstand: die kleinste Zahl von Einfügungen, Löschungen und Ersetzungen einzelner Zeichen.

```
::tclutils::tufuzzy::distance       a b              ?-nocase b?
::tclutils::tufuzzy::searchDistance pattern text     ?-nocase b?
::tclutils::tufuzzy::similarity     a b              ?-nocase b?
::tclutils::tufuzzy::subsequence    pattern text     ?-nocase b?
::tclutils::tufuzzy::bestMatch      pattern candidates ?-nocase b?
```

Befehl	Rückgabe
distance	Abstand der beiden Zeichenketten, ganze Zahl
searchDistance	kleinster Abstand von pattern zu irgendeinem Ausschnitt von text
similarity	1.0 -abstand /längere Länge, zwischen 0.0 und 1.0; zwei leere Ketten ergeben 1.0
subsequence	1, wenn die Zeichen von pattern in dieser Reihenfolge in text vorkommen, sonst 0
bestMatch	der Kandidat mit dem kleinsten Abstand; bei Gleichstand der erste; leere Liste ergibt ""

Option	Vorgabe	Wirkung
-nocase	0	beide Seiten vorher mit string tolower

`subsequence` ist die Suche, wie sie Auswahllisten im Stil von fzf verwenden: `tcu` passt auf `tclutils`, `utc` nicht. Die Rechnung läuft über Zeichen, nicht über Bytes; Umlaute zählen als ein Zeichen.

Fehler	Meldung	-errorcode
unbekannte Option	unknown option -x	TCLUTILS TUFUZZY OPTION
Option ohne Wert	missing value for option -x	TCLUTILS TUFUZZY OPTION
überzählige Argumente	unexpected arguments: …	TCLUTILS TUFUZZY ARGS

Fehler	Meldung	-errorcode
-nocase kein Wahrheitswert	-nocase requires a boolean value	TCLUTILS COMMON BOOLEAN -nocase

```
package require tclutils::tufuzzy
::tclutils::tufuzzy::distance kitten sitting     ;# -> 3
::tclutils::tufuzzy::distance Haus haus -nocase 1
                                                 ;# -> 0
::tclutils::tufuzzy::searchDistance gama "gamma line"
                                                 ;# -> 1
::tclutils::tufuzzy::similarity kitten sitting
                                              ;# -> 0.5714285714285714
::tclutils::tufuzzy::subsequence tcu tclutils    ;# -> 1
::tclutils::tufuzzy::bestMatch tujion {tujoin tupaste tucomm}
                                                 ;# -> tujoin
```

6.11 Ungefähr suchen: tuagrep

tuagrep ist ein grep mit Fehlertoleranz, aufgebaut auf `tufuzzy::searchDistance`. Eine Zeile passt, wenn der Abstand des Musters zu einem Ausschnitt der Zeile höchstens `-maxdist` ist. Vorbild ist Unix `agrep`.

```
::tclutils::tuagrep::search text     pattern ?option wert ...?
::tclutils::tuagrep::match  line     pattern ?option wert ...?
::tclutils::tuagrep::file   filename pattern ?option wert ...?
::tclutils::tuagrep::files  fileList pattern ?option wert ...?
```

Option	Vorgabe	Wirkung
-maxdist	1	größter erlaubter Abstand, ganze Zahl ab 0; 0 heißt exakter Teilstring
-nocase	0	ohne Groß/Klein
-invert	0	die Zeilen ohne Treffer
-linenumbers	0	jede Zeile als {nummer text}, Nummer ab 1
-count	0	nur die Anzahl
-filenames	0	nur file: Treffer als {datei treffer}
-fileswithmatches	0	nur file, files: nur die Pfade mit Treffer

Befehl	Rückgabe
search	Liste der Treffer, mit -count 1 die Anzahl
match	1 oder 0 für eine Zeile; beachtet -maxdist, -nocase, -invert
file	wie search; mit -fileswithmatches 1 der Pfad oder ""

Befehl	Rückgabe
files	Treffer aller Dateien, immer mit Dateinamen; mit -count 1 ein Dict pfad anzahl; mit -fileswithmatches 1 die Liste der Pfade

Die Optionen gleichen denen von `tugrep` aus Text und Dateien. `tuagrep` kennt kein `-max`, kein `-word` und kein `-wholeline`, und das Muster ist immer literal: keine regulären Ausdrücke. `agrep` erlaubt dagegen Muster mit Fehlertoleranz und Gewichte für Einfügen, Löschen und Ersetzen; `-maxdist` entspricht `agrep -k` bzw. `-1` bis `-9`.

`search` zerlegt den Text an `\n` in dieselben Zeilen, die `file` liest: Ein `\n` am Ende schließt die letzte Zeile ab und beginnt keine leere; ein leerer Text hat keine Zeile. Ein leeres Muster passt auf jede Zeile. Ein Text hat keinen Dateinamen; `search` und `match` nehmen `-filenames` und `-fileswithmatches` an, lassen sie aber ohne Wirkung. `files` setzt den Dateinamen immer dazu.

Fehler	Meldung	-errorcode
unbekannte Option	unknown option -x	TCLUTILS TUAGREP OPTION
Option ohne Wert	missing value for option -x	TCLUTILS TUAGREP OPTION
überzählige Argumente	unexpected arguments: …	TCLUTILS TUAGREP ARGS
-maxdist keine ganze Zahl ab 0	-maxdist must be a non-negative integer	TCLUTILS TUAGREP MAXDIST
Schalter kein Wahrheitswert	-invert requires a boolean value	TCLUTILS COMMON BOOLEAN -name

Eine Datei, die sich nicht öffnen lässt, meldet `open` mit `POSIX ENOENT …`.

```
package require tclutils::tuagrep
set text "gamma line\nbeta line\ndelta"
::tclutils::tuagrep::search $text gama   ;# -> {gamma line}
::tclutils::tuagrep::search $text gama -maxdist 0
                                         ;# ->
::tclutils::tuagrep::search $text gama -linenumbers 1
                                         ;# -> {1 {gamma line}}
::tclutils::tuagrep::search $text gama -invert 1
                                         ;# -> {beta line} delta
::tclutils::tuagrep::match Hauptstrasse strase
                                         ;# -> 1
::tclutils::tuagrep::search "eins\nzwei\n" xyz -invert 1
                                         ;# -> eins zwei
::tclutils::tuagrep::search $text gama -maxdist x
    ;# Fehler: -maxdist must be a non-negative integer
```

Mit Dateien; `a.txt` enthält `gamma line` und `beta`, `b.txt` enthält `nichts` und `gamma`, `c.txt` enthält `leer`:

```
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama
    ;# -> {a.txt {gamma line}} {b.txt gamma}
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama -count 1
                                          ;# -> a.txt 1 b.txt 1 c.txt 0
::tclutils::tuagrep::files {a.txt b.txt c.txt} gama \
    -fileswithmatches 1                   ;# -> a.txt b.txt
```

Nächstes Kapitel: Zeichen, Zahlen, Datum und Pfade

7 Zeichen, Zahlen, Datum und Pfade

Vier Module für das, was der Tcl-Kern nur umständlich oder gar nicht liefert: sichtbare Zeichen (tustr), Beträge (tunum), Kalenderdaten (tudate) und Pfade ohne Dateisystem (tupath). Alle vier liegen im Umbrella, keines braucht Tk.

7.1 tustr

`tustr` ergänzt `string`. Was der Kern kann (`string reverse`, `string map`, `string repeat`), bleibt beim Kern.

Befehl	Wirkung
isEmpty s	1 bei leerem String
truncate s n ?ende?	kürzt auf n Zeichen; Vorgabe für ende ist ..., die Zeichen zählen mit
padLeft, padRight, center	füllen auf eine Breite; Vorgabezeichen ist das Leerzeichen
startsWith, endsWith	1, wenn der Rand passt
removePrefix, removeSuffix	schneiden den Rand ab, sonst unverändert
splitTrim s ?sep?	trennt, entfernt Leerzeichen, wirft leere Stücke weg; Vorgabe für sep ist das Leerzeichen
toCamel, toSnake	my-cool_var wird myCoolVar, myCoolVar wird my_cool_var; Buchstaben jeder Schrift zählen
slugify s	ASCII-Kleinbuchstaben: Umlaute umschrieben, sonst alles außer a–z und Ziffern zu -
capitalize s	erster Buchstabe groß
count s teil	nicht überlappende Treffer
graphemes s	Liste der sichtbaren Zeichen
isPrintable zeichen	1, wenn genau ein druckbares Zeichen vorliegt

`truncate`, `padLeft`, `padRight` und `center` rechnen mit `string length`, nicht mit `graphemes`. Eine negative Breite oder Länge ergibt `{TCLUTILS TUSTR ARG}`.

```
package require tclutils::tustr
::tclutils::tustr::truncate HalloWelt 6          ;# -> Hal...
::tclutils::tustr::toCamel my-cool_var          ;# -> myCoolVar
::tclutils::tustr::toSnake myCoolVar            ;# -> my_cool_var
::tclutils::tustr::splitTrim { a , , b } ,      ;# -> a b
::tclutils::tustr::count ababa ba               ;# -> 2
::tclutils::tustr::slugify "Hallo, Welt!"       ;# -> hallo-welt
::tclutils::tustr::slugify "Größe"              ;# -> groesse
::tclutils::tustr::toCamel "Jörg Müller"        ;# -> jörgMüller
```

`slugify` liefert ASCII für Dateinamen und URLs. Lateinische Buchstaben schreibt es vorher um: ä wird `ae`, ö `oe`, ü `ue`, ß `ss`, Akzente fallen weg (é wird `e`, ł wird `l`). Was keine ASCII-Form hat, etwa `??`, fällt weg. `toCamel` und `toSnake` lassen Buchstaben jeder Schrift stehen (seit 0.3; bis 0.2 wurde "Größe" zu `gr-e` und `toCamel "Jörg Müller"` zu `jRgMLler`).

7.1.1 Sichtbare Zeichen

`graphemes` zerlegt einen String in sichtbare Zeichen, ein vereinfachtes UAX #29. Basiszeichen mit kombinierenden Akzenten, Emoji-ZWJ-Folgen, Flaggenpaare und CR LF bleiben zusammen. Hangul-Silben und indische Konjunkte deckt es nicht ab. `[join [graphemes $s] ""]` ergibt wieder `$s`.

`isPrintable` prüft genau ein Zeichen nach Unicode, nicht nur ASCII: 0 für C0/C1-Steuerzeichen und DEL, 1 für U+00A0. Ein leerer String oder mehr als ein Zeichen wirft `{TCLUTILS TUSTR ARG}` mit `isPrintable expects exactly one character`. Ersatzpaare, die Tcl 8.6 aus einem Kanal liefert, zählen bei beiden Befehlen als ein Zeichen.

```
llength [::tclutils::tustr::graphemes "e\u0301x"]   ;# -> 2
string length "e\u0301x"                           ;# -> 3
::tclutils::tustr::isPrintable \u00e4               ;# -> 1
::tclutils::tustr::isPrintable \x07                 ;# -> 0
```

7.2 tupath

`tupath` liegt um `file`. `clean` und `relative` fragen das Dateisystem nicht; `normalize` und `readlink` tun es.

Befehl	Wirkung
normalize pfad	absolut, löst ., .. und symbolische Links (file normalize)
clean pfad	., .. und // nur lexikalisch; ein leerer Pfad wird .
isAbsolute pfad	1 oder 0
components pfad	file split
join teile…	file join
relative basis ziel	Weg von der Basis zum Ziel, beide absolut oder beide relativ
commonPath liste	längstes gemeinsames Verzeichnis, sonst ""

Befehl	Wirkung
readlink pfad	Ziel eines symbolischen Links

```
package require tclutils::tupath
::tclutils::tupath::clean a/./b/../c             ;# -> a/c
::tclutils::tupath::clean /a/../../b             ;# -> /b
::tclutils::tupath::clean ""                     ;# -> .
::tclutils::tupath::commonPath {/a/b/c /a/b/d}   ;# -> /a/b
::tclutils::tupath::relative /a/b /a/b/c/d       ;# -> c/d
::tclutils::tupath::relative /a/b /a/b           ;# -> .
```

Seit tclutils 0.64.0 behält ein absoluter Pfad seine Wurzel: `..` endet dort. Unter Windows gilt das laut Code auch für `C:/`, `/` (laufwerksrelativ) und `//server/share`. Die Modulversion blieb 0.1.

`relative` mit einem relativen und einem absoluten Pfad wirft `{TCLUTILS TUPATH RELATIVE}`. `commonPath` ohne Pfad wirft `{TCLUTILS TUPATH ARGS}`. `readlink` auf einer gewöhnlichen Datei wirft `{TCLUTILS TUPATH NOTLINK}`. Teilen sich die Pfade kein erstes Stück, liefert `commonPath ""`.

7.3 tudate

`tudate` rechnet in der lokalen Zeitzone, auf `clock`.

Befehl	Wirkung
parse text ?-format fmt?	Sekunden seit der Epoche; ohne -format eine Liste von Formaten, danach ein freier Versuch
iso sekunden ?-time 0?	yyyy-mm-dd, mit -time 1 zusätzlich THH:MM:SS
add sekunden zahl einheit	Kalenderaddition; Einheit second, minute, hour, day, week, month, year, auch im Plural
diff a b ?-unit days?	a minus b, ganzzahlig; Einheiten seconds, minutes, hours, days, weeks
relative sekunden ?-base sek?	englische Tagesformel; ohne -base ist die Basis jetzt
today	heutiges Datum als ISO-Text
zones	sortierte Namen für clock format -timezone

```
package require tclutils::tudate
set t [::tclutils::tudate::parse 15.03.2024]
::tclutils::tudate::iso $t                              ;# ->
    2024-03-15
::tclutils::tudate::iso [::tclutils::tudate::add $t 1 month]
                                                        ;# ->
    2024-04-15
```

```
::tclutils::tudate::relative $t -base $t                ;# -> today
::tclutils::tudate::relative [::tclutils::tudate::add $t 3 day] -base
    $t
                                                        ;# -> in 3 days
expr {"Europe/Berlin" in [::tclutils::tudate::zones]}   ;# -> 1
```

Ohne `-format` probiert `parse` der Reihe nach ISO-Datum und ISO-Zeitpunkt (auch ohne Sekunden und als `YYYYMMDD`), `d.m.Y` (auch mit Uhrzeit), `d/m/Y`, `Y/m/d`, `m/d/Y`, `H:M:S`, `H:M` und englische Monatsnamen (`29 Oct 2025`, `Oct 29 2025`). `02/03/2026` ist der

1. März; `m/d/Y` greift erst, wenn `d/m/Y` kein Datum ergibt, so wird

`10/29/2026` der 29. Oktober. Eine Zeichenkette nur aus Ziffern und Trennzeichen geht nie an den freien Versuch.

Einen Tag oder Monat, den es nicht gibt, lehnt `parse` ab, mit und ohne `-format`, unter Tcl 8.6 wie unter 9.0: `2026-02-29`, `31.04.2026`, `29/02/2026` und `Feb 30 2026` ergeben `{TCLUTILS TUDATE PARSE}`. `clock scan` allein macht unter 8.6 daraus den 1. März, den 1. Mai und den 2. Mai 2028; `parse` formatiert jeden Treffer zurück und verwirft ihn, wenn die Zahlen nicht stimmen. Bis tclutils 0.68.0 rutschten diese Tage unter 8.6 durch.

```
::tclutils::tudate::iso [::tclutils::tudate::parse 10/29/2026]
                                                        ;# ->
    2026-10-29
catch {::tclutils::tudate::parse 2026-02-29} m o
dict get $o -errorcode                                  ;# -> TCLUTILS
    TUDATE PARSE
```

`relative` antwortet englisch: `today`, `tomorrow`, `yesterday`, `in N days`, `N days ago`. Ein unlesbares Datum ergibt `{TCLUTILS TUDATE PARSE}`, eine unbekannte Einheit `{TCLUTILS TUDATE UNIT}`.

`diff -unit days` teilt die Sekunden durch 86400 und schneidet ab. Es zählt keine Kalendertage. Vom 15. März 2024 zum 15. April 2024 liefert es 30, nicht 31: in der lokalen Zone (CEST, TZ nicht gesetzt) fehlt durch die Sommerzeit eine Stunde. `add` selbst geht über `clock add` und landet auf dem 15. April.

`zones` liest die erste Quelle, die Einträge hat: Tcls tzdata, sonst `/usr/share/zoneinfo`. Namen aus dem Systemverzeichnis prüft es mit `clock`, weil Tcl 8.6 ohne eigene tzdata Namen wie `Etc/GMT+1` ablehnt. `UTC` ist immer dabei. Die Liste mischt die beiden Quellen nicht.

7.4 tunum

`tunum` liest Beträge, wie sie in Tabellen stehen, und schreibt sie deutsch. `tunumfmt` ist ein anderes Modul: das liest Größen wie `1.5K`, `2Mi` und `2KiB`.

Befehl	Wirkung
parse text ?-default {}? ?-locale auto?	Tcl-Double, oder -default, wenn nichts erkennbar ist
sum werte ?-default 0?	Summe, unlesbare Einträge fallen weg; immer Locale auto

Befehl	Wirkung
isNumber text	1, wenn parse mit auto einen Wert findet
format zahl ?-locale de? ?-decimals 2?	Tausenderpunkt, Dezimalkomma

Locale `auto` (Vorgabe von `parse`): ein Komma mit Ziffern am Ende ist das Dezimalkomma, Punkte sind Tausender. Sonst ist der Punkt dezimal und die Kommas sind Tausender. `€`, `$`, `£`, `¥` und Leerzeichen fallen weg.

Locale `de-strict`: der Punkt ist immer ein Tausender und fällt weg, das Komma ist dezimal, Währungszeichen bleiben stehen. `956.06` wird damit `95606.0`. Wer den Punkt als Dezimalzeichen meint, nimmt `auto`.

```
package require tclutils::tunum
::tclutils::tunum::parse "1.234,56"                ;# -> 1234.56
::tclutils::tunum::parse "1,234.56"                ;# -> 1234.56
::tclutils::tunum::parse "12 \u20AC"               ;# -> 12.0
::tclutils::tunum::parse "abc"                     ;# -> (leer)
::tclutils::tunum::sum {1,50 2,00 abc 4,20}        ;# -> 7.7
::tclutils::tunum::parse 956.06 -locale de-strict  ;# -> 95606.0
::tclutils::tunum::format 1234.5                   ;# -> 1.234,50
::tclutils::tunum::format 1234.5 -decimals 0       ;# -> 1.235
::tclutils::tunum::format 2.675                    ;# -> 2,68
::tclutils::tunum::format -2.5 -decimals 0         ;# -> -3
format %.2f 2.675                                  ;# -> 2.67
```

`format` rundet seit Modul 0.4 auf den Dezimalziffern halb von null weg, auf jeder Plattform gleich. Bis tclutils 0.63.0 lief das über `format %.*f`, und dort entschied die C-Laufzeit. Das abweichende Verhalten der Windows-Laufzeit vor 0.4 steht im CHANGELOG. Eine Währung hängt der Aufrufer selbst an: `"[::tclutils::tunum::format $n] EUR"`.

Die Prozedur heißt `format` und liegt im Namensraum `tclutils::tunum`. Der Aufruf bleibt `::tclutils::tunum::format`, dann verdeckt sie Tcls `format` nicht. `-decimals` unter 0 ergibt `{TCLUTILS TUNUM DECIMALS}`, eine andere Locale als `de` ergibt `{TCLUTILS TUNUM LOCALE}`. `parse -locale` kennt `auto` und `de-strict`.

7.5 Merkliste

- Sichtbare Länge: `tustr::graphemes`, nicht `string length`.
- `slugify` liefert ASCII und schreibt Umlaute um: "Größe" wird `groesse`.
- `tupath::clean` und `relative` lesen keine Datei; `..` endet an der Wurzel.
- `tudate::diff -unit days` teilt Sekunden; `relative` antwortet englisch.
- Beträge: `tunum::format`. `de-strict` behandelt jeden Punkt als Tausender.
- Bundesweite Feiertage: Kapitel Notizen und Kalender.

Nächstes Kapitel: Datenformate

8 Datenformate

CSV, JSON und INI sind Pure Tcl und brauchen kein Fremdpaket. Beträge liest `tunum` im Kapitel

Zeichen, Zahlen, Datum und Pfade, den Text einer PDF das Kapitel Text aus PDF-Dateien. Base64, Digests und Dicts stehen im Kapitel Kodierung, Hash und Vergleich. Markdown, iCalendar, vCard und Notizen stehen im Kapitel Notizen und Kalender.

8.1 CSV: tucsv

tucsv zerlegt Zeilen, fügt sie wieder zusammen und liest eine Tabelle in Dicts. Es ist kein Import-Rahmen.

Befehl	Wirkung
parseLine zeile	eine Zeile zur Liste; -delimiter (Vorgabe Komma), -quote (Vorgabe "), -trim (Vorgabe 0)
joinLine felder	eine Liste zur Zeile; dieselben -delimiter und -quote, dazu -alwaysQuote (Vorgabe 0)
parse text ?optionen?	Zeilen zu einer Liste von Feldlisten; ein Feld darf einen Zeilenumbruch enthalten
text zeilen ?optionen?	Feldlisten zurück zum Text; zusätzlich -newline, -bom, -profile
dicts zeilen	erste Zeile wird Kopf, jede weitere ein Dict
file pfad ?optionen?	Datei wie parse; zusätzlich -encoding (Vorgabe utf-8), -profile
writeFile pfad zeilen ?optionen?	Datei schreiben; wie text, dazu -encoding

Ein Feld mit Komma, Anführungszeichen oder Zeilenumbruch wird in Anführungszeichen gesetzt. Ein Anführungszeichen im Feld verdoppelt sich. `-alwaysQuote 1` setzt jedes Feld in Anführungszeichen.

```
package require tclutils::tucsv
::tclutils::tucsv::parseLine {a,"b,c","d""e"}
                                          ;# -> a {b,c} {d"e}
::tclutils::tucsv::joinLine [list a {b,c} {d"e}]
                                          ;# -> a,"b,c","d""e"
set zeilen [::tclutils::tucsv::parse "name;city\nAlice;Berlin"
    -delimiter {;}]
                                          ;# -> {name city} {Alice
    Berlin}
::tclutils::tucsv::dicts $zeilen          ;# -> {name Alice city
    Berlin}
::tclutils::tucsv::parse "a,\"b\nc\",d"   ;# -> a {b\nc} d
```

`-strict 1` ist die Vorgabe. Ein nicht geschlossenes Anführungszeichen wirft `{TCLUTILS TUCSV QUOTE}` mit `unterminated quoted field`. `-strict 0` nimmt den Rest als Feld und wirft nicht. Ein führendes UTF-8-BOM entfernt `parse` von selbst, auch das aus `-profile excel-de`.

Für Excel gibt es seit `tucsv` 0.2 Profile. `-profile excel-de` setzt Semikolon, CRLF und ein

UTF-8-BOM, damit ein deutsches Excel Umlaute und Spalten richtig erkennt. `-profile rfc4180` setzt Komma und CRLF. Eine Option nach dem Profil gilt vor dem Profil. Ein unbekanntes Profil wirft `{TCLUTILS TUCSV PROFILE <name>}`.

```
package require tclutils::tucsv 0.2
set zeilen {{Name Ort} {Müller Köln} {{A, B} Essen}}
::tclutils::tucsv::writeFile liste.csv $zeilen -profile excel-de
                                              ;# BOM, "Name;Ort" CRLF ...
::tclutils::tucsv::file liste.csv -delimiter {;}
                                              ;# -> {Name Ort} {Müller
    Köln} {{A, B} Essen}
::tclutils::tucsv::text {{a b}} -profile excel-de -delimiter ,
                                              ;# BOM + a,b
```

`file` und `writeFile` lesen und schreiben UTF-8, wenn `-encoding` nichts anderes sagt. `writeFile` schreibt die Zeilenenden aus `-newline` unverändert; ohne `-newline` gilt die Vorgabe der Plattform.

Felder ohne Anführungszeichen, nur am Trennzeichen geschnitten, gehören zu `tucut` im Kapitel Text und Dateien.

8.2 JSON: tujson

tujson liest und schreibt JSON. Es ist kein JSON5: keine Kommentare, keine nachhängenden Kommas, kein JSONPath, kein Strom.

`parse` und `fromJson` liefern Tcl-Werte. Dabei geht der Unterschied von Objekt, Array und Skalar verloren.

JSON	Tcl aus parse
Objekt	Dict, Schlüssel in der Reihenfolge des Texts
Array	Liste
String	String
Zahl	der Zifferntext aus der Quelle
true / false	1 / 0
null	leerer String

Steht ein Schlüssel in einem Objekt zweimal, gilt der letzte Wert, wie in JavaScript; RFC 8259 verlangt nur, dass Namen eindeutig sein sollten.

```
package require tclutils::tujson
set d [::tclutils::tujson::parse
    {{"name":"Gregor","items":[1,2],"ok":true,"x":null}}]
dict get $d name                              ;# -> Gregor
dict get $d items                             ;# -> 1 2
dict get $d ok                                ;# -> 1
dict get $d x                                 ;# -> (leer)
```

Ein fehlerhafter Text wirft `{TCLUTILS TUJSON PARSE offset}`. Bei `{` allein lautet die Meldung `invalid JSON at offset 1: expected object key string`.

`parseTyped` behält die Art. Jeder Knoten ist `{art wert}`:

Art	Wert
object	Dict, Schlüssel auf einen Knoten
array	Liste von Knoten
string	der String
number	die Ziffern aus der Quelle
boolean	true oder false
null	leer

```
::tclutils::tujson::parseTyped {{"tags":["x"],"n":1}}
    ;# -> object {tags {array {{string x}}} n {number 1}}
```

`toJson` schreibt so einen Baum zurück. Die Konstruktoren `str`, `num`, `bool`, `null`, `obj` und `arr` bauen ihn von Hand. `-indent N` mit N > 0 bricht Zeilen um, je Stufe N Leerzeichen. Die Vorgabe 0 ist eine Zeile. Eine unbekannte Art wirft `{TCLUTILS TUJSON TYPE}`.

```
set doc [::tclutils::tujson::obj [list \
    name [::tclutils::tujson::str Alice] \
    n    [::tclutils::tujson::num 1] \
    z    [::tclutils::tujson::null]]]
::tclutils::tujson::toJson $doc
    ;# -> {"name":"Alice","n":1,"z":null}
```

`quote` und `escape` setzen einen String in JSON-Anführungszeichen. `validate` prüft, ohne einen Wert zu bauen. `minify` und `pretty` formatieren Text, `fileMinify` und `filePretty` eine Datei. Auf der Kommandozeile: `tclsh bin/tujson.tcl validate|minify|pretty|parse datei`.

`true` als `1` und `null` als leer sind Absicht von `parse`. Wer die Art braucht, nimmt `parseTyped`.

8.3 INI: tuini

tuini liest INI in ein Dict: Abschnitt auf ein Dict von Schlüssel zu Wert. Schlüssel vor der ersten `[section]` liegen im Abschnitt `""`. Die Reihenfolge bleibt. Kommentare mit `#` oder `;` fallen weg.

Befehl	Wirkung
parse text	das Dict
toIni dict	INI-Text, schlüssel=wert ohne Leerzeichen um =
sections dict	die Abschnittsnamen

Befehl	Wirkung
keys dict ?abschnitt?	die Schlüssel
get dict abschnitt schlüssel ?vorgabe?	der Wert, oder die Vorgabe
has dict abschnitt schlüssel	1 oder 0
setValue dict abschnitt schlüssel wert	eine Kopie mit dem neuen Wert
addSection dict abschnitt	Kopie, Abschnitt ist da
removeKey, removeSection	Kopie ohne Schlüssel oder Abschnitt

`setValue` ändert das übergebene Dict nicht.

```
package require tclutils::tuini
set ini [::tclutils::tuini::parse "# k\n\n\[lager\]\nort = Nord\n"]
::tclutils::tuini::sections $ini             ;# -> lager
::tclutils::tuini::get $ini lager ort        ;# -> Nord
::tclutils::tuini::get $ini lager fehlt -    ;# -> -
set ini [::tclutils::tuini::setValue $ini lager gang 3]
::tclutils::tuini::toIni $ini                ;# ->
    \[lager\]\nort=Nord\ngang=3
```

Steht ein Schlüssel in einem Abschnitt zweimal, gilt der letzte Wert; Windows (`GetPrivateProfileString`) nimmt den ersten. Leerzeichen am Rand von Schlüssel und Wert fallen weg, ein Wert kann also nicht auf ein Leerzeichen enden.

Ein nicht geschlossener Abschnitt wirft `{TCLUTILS TUINI SYNTAX}`, etwa `unterminated section header: [offen`. Was INI nicht fassen kann, lehnen `setValue`, `addSection` und `toIni` seit 0.2 mit `{TCLUTILS TUINI VALUE}` ab: einen Zeilenumbruch im Wert; im Schlüssel =, einen Zeilenumbruch, `;`, `#` oder `[` am Anfang, Leerzeichen am Rand; im Abschnittsnamen `]`, einen Zeilenumbruch, Leerzeichen am Rand. Bis 0.1 schrieb `toIni` solchen Text, und `parse` scheiterte daran.

```
catch {::tclutils::tuini::setValue {} lager notiz "a\nb"} m
set m                                        ;# -> value of lager.notiz:
    no line breaks in INI
```

`tusettings`, Kapitel tusettings, prüft dieselben Grenzen schon beim Speichern und lehnt zusätzlich Leerzeichen am Rand eines Werts ab.

8.4 XML: tuxml

tuxml schreibt kleine Elemente. Es prüft kein Dokument und kennt kein XPath. `escape` setzt &, < und > um; mit `-quotes 1` auch Anführungszeichen. `unescape` löst die vordefinierten Entitäten und `&#…;` auf. `textElement` maskiert den Text, `element` fügt ihn roh ein.

```
package require tclutils::tuxml
::tclutils::tuxml::declaration               ;# -> <?xml version="1.0"
    encoding="UTF-8"?>
::tclutils::tuxml::textElement lager {ort Nord} {a < b}
                                             ;# -> <lager ort="Nord">a
    &lt; b</lager>
```

Ein Name, der nicht mit Buchstabe oder `_` beginnt, wirft `{TCLUTILS TUXML NAME <name>}`, ein Attributname `{TCLUTILS TUXML ATTR <name>}`.

8.5 SQLite: tusqlite

tusqlite öffnet keine Datenbank und lädt `sqlite3` nicht. Der Aufrufer übergibt ein offenes Handle. `insert` baut die Anweisung in einem eigenen Geltungsbereich, damit ein Wert nicht an eine Variable des Aufrufers bindet. `rows` liefert Dicts in der Reihenfolge der Zeilen. `value` nimmt die erste Spalte der ersten Zeile, sonst die Vorgabe. `value db sql -default d -params dict` hält Vorgabe und Platzhalter auseinander; die Stellungsform `value db sql vorgabe ?dict?` gilt weiter. Ohne Optionen wurde `value db $sql {id 1}` die Vorgabe, und das Dict kam zurück, wenn keine Zeile passte.

Seit `tusqlite` 0.2 nehmen `rows` und `value` ein Dict mit Werten für die Platzhalter `:name`. Die Abfrage läuft ebenfalls in einem eigenen Geltungsbereich: Variablen des Aufrufers erreicht sie nicht, ein Platzhalter ohne Wert im Dict wird SQL-NULL. `update` und `delete` nehmen die Bedingung als Dict (`spalte wert`, mit UND verknüpft) und geben die Zahl der geänderten Zeilen zurück. Ein leeres Bedingungs-Dict lehnen sie ab, damit ein Fehler nicht alle Zeilen trifft.

Ein Wert gleich `[tusqlite::null]` wird SQL-NULL. `rows` gibt diese Zelle als leere Zeichenkette zurück. `quoteId` setzt einen Namen in Anführungszeichen und verdoppelt ein Anführungszeichen darin.

```
package require sqlite3
package require tclutils::tusqlite
sqlite3 db :memory:
db eval {create table lager (ort text, gang integer)}
::tclutils::tusqlite::insert db lager {ort Nord gang 3}
                                             ;# -> 1
::tclutils::tusqlite::rows db {select ort, gang from lager}
                                             ;# -> {ort Nord gang 3}
::tclutils::tusqlite::value db {select gang from lager where ort =
    'x'} -
                                             ;# -> -
::tclutils::tusqlite::insert db lager {ort Sued gang 5}
                                             ;# -> 2
::tclutils::tusqlite::rows db {select ort, gang from lager where gang
    > :min} {min 4}
                                             ;# -> {ort Sued gang 5}
::tclutils::tusqlite::value db {select gang from lager where ort = :o}
    - {o Nord}
                                             ;# -> 3
```

```
::tclutils::tusqlite::update db lager {gang 4} {ort Nord}
                                              ;# -> 1
::tclutils::tusqlite::delete db lager {ort Sued}
                                              ;# -> 1
```

Ein leeres Dict bei `insert` oder bei den neuen Werten von `update` wirft `{TCLUTILS TUSQLITE EMPTY}`, ein leeres Bedingungs-Dict `{TCLUTILS TUSQLITE WHERE}`, ein Platzhaltername mit anderen Zeichen als Buchstaben, Ziffern und `_` `{TCLUTILS TUSQLITE PARAM <name>}`.

8.6 LDIF: tuldif

tuldif liest LDIF nach RFC 2849 in eine Liste von Einträgen und schreibt sie zurück. Ein Eintrag ist eine geordnete Liste von Paaren `{attribut wert}`. `dn` ist ein Attribut wie jedes andere, ein mehrwertiges Attribut steht mehrmals in der Liste.

Befehl	Wirkung
parse text ?-binary namen?	Liste der Einträge; eine Leerzeile trennt zwei Einträge
toLdif einträge ?-binary namen?	LDIF-Text, nach jedem Eintrag eine Leerzeile
dn eintrag	Wert von dn, sonst die leere Zeichenkette
get eintrag attribut	alle Werte des Attributs; Groß-und Kleinschreibung zählt nicht
attributes eintrag	Attributnamen in der Reihenfolge des ersten Auftretens
toDict eintrag	Dict Attribut auf Liste der Werte
addAttr eintrag attribut wert	Kopie mit angehängtem Paar
setAttr eintrag index attribut wert	Kopie, das Paar an index ersetzt
removeAttr eintrag index	Kopie ohne das Paar an index

`parse` übergeht Kommentarzeilen mit `#` und die Zeile `version:`. Eine Zeile, die mit einem Leerzeichen beginnt, setzt die vorige fort. Ein Wert `attr:: …` wird über `tubase64` dekodiert und als UTF-8 gelesen, denn LDIF-Werte sind nach RFC 2849 UTF-8. Bytes, die kein gültiges UTF-8 ergeben, bleiben Bytes, ein Zeichen je Byte. Bei `attr:< url` ist die URL selbst der Wert; eine Datei wird nicht gelesen.

`toLdif` schreibt einen Wert als Base64, wenn er mit Leerzeichen, `:` oder `<` beginnt, mit einem Leerzeichen endet oder ein Zeichen außerhalb von ASCII 32–126 enthält. Vor Base64 wird der Wert nach UTF-8 gewandelt; `Hällo` wird `SMOkbGxv`, ein Zeichen über 255 wie `€` geht ebenso. Lange Zeilen bricht `toLdif` nicht um. Kommentare aus der Eingabe sind nach `parse` weg.

Binärattribute (seit 0.3) haben Bytes als Wert, ein Zeichen je Byte. `parse` lässt die dekodierten Bytes stehen, auch wenn sie zufällig gültiges UTF-8 sind, und `toLdif` schreibt sie unverändert als Base64. Bis 0.2 wandelte `toLdif` sie erst nach UTF-8, und die Base64-Zeile trug andere Bytes. Binär sind `jpegPhoto`, `photo`, `audio`, `thumbnailPhoto`, die Zertifikats- und

Sperrlisten-Attribute (`userCertificate`, `cACertificate`, …), `objectGUID`, `objectSid`, jedes Attribut mit der Option `;binary` und die Namen aus `-binary` an `parse` und `toLdif`. Ein Zeichen über `\xFF` in einem Binärwert wirft `{TCLUTILS TULDIF BINARY}`.

```
package require tclutils::tuldif
set ldif "dn: cn=Alice,dc=ex\ncn: Alice\nmail: a@ex.com\n"
append ldif "mail: alice@ex.com\n\ndn: cn=Bob,dc=ex\ncn: Bob\n"
set e [::tclutils::tuldif::parse $ldif]
llength $e                                  ;# -> 2
set alice [lindex $e 0]
::tclutils::tuldif::dn $alice               ;# -> cn=Alice,dc=ex
::tclutils::tuldif::get $alice MAIL         ;# -> a@ex.com alice@ex.com
::tclutils::tuldif::attributes $alice       ;# -> dn cn mail
::tclutils::tuldif::toDict $alice
    ;# -> dn cn=Alice,dc=ex cn Alice mail {a@ex.com alice@ex.com}
set e2 [::tclutils::tuldif::parse "dn: x\ncn:: SGFsbG8=\n"]
::tclutils::tuldif::get [lindex $e2 0] cn ;# -> Hallo
::tclutils::tuldif::toLdif [list {{dn x} {cn { Lager}}}]
                                            ;# -> dn: x\ncn:: IExhZ2Vy\n
::tclutils::tuldif::toLdif [list {{dn x} {cn {Lager }}}]
                                            ;# -> dn: x\ncn:: TGFnZXIg\n
::tclutils::tuldif::toLdif [list {{dn x} {cn Hällo}}]
                                            ;# -> dn: x\ncn:: SMOkbGxv\n
set e3 [::tclutils::tuldif::parse "dn: x\ncn:: SMOkbGxv\n"]
::tclutils::tuldif::get [lindex $e3 0] cn ;# -> Hällo
```

Ein Binärwert, den `parse` als Bytes liefert, geht durch `toLdif` erneut über UTF-8 und kommt deshalb nicht Byte für Byte gleich heraus. `tuldif` ist für Textwerte gedacht.

Eine Zeile ohne Doppelpunkt wirft `{TCLUTILS TULDIF SYNTAX}` mit `LDIF line without ':' -> zeile`. `tuldif` liest nur Inhaltssätze. Ein Änderungssatz wirft beim Wort `changetype` den Code `{TCLUTILS TULDIF CHANGETYPE}` mit `LDIF change records are not supported (changetype: modify)`.

```
set aend "dn: cn=A\nchangetype: modify\nreplace: mail\n"
catch {::tclutils::tuldif::parse $aend} m o
dict get $o -errorcode            ;# -> TCLUTILS TULDIF CHANGETYPE
```

8.7 Lesezeichen: tubookmark

tubookmark liest und schreibt das Netscape-Lesezeichenformat, die HTML-Datei, über die Browser Lesezeichen importieren und exportieren. Ein Lesezeichen ist ein Dict mit `title`, `url`, `folder`, `folders`, `tags` und `adddate`. `folders` ist die Liste der Ordnernamen von oben, leer für die oberste Ebene; `folder` dasselbe mit `/` verbunden, zum Anzeigen. `tags` ist eine Liste.

Befehl	Wirkung
parse html	flache Liste der Lesezeichen-Dicts

Befehl	Wirkung
serialize lesezeichen ?-title text?	die HTML-Datei als Text

Option	Vorgabe	Wirkung
-title	Bookmarks	Inhalt von <TITLE> und <H1>

`parse` liest das HTML Tag für Tag, nicht Zeile für Zeile. Jedes `<A …>…</A>` ist ein Lesezeichen, auch wenn mehrere Links in einer Zeile stehen. Ordner ergeben sich aus `<H3>` und den geschachtelten `<DL>`. Entitäten in Titel, `HREF`, `TAGS` und `ADD_DATE` löst `tuxml::unescape` auf. `TAGS` wird am Komma geteilt, `ADD_DATE` bleibt Text. Werte dürfen in doppelten oder einfachen Anführungszeichen oder ohne stehen (`HREF='…'`, seit 0.3). Steht nach einem Link ein `<DD>`, kommt sein Text in den Schlüssel `description`. Liegt das Lesezeichen in dem Ordner mit `PERSONAL_TOOLBAR_FOLDER="true"` (der Lesezeichenleiste) oder darunter, trägt `toolbar` den Pfad dieses Ordners als Liste. Beide Schlüssel gibt es nur, wenn die Datei sie hat; `serialize` schreibt sie zurück. Andere Attribute wie `ICON`, die Daten der Ordner und ein `<DD>` nach einem Ordner fallen weg.

`serialize` behält die Reihenfolge der Liste: Ein Ordner steht dort, wo sein erstes Lesezeichen steht, Ordner und Links bleiben gemischt. Den Pfad nimmt es aus `folders`, sonst teilt es `folder` am `/`; ein Ordnername mit `/` bleibt so ein Ordner. Bis 0.2 sortierte `serialize` die Ordner, stellte sie vor die Links und machte aus `A/B` zwei Ordner. Titel und Ordnername werden maskiert (&, <, >), die Attributwerte `HREF`, `TAGS` und `ADD_DATE` zusätzlich `"` als `"`. Jede Zeile des Kopfs beginnt in der ersten Spalte.

```
package require tclutils::tubookmark
set html "<DL><p>\n<DT><A HREF=\"https://tcl.tk/\" TAGS=\"tcl,lang\">"
append html "Tcl &amp; Tk</A>\n<DT><H3>Dev</H3>\n<DL><p>\n"
append html "<DT><A HREF=\"https://core.tcl-lang.org/\">Fossil</A>\n"
append html "</DL><p>\n</DL><p>\n"
set bms [::tclutils::tubookmark::parse $html]
llength $bms                                ;# -> 2
dict get [lindex $bms 0] title              ;# -> Tcl & Tk
dict get [lindex $bms 0] tags               ;# -> tcl lang
dict get [lindex $bms 1] folder             ;# -> Dev
set neu [::tclutils::tubookmark::serialize $bms -title Lager]
llength [::tclutils::tubookmark::parse $neu]
                                            ;# -> 2
set zwei "<DT><A HREF=\"a\">A</A><DT><A HREF=\"b\">B</A>"
llength [::tclutils::tubookmark::parse $zwei]
                                            ;# -> 2
set bm [dict create title T url u folder "" tags {a"b} adddate 1]
set h [::tclutils::tubookmark::serialize [list $bm]]
string match {*TAGS="a&quot;b"*} $h         ;# -> 1
set t [dict get [lindex [::tclutils::tubookmark::parse $h] 0] tags]
lindex $t 0                                 ;# -> a"b
```

Eigene Fehlercodes gibt es nicht. Eine unbekannte Option an `serialize` wirft `{TCLUTILS COMMON OPTION -name}`.

8.8 Was hier keine eigene Liste hat

TDBC und MaxMind stehen im Kapitel TDBC und MaxMind. ODF und die PDF-Struktur stehen im Kapitel ODF und PDF-Struktur.

Die Ansicht derselben Daten in der Oberfläche liegt in *tkutils* (`tkujson`, `tkuical`). `tupdf` liest die Struktur, `tupdftext` den Text.

8.9 Merkliste

- CSV mit Anführungszeichen: `tucsv`. Schneiden an einem Trennzeichen: `tucut`.
- `tujson::parse` macht `true` zu `1` und `null` zu leer. Die Art behält `parseTyped`.
- `tuini::setValue` gibt ein neues Dict zurück.
- INI-Kommentare sind `#` und `;`. Um das Gleichheitszeichen schreibt `toIni` keine Leerzeichen.
- `tuxml` maskiert in `textElement`. `element` setzt den Inhalt roh ein.
- `tusqlite` bindet SQL-NULL über `null`. Aus `rows` kommt die leere Zeichenkette.
- `tusqlite::rows` und `value` binden `:name` aus dem Dict, nicht aus Variablen.
- CSV für ein deutsches Excel: `tucsv::writeFile … -profile excel-de`.
- `tuldif`: Eintrag = Liste von Paaren. Base64-Werte sind UTF-8; Änderungssätze werfen `CHANGETYPE`.
- `tubookmark`: jedes `<A>` ein Lesezeichen; den Ordnerpfad trägt `folders` als Liste.

Nächstes Kapitel: TDBC und MaxMind

9 TDBC und MaxMind

tutdbc 0.1 spricht eine TDBC-Verbindung an, die der Aufruf schon geöffnet hat. Das Modul lädt keinen Treiber und steht nicht im Umbrella. tummdb 0.1 liest eine MaxMind-DB der Fassung 2.0. Es liegt im Umbrella und baut auf `tubin`.

9.1 TDBC: tutdbc

```
package require tclutils::tutdbc
::tclutils::tutdbc::insert  verbindung tabelle dict
::tclutils::tutdbc::rows    verbindung sql ?bindungen?
::tclutils::tutdbc::value   verbindung sql ?bindungen? ?vorgabe?
::tclutils::tutdbc::execute verbindung sql ?bindungen?
::tclutils::tutdbc::transaction verbindung skript
::tclutils::tutdbc::quoteId name
::tclutils::tutdbc::null
```

Platzhalter im SQL heißen `:name`. Die Bindungen sind ein Dict. `rows` liefert eine Liste von Dicts. Eine SQL-NULL-Zelle ist die leere Zeichenkette, die Spalte bleibt im Dict. `value` nimmt die erste Spalte der ersten Zeile, sonst die Vorgabe. `insert` gibt die Zahl der betroffenen Zeilen zurück. Ein leeres Dict wirft `{TCLUTILS TUTDBC EMPTY}`.

`null` ist der Sentinel für SQL-NULL. Derselbe Effekt tritt ein, wenn der Schlüssel im Dict fehlt. Eine gespeicherte `0` bleibt `0`.

```
package require tdbc::sqlite3
package require tclutils::tutdbc
tdbc::sqlite3::connection create db :memory:
db allrows {create table lager (ort text, gang integer)}
::tclutils::tutdbc::insert db lager {ort Nord gang 3}
                                          ;# -> 1
::tclutils::tutdbc::rows db {select ort, gang from lager}
                                          ;# -> {ort Nord gang 3}
::tclutils::tutdbc::value db {select gang from lager where ort = :o}
    {o x} -
                                          ;# -> -
::tclutils::tutdbc::quoteId {a"b}         ;# -> "a""b"
```

`insert` mit `gang` auf `[null]` schreibt SQL-NULL. Nach `insert db lager [list ort Sued gang [null]]` hat die Zeile für `Sued` den Schlüssel `gang` mit leerem Wert. `gang 0` kommt als `0` zurück.

Ein Fehler im Skript von `transaction` macht die Änderungen rückgängig und wirft den ursprünglichen Fehler weiter. Nach einem Insert und einem abgebrochenen zweiten Insert bleibt `count(*)` bei 1, die Meldung ist `abbruch`, der `-errorcode` ist der des abgebrochenen Skripts.

9.2 MaxMind: tummdb

```
::tclutils::tummdb::open     pfad
::tclutils::tummdb::metadata handle
::tclutils::tummdb::lookup   handle ip
::tclutils::tummdb::close    handle
```

`open` und `close` stehen nicht in `namespace export`. Der Aufruf bleibt qualifiziert, sonst träfe er Tcls `open` und `close`. `open` liest die Datei ganz in den Speicher und gibt ein Handle wie `tummdb1` zurück. `metadata` ist das Dict aus dem Dateiende. `lookup` liefert den Datensatz oder die leere Zeichenkette, wenn die Adresse keinen Satz hat.

Eine Datei ohne die Markierung `MaxMind.com` wirft `{TCLUTILS TUMMDB FORMAT}`. `metadata` und `lookup` mit einem unbekannten Handle werfen `{TCLUTILS TUMMDB HANDLE}`. `close` auf einem solchen Handle gibt die leere Zeichenkette zurück und wirft nicht.

Weitere Codes aus dem Modul: `RECORDSIZE`, wenn die Knotenbreite nicht getragen wird, und TYPE, wenn ein Datenfeld eine unbekannte Art hat.

9.3 Merkliste

- `tutdbc` lädt keinen Treiber. Die Verbindung öffnet der Aufruf.
- SQL-NULL bleibt eine Spalte mit leerem Wert. `0` bleibt `0`.
- `transaction` rollt bei Fehler zurück und reicht den Fehler weiter.
- `tummdb::open` und `close` immer mit Namensraum schreiben.

Nächstes Kapitel: Kodierung, Hash und Vergleich

10 Kodierung, Hash und Vergleich

Base64 und Base32, CRC, die Digests und xxHash32, der Wechsel der Kodierung, UUIDs, Query-Strings, verschachtelte Dicts, ein Zeilenvergleich und das Anwenden eines Diffs. Alles liegt im Umbrella. CRC nutzt `zlib` aus Tcl selbst, die Digests sind reines Tcl und brauchen kein tcllib.

10.1 Base64: tubase64

`encode` und `decode` gehen über `binary encode/decode base64`. `encodeFile` liest die Datei binär, `decodeFile` den Text.

Option	Vorgabe	Wirkung
-maxlen	0	Zeilenumbruch nach so vielen Zeichen; 0 lässt die Zeile ganz
-encoding	binary	encode: binary nimmt jedes Zeichen als ein Byte, utf-8 wandelt Text zuerst; decode: utf-8 macht aus den Bytes wieder Text

```
package require tclutils::tubase64
::tclutils::tubase64::encode hallo          ;# -> aGFsbG8=
::tclutils::tubase64::decode aGFsbG8=       ;# -> hallo
::tclutils::tubase64::encode hallohallo -maxlen 4
                                          ;# -> aGFs\nbG9o\nYWxs\nbw==
```

`-maxlen -1` wirft `{TCLUTILS TUBASE64 MAXLEN}`. Ein Text ohne gültiges Base64, etwa `!!!`, kommt als leere Zeichenkette zurück, ohne Fehlercode.

`encode` nimmt ohne Option Bytes. Ein Zeichen über 255 wirft unter Tcl 8.6 wie unter 9.0 `{TCLUTILS TUBASE64 BYTES}`; bis tclutils 0.68.0 behielt Tcl 8.6 still nur das untere Byte. Text gehört mit `-encoding utf-8` hinein:

```
::tclutils::tubase64::encode "Jörg €" -encoding utf-8   ;# ->
    SsO2cmcg4oKs
::tclutils::tubase64::decode SsO2cmcg4oKs -encoding utf-8 ;# -> Jörg €
::tclutils::tubase64::encode Jörg                        ;# -> SvZyZw==
```

Das letzte Beispiel kodiert `ö` als ein Byte `f6`. `tuhash` und `tuxxhash` wandeln Text dagegen von sich aus nach UTF-8; sie werden meist auf Text angewandt, `tubase64`, `tubase32` und `tucrc` meist auf Datei- und Bilddaten.

10.2 Base32: tubase32

tubase32 kodiert nach RFC 4648, wahlweise mit dem Alphabet base32hex. Tcl selbst bringt nur Base64 mit; die Umsetzung ist reines Tcl. `encodeFile` liest die Datei binär, `decodeFile` den Text.

Befehl	Wirkung
encode daten ?-pad b? ?-hex b?	Base32-Text in Großbuchstaben

Befehl	Wirkung
decode text ?-hex b?	die Bytes
encodeFile pfad ?-pad b? ?-hex b?	die Bytes der Datei kodiert
decodeFile pfad ?-hex b?	den Text der Datei dekodiert

Option	Vorgabe	Wirkung
-pad	1	füllt mit = auf eine volle Gruppe von acht Zeichen; 0 lässt es weg
-hex	0	1 nimmt das Alphabet 0–9, A–V (base32hex, RFC 4648 §7)

`decode` übergeht `=`, Leerzeichen, Tabulator und Zeilenumbruch und nimmt Klein- wie Großbuchstaben. Das Ende prüft `decode` wie RFC 4648 §3.5: Die Zahl der Zeichen muss eine sein, die `encode` erzeugt (ein Vielfaches von 8 plus 0, 2, 4, 5 oder 7), und die Bits nach dem letzten ganzen Byte müssen 0 sein. `decode MY` ergibt `f`, `decode MZ` ist ein Fehler.

```
package require tclutils::tubase32
::tclutils::tubase32::encode foobar        ;# -> MZXW6YTBOI======
::tclutils::tubase32::encode foobar -pad 0 ;# -> MZXW6YTBOI
::tclutils::tubase32::encode foobar -hex 1 ;# -> CPNMUOJ1E8======
::tclutils::tubase32::decode mzxw6ytboi    ;# -> foobar
::tclutils::tubase32::encode [encoding convertto utf-8 Größe]
                                          ;# -> I5ZMHNWDT5SQ====
::tclutils::tubase32::decode MY            ;# -> f
catch {::tclutils::tubase32::decode MZ} m o
dict get $o -errorcode                     ;# -> TCLUTILS TUBASE32 BITS
```

Die Eingabe von `encode` sind Bytes. Ein Zeichen über 255 wirft unter Tcl 9.0 `{TCL VALUE BYTES}`; unter Tcl 8.6 bleibt still nur das untere Byte. Text gehört deshalb vorher durch `encoding convertto utf-8`.

Anders als `tubase64` liefert `tubase32` bei ungültigem Text keine leere Zeichenkette, sondern einen Fehler. Ein Zeichen außerhalb des Alphabets wirft `{TCLUTILS TUBASE32 CHAR}`, etwa `invalid base32 character: "1"`. Eine unmögliche Länge wie `MZX` wirft `{TCLUTILS TUBASE32 LENGTH}`, Bits ungleich 0 am Ende wie bei `MZ` `{TCLUTILS TUBASE32 BITS}`. Ein ungültiges Wahrheitswort an `-pad` oder `-hex` wirft `{TCLUTILS COMMON BOOLEAN -pad}` bzw. `-hex`.

10.3 CRC: tucrc

`crc32` und `adler32` reichen die Bytes an `zlib` weiter. Ein zweites Argument ist der Startwert für eine fortgesetzte Summe. Danach darf `-encoding binary|utf-8` stehen, wie bei `tubase64`: ohne Option ist ein Zeichen über 255 `{TCLUTILS TUCRC BYTES}`, mit `utf-8` gleicht die Summe der von `crc32File` einer UTF-8-Datei mit diesem Text. `hex32` macht daraus acht Hex-Zeichen in Großbuchstaben. `crc32File` und `adler32File` lesen die Datei binär.

```
package require tclutils::tucrc
::tclutils::tucrc::crc32 hallo              ;# -> 3111268817
::tclutils::tucrc::hex32 [::tclutils::tucrc::crc32 hallo]
                                           ;# -> B97231D1
::tclutils::tucrc::adler32 hallo            ;# -> 102498833
```

Ein falsches Argument meldet `wrong # args` mit `{TCLUTILS TUCRC OPTION}`, eine unbekannte Kodierung `{TCLUTILS TUCRC ENCODING}`.

10.4 Digests: tuhash

`sha256`, `sha1` und `md5` sind eigene Umsetzungen von FIPS 180-4 und RFC 1321. Die Dateibefehle hashen die Bytes der Datei. Am Text hängt `-encoding`:

Wert	Wirkung
utf-8	Vorgabe; der Text wird zuerst nach UTF-8 gewandelt
binary	jedes Zeichen ist ein Byte 0–255

```
package require tclutils::tuhash
::tclutils::tuhash::sha256 ""               ;# ->
    e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
::tclutils::tuhash::sha256 abc              ;# ->
    ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
::tclutils::tuhash::md5 ""                  ;# ->
    d41d8cd98f00b204e9800998ecf8427e
::tclutils::tuhash::sha1 ""                 ;# ->
    da39a3ee5e6b4b0d3255bfef95601890afd80709
```

Eine andere Kodierung wirft `{TCLUTILS TUHASH ENCODING}`. Dieselben leeren Digests sind die Prüfvektoren der Normen.

10.5 xxHash32: tuxxhash

tuxxhash rechnet XXH32, einen schnellen Hash ohne kryptographischen Anspruch, in reinem Tcl. Er dient zum Erkennen von Änderungen und zum Vorsortieren möglicher Duplikate, nicht zum Schutz gegen Fälschung. Varianten mit 64 oder 128 Bit gibt es nicht.

Befehl	Wirkung
xxh32 daten ?seed? ?-encoding e?	acht Hex-Zeichen in Kleinbuchstaben
xxh32file pfad ?seed?	dasselbe für die Bytes der ganzen Datei

`seed` ist eine ganze Zahl, Vorgabe `0`. Davon zählen die unteren 32 Bit. Wie bei `tuhash` hängt am Text `-encoding`; es steht nach `seed`:

Wert	Wirkung
utf-8	Vorgabe; der Text wird zuerst nach UTF-8 gewandelt

Wert	Wirkung
binary	jedes Zeichen ist ein Byte 0–255

```
package require tclutils::tuxxhash
::tclutils::tuxxhash::xxh32 ""              ;# -> 02cc5d05
::tclutils::tuxxhash::xxh32 hello           ;# -> fb0077f9
::tclutils::tuxxhash::xxh32 hello 1         ;# -> fcfffba9
::tclutils::tuxxhash::xxh32 Größe           ;# -> 3a44ca22
set b [encoding convertto utf-8 Größe]
::tclutils::tuxxhash::xxh32 $b 0 -encoding binary
                                          ;# -> 3a44ca22
```

`xxh32 Größe` hasht die sieben UTF-8-Bytes und stimmt mit `xxh32file` einer UTF-8-Datei mit diesem Text überein. Ein Zeichen über 255 wie € geht ebenso. `-encoding binary` nimmt die Zeichen als Bytes, etwa für Daten aus `binary format` oder einem Kanal mit `-translation binary`.

Ein `seed`, der keine ganze Zahl ist, wirft `{TCLUTILS TUXXHASH SEED}`. Ein anderer Wert an `-encoding` wirft `{TCLUTILS TUXXHASH ENCODING}`, ein Zeichen über 255 bei `-encoding binary` `{TCLUTILS TUXXHASH BYTES}`, ein anderes Argument als `-encoding` `{TCLUTILS TUXXHASH OPTION}`. Eine Datei, die sich nicht öffnen lässt, wirft `{TCLUTILS TUXXHASH IO}`. Mit 32 Bit treten in großen Beständen zufällige Kollisionen auf; gleiche Werte sind nur Kandidaten für einen Vergleich der Bytes oder mit `sha256`.

10.6 Kodierung wechseln: tuiconv

tuiconv wandelt zwischen den Kodierungen, die Tcl kennt, und liest und schreibt Dateien mit ausdrücklicher Kodierung. Die Kanäle sind dabei binär; die Umwandlung geschieht mit `encoding convertfrom` und `encoding convertto`.

Befehl	Wirkung
encodings	die Namen aus encoding names, sortiert
convert bytes von nach ?-replace b?	Bytes in von zu Bytes in nach
readFile pfad kodierung ?-replace b?	der Text der Datei
writeFile pfad text kodierung ?-replace b?	schreibt die Datei, gibt pfad zurück
convertFile ein aus von nach ?-replace b?	liest ein, schreibt aus, gibt aus zurück

Ein Name muss genau in `encoding names` stehen, etwa `utf-8`, `iso8859-1` oder `cp1252`. Schreibweisen wie `utf8` oder `latin1` werfen `{TCLUTILS TUICONV ENCODING name}` mit `unknown encoding "utf8"`. Die Liste ist unter Tcl 9.0 länger als unter 8.6.

```
package require tclutils::tuiconv
set utf8 [encoding convertto utf-8 Größe]
set cp [::tclutils::tuiconv::convert $utf8 utf-8 cp1252]
binary encode hex $cp                        ;# -> 4772f6df65
set zurueck [::tclutils::tuiconv::convert $cp cp1252 utf-8]
encoding convertfrom utf-8 $zurueck          ;# -> Größe
```

Seit 0.2 ist die Umwandlung unter Tcl 8.6 und 9.0 gleich streng. Ein Zeichen, das die Zielkodierung nicht kennt, etwa `ł` nach `cp1252`, wirft `{TCLUTILS TUICONV UNMAPPABLE cp1252}`; Bytes, die in der Quellkodierung ungültig sind, werfen `{TCLUTILS TUICONV INVALID name}`. `-replace 1` an allen Befehlen ersetzt stattdessen, beim Schreiben durch `?`. `writeFile` wandelt vor dem Öffnen um: Bei einem Fehler behält die Datei ihren Inhalt. Bis 0.1 setzte Tcl 8.6 still `?` ein, Tcl 9.0 warf einen Tcl-Fehler ohne eigenen Code.

```
catch {::tclutils::tuiconv::convert [encoding convertto utf-8 ł] utf-8
    cp1252} m
set m             ;# -> character U+0142 at index 0 cannot be written
    in cp1252
binary encode hex [::tclutils::tuiconv::convert \
    [encoding convertto utf-8 ł] utf-8 cp1252 -replace 1]
                                             ;# -> 3f
```

10.7 UUID: tuuuid

v4 ist zufällig, v7 trägt die Unix-Zeit in Millisekunden in den ersten 48 Bit und danach Zufall (RFC 9562). `generate` nimmt `-version 4` oder `7`, Vorgabe `4`. `nil` ist die Null-UUID. `validate` prüft die Form `8-4-4-4-12`, Groß- und Kleinschreibung. `version` liest die Ziffer an Stelle 14.

```
package require tclutils::tuuuid
::tclutils::tuuuid::nil                      ;# ->
    00000000-0000-0000-0000-000000000000
::tclutils::tuuuid::validate [::tclutils::tuuuid::nil]
                                             ;# -> 1
::tclutils::tuuuid::version [::tclutils::tuuuid::nil]
                                             ;# -> 0
::tclutils::tuuuid::version [::tclutils::tuuuid::v4]
                                             ;# -> 4
::tclutils::tuuuid::version [::tclutils::tuuuid::v7]
                                             ;# -> 7
```

`version nein` wirft `{TCLUTILS TUUUID INVALID}`. `generate -version 9` wirft `{TCLUTILS TUUUID VERSION}`. Die Zufallsbytes kommen von `/dev/urandom`, sonst von `rand()`.

10.8 Query-Strings: tuurl

Nicht reserviert bleiben Buchstaben, Ziffern und `-._~`. Alles andere wird über die UTF-8-Bytes

mit `%` kodiert.

Befehl	-plus	Wirkung
encode, decode	0	Leerzeichen wird %20; + bleibt +
buildQuery, parseQuery	1	Leerzeichen wird +, und + wird beim Lesen wieder zum Leerzeichen

```
package require tclutils::tuurl
::tclutils::tuurl::encode "a b"             ;# -> a%20b
::tclutils::tuurl::encode "a b" -plus 1     ;# -> a+b
::tclutils::tuurl::encode Größe             ;# -> Gr%C3%B6%C3%9Fe
::tclutils::tuurl::decode %C3%BC            ;# -> ü
::tclutils::tuurl::buildQuery {ort {Nord Lager} gang 3}
                                            ;# -> ort=Nord+Lager&gang=3
::tclutils::tuurl::parseQuery {ort=Nord+Lager&gang=3}
                                            ;# -> ort {Nord Lager} gang 3
```

Ein Schlüssel ohne `=` bekommt den leeren Wert. Steht ein Schlüssel mehrmals in der Query (`a=1&a=2`, etwa bei Checkboxen), behält `parseQuery` nur den letzten Wert; wer alle braucht, teilt die Query selbst am `&`. Ein ungültiges Wahrheitswort an `-plus` wirft `{TCLUTILS COMMON BOOLEAN -plus}`.

10.9 Verschachtelte Dicts: tudict

Der Kern kann `dict get $d a b` schon. tudict ergänzt den Weg mit Vorgabe, die Blattpfade, eine flache Form, das tiefe Mischen und das Umdrehen.

Befehl	Wirkung
getOr dict vorgabe schlüssel …	Wert oder die Vorgabe, wenn ein Schlüssel fehlt
paths dict	Blattpfade, jeder Pfad eine Liste von Schlüsseln
flatten dict ?sep?	ein flaches Dict, Schlüssel mit . verbunden
mergeDeep dict …	von links nach rechts; verschachtelte Dicts mischen, Skalare überschreiben
invert dict	Schlüssel und Wert tauschen; bei doppeltem Wert bleibt der letzte

Die Vorgabe steht vor dem Pfad.

```
package require tclutils::tudict
::tclutils::tudict::getOr {ui {theme dark}} - ui theme
                                            ;# -> dark
::tclutils::tudict::getOr {ui {theme dark}} - ui fehlt
                                            ;# -> -
::tclutils::tudict::paths {a {b {c 1}} d 2}
                                            ;# -> {a b c} d
::tclutils::tudict::flatten {a {b {c 1}} d 2}
```

```
                                          ;# -> a.b.c 1 d 2
::tclutils::tudict::invert {a 1 b 2}        ;# -> 1 a 2 b
::tclutils::tudict::invert {a 1 b 1}        ;# -> 1 b
```

Ohne Schlüssel wirft `getOr {TCLUTILS TUDICT ARG}`. Ein Wert, der wie ein Dict aussieht, wird eins: `flatten` von `{name {a b}}` ergibt `name.a b`, weil `a b` gerade Länge hat. `mergeDeep` steht mit einem Beispiel im Anhang Praxis.

10.10 Zeilenvergleich: tudiff

`text` liefert eine Liste von `equal`, `delete` und `insert`, jeweils mit der Zeile. `unifiedText` setzt daraus einen einheitlichen Diff, `contextText` die Kontextform. `files`, `unified` und `context` lesen zwei Dateien. `directory` vergleicht zwei Bäume.

Option	Vorgabe	Wirkung
-maxcells	2000000	Obergrenze für die Tabelle, Zeilen alt mal Zeilen neu
-fromlabel, -tolabel	old, new	Kopfzeilen von unifiedText und contextText

```
package require tclutils::tudiff
::tclutils::tudiff::text "a\nb" "a\nc"      ;# -> {equal a} {delete b}
    {insert c}
::tclutils::tudiff::unifiedText "a\nb" "a\nc"
                                            ;# -> --- old\n+++ new\n@@
    -1,2 +1,2 @@\n a\n-b\n+c
```

`-maxcells 0` wirft `{TCLUTILS TUDIFF OPTION MAXCELLS}`. Zwei Texte mit je zwei Zeilen und `-maxcells 1` werfen `{TCLUTILS TUDIFF TOO_LARGE}`, weil 2 × 2 die Grenze übersteigt.

Zeilenenden zählen nach Vorgabe nicht mit: CRLF, CR und LF sind gleich, ein fehlendes letztes Newline auch (`-eol ignore`). `-eol strict` (seit 0.2) vergleicht jede Zeile mit ihrem Ende. Die Operationen tragen dann das Ende mit, und `unifiedText` markiert ein fehlendes Newline wie GNU diff. `files`, `unified` und `context` nehmen dazu `-encoding`.

```
::tclutils::tudiff::text "a\r\nb" "a\nb\n"              ;# -> {equal
    a} {equal b}
::tclutils::tudiff::unifiedText "a\nb\n" "a\nb" -eol strict
;# -> --- old\n+++ new\n@@ -1,2 +1,2 @@\n a\n-b\n+b\n\\ No newline at
    end of file
```

10.11 Diff anwenden: tupatch

tupatch ist das Gegenstück zu `tudiff`: es wendet einen einheitlichen Diff auf einen Text an. Das kann die Ausgabe von `unifiedText` sein, aber auch `diff -u` oder `git diff` mit mehreren Abschnitten.

Befehl	Wirkung
text quelle patch ?optionen?	der geänderte Text
fromFile pfad patch ?optionen?	wie text, die Quelle aus der Datei
parse patch	Liste der Abschnitte als Dicts

Option	Vorgabe	Wirkung
-reverse	0	1 wendet den Diff rückwärts an und nimmt ihn so zurück
-strict	1	prüft Kontext-und Löschzeilen gegen die Quelle; 0 folgt blind dem Diff

Ein Abschnitt aus `parse` hat `oldStart`, `oldLen`, `newStart`, `newLen`, `body`, `oldNoNl` und `newNoNl`. `body` ist eine Liste von Paaren aus Art (`, `+`, `-`) und Zeile. Kopfzeilen wie `---`, `+++`, `diff` und `index` fallen weg. `oldNoNl` und `newNoNl` sind 1, wenn `\ No newline at end of file` hinter der letzten alten bzw. neuen Zeile steht. Eine leere Zeile beendet einen Abschnitt.

Jeder Abschnitt gilt an der Zeile aus seinem Kopf. Eine Verschiebung sucht `tupatch` nicht. Die Zeilen werden mit `\n` verbunden. Endet die Quelle auf `\n`, endet auch das Ergebnis darauf; die Markierung `\ No newline at end of file` wirkt wie bei `diff -u` (seit 0.3; bis 0.2 fehlte das letzte `\n` im Ergebnis immer). Ohne Abschnitt im Diff kommt die Quelle unverändert zurück.

```
package require tclutils::tudiff
package require tclutils::tupatch
set alt "alpha\nbeta\ngamma"
set neu "alpha\nBETA\ngamma"
set p [::tclutils::tudiff::unifiedText $alt $neu]
::tclutils::tupatch::text $alt $p           ;# -> alpha\nBETA\ngamma
::tclutils::tupatch::text $neu $p -reverse 1
                                          ;# -> alpha\nbeta\ngamma
llength [::tclutils::tupatch::parse $p]     ;# -> 1
```

Passt eine Zeile nicht, wirft `text {TCLUTILS TUPATCH CONTEXT}`, etwa bei der Quelle `alpha\nbeta\nGAMMA`: `patch does not apply: expected "gamma" but found "GAMMA" at source line 3`. Ein Abschnitt hinter dem Ende der Quelle oder einer, der einen früheren überlappt, wirft `{TCLUTILS TUPATCH RANGE}`. Eine unbekannte Option wirft `{TCLUTILS COMMON OPTION -name}`. `-reverse` und `-strict` nehmen jeden Wahrheitswert (`1`, `yes`, `true`, `on` und die Gegenstücke); `-strict foo` wirft `{TCLUTILS COMMON BOOLEAN -strict}`.

10.12 Merkliste

- Base64 ohne Zeilenumbruch: `-maxlen 0`. Ungültiger Text wird leer.
- Digests von Text sind UTF-8. Dateien hashen die Bytes.
- `tuurl`: am einzelnen Wert ist `-plus` aus, an der Query an.
- `getOr`: erst das Dict, dann die Vorgabe, dann die Schlüssel.
- `tudiff` hält die ganze Vergleichstabelle im Speicher.
- Base32: `tubase32`, Eingabe sind Bytes; `-hex 1` für base32hex. Ungültiger Text wirft einen

Fehler.
- xxh32 hasht Text als UTF-8 wie `tuhash`; `-encoding binary` für Bytes.
- `tuiconv` kennt nur die Namen aus `encoding names`. Nicht darstellbare Zeichen und ungültige Bytes sind ein Fehler; `-replace 1` ersetzt.
- `tupatch` wendet ohne Verschiebung an. `-reverse 1` nimmt den Diff zurück.

Nächstes Kapitel: Binärdaten und Dateiinformationen

11 Binärdaten und Dateiinformationen

Die Module dieses Kapitels zeigen Bytes an, suchen und ändern sie in einer Datei, vergleichen zwei Dateien und lesen Größe und Metadaten. `tucode` liefert dazu die Zeichentabellen für ASCII und Latin-1. Alle Module sind reines Tcl.

Gruppe	Module
Ansehen	tuhexdump, tuod, tustrings
Ändern	tuhexedit
Vergleichen	tucmp
Größe und Metadaten	tusize, tustat
Zeichentabellen	tucode

Die Befehle mit Daten als Argument erwarten Bytes: einen Tcl-String, in dem jedes Zeichen einen Wert von 0 bis 255 hat. So einen String liefert `read` auf einem Kanal mit `-translation binary`, und so lesen auch die `file`-Varianten der Module ihre Datei. Ein Text mit Zeichen über 255, etwa `€`, ist kein Byte-String. `tuhexdump`, `tuod`, `tustrings`, `tucmp` und `tuhexedit::writeBytes` prüfen das vorab und werfen unter Tcl 8.6 und 9 gleich `{TCLUTILS <MODUL> BYTES}`, etwa `{TCLUTILS TUOD BYTES}`. Text wird deshalb vorher mit `encoding convertto utf-8` in Bytes gewandelt.

```
package require tclutils::tuod
::tclutils::tuod::bytes [encoding convertto utf-8 €]
                                          ;# -> 226 130 172
catch {::tclutils::tuod::bytes €} msg opts ;# -> 1
dict get $opts -errorcode         ;# -> TCLUTILS TUOD BYTES
```

Prüfsummen, Hashes, Base64 und der Zeilenvergleich stehen im Kapitel Kodierung, Hash und Vergleich, die Erkennung des Dateityps mit `tufile` im Kapitel Text und Dateien.

11.1 Hexdump: tuhexdump

tuhexdump gibt Bytes als Zeilen mit Adresse, Hex-Werten und ASCII-Spalte aus, in der Form von `hexdump -C`.

```
::tclutils::tuhexdump::data bytes ?option wert ...?
::tclutils::tuhexdump::file pfad  ?option wert ...?
```

Option	Vorgabe	Wirkung
-width	16	Bytes je Zeile, ganze Zahl ab 1
-offset	0	erstes Byte; die Adresse zählt ab hier
-length	-1	Anzahl der Bytes; negativ heißt bis zum Ende

Rückgabe ist ein String, Zeilen getrennt durch `\n`, ohne `\n` am Ende. Jede Zeile hat die Adresse mit acht Hex-Stellen, die Bytes in Kleinbuchstaben und zwischen `|` die Zeichen 32 bis 126; jedes andere Byte steht dort als `.`. Die letzte Zeile wird mit Leerzeichen auf die volle Breite aufgefüllt. Leere Daten ergeben einen leeren String.

Abweichungen von `hexdump -C` und `xxd`: keine Lücke nach dem achten Byte, keine Zusammenfassung gleicher Zeilen mit `*`, keine Schlusszeile mit der Endadresse.

```
package require tclutils::tuhexdump
set daten "Hallo, Welt!\n\x00\x01\xff"
puts [::tclutils::tuhexdump::data $daten -width 8]
```

```
00000000  48 61 6c 6c 6f 2c 20 57  |Hallo, W|
00000008  65 6c 74 21 0a 00 01 ff  |elt!....|
```

```
::tclutils::tuhexdump::data "Hallo, Welt!" -offset 7 -length 4 -width 4
                              ;# -> 00000007  57 65 6c 74  |Welt|
```

Bei Standardbreite ist eine Zeile 77 Zeichen lang.

Fehler	-errorcode
-width keine ganze Zahl ab 1	{TCLUTILS COMMON INTEGER -width}
-offset negativ oder keine ganze Zahl	{TCLUTILS TUHEXDUMP OFFSET}
-length keine ganze Zahl	{TCLUTILS TUHEXDUMP LENGTH}
Zeichen über 255 in den Daten	{TCLUTILS TUHEXDUMP BYTES}
unbekannte Option, Option ohne Wert	{TCLUTILS COMMON OPTION -name}

11.2 Oktal, dezimal, Zeichen: tuod

tuod gibt Bytes einzeln als Oktal-, Hex- oder Dezimalzahl oder als Zeichen aus. `bytes` liefert die Bytewerte als Liste ohne Vorzeichen; `binary scan` mit `c` allein ergäbe für `\xff` den Wert `-1`.

```
::tclutils::tuod::bytes bytes ?option wert ...?
::tclutils::tuod::data  bytes ?option wert ...?
::tclutils::tuod::file  pfad  ?option wert ...?
```

Option	Vorgabe	Wirkung
-format	octal	octal (%03o), hex (%02x), decimal (%03d), char
-width	16	Bytes je Zeile, mindestens 1

Option	Vorgabe	Wirkung
-offset	0	erstes Byte
-length	-1	Anzahl der Bytes; negativ heißt bis zum Ende
-address	hex	hex (8 Stellen), octal (7), decimal (8), none

`-format char` zeigt die Zeichen 32 bis 126 und sonst `.`. Zwischen Adresse und Werten stehen zwei Leerzeichen, zwischen den Werten eines. `bytes` wertet nur `-offset` und `-length` aus und nimmt die übrigen Optionen ohne Wirkung an.

Abweichungen von `od`: Unix `od` zeigt ohne Option Zwei-Byte-Wörter oktal und die Adresse oktal. `tuod` zeigt jedes Byte einzeln und die Adresse hex; `-address octal` ergibt die Adresse wie bei `od`. Statt `\n` und `\0` wie bei `od -c` steht `.`. Gleiche Zeilen werden nicht zu `*` zusammengefasst, eine Schlusszeile mit der Endadresse fehlt.

```
package require tclutils::tuod
::tclutils::tuod::bytes "A\x00\xff"       ;# -> 65 0 255
::tclutils::tuod::data ABC                ;# -> 00000000  101 102 103
::tclutils::tuod::data "Hallo\n" -format hex -address none
                                          ;# -> 48 61 6c 6c 6f 0a
::tclutils::tuod::data abcdef -format char -offset 2 -length 3
                                          ;# -> 00000002  c d e
set daten "Hallo\n\x00\xff"
puts [::tclutils::tuod::data $daten -format char \
    -address octal -width 4]
```

```
0000000  H a l l
0000004  o . . .
```

Fehler	-errorcode
unbekanntes -format	{TCLUTILS TUOD FORMAT wert}
unbekannte -address	{TCLUTILS TUOD ADDRESS wert}
-width keine ganze Zahl ab 1	{TCLUTILS TUOD WIDTH}
-offset negativ oder keine ganze Zahl	{TCLUTILS TUOD OFFSET}
-length keine ganze Zahl	{TCLUTILS TUOD LENGTH}
Zeichen über 255 in den Daten	{TCLUTILS TUOD BYTES}
unbekannte Option, Option ohne Wert	{TCLUTILS COMMON OPTION -name}

`data` und `file` prüfen `-format` und `-address` vor dem ersten Byte, ein falscher Wert fällt also auch bei leeren Daten auf.

11.3 Lesbare Zeichenketten: tustrings

tustrings sucht in Binärdaten Folgen druckbarer ASCII-Zeichen, wie Unix `strings`.

```
::tclutils::tustrings::extract bytes ?-minlength n?
::tclutils::tustrings::file    pfad  ?-minlength n?
```

Option	Vorgabe	Wirkung
-minlength	4	kürzere Folgen werden verworfen; ganze Zahl ab 1

Druckbar sind die Bytes 32 bis 126. Jedes andere Byte beendet eine Folge, auch Tabulator und Zeilenende. Rückgabe ist eine Tcl-Liste der Folgen in der Reihenfolge des Vorkommens, ohne Position.

Abweichungen von `strings`: GNU `strings` zählt den Tabulator zur Zeichenkette, `tustrings` trennt dort. Eine Ausgabe der Position wie `strings -t` gibt es nicht. Umlaute sind auch in Latin-1 keine druckbaren ASCII-Zeichen und trennen ebenfalls.

```
package require tclutils::tustrings
set bin "\x00\x01%PDF-1.7\x00ab\x00Hallo Welt\n\xffKey\tWert"
::tclutils::tustrings::extract $bin  ;# -> %PDF-1.7 {Hallo Welt} Wert
::tclutils::tustrings::extract $bin -minlength 2
                          ;# -> %PDF-1.7 ab {Hallo Welt} Key Wert
::tclutils::tustrings::extract "Größe 12"  ;# -> {e 12}
```

Eine Option `-encoding` gibt es nicht; druckbar sind immer die Bytes 32 bis 126.

Fehler	-errorcode
-minlength keine ganze Zahl ab 1	{TCLUTILS COMMON INTEGER -minlength}
Zeichen über 255 in den Daten	{TCLUTILS TUSTRINGS BYTES}
unbekannte Option, Option ohne Wert	{TCLUTILS COMMON OPTION -name}

11.4 Lesen, suchen, patchen: tuhexedit

tuhexedit liest Bytes an einer Stelle, sucht Byte- und Textmuster und überschreibt Bytes in der Datei. Es ist eine Bibliothek, kein Editor mit Oberfläche. Unix hat dafür kein einzelnes Werkzeug; das Überschreiben an fester Stelle entspricht `dd ... conv=notrunc`.

```
::tclutils::tuhexedit::readBytes  pfad offset länge
::tclutils::tuhexedit::writeBytes pfad offset bytes ?sicherung?
::tclutils::tuhexedit::patch      pfad offset hex ?-backup 1?
::tclutils::tuhexedit::backup     pfad ?zielname?
::tclutils::tuhexedit::findHex    pfad hex
::tclutils::tuhexedit::findString pfad text ?kodierung?
::tclutils::tuhexedit::dump       pfad ?option wert ...?
```

Befehl	Rückgabe
readBytes	die Bytes; am Dateiende weniger, hinter dem Ende leer
writeBytes, patch	der Pfad, wie übergeben
backup	der Name der Kopie; Vorgabe pfad.bak

Befehl	Rückgabe
findHex, findString	Liste der Offsets ab 0, auch überlappend
dump	Hexdump wie tuhexdump::file, mit dessen Optionen

Ein Offset ist eine ganze Zahl ab 0 oder Hex mit `0x`, etwa `0x100`; `0x` ohne gültige Hex-Ziffer ist ein Fehler. Hex-Muster für `findHex` und `patch` dürfen Leerzeichen,`:`, `-`,`,` und `0x` vor jedem Byte enthalten; `{50 4B 03 04}`, `504b0304` und `0x50,0x4B,0x03,0x04` sind dasselbe Muster. `findString` wandelt den Text vor der Suche mit der Kodierung in Bytes, Vorgabe `utf-8`. Ein `ä` in einer Latin-1-Datei findet deshalb nur `findString pfad ä iso8859-1`. Ein Zeichen, das die Kodierung nicht darstellen kann, ist ein Fehler.

`writeBytes` und `patch` legen vorher eine Kopie `pfad.bak` an und überschreiben eine vorhandene Kopie. `writeBytes` mit `0` als viertem Argument und `patch` mit `-backup 0` lassen die Kopie weg. Die Datei wird an der Stelle überschrieben, nichts wird eingefügt oder entfernt; die Länge der Datei bleibt gleich. Reichen die Bytes ab dem Offset über das Dateiende hinaus, wirft der Befehl einen Fehler, bevor er die Kopie anlegt; Datei und Kopie bleiben dann unverändert. `findHex`, `findString` und `patch` lesen die ganze Datei, `readBytes` nur den angegebenen Teil.

```
package require tclutils::tuhexedit
set fh [open probe.bin wb]
puts -nonewline $fh ABCDEFGH
close $fh
::tclutils::tuhexedit::readBytes probe.bin 2 3    ;# -> CDE
::tclutils::tuhexedit::findString probe.bin CD    ;# -> 2
::tclutils::tuhexedit::findHex probe.bin {47 48}  ;# -> 6
::tclutils::tuhexedit::patch probe.bin 0x2 {58 59}
                                                  ;# -> probe.bin
::tclutils::tuhexedit::readBytes probe.bin 0 8    ;# -> ABXYEFGH
::tclutils::tuhexedit::readBytes probe.bin.bak 0 8
                                                  ;# -> ABCDEFGH
::tclutils::tuhexedit::dump probe.bin -width 8
              ;# -> 00000000  41 42 58 59 45 46 47 48  |ABXYEFGH|
catch {::tclutils::tuhexedit::patch probe.bin 7 {58 59}} msg opts
                                                  ;# -> 1
dict get $opts -errorcode     ;# -> TCLUTILS TUHEXEDIT RANGE
```

Fehler	-errorcode
Offset negativ, keine Zahl oder 0x ohne Hex-Ziffer	{TCLUTILS TUHEXEDIT OFFSET}
Länge negativ oder keine Zahl	{TCLUTILS TUHEXEDIT LENGTH}
Schreiben über das Dateiende	{TCLUTILS TUHEXEDIT RANGE}
writeBytes mit Zeichen über 255	{TCLUTILS TUHEXEDIT BYTES}
Zeichen nicht in der Kodierung (findString)	{TCLUTILS TUHEXEDIT ENCODING}

Fehler	-errorcode
leeres Suchmuster	{TCLUTILS TUHEXEDIT NEEDLE}
Hex mit ungerader Stellenzahl	{TCLUTILS TUBIN HEX ODD}
Hex mit anderen Zeichen	{TCLUTILS TUBIN HEX BADCHAR}
unbekannte Option an patch	{TCLUTILS COMMON OPTION -name}
unbekannte Kodierung	{TCL LOOKUP ENCODING name}
Datei fehlt	{POSIX ENOENT ...}

`writeBytes` legt keine neue Datei an; eine fehlende Datei wirft `{POSIX ENOENT ...}`. Alle Prüfungen laufen vor dem Schreiben, ein abgelehnter Aufruf ändert nichts.

11.5 Byteweise vergleichen: tucmp

tucmp vergleicht zwei Byte-Strings oder zwei Dateien und meldet die erste Abweichung, wie Unix `cmp`.

```
::tclutils::tucmp::data            bytes1 bytes2
::tclutils::tucmp::files           pfad1 pfad2
::tclutils::tucmp::equal           pfad1 pfad2
::tclutils::tucmp::firstDifference pfad1 pfad2
```

`data` und `files` liefern ein Dict:

Schlüssel	Inhalt
equal	1 bei gleichen Daten, sonst 0
offset	erste abweichende Stelle ab 0; -1 bei gleichen Daten
byte1, byte2	Bytewert 0–255 an offset; leer, wo die Daten zu Ende sind
size1, size2	Anzahl der Bytes beider Daten

`equal` liefert nur `equal`, `firstDifference` nur `offset`. Ist eine Seite der Anfang der anderen, steht `offset` auf der Länge der kürzeren Seite und deren Byte ist leer.

Abweichungen von `cmp`: Der Offset zählt ab 0, `cmp` zählt Byte und Zeile ab 1. Eine Zeilennummer gibt es nicht, das Ergebnis ist ein Dict statt eines Exit-Status. `files` liest beide Dateien ganz in den Speicher.

```
package require tclutils::tucmp
dict get [::tclutils::tucmp::data abc abc] equal    ;# -> 1
dict get [::tclutils::tucmp::data abc abc] offset   ;# -> -1
::tclutils::tucmp::data abc abx
::tclutils::tucmp::data abc abcd
```

```
equal 0 offset 2 byte1 99 byte2 120 size1 3 size2 3
equal 0 offset 3 byte1 {} byte2 100 size1 3 size2 4
```

`data` vergleicht Bytes. Ein Zeichen über 255 ist ein Fehler, auch wenn beide Strings gleich sind;

Text wird vorher mit `encoding convertto` gewandelt.

```
set cmp [::tclutils::tucmp::data [encoding convertto utf-8 a€] \
    [encoding convertto utf-8 aä]]
dict get $cmp byte1                         ;# -> 226
dict get $cmp size1                         ;# -> 4
```

Fehler	-errorcode
Zeichen über 255 in data	{TCLUTILS TUCMP BYTES}
Datei fehlt	{POSIX ENOENT ...}

11.6 Größen: tusize

tusize summiert Dateigrößen unter einem Verzeichnis und schreibt Bytezahlen lesbar, ein kleines `du`.

```
::tclutils::tusize::bytes   pfad
::tclutils::tusize::entries verzeichnis
::tclutils::tusize::human   n ?-si 0|1?
```

Befehl	Rückgabe
bytes	Größe der Datei, oder die Summe aller Dateien darunter
entries	Liste {bytes pfad} je direktem Eintrag, nach Pfad sortiert
human	Zahl mit Einheit

Option human	Vorgabe	Wirkung
-si	0	0: Basis 1024, B KiB MiB GiB TiB PiB EiB ZiB YiB; 1: Basis 1000, B kB MB GB TB PB EB ZB YB

`bytes` nimmt Punkt-Dateien mit. Gezählt wird die Dateigröße, nicht der belegte Platz auf dem Datenträger, und die Verzeichnisse selbst zählen nicht mit. Unter der Basis gibt `human` die ganze Zahl mit `B` aus, sonst den Wert mit einer Nachkommastelle. Über `YiB` (YB) geht es nicht hinaus, größere Werte bleiben in dieser Einheit.

Die Einheit nennt immer ihre Basis: `KiB` ist 1024, `kB` ist 1000. 1500 und 1536 Bytes sind beide `1.5 KiB`, weil nur eine Nachkommastelle steht. `tunumfmt::toHuman` schreibt dagegen wie `numfmt` das bloße `K` für 1000 (`1.5K`); für Bytes passt dort `-to iec` (`1.5Ki`).

In einen symbolischen Link steigt `bytes` nicht ab und folgt ihm nicht. Gezählt wird der Link selbst mit `file lstat`; unter Unix ist seine Größe die Länge des Zielpfads. Das gilt für Links auf Dateien, auf Verzeichnisse und für Links ohne Ziel. Ein Link im Kreis führt so nicht in eine Schleife.

Abweichungen von `du`: `du` zählt ohne Option belegte Blöcke und die Verzeichnisse selbst. `tusize::bytes` liegt näher an `du -sb`, ohne die Verzeichniseinträge.

```
package require tclutils::tusize
file mkdir projekt/sub
foreach {pfad n} {projekt/a.txt 100 projekt/sub/b.bin 3000} {
    set fh [open $pfad wb]
    puts -nonewline $fh [string repeat x $n]
    close $fh
}
::tclutils::tusize::bytes projekt        ;# -> 3100
::tclutils::tusize::entries projekt
                          ;# -> {100 projekt/a.txt} {3000 projekt/sub}
::tclutils::tusize::human 500            ;# -> 500 B
::tclutils::tusize::human 1536           ;# -> 1.5 KiB
::tclutils::tusize::human 1500 -si 1     ;# -> 1.5 kB
::tclutils::tusize::human 5368709120     ;# -> 5.0 GiB
::tclutils::tusize::human [expr {2**70}] ;# -> 1.0 ZiB
```

An einem echten Baum hängt die Summe vom Inhalt ab; das Ergebnis von `entries` hat dann dieselbe Form, eine Liste von Paaren aus Zahl und Pfad.

Fehler	-errorcode
Pfad fehlt (bytes)	{TCLUTILS TUSIZE NOTFOUND}
kein Verzeichnis oder fehlt (entries)	{TCLUTILS TUSIZE NOTDIR}
n keine ganze Zahl (human)	{TCLUTILS TUSIZE VALUE}
-si kein Wahrheitswert	{TCLUTILS COMMON BOOLEAN -si}
unbekannte Option	{TCLUTILS COMMON OPTION -name}

11.7 Metadaten: tustat

tustat liefert die Werte von `file stat` als Dict und formatiert eine Auswahl davon als Text, ähnlich Unix `stat`.

```
::tclutils::tustat::file   pfad
::tclutils::tustat::files  pfade
::tclutils::tustat::render dict ?-timeformat format?
```

`file` liefert ein Dict mit den Schlüsseln aus `file stat` (`atime`, `blksize`, `blocks`, `ctime`, `dev`, `gid`, `ino`, `mode`, `mtime`, `nlink`, `size`, `uid`) und dazu `path` (wie übergeben), `normalized` (Ergebnis von `file normalize`), `type` (Ergebnis von `file type`: `file`, `directory`, `link` und so weiter) und `modeOctal` (`mode` in der gewohnten Oktalform, etwa `100644`). Bei einem Link kommt `target` hinzu, das Ergebnis von `file readlink`. `files` liefert eine Liste solcher Dicts in der Reihenfolge der Pfade.

Option render	Vorgabe	Wirkung
-timeformat	%Y-%m-%d %H:%M:%S	Format für clock format, Ortszeit

`render` gibt die Felder `path`, `type`, `size`, `mtime`, `mode`, `ino`, `dev`, `uid` und `gid` in dieser

Reihenfolge aus, je Zeile der Name auf zehn Zeichen und der Wert. Fehlt ein Feld im Dict, fehlt die Zeile. Andere Felder gibt `render` nicht aus.

Einem Link folgt `file` nicht, wie Unix `stat` ohne `-L`: Größe, Rechte und Zeiten kommen aus `file lstat` und gehören zum Link selbst. Ein Link ohne Ziel ist deshalb kein Fehler. `mode` ist eine Dezimalzahl mit Dateityp und Rechten; `render` gibt sie so aus.

```
package require tclutils::tustat
set d [::tclutils::tustat::file projekt/a.txt]
dict get $d size                          ;# -> 100
dict get $d type                          ;# -> file
dict get $d path                          ;# -> projekt/a.txt
llength [::tclutils::tustat::files {projekt/a.txt projekt}]
                                          ;# -> 2
file link -symbolic projekt/l.txt a.txt
set l [::tclutils::tustat::file projekt/l.txt]
dict get $l type                          ;# -> link
dict get $l target                        ;# -> a.txt
puts [::tclutils::tustat::render $d]
```

Die Ausgabe von `render` hat diese Form; Zeit, Rechte und Nummern hängen von Datei und Rechner ab:

```
path       projekt/a.txt
type       file
size       100
mtime      JJJJ-MM-TT hh:mm:ss
mode       Dezimalzahl
ino        Dezimalzahl
dev        Dezimalzahl
uid        Dezimalzahl
gid        Dezimalzahl
```

`file` und `files` haben keine eigenen Fehlercodes; ein fehlender Pfad wirft `{POSIX ENOENT ...}` aus `file type`. Eine unbekannte Option an `render` wirft `{TCLUTILS COMMON OPTION -name}`.

11.8 Zeichentabellen: tucode

tucode gibt Tabellen der Zeichen 0 bis 255 aus, ASCII und Latin-1, dazu ausgewählte Zeichengruppen, und schlägt einzelne Zeichen nach. Das Unix-Vorbild sind die Handbuchseiten `ascii(7)` und `iso_8859-1(7)`.

```
::tclutils::tucode::lookup     wert
::tclutils::tucode::lookupChar zeichen
::tclutils::tucode::table      von bis ?option wert ...?
::tclutils::tucode::render     von bis ?option wert ...?
::tclutils::tucode::ascii      ?option wert ...?
::tclutils::tucode::latin1     ?option wert ...?
::tclutils::tucode::ansi       ?option wert ...?
::tclutils::tucode::signs      ?gruppe ...? ?option wert ...?
::tclutils::tucode::groups
```

`lookup` nimmt eine Dezimalzahl, Hex mit `0x`, einen Codepunkt mit `U+` (`U+00E4`, auch klein u+e4) oder ein Zeichen aus Latin-1. Bei mehreren Zeichen gilt das erste. Ein leerer Wert, ein Zeichen über 255 und ein Codepunkt über `U+00FF` sind Fehler. `lookupChar` ist derselbe Befehl. Rückgabe ist ein Dict:

Schlüssel	Inhalt
code	Wert 0–255
hex, oct	zwei Hex-Stellen groß, drei Oktalstellen
glyph	Anzeige: ^@ bis ^_ und ^? für Steuerzeichen, SP für das Leerzeichen, . für 128–159 und das weiche Trennzeichen
name	Kurzname: NUL, DEL, nbsp, ssharp, auml; bei 32–126 das Zeichen selbst
char	das Zeichen

`table` und `render` sind derselbe Befehl für den Bereich `von` bis `bis`, beide aus 0 bis 255. `ascii` ist `0 127`, `latin1` ist `128 255`, `ansi` ist `latin1`. Die Einträge laufen spaltenweise von oben nach unten.

Option	Vorgabe	Wirkung
-columns	4	Spalten nebeneinander
-showName	1	Kurzname je Eintrag
-compact	0	1: Matrix mit 16 Zeichen je Zeile, nur die Anzeige

In der vollen Tabelle wird jeder Name auf die Länge des längsten Namens im Bereich aufgefüllt, so stehen die Spalten bündig. Mit vier Spalten ist sie für ASCII 73 und für Latin-1 85 Zeichen breit.

`signs` gibt Gruppen mit Titel, Linie und einem Eintrag je Zeile aus; ohne Gruppe alle. `groups` liefert die Namen sortiert. Die Gruppen `controls`, `whitespace`, `quotes`, `dashes`, `german`, `currency`, `math` und `latin1hi` enthalten Latin-1-Zeichen; `arrows`, `boxdraw` und `boxdraw2` enthalten Unicode-Zeichen über 255 und zeigen sie als `U+XXXX`. `signs` nimmt die Optionen vor oder nach den Gruppen an; wirksam ist dort `-showName`.

```
package require tclutils::tucode
::tclutils::tucode::lookup 0xDF
```

```
code 223 hex DF oct 337 glyph ß name ssharp char ß
```

```
dict get [::tclutils::tucode::lookup 65] name       ;# -> A
dict get [::tclutils::tucode::lookup ä] code        ;# -> 228
dict get [::tclutils::tucode::lookup U+00E4] name   ;# -> auml
dict get [::tclutils::tucode::lookup 0] glyph       ;# -> ^@
dict get [::tclutils::tucode::lookup 160] name      ;# -> nbsp
puts [::tclutils::tucode::table 0 7 -columns 2]
puts [::tclutils::tucode::table 64 95 -compact 1]
puts [::tclutils::tucode::signs currency]
puts [::tclutils::tucode::signs dashes -showName 0]
```

```
Dec Hex  Chr  Name
----------------------------------------
  0 00  ^@   NUL |   4 04  ^D   EOT
  1 01  ^A   SOH |   5 05  ^E   ENQ
  2 02  ^B   STX |   6 06  ^F   ACK
  3 03  ^C   ETX |   7 07  ^G   BEL

    0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
40  @  A  B  C  D  E  F  G  H  I  J  K  L  M  N  O
50  P  Q  R  S  T  U  V  W  X  Y  Z  [  \  ]  ^  _

Currency symbols
----------------------------------------
 36 24  $    $
162 A2  ¢    cent
163 A3  £    pound
164 A4  ¤    curren
165 A5  ¥    yen
Dashes and hyphen
----------------------------------------
 45 2D  -
173 AD  .
```

In der kompakten Form ist jede Zelle drei Zeichen breit; die Ziffern der Kopfzeile stehen über ihren Spalten.

Fehler	**-errorcode**
Wert außerhalb 0–255, von > bis, keine ganze Zahl	{TCLUTILS TUCODE RANGE}
leerer Wert, Zeichen über 255 (lookup €), U+ über U+00FF	{TCLUTILS TUCODE LOOKUP}
unbekannte Gruppe	{TCLUTILS TUCODE GROUP}
-columns keine ganze Zahl ab 1	{TCLUTILS COMMON INTEGER -columns}

Fehler	-errorcode
-showName, -compact kein Wahrheitswert	{TCLUTILS COMMON BOOLEAN -name}
unbekannte Option, Option ohne Wert	{TCLUTILS COMMON OPTION -name}

11.9 Merkliste

- Daten-Befehle erwarten Bytes 0–255. Text vorher mit `encoding convertto utf-8` wandeln; sonst folgt `{TCLUTILS <MODUL> BYTES}`.
- `tuhexdump` und `tuod` fassen gleiche Zeilen nicht zusammen.
- `tustrings` trennt am Tabulator.
- `patch` und `writeBytes` legen ohne `-backup 0` bzw. `0` eine Kopie `.bak` an, fügen nichts ein und schreiben nicht über das Dateiende.
- `tucmp` zählt den Offset ab 0, nicht ab 1 wie `cmp`.
- `tusize` zählt Dateigrößen, nicht Blöcke; Links mit ihrer eigenen Größe.
- `tustat::file` folgt Links nicht; `target` nennt das Ziel.

Nächstes Kapitel: Notizen und Kalender

12 Notizen und Kalender

Markdown, iCalendar mit Wiederholungsregeln, vCard, ein Notizbaum im Speicher und in SQLite, ein Monatskalender als Text und die bundesweiten Feiertage. Alles liegt im Umbrella und ist reines Tcl; `tunotesdb` braucht beim Aufruf `sqlite3` mit FTS5. `tuvcard` nutzt dazu `tubase64`.

12.1 Markdown: tumd

tumd setzt eine Teilmenge von CommonMark in ein HTML-Fragment, ohne `<html>`-Rahmen. Überschriften sind ATX (# bis `######`). Dazu kommen Absätze, eingezäunter Code, Zitate, flache Listen und waagerechte Linien. Im Fließtext: Code in Gravis, `**fett**`, `*kursiv*`, Links, Bilder und Autolinks. Verschachtelte Listen, Setext-Überschriften, Tabellen und Referenzlinks bleiben stehen, wie sie sind, und werden nicht als solche erkannt. Eine eingerückte Liste wird eine flache Liste.

`frontmatter` trennt einen YAML-Kopf zwischen zwei Zeilen `---`. Fehlt der schließende Strich, bleibt der Kopf leer und der Text ganz. `toHtml` lässt den Kopf weg. `headings` liefert Paare aus Stufe und Text. `toc` baut eine Liste mit Ankern, `toHtml` setzt dieselben als `id`. Der Anker entsteht wie bei GitHub: kleingeschrieben, `*` und Gravis fallen weg, ebenso alles außer Buchstaben, Ziffern, `_`, `-` und Leerzeichen; Leerzeichen werden `-`. Buchstaben jeder Schrift bleiben. Ein zweiter gleicher Anker bekommt `-1`, ein dritter `-2` (seit 0.2; bis 0.1 blieben nur `a`–`z` und Ziffern, und `Jörg` und `Jrg` hatten denselben Anker `jrg`).

```
package require tclutils::tumd
::tclutils::tumd::toHtml "# Größe\n\nEin **Regal** und `x < y`."
                                              ;# -> <h1
    id="größe">Größe</h1>\n<p>Ein <strong>Regal</strong> und <code>x
    &lt; y</code>.</p>
```

```
::tclutils::tumd::headings "# Größe"       ;# -> {1 Größe}
::tclutils::tumd::toc "# Größe"            ;# -> - [Größe](#größe)
::tclutils::tumd::frontmatter "---\ntitle: Lager\n---\n# Lager\n"
                                          ;# -> {title: Lager} {#
    Lager\n}
::tclutils::tumd::toHtml "- a\n  - b"      ;# ->
    <ul>\n<li>a</li>\n<li>b</li>\n</ul>
```

Der Anker von "Größe" ist `größe`. `tustr::slugify` macht daraus `groesse`.

12.2 iCalendar: tuical

`parse` liest den Text in eine Liste von Komponenten. Eine Komponente ist ein Dict mit `type`, `props` und `components`. Eine Eigenschaft ist ein Dict mit `name`, `value` und `params`. Der Wert bleibt roh, wie in der Datei. `escapeText` und `unescapeText` setzen `\,;,,` und den Zeilenumbruch nach RFC 5545. `eventInfo`, `todoInfo` und `journalInfo` lösen die Textfelder auf. Eine Fortsetzungszeile beginnt mit Leerzeichen oder Tabulator. `BEGIN` und `END` dürfen klein geschrieben sein. Ein Parameterwert in Anführungszeichen darf `;` und `:` enthalten (`ALTREP="http://x"`); die Anführungszeichen fallen beim Lesen weg und kommen beim Schreiben wieder dazu, wo der Wert sie braucht. `setProperty` und `addProperty` lehnen einen Wert mit Zeilenumbruch ab (`{TCLUTILS TUICAL VALUE}`), weil der rohe Umbruch die Zeile beenden würde; Text gehört vorher durch `escapeText`.

`events`, `todos` und `journals` suchen `VEVENT`, `VTODO` und `VJOURNAL` auch in verschachtelten Komponenten. `newComponent` baut eine Komponente aus einer flachen Liste `Name Wert`. `toIcs` schreibt sie zurück, jede Zeile mit CR/LF, seit 0.2 auch die letzte. Lange Zeilen faltet `toIcs` nach 75 Byte, nie mitten in einem UTF-8-Zeichen; bis tclutils 0.68.0 zählte es Zeichen, eine Zeile aus Umlauten wurde so 142 Byte lang.

```
package require tclutils::tuical
set ics "BEGIN:VCALENDAR\nBEGIN:VEVENT\nUID:1\nSUMMARY:Lager\\,
    Nord\nDTSTART:20240315T090000\nEND:VEVENT\nEND:VCALENDAR\n"
set ev [lindex [::tclutils::tuical::events [::tclutils::tuical::parse
    $ics]] 0]
dict get [::tclutils::tuical::eventInfo $ev] summary
                                          ;# -> Lager, Nord
dict get [::tclutils::tuical::eventInfo $ev] dtstart
                                          ;# -> 20240315T090000
::tclutils::tuical::escapeText "a;b\nc"    ;# -> a\;b\nc
```

Eine Zeile ohne Doppelpunkt und ein `END` ohne `BEGIN` werfen `{TCLUTILS TUICAL SYNTAX}`. Eine Eigenschaftsliste mit ungerader Länge wirft `{TCLUTILS TUICAL ARG}`.

12.3 Wiederholungen: turrule

turrule rechnet eine Wiederholungsregel `RRULE` aus iCalendar (RFC 5545) in einzelne Termine um. Gerechnet wird in UTC.

Befehl	Wirkung
parse rrule	Dict der Teile; Schlüssel in Großbuchstaben, Werte wie geschrieben
occurrences ?optionen?	Liste der Termine
eventsInRange ics von bis	Liste von Dicts uid summary start für alle VEVENT im Zeitraum; lädt tuical

Option	Vorgabe	Wirkung
-dtstart		Beginn, Pflicht; JJJJ-MM-TT, JJJJMMTT, auch mit Uhrzeit THHMMSS oder THH:MM:SS; ein Z am Ende fällt weg
-rule		die Regel ohne RRULE:
-from	-dtstart	frühere Termine erscheinen nicht
-to		spätere Termine erscheinen nicht
-count	0	höchstens so viele Termine; 0 ohne Grenze

Teil der Regel	Umfang
FREQ	DAILY, WEEKLY, MONTHLY, YEARLY
INTERVAL	Abstand der Perioden, Vorgabe 1
COUNT	Zahl der Termine, gezählt ab -dtstart
UNTIL	spätester Zeitpunkt; als Datum ohne Uhrzeit der ganze Tag
BYDAY	Wochentage MO bis SU; MONTHLY, YEARLY: auch mit Ordnungszahl wie 1MO, 4TH oder -1FR
BYMONTHDAY	Tage des Monats, negativ vom Monatsende; nicht mit WEEKLY
BYMONTH	Monate 1–12; YEARLY nimmt diese Monate, sonst grenzt es ein
WKST	nur MO

YEARLY rechnet so: Mit `BYMONTH` gilt jeder genannte Monat, eine Ordnungszahl in `BYDAY` zählt im Monat (`BYMONTH=11;BYDAY=4TH` ist der vierte Donnerstag im November). Ohne `BYMONTH` gilt `BYMONTHDAY` in jedem Monat, und eine Ordnungszahl in `BYDAY` zählt im Jahr (`BYDAY=20MO` ist der 20. Montag). Ohne diese Teile bleibt der Jahrestag von `-dtstart`. Stehen `BYDAY` und `BYMONTHDAY` zusammen, bleiben nur Tage, die beide erfüllen (`BYDAY=FR;BYMONTHDAY=13`). Ein Monatstermin am 31. überspringt Monate ohne 31. Tag, ein Jahrestermin am 29. Februar die Jahre ohne Schalttag.

Andere Teile wie `BYSETPOS`, `BYWEEKNO`, `BYYEARDAY`, `BYHOUR` oder ein `WKST` außer `MO` werfen `{TCLUTILS TURRULE UNSUPPORTED}`, statt wegzufallen.

`COUNT` zählt ab `-dtstart`; `-from` und `-to` schneiden nur die Ausgabe zu. Ein `-dtstart` ohne Uhrzeit liefert Daten, eines mit Uhrzeit liefert `JJJJ-MM-TTTHH:MM:SS` mit derselben Uhrzeit. `UNTIL` ohne Uhrzeit schließt den ganzen Tag ein: ein Termin um 09:00 am Tag von `UNTIL` gehört dazu.

```
package require tclutils::turrule
::tclutils::turrule::parse "FREQ=WEEKLY;BYDAY=TU,TH;COUNT=4"
    ;# -> FREQ WEEKLY BYDAY TU,TH COUNT 4
::tclutils::turrule::occurrences -dtstart 2025-10-01 \
    -rule "FREQ=WEEKLY;BYDAY=TU,TH;COUNT=4"
    ;# -> 2025-10-02 2025-10-07 2025-10-09 2025-10-14
::tclutils::turrule::occurrences -dtstart 2025-01-31 \
    -rule "FREQ=MONTHLY;BYMONTHDAY=-1" -count 3
    ;# -> 2025-01-31 2025-02-28 2025-03-31
::tclutils::turrule::occurrences -dtstart 20251002T090000 \
    -rule "FREQ=DAILY;INTERVAL=2" -count 2
    ;# -> 2025-10-02T09:00:00 2025-10-04T09:00:00
set ics "BEGIN:VCALENDAR\nBEGIN:VEVENT\nUID:4\nSUMMARY:Standup\n"
append ics "DTSTART:20251002T090000\nRRULE:FREQ=WEEKLY\n"
append ics "END:VEVENT\nEND:VCALENDAR\n"
set ev [::tclutils::turrule::eventsInRange $ics 2025-10-01 2025-10-10]
lmap e $ev {dict get $e start}
    ;# -> 2025-10-02T09:00:00 2025-10-09T09:00:00
::tclutils::turrule::occurrences -dtstart 2025-01-01 \
    -rule "FREQ=YEARLY;BYMONTH=11;BYDAY=4TH;COUNT=2"
    ;# -> 2025-11-27 2026-11-26
::tclutils::turrule::occurrences -dtstart 2025-01-01 \
    -rule "FREQ=MONTHLY;BYDAY=FR;BYMONTHDAY=13;COUNT=2"
    ;# -> 2025-06-13 2026-02-13
::tclutils::turrule::occurrences -dtstart 2025-01-06T09:00:00 \
    -rule "FREQ=WEEKLY;UNTIL=20250120"
    ;# -> 2025-01-06T09:00:00 2025-01-13T09:00:00 2025-01-20T09:00:00
```

Ohne `-to`, `-count`, `COUNT` und `UNTIL` wirft `occurrences` `{TCLUTILS TURRULE UNBOUNDED}`. Eine Regel ohne `FREQ` oder mit einer anderen Frequenz als den vier oben wirft `{TCLUTILS TURRULE FREQ}`, ein unlesbares Datum `{TCLUTILS TURRULE DATE}`. Ein unbekanntes Wochentagskürzel wie `XX`, die Ordnungszahl 0 oder eine Ordnungszahl bei `DAILY` und `WEEKLY` werfen `{TCLUTILS TURRULE BYDAY}`, ein Monat außerhalb 1–12 in `BYMONTH` oder ein Tag außerhalb ±1–31 in `BYMONTHDAY` `{TCLUTILS TURRULE VALUE}`.

12.4 vCard: tuvcard

`parse` liefert eine Liste von Karten. Eine Karte ist eine Liste von Eigenschafts-Dicts, derselbe Aufbau wie bei `tuical`. `property` gibt den ersten Wert eines Namens, `get` alle Werte, `fullName` die Eigenschaft `FN`. `toVcf` schreibt CR/LF, auch nach der letzten Zeile, und faltet wie `tuical` nach 75 Byte. Groß- und Kleinschreibung des Namens spielt keine Rolle. Ein Gruppenpräfix zählt nicht zum Namen: `item1.FN:Ada` findet `fullName`, geschrieben bleibt es `item1.FN`. `TYPE="work;cell"` ist ein Wert `work;cell`. Ein Zeilenumbruch im Wert von `addProperty` oder `setProperty` ist `{TCLUTILS TUVCARD VALUE}`.

```
package require tclutils::tuvcard
set vcf "BEGIN:VCARD\nFN:Ada\nN:Lovelace;Ada;;;\nTEL:1\nEND:VCARD\n"
set card [lindex [::tclutils::tuvcard::parse $vcf] 0]
::tclutils::tuvcard::fullName $card          ;# -> Ada
::tclutils::tuvcard::get $card TEL           ;# -> 1
::tclutils::tuvcard::names $card             ;# -> FN N TEL
```

Eine Zeile ohne Doppelpunkt wirft `{TCLUTILS TUVCARD SYNTAX}`.

12.5 Notizbaum: tunotes

Ein Speicher ist ein Dict von Id auf Notiz. Die Notiz hat `id`, `parent_id`, `title`, `content`, `created`, `modified` und `tags`. `create`, `update` und `delete` bekommen den Namen der Variablen und ändern sie. `get`, `search`, `count`, `roots` und `children` bekommen den Wert. Die Id ist `clock microseconds`.

`search` vergleicht Titel und Inhalt mit `string match`, ohne Rücksicht auf Großbuchstaben. Ein Stern in der Frage ist ein Musterzeichen. `toJson` und `fromJson` gehen über `tujson`. Fehlende Felder füllt `fromJson` leer auf. `load` einer fehlenden Datei ergibt einen leeren Speicher. `save` schreibt die Datei immer als UTF-8 (0.2). `load` liest eine Datei, die kein gültiges UTF-8 ist, in der Systemkodierung; so hat 0.1 sie geschrieben. `delete` mit dem dritten Argument `1` (Vorgabe) entfernt den Zweig, mit `0` hängen die Kinder an der Wurzel.

```
package require tclutils::tunotes
set store [dict create]
set id [::tclutils::tunotes::create store Lager "Regal 3" {ort}]
dict get [::tclutils::tunotes::get $store $id] title
                                             ;# -> Lager
dict get [::tclutils::tunotes::get $store $id] tags
                                             ;# -> ort
::tclutils::tunotes::count $store              ;# -> 1
set gelesen [::tclutils::tunotes::fromJson {{"7":{"title":"Lager"}}}]
dict get [dict get $gelesen 7] title         ;# -> Lager
dict get [dict get $gelesen 7] parent_id     ;# ->
```

Eine unbekannte Id wirft `{TCLUTILS TUNOTES NOTFOUND}`, ebenso ein `create` mit fehlendem Elter.

12.6 Notizen in SQLite: tunotesdb

tunotesdb speichert dieselben Notizen wie `tunotes` in SQLite, je Notiz eine Zeile, mit einem FTS5-Index für die Volltextsuche. Wie `tusqlite` lädt das Modul `sqlite3` nicht: der Aufrufer öffnet die Datenbank und übergibt das Handle. Das Notiz-Dict hat denselben Aufbau wie in `tunotes`; `parent_id` ist bei einer Wurzel leer.

`init db` legt Tabelle, FTS5-Index und drei Trigger an, falls sie fehlen, und schaltet `journal_mode = WAL` und `foreign_keys` ein. Ein wiederholter Aufruf schadet nicht.

Befehl	Wirkung
create db titel inhalt ?tags? ?elter?	die neue Id aus clock microseconds
update db id titel inhalt ?tags? ?elter?	die Id; elter hat die Vorgabe KEEP und bleibt dann
get db id, exists db id	das Notiz-Dict bzw. 1 oder 0
delete db id ?kaskade?	1 (Vorgabe) löscht den Zweig, 0 macht die Kinder zu Wurzeln
move db id elter	die Id; eine Notiz unter sich selbst oder ein Kind wird abgelehnt
ids db, count db	alle Ids bzw. ihre Zahl
roots db, children db elter	Wurzeln bzw. direkte Kinder; children db "" sind die Wurzeln
descendants, ancestors, parent, path, depth, siblings, subtree	der Baum über WITH RECURSIVE; path beginnt an der Wurzel
search db frage ?optionen?	Treffer-Dicts id title snippet rank, bester zuerst
byTag db tag, tags db	Ids mit dem Tag bzw. alle Tags, sortiert
toStore db, fromStore db store ?-clear 1?	die Brücke zu einem tunotes-Speicher

Option von search	Vorgabe	Wirkung
-limit	50	höchstens so viele Treffer; 0 für alle
-snippet	10	Wörter Kontext im Ausschnitt; der Treffer steht in [ ]

Die Frage geht unverändert an FTS5: mehrere Wörter heißen UND, `"wort folge"` ist eine Phrase, `wort*` ein Präfix, `OR` und `NOT` gelten wie geschrieben. Der Tokenizer `unicode61` mit `remove_diacritics 2` sucht ohne Rücksicht auf Groß- und Kleinschreibung und auf Akzente. Tags liegen als Tcl-Liste in einer Spalte; `byTag` vergleicht in Tcl.

```
package require sqlite3
package require tclutils::tunotesdb
sqlite3 db :memory:
::tclutils::tunotesdb::init db
set a [::tclutils::tunotesdb::create db Lager "Regal 3 Nord" ort]
set b [::tclutils::tunotesdb::create db Regal "Fach 2, Schrauben" \
    {} $a]
::tclutils::tunotesdb::count db             ;# -> 2
expr {[::tclutils::tunotesdb::parent db $b] == $a}
                                           ;# -> 1
::tclutils::tunotesdb::depth db $b          ;# -> 1
set h [lindex [::tclutils::tunotesdb::search db schraube*] 0]
dict get $h snippet                         ;# -> Fach 2, [Schrauben]
::tclutils::tunotesdb::tags db              ;# -> ort
```

`fromStore` liest den Speicher in einer Transaktion ein. Eine Notiz mit vorhandener Id wird an Ort und Stelle geändert; ihre Kinder in der Datenbank bleiben. Die Reihenfolge im Speicher spielt keine Rolle, ein Kind darf vor seinem Elter stehen: die Prüfung der Fremdschlüssel wartet bis zum Ende der Transaktion. Jeder `parent_id` muss im Speicher oder in der Datenbank stehen, sonst wirft `fromStore NOTFOUND` und nimmt alles zurück.

```
set st [::tclutils::tunotesdb::toStore db]
dict set st $a title Lagerhalle
::tclutils::tunotesdb::fromStore db [dict create $a [dict get $st $a]]
                                           ;# -> 1
::tclutils::tunotesdb::count db             ;# -> 2
expr {[::tclutils::tunotesdb::path db $b] eq [list $a $b]}
                                           ;# -> 1
```

Der `-errorcode` ist `{TCLUTILS TUNOTESDB grund}`:

Grund	Fall
NODB	das Handle ist kein brauchbarer sqlite3-Befehl
NOFTS5	SQLite ohne FTS5
NOTFOUND	keine Notiz mit der Id, oder kein solcher Elter (auch ein parent_id an fromStore)
CYCLE	move unter sich selbst oder einen Nachfahren
BADQUERY	FTS5 kann die Frage nicht lesen, etwa "offen
BADOPTION	unbekannte Option oder -limit keine Zahl ≥ 0
BADNOTE	einer Notiz an fromStore fehlt ein Feld

12.7 Monatskalender: tucal

tucal gibt Kalender als Text aus, wie das Kommando `cal`. Das Gegenstück als PNG ist `tumonthpng` im Kapitel Bilder ohne Tk.

Befehl	Wirkung
month ?monat? ?jahr? ?optionen?	ein Monat; ohne Angabe der laufende Monat, ohne Jahr das laufende Jahr (UTC)
now ?…?	dasselbe wie month
render jahr monat ?optionen?	ein Monat, das Jahr zuerst
three ?monat? ?jahr? ?optionen?	Vormonat, Monat und Folgemonat nebeneinander
year jahr ?optionen?	das ganze Jahr in vier Zeilen zu je drei Monaten

Eine einzelne vierstellige Zahl ab 1900 an `month` gilt als Jahr und liefert `year`. In `year` laufen die Monate zeilenweise wie bei `cal`: die erste Zeile zeigt Januar, Februar und März.

Option	Vorgabe	Wirkung
-mondayfirst	0	1 beginnt die Woche am Montag
-weeknumbers	0	1 setzt die Kalenderwoche links davor; mit Montag zuerst nach ISO (%V), sonst %U
-iso	0	1 setzt -mondayfirst 1 und -weeknumbers 1
-locale		Locale von clock format für Monatsnamen und Wochentage, etwa de_DE; leer bleibt es englisch (Su Mo Tu …)

Jede Option braucht einen Wert; `-iso` allein reicht nicht. Die Rückgabe ist Text mit `\n`, ohne Zeilenumbruch am Ende. Eine Tageszelle ist in `month`, `render` und `three` drei Zeichen breit, in `year` zwei; ein Monat ist damit 27 bzw. 20 Zeichen breit, mit Kalenderwoche 4 Zeichen mehr. Der Monatstitel steht mittig über den Tagesspalten. `three` und `year` setzen die Monate mit zwei Leerzeichen Abstand nebeneinander; das Jahr in `year` steht mittig über der ganzen Breite. Keine Zeile endet mit einem Leerzeichen.

```
package require tclutils::tucal
puts [::tclutils::tucal::month 5 2026 -locale de_DE -iso 1]
set kal [::tclutils::tucal::month 2 2026]
llength [split $kal \n]                      ;# -> 6
set titel [lindex [split $kal \n] 0]
string first F $titel                        ;# -> 7
string trim $titel                           ;# -> February 2026
set j [split [::tclutils::tucal::year 2026] \n]
regexp -all -inline {[A-Z][a-z]+} [lindex $j 2]
                                            ;# -> January February March
```

Die Ausgabe von `puts`:

```
             Mai 2026
     Mo  Di  Mi  Do  Fr  Sa  So
18                    1   2   3
19    4   5   6   7   8   9  10
```

```
20   11  12  13  14  15  16  17
21   18  19  20  21  22  23  24
22   25  26  27  28  29  30  31
```

Ein Monat außerhalb 1–12 wirft `{TCLUTILS TUCAL MONTH}`, ein Jahr kleiner 1 `{TCLUTILS TUCAL YEAR}`. Mehr als zwei Zahlen werfen `{TCLUTILS TUCAL ARGS}`, eine Option ohne Wert `{TCLUTILS TUCAL OPTION}`, eine unbekannte Option `{TCLUTILS COMMON OPTION -name}`.

12.8 Feiertage: tuholiday

`easter` rechnet den Ostersonntag im gregorianischen Kalender (Meeus/Jones/Butcher) und gibt ein ISO-Datum. `holidays` liefert ein Dict Datum auf Name, sortiert. `-region` kennt nur `de`: die Tage, die in allen Ländern gelten, dazu Ostersonntag und Pfingstsonntag. Heilige Drei Könige, Fronleichnam, Allerheiligen und der Reformationstag fehlen. `isHoliday` gibt den Namen oder die leere Zeichenkette.

Abstand zu Ostern	Name
−2	Karfreitag
0	Ostersonntag
+1	Ostermontag
+39	Christi Himmelfahrt
+49	Pfingstsonntag
+50	Pfingstmontag

Fest liegen Neujahr, der Tag der Arbeit, der Tag der Deutschen Einheit und die beiden Weihnachtstage. 2024 hat das Dict elf Einträge. Oster- und Pfingstsonntag sind nur in einzelnen Ländern gesetzliche Feiertage; `-sundays 0` (seit 0.2, für `holidays` und `isHoliday`) lässt sie weg und gibt die neun Tage, die überall gelten.

```
package require tclutils::tuholiday
::tclutils::tuholiday::easter 2024           ;# -> 2024-03-31
::tclutils::tuholiday::easter 2025           ;# -> 2025-04-20
::tclutils::tuholiday::isHoliday 2024-03-31  ;# -> Ostersonntag
::tclutils::tuholiday::isHoliday 2024-03-15  ;# ->
::tclutils::tuholiday::isHoliday 2024-03-31 -sundays 0  ;# ->
dict size [::tclutils::tuholiday::holidays 2024 -sundays 0]  ;# -> 9
```

`easter xyz` wirft `{TCLUTILS TUHOLIDAY YEAR}`. `-region at` wirft `{TCLUTILS TUHOLIDAY REGION}`, `-sundays vielleicht` `{TCLUTILS TUHOLIDAY SUNDAYS}`.

12.9 Kalenderansichten: tical

Die Module dieses Kapitels liefern Daten und Text: Wiederholungen, Feiertage, iCalendar, einen Monat wie `cal`. Ansichten für Monat, Woche und Tag mit Renderern für Terminal, Tk-Canvas und PDF bietet die eigenständige Bibliothek tical 1.1. Sie bringt eigene Feiertage und einen

iCalendar-Export (mit RRULE) mit; ihr Kern braucht tclutils nicht. Ihr optionaler Import (`tical::io::import`) liest iCalendar mit `tuical` und expandiert Wiederholungen mit `turrule`. Beschrieben ist sie im Buch *Utils-Stack*.

12.10 Merkliste

- `tumd` ist eine Teilmenge. Der Anker von "Größe" ist `größe`.
- iCalendar- und vCard-Werte bleiben roh. `eventInfo` löst Text auf.
- `toIcs` und `toVcf` schreiben CR/LF, auch nach der letzten Zeile.
- `tunotes`: ändern über den Variablennamen, lesen über den Wert.
- Feiertage: nur `de`, elf Namen, keine landesspezifischen Tage.
- `turrule` braucht eine Grenze: `-to`, `-count`, `COUNT` oder `UNTIL`. Nicht unterstützte Teile werfen `UNSUPPORTED`.
- `tunotesdb`: Handle selbst öffnen, dann `init`. `fromStore` ändert vorhandene Notizen an Ort und Stelle.
- `tucal` ist Text wie `cal`; als PNG `tumonthpng`.

Nächstes Kapitel: Text aus PDF-Dateien

13 Text aus PDF-Dateien

tupdftext 0.1 ist neu in tclutils 0.67.0. Es liest den Text einer PDF Seite für Seite, in reinem Tcl. Die Ströme packt Tcls `zlib` aus. poppler und eine C-Erweiterung braucht es nicht. Gedacht ist es für die Volltextsuche: Wörter, Leerzeichen und Zeilenwechsel. Spalten und Tabellen kommen in der Reihenfolge des Inhaltsstroms.

Die Struktur einer PDF (Version, Objekte, Trailer, Metadaten, ZUGFeRD) liest das Schwestermodul `tupdf`. `tupdftext` liest den Text.

13.1 Befehle

```
package require tclutils::tupdftext
::tclutils::tupdftext::pages   datei ?-notext error|empty?
::tclutils::tupdftext::text    datei ?-separator s? ?-notext
    error|empty?
::tclutils::tupdftext::summary datei
```

`pages` liefert eine Liste, eine Zeichenkette je Seite, in Seitenfolge. Zeilen sind mit `\n` getrennt, Folgen von Leerzeichen zusammengezogen, Ligaturen (fi, fl) ausgeschrieben, weiche Trennstriche entfernt.

`text` hängt die Seiten mit `-separator` aneinander. Die Vorgabe ist der Seitenvorschub `\f`, wie bei `pdftotext`. Eine einzelne Seite hat keinen Trenner.

`summary` liefert ein Dict:

Schlüssel	Bedeutung
pages	Zahl der Seiten
encrypted	1, wenn ein Eintrag /Encrypt da ist

Schlüssel	Bedeutung
version	Version aus der Kopfzeile, etwa 1.4
textPages	Seiten mit mindestens einem Zeichen
unmapped	Glyphen, die als U+FFFD kamen (Zwei-Byte-Schrift ohne /ToUnicode)
fonts	Liste von Dicts name, subtype, toUnicode

Mit `-notext empty` wird aus einer PDF ohne Text eine Liste leerer Zeichenketten, eine je Seite. Ohne die Option ist das der Fehler `NOTEXT`.

Der Befehl heißt `summary`, nicht `info`. Eine Prozedur `info` im Namensraum des Moduls hätte Tcls `info` für alle Prozeduren des Moduls verdeckt.

```
set seiten [::tclutils::tupdftext::pages $datei]
llength $seiten                                          ;# -> 1
lindex $seiten 0                                         ;# -> Lager
    Nord, Regal 3\nGroesse: 12 Stueck
dict get [::tclutils::tupdftext::summary $datei] version ;# -> 1.4
dict get [::tclutils::tupdftext::summary $datei] unmapped
                                                         ;# -> 0
::tclutils::tupdftext::text $zweiSeiten                  ;# -> Seite
    eins\fSeite zwei
```

Die Kommentare gelten für zwei Dateien aus pdf4tcl: eine Seite Helvetica mit den beiden Zeilen oben, und zwei Seiten mit "Seite eins" und "Seite zwei". Dieselbe leere Seite ohne die Option wirft `{TCLUTILS TUPDFTEXT NOTEXT}`; mit `-notext empty` ist die Liste ein leeres Element. Eine fehlende Datei ergibt `FILE`, eine Datei ohne `%PDF--Kopf` `FORMAT`. Tcl 8.6 und Tcl 9 liefern denselben Text.

13.2 Fehler

Jeder Fehler trägt `-errorcode {TCLUTILS TUPDFTEXT grund}`:

Grund	Wann
FILE	die Datei ist nicht lesbar
FORMAT	kein %PDF--Kopf, keine Objekte oder keine Seiten
ENCRYPTED	die Datei ist verschlüsselt; auch ein leeres Benutzerpasswort wird nicht gelesen
NOTEXT	keine Seite hat Text: ein Scan ohne OCR, oder nur Bilder
OPTION	unbekannte Option oder fehlender Wert

13.3 Was ankommt

Objekte sucht es als `N G obj` am Zeilenanfang. Die Querverweistabelle braucht es nicht; eine beschädigte stört nicht. Eine spätere Definition derselben Nummer gilt, wie bei einem

inkrementellen Update. Objekte in Objektströmen (`/ObjStm`) füllen Nummern, die nicht direkt dastehen.

Filter: `FlateDecode` (auch mit PNG-Predictor), `ASCIIHexDecode`, `ASCII85Decode`, `LZWDecode`, und Ketten davon. Seiten kommen über den Seitenbaum ab `/Root`, mit geerbten `/Resources`. Ohne brauchbaren Baum: alle Objekte `/Type /Page` in Dateireihenfolge.

Textoperatoren: `Tj`, `TJ`, `'`, `"` mit `Tf`, `Tc`, `Tw`, `Tz`, `TL`, `Ts`, `Td`, `TD`, `Tm`, `T*`, `q`, `Q`, `cm`. Form-XObjects (`Do`) mit eigenen Ressourcen und eigener Matrix, bis Tiefe 8; ein Formular, das sich selbst zeichnet, einmal. Eingebettete Bilder werden übersprungen.

Schriften: Type0 (Zwei-Byte-Codes, `Identity-H`/`-V` oder eine eingebettete CMap), Type1, TrueType, Type3. Unicode aus `/ToUnicode` (`bfchar`, `bfrange`), sonst aus `/Encoding` (`WinAnsiEncoding`, `MacRomanEncoding`, `StandardEncoding`, `PDFDocEncoding` und `/Differences` mit Glyphennamen). Leerzeichen und Zeilenwechsel kommen aus den Positionen: ein Sprung um eine halbe Zeile nach unten ist eine neue Zeile, eine Lücke von mehr als 0,15 em in der Zeile ein Leerzeichen.

13.4 Was es nicht tut

- entschlüsseln (ENCRYPTED)
- Text aus Bildern lesen
- eine Zwei-Byte-Schrift ohne `/ToUnicode` abbilden (U+FFFD, gezählt in `unmapped`)
- Spalten auseinanderhalten
- Wörter zusammensetzen, die am Zeilenende getrennt sind (`pdftotext` tut das)

Im Vergleich mit `pdftotext` (poppler 24.02) an 13 PDFs, festgehalten in `docs/tupdftext.md`: bei elf Dateien 99,5 bis 100 % der Wörter, bei zwei Formelseiten 91 %. Ein Buch mit 108 Seiten braucht unter Tcl 8.6.14 etwa 3,6 s. Tcl 9.0.4 liefert dort denselben Text; Zeichen außerhalb der BMP sind unter Tcl 9 ein Zeichen und unter Tcl 8.6 ein Ersatzpaar. Dieser Vergleich ist hier nicht erneut gelaufen. Eine verschlüsselte Datei ist in diesem Lauf nicht erzeugt; die Modulbeschreibung nennt dafür `ENCRYPTED`.

13.5 Kommandozeile

```
tclsh bin/tupdf.tcl text doc.pdf
tclsh bin/tupdf.tcl text doc.pdf -page 3
```

`book-search.tcl` und mdserver nutzen das Modul für den Text in PDFs.

13.6 Merkliste

- `pages` je Seite, `text` mit `\f` dazwischen, `summary` für die Kennzahlen.
- Scan und verschlüsselte Datei sind Fehler, kein stiller Leerstring.
- `-notext empty` macht aus dem Fehler eine leere Seitenzeichenkette.
- Spalten, Silbentrennung am Zeilenende und Text in Bildern bleiben draußen.

Nächstes Kapitel: ODF und PDF-Struktur

14 ODF und PDF-Struktur

tupdf 0.2 liest die Struktur einer PDF: Kopfzeile, Objektnummern, Seitenmarker, Trailer, Infos und einen Hinweis auf ZUGFeRD. Den Text der Seiten liest `tupdftext`, Kapitel Text aus PDF-Dateien .

tuodf 0.1 schreibt und liest ein schlichtes OpenDocument mit Absätzen. Beide liegen im Umbrella.

14.1 PDF: tupdf

```
package require tclutils::tupdf
::tclutils::tupdf::version  datei
::tclutils::tupdf::objects  datei
::tclutils::tupdf::object   datei nummer
::tclutils::tupdf::trailer  datei
::tclutils::tupdf::metadata datei
::tclutils::tupdf::summary  datei
::tclutils::tupdf::zugferd  datei
```

`version` nimmt die Zahl hinter `%PDF-` in den ersten 1024 Bytes. `objects` liefert die Nummern aus `N G obj`, sortiert und einmalig. Komprimierte Objektströme packt das Modul mit `zlib` aus, bevor es zählt. `object` gibt den Rohtext von `id gen obj` bis `endobj`. `pages` in `summary` zählt `/Type /Page` und lässt `/Pages` aus.

Die folgende Datei hat den Kopf `%PDF-1.4`, vier Objekte, eine Seite und ein Info-Objekt mit `Title (Lager)` und `Author (Nord)`.

```
::tclutils::tupdf::version $datei                    ;# -> 1.4
::tclutils::tupdf::objects $datei                    ;# -> 1 2 3 4
dict get [::tclutils::tupdf::summary $datei] pages   ;# -> 1
dict get [::tclutils::tupdf::metadata $datei] Title  ;# -> Lager
```

`summary` trägt zusätzlich `version`, `size`, `objects`, `encrypted`, `linearized`, `acroform` und `zugferdDetected`. Vorhandene Infofelder liegen in demselben Dict. Hier sind `encrypted`, `linearized` und `acroform` 0, `zugferdDetected` ebenfalls 0.

`zugferd` ist eine Suche im Dateitext, keine Konformitätsprüfung. Ohne Anhang und ohne die Marker bleibt das Dict leer in den Listen:

```
dict get [::tclutils::tupdf::zugferd $datei] detected ;# -> 0
dict get [::tclutils::tupdf::zugferd $datei] profile  ;# -> {}
```

Die Schlüssel sind `detected`, `profile`, `attachmentNames`, `mimeTypes` und `hints`. Ein Treffer setzt `profile` auf `factur-x`, `zugferd` oder `unknown`.

Code	Wann
{TCLUTILS TUPDF FORMAT}	kein %PDF--Kopf, oder endobj fehlt nach dem Objekt
{TCLUTILS TUPDF ARG}	die Objektnummer ist keine ganze Zahl

Code	Wann
{TCLUTILS TUPDF NOTFOUND}	die Nummer steht nach dem Aufblasen nicht im Text

`object $datei x` wirft ARG. `object $datei 9` an der Datei oben wirft `NOTFOUND`.

14.2 ODF: tuodf

```
package require tclutils::tuodf
::tclutils::tuodf::createText datei absätze ?-title T? ?-creator C?
::tclutils::tuodf::parts      datei
::tclutils::tuodf::part       datei name
::tclutils::tuodf::paragraphs datei
::tclutils::tuodf::text       datei
```

`createText` gibt den Dateipfad zurück. Der Behälter ist ein ZIP. `mimetype` steht als erster Eintrag und bleibt unkomprimiert. `-title` und `-creator` landen in `meta.xml`.

```
set z [::tclutils::tuodf::createText lager.odt \
    [list {Lager Nord} {a < b} {}] -title Lager -creator Nord]
::tclutils::tuodf::parts $z
    ;# -> mimetype content.xml styles.xml meta.xml
    META-INF/manifest.xml
::tclutils::tuodf::part $z mimetype
    ;# -> application/vnd.oasis.opendocument.text
::tclutils::tuodf::paragraphs $z          ;# -> {Lager Nord} {a < b} {}
::tclutils::tuodf::text $z                ;# -> Lager Nord\na < b\n
```

`a < b` kommt als Text zurück. Im XML steht `<`. Ein leerer Eintrag in der Liste ist ein leerer Absatz. `text` hängt die Absätze mit `\n` aneinander, deshalb endet das Ergebnis mit einem Zeilenumbruch. `meta.xml` enthält `<dc:title>Lager</dc:title>` und `<dc:creator>Nord</dc:creator>`.

`paragraphs` liest `text:p`. Tabellen und andere Blöcke übergeht es. Eine fehlende ZIP-Datei meldet `tuzip`, nicht einen eigenen Code von `tuodf`.

14.3 Merkliste

- Den Seitentext liest `tupdftext`. `tupdf` liefert Version, Objekte, Infos.
- `/Pages` zählt nicht als Seite. `zugferd` prüft die Datei nicht auf Norm.
- `tuodf` schreibt Absätze. `mimetype` ist der erste, unkomprimierte Eintrag.
- Ein leerer Absatz am Ende hängt an `text` einen Zeilenumbruch.

Nächstes Kapitel: Diagramme

15 Diagramme

Kästen, Pfeile, Balken und Kreise entstehen in reinem Tcl, ohne Browser und ohne Node. Die Eingabe ist ein Text in einer Teilmenge der Mermaid-Schreibweise, die Ausgabe SVG oder PNG.

Für Knoten-Kanten-Graphen ist das Modell ein Dict von tudiagram: `layout` rechnet die Kästen in Pixeln, `toSvg` und `toPng` zeichnen daraus. Die übrigen Arten haben je ein eigenes Modul mit eigenem Modell und eigener Zeichnung.

tuflow 0.3 ist der Eingang für alle Arten. Beim Laden holt `tuflow` nur `tudiagram` 0.5. Das Modul einer Art kommt beim ersten Aufruf mit diesem Schlüsselwort. Die Liste steht auch im Kapitel Umbrella und Module.

Art	Schlüsselwort	Modul	parse liefert Graph	PNG-Optionen
Flussdiagramm	flowchart, graph	tuflow	ja	-scale, -fontfile
Zustände	stateDiagram, stateDiagram-v2	tustate	ja	-scale, -fontfile
Klassen	classDiagram, classDiagram-v2	tuclass	ja	-scale, -fontfile
Entitäten	erDiagram	tuer	ja	-scale, -fontfile
Anforderungen	requirementDiagram	turequirement	ja	-scale, -fontfile
Mindmap	mindmap	tumindmap	ja	-scale, -fontfile
C4	C4Context, C4Container, C4Component, C4Dynamic, C4Deployment	tuc4	ja	-scale, -fontfile
Blöcke	block-beta	tublock	ja	-scale, -fontfile
Git-Verlauf	gitGraph	tugit	ja	-scale, -fontfile
Architektur	architecture, architecture-beta	tuarchitecture	ja	-scale, -fontfile
Sequenz	sequenceDiagram	tusequence	nein, UNSUPPORTED	Fläche aus dem Inhalt
Gantt	gantt	tugantt	nein	Breite 800, Höhe ab 400 nach Aufgaben
Kreis	pie	tupie	nein	480 × 320, -legend
XY	xychart-beta	tuxychart	nein	520 × 340
Quadrant	quadrantChart	tuquadrant	nein	420 × 420
Journey	journey	tujourney	nein	600 × 360

Art	Schlüsselwort	Modul	parse liefert Graph	PNG-Optionen
Zeitleiste	timeline	tutimeline	nein	640 × 380
Sankey	sankey, sankey-beta	tusankey	nein	700 × 400
Radar	radar, radar-beta	turadar	nein	600 × 600
Treemap	treemap, treemap-beta	tutreemap	nein	640 × 400
Bitfelder	packet, packet-beta	tupacket	nein	Breite 640, Höhe aus den Zeilen
Kanban	kanban	tukanban	nein	Fläche aus Spalten und Karten

Die Maße in der letzten Spalte sind die Vorgaben von `-width` und `-height`; jede Art ohne Graph nimmt dazu `-scale` und `-fontfile`. Mermaid kennt weitere Arten (`zenuml`, `xychart` ohne `-beta`). Ein solcher Text geht als Flussdiagramm durch, das erste Wort wird ein Knoten.

15.1 Vom Text zur Datei: tuflow

```
::tclutils::tuflow::parse    text                ;# -> tudiagram-Modell
::tclutils::tuflow::toSvg    text ?opt wert ...? ;# -> SVG-Text
::tclutils::tuflow::toPng    text ?opt wert ...? ;# -> PNG-Bytes
::tclutils::tuflow::writeSvg text datei ?opt wert ...?  ;# -> datei
::tclutils::tuflow::writePng text datei ?opt wert ...?  ;# -> datei
```

`toSvg` und `toPng` lesen das erste Wort der ersten Zeile, die weder leer noch ein `%%`-Kommentar ist, ohne Rücksicht auf Groß- und Kleinschreibung. Steht dort eine Art ohne Graph, ruft die Fassade `parse` und `toSvg` oder `toPng` des Moduls dieser Art. Sonst ruft sie das eigene `parse` und zeichnet das Modell mit `tudiagram`. `writeSvg` schreibt UTF-8 mit LF, `writePng` binär; beide geben den Dateinamen zurück. Bei allen Diagrammmodulen geht das Schreiben über eine temporäre Datei (`common::replaceFile`): Scheitert es, bleibt die alte Datei stehen, und kein Kanal bleibt offen.

```
package require tclutils::tuflow
set t {flowchart LR
    A --> B
}
string range [::tclutils::tuflow::toSvg $t] 0 3   ;# -> <svg
string range [::tclutils::tuflow::toPng $t] 1 3   ;# -> PNG
::tclutils::tuflow::writePng $t fluss.png         ;# -> fluss.png
```

Option	Vorgabe	Wirkung
-scale	1	ganze Zahl ab 1; PNG: Fläche und Zeichnung mal Faktor
-fontfile	leer	TTF- oder OTF-Datei für die PNG-Schrift

Option	Vorgabe	Wirkung
-width	je Art	Breite in Pixeln, nur Arten ohne Graph
-height	je Art	Höhe in Pixeln, nur Arten ohne Graph
-legend	1	nur pie: 0 lässt die Legende weg

Für einen Graphen nimmt `toPng` nur `-scale` und `-fontfile`, `toSvg` gar keine Option. Für die anderen Arten gilt `-scale` auch im SVG: Die Fläche bleibt bei `-width` × `-height`, Schrift, Linien und Abstände werden um den Faktor größer. Eine Option, die die Art nicht kennt, lässt die Fassade ohne Fehler fallen; `-scale 0` dagegen wirft den Fehler `ARG` der Art, beim Graphen `{TCLUTILS TUDIAGRAM ARG}`. `-width 0` bei einer festen Fläche wirft `{TCLUTILS COMMON INTEGER width}` aus der Zeichenfläche.

```
set p "pie\n\"a\" : 1\n"
set a [::tclutils::tuflow::toPng $p -farbe rot]
string equal $a [::tclutils::tuflow::toPng $p]   ;# -> 1
catch {::tclutils::tuflow::toPng $p -scale 0} m o
dict get $o -errorcode                    ;# -> TCLUTILS TUPIE ARG
```

Das Modul einer Art direkt aufzurufen lohnt sich in drei Fällen: wenn das Modell gebraucht wird (`parse` jeder Art liefert ein Dict), wenn eine unbekannte Option als Fehler auffallen soll (die Module prüfen streng mit `{TCLUTILS COMMON OPTION ...}`), und wenn ein `tudiagram` -Modell vor dem Zeichnen geändert wird, etwa mit `setMeta -theme`. Jedes Modul ohne Graph hat dieselben fünf Befehle: `parse text` und `toSvg`, `toPng`, `writeSvg`, `writePng` mit dem Modell statt des Texts.

`tuflow::parse` liefert für eine Art ohne Graph den Fehler `{TCLUTILS TUFLOW UNSUPPORTED}`, auch für `sankey` ohne `-beta`.

```
catch {::tclutils::tuflow::parse "pie\n\"a\" : 1\n"} m o
dict get $o -errorcode               ;# -> TCLUTILS TUFLOW UNSUPPORTED
catch {::tclutils::tuflow::parse "sankey\na,b,1\n"} m o
dict get $o -errorcode               ;# -> TCLUTILS TUFLOW UNSUPPORTED
```

15.2 Modell: tudiagram

```
::tclutils::tudiagram::create   ?-title t? ?-direction d? ?-theme t?
                                ?-fontfile f?
::tclutils::tudiagram::addNode  d id ?-label t? ?-shape s?
                                ?-style dict?
::tclutils::tudiagram::addEdge  d von nach ?-label t? ?-style s?
                     ?-arrow a? ?-startMark m? ?-endMark m?
::tclutils::tudiagram::addGroup d id ?-label t? ?-members ids?
::tclutils::tudiagram::setMeta  d ?-schlüssel wert ...?
::tclutils::tudiagram::validate d           ;# -> Liste der Probleme
::tclutils::tudiagram::layout   d           ;# -> Dict mit Koordinaten
::tclutils::tudiagram::theme    name        ;# -> Farb-Dict
::tclutils::tudiagram::toSvg    d
```

```
::tclutils::tudiagram::toPng    d ?-scale n?
::tclutils::tudiagram::writeSvg d datei
::tclutils::tudiagram::writePng d datei ?-scale n?
```

`create` liefert das Dict mit den Schlüsseln `version`, `meta`, `nodes`, `edges` und `groups`. `meta` hält `title`, `direction`, `theme`, `fontfile`, `nodeGap` 30, `rankGap` 70 und `padding` 20. Die Richtung ist `LR`, `TB`, `RL` oder `BT`, Vorgabe `LR`. Themen sind `default`, `pipeline` und `mono`. Statt des Namens geht auch ein Dict mit mindestens `fill`; fehlende Farben kommen aus `default`. Jeder andere Wert wirft `{TCLUTILS TUDIAGRAM THEME}`, in `theme`, `create` und `setMeta`. Der Titel steht mittig über dem Graphen: `layout` schiebt alle Knoten und Kanten um das Titelband nach unten (`titleH` in `meta`, 26 Pixel) und verbreitert die Fläche, wenn der Titel breiter ist als der Graph. `addNode`, `addEdge` und `addGroup` geben das erweiterte Dict zurück, sie ändern das übergebene nicht. `setMeta` setzt beliebige Schlüssel in `meta` und prüft nur `-direction` und `-theme`.

Form	Namen
Knoten -shape	box (Vorgabe), rounded, dot, circle, stadium, diamond, hexagon, cylinder
Knoten -style	Dict mit fill, stroke, text als #rrggbb; fehlende Schlüssel nimmt das Thema
Kante -style	solid (Vorgabe), dashed, dotted, thick
Kante -arrow	end (Vorgabe), none, both, start
Kante -startMark, -endMark	none (Vorgabe), exactlyOne, zeroOrOne, oneOrMany, zeroOrMany

Ein Knoten ohne `-label` trägt seine Id. Ein `dot` zeigt keine Beschriftung. Die Marken an den Kantenenden sind Krähenfüße wie im ER-Diagramm. `addEdge` prüft weder die Enden noch die Werte von `-style` und `-arrow`.

`validate` wirft nicht. Eine leere Liste heißt: es gibt Knoten, und jede Kante nennt bekannte Enden. `layout` setzt an jedem Knoten `x`, `y`, `width` und `height`, an jeder Kante `points`, `arrowStart` und `back`, in `meta` die Gesamtmaße `width` und `height` sowie die Höhe des Titelbands `titleH`. Die Schriftmetrik ist das feste 6×8-Raster, auch ohne Zeichenfläche. Eine Kante von einem Knoten zu sich selbst nimmt die Rangfolge nicht auf und wird nicht gezeichnet. Ein Zyklus wird an einer Kante gebrochen, die dann rückwärts läuft (`back` 1).

```
package require tclutils::tudiagram
set d [::tclutils::tudiagram::create -title Lager -direction LR]
set d [::tclutils::tudiagram::addNode $d A -label Eingang]
set d [::tclutils::tudiagram::addNode $d B -label Pruefung \
    -shape diamond]
set d [::tclutils::tudiagram::addEdge $d A B -label ja]
::tclutils::tudiagram::validate $d           ;# -> (leer)
set L [::tclutils::tudiagram::layout $d]
dict keys [lindex [dict get $L nodes] 0]
                        ;# -> id label shape style x y width height
dict get [dict get $L meta] width            ;# -> 394
dict get [dict get $L meta] titleH           ;# -> 26
```

Nach `layout` steht `A` bei x 20, y 46, 112×44, und `B` bei x 202, y 46, 172×56; ohne Titel stehen beide bei y 20. `toSvg` beginnt mit `<svg xmlns="http://www.w3.org/2000/svg"`.

Gruppen sind Rahmen mit Beschriftung um die Knoten in `-members`. Ein Name in `-members`, der kein Knoten ist, fällt beim Zeichnen weg. Liegt eine Gruppe ganz in einer anderen, bekommt die äußere mehr Abstand. Mit Gruppen ordnet `layout` die Mitglieder jeder Gruppe in ein eigenes Band, damit sich die Rahmen nicht überlappen.

Eine zweite Id `A` wirft `{TCLUTILS TUDIAGRAM DUPID}`. Die Richtung `XX` wirft `{TCLUTILS TUDIAGRAM DIR}`, eine Form `triangle` `{TCLUTILS TUDIAGRAM ARG}`. Ein Modell ohne Knoten ergibt aus `validate` den Eintrag `EMPTY no nodes`, `layout` wirft dann `{TCLUTILS TUDIAGRAM EMPTY}`. Eine Kante nach `Z` und `Y` hängt an `validate` je einen Eintrag `NONODE` für beide Enden.

```
catch {::tclutils::tudiagram::create -theme neon} m o
dict get $o -errorcode               ;# -> TCLUTILS TUDIAGRAM THEME
```

15.3 Flussdiagramm: tuflow parse

`parse` liest `graph` und `flowchart` und liefert ein `tudiagram`-Modell. Die Kopfzeile ist optional, Groß- und Kleinschreibung zählen nicht (`FLOWCHART td`, `Graph LR`); ohne Richtung gilt `LR`, `TD` wird `TB`. Eine Id besteht aus Buchstaben, Ziffern und `_`, Umlaute eingeschlossen: `Büro[Amt]` ist der Knoten `Büro` (bis tclutils 0.68.0 nur ASCII, und die Zeile fiel weg). Die Form steckt in der Klammer, Anführungszeichen um die Beschriftung fallen weg.

Schreibweise	Form
A allein	box, Beschriftung A
A[Text] und A[[Text]]	box
A(Text)	rounded
A([Text])	stadium
A((Text))	circle
A{Text}	diamond

Schreibweise	Form
A{{Text}}	hexagon
A[(Text)]	cylinder

Pfeil	-style	-arrow
-->, --o, --x	solid	end
---	solid	none
-.->	dotted	end
-.-	dotted	none
==>	thick	end
===	thick	none

Die Beschriftung steht als `A -->|Text| B`, `A -- Text --> B`, `A -. Text.-> B` oder `A == Text ==> B`. Ketten `A --> B --> C` gelten. `&` verbindet mehrere Knoten auf einer Seite: `A & B --> C & D` ergibt eine Kante für jedes Paar, hier vier. `%%` ist ein Kommentar, Anweisungen dürfen mit `;` getrennt sein; ein `;` in einer Beschriftung (`A[Jörg;Büro]`, `-->|a;b|`) oder in Anführungszeichen trennt nicht. Wird eine Id zweimal mit Form genannt, gilt die letzte. `subgraph name [Beschriftung]` bis `end` wird eine Gruppe mit den Knoten darin, auch verschachtelt; `direction` darin entfällt. Eine Kante auf die Gruppen-Id entfällt.

Nicht gelesen werden:

- `style`, `classDef`, `class`, `click` und `linkStyle`: übergangen;
- `A:::klasse`: die Klasse entfällt, der Knoten bleibt;
- `A[/Text/]` und die anderen Trapez- und Parallelogrammformen: `box` mit der Beschriftung `/Text/`;
- `---o` und `---x` zeichnen eine gewöhnliche Spitze, keinen Kreis und kein Kreuz.

```
package require tclutils::tuflow
set src {flowchart TD
    A[Eingang] --> B{Pruefung}
    B -->|ja| C[(Lager)]
}
set m [::tclutils::tuflow::parse $src]
dict get [dict get $m meta] direction        ;# -> TB
set e [lindex [dict get $m edges] 1]
list [dict get $e label] [dict get $e style] ;# -> ja solid
set w [::tclutils::tuflow::parse "Graph RL\nA & B --> C\n"]
dict get [dict get $w meta] direction        ;# -> RL
lmap k [dict get $w edges] {dict get $k from}  ;# -> A B
```

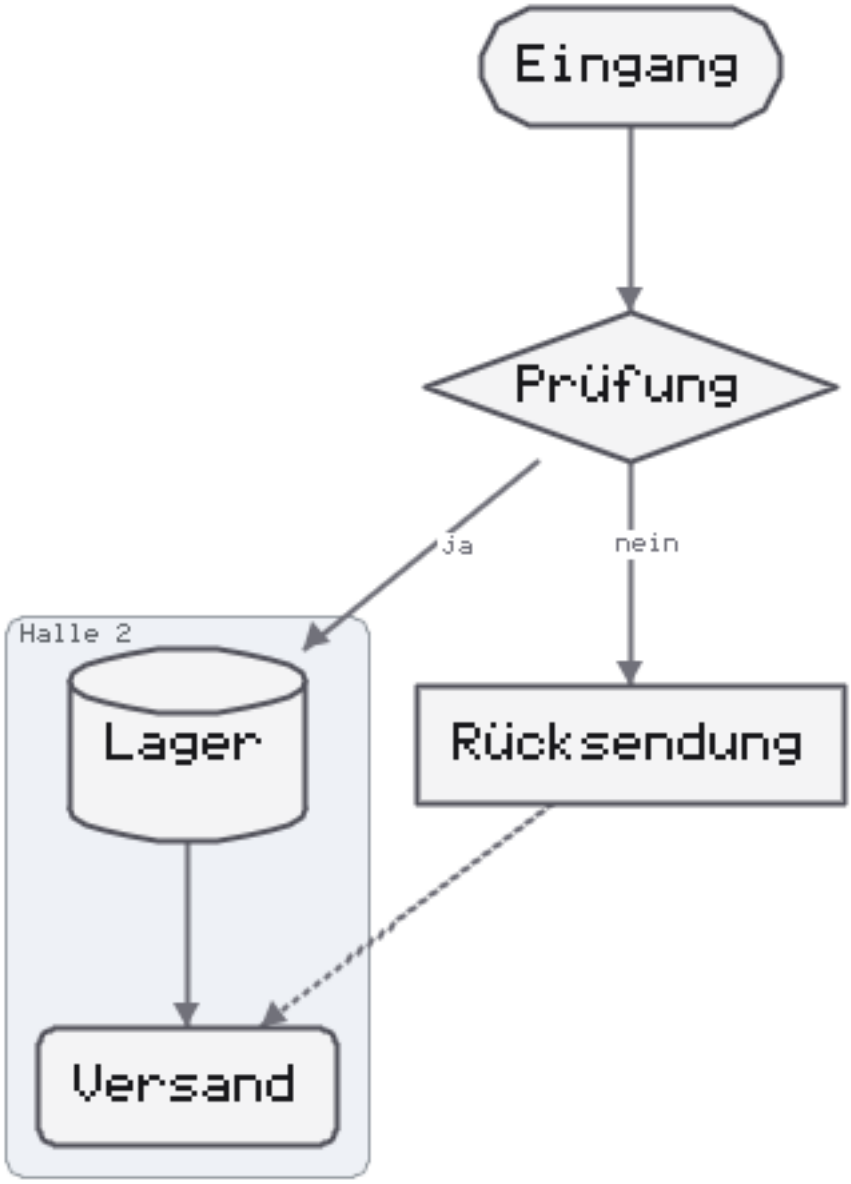
Die Knoten sind `A` als `box` "Eingang", `B` als `diamond` "Pruefung" und `C` als `cylinder` "Lager". Eine Kante ist ein Dict mit `from`, `to`, `label`, `style`, `arrow`, `startMark` und `endMark`.

```
set g {flowchart LR
    subgraph kasten [Kasten]
        A[a] --> B[b]
    end
}
dict get [::tclutils::tuflow::parse $g] groups
                 ;# -> {id kasten label Kasten members {A B}}
```

`A-3` wirft `{TCLUTILS TUFLOW BADID}`. Ein Text ohne Knoten wirft `{TCLUTILS TUFLOW EMPTY}`.

Das folgende Skript erzeugt das Bild darunter.

```
set fluss {flowchart TB
    A([Eingang]) --> B{Prüfung}
    B -->|ja| C[(Lager)]
    B -->|nein| D[Rücksendung]
    subgraph halle [Halle 2]
        C --> E(Versand)
    end
    D -.-> E
}
::tclutils::tuflow::writePng $fluss 056-fluss.png  ;# -> 056-fluss.png
```



Flussdiagramm mit Gruppe, 370 × 494 Pixel

15.4 Zustandsdiagramm: tustate

tustate liest `stateDiagram` und `stateDiagram-v2`. Ein Zustand wird ein `rounded`-Knoten, ein Übergang eine Kante mit Spitze. Die Richtung ist `TB`.

Schreibweise	Ergebnis
A --> B und A --> B : Text	Kante, Text als Beschriftung
[*] --> A	Startpunkt __start als dot
A --> [*]	Endpunkt __end als dot

Schreibweise	Ergebnis
A : Text	Beschriftung des Zustands A; die Id darf Umlaute haben (Büro : Jörg)
A allein	Zustand ohne Kante
state X <<fork>>, <<join>>	Zustand X als rounded
direction LR	LR; RL wird LR, BT und TD werden TB

Nicht gelesen werden:

- `state X { … }`: der Rahmen entfällt, die Übergänge darin bleiben;
- `note …: Text` und `note … end note`: übergangen;
- `state "Lang" as L`: übergangen, `L` entsteht erst bei der ersten Kante;
- eine Kante von `A` nach `A`: im Modell, nicht gezeichnet.

```
set st {stateDiagram-v2
    [*] --> Bereit
    Bereit --> Laeuft : start
    Laeuft --> [*]
    Bereit : wartet
}
set m [::tclutils::tuflow::parse $st]
lmap n [dict get $m nodes] {dict get $n label}
                                  ;# -> __start wartet Laeuft __end
lmap n [dict get $m nodes] {dict get $n shape}
                                  ;# -> dot rounded rounded dot
```

Ein Text ohne Zustand wirft `{TCLUTILS TUSTATE EMPTY}`.

15.5 Klassendiagramm: tuclass

tuclass liest `classDiagram` und `classDiagram-v2`. Jede Klasse wird ein `box`-Knoten mit drei Teilen: Name, Attribute, Methoden. Eine Zeile `----------` trennt die Teile. Ein Mitglied mit `(` ist eine Methode, alle anderen sind Attribute. Die Richtung ist `TB`.

Schreibweise	Ergebnis
class X { … } über mehrere Zeilen	Klasse mit einem Mitglied je Zeile
class X { +a; +b() }	Block auf einer Zeile, ; trennt die Mitglieder; in Anführungszeichen nicht (0.3)
class X, class X { }	Klasse ohne Mitglieder
X : +int alter, X : +laut()	Mitglied von X
A <|-- B	Kante B nach A, Beschriftung extends
A *-- B	Kante A nach B, composition
A o-- B	aggregation

Schreibweise	Ergebnis
A ..> B	dependency, gestrichelt
A ..|> B	extends, gestrichelt
A --> B	ohne Beschriftung
A --> B : Text	Text als Beschriftung
A "1" --> "*" B	Kante A nach B; die Kardinalitäten entfallen

Steht `<` vorn im Pfeil, zeigt die Kante zur linken Klasse. Jede Kante hat eine einfache Spitze; die UML-Spitzen (Dreieck, Raute) zeichnet das Modul nicht. Ein Mitglied vor der schließenden Klammer (`+b()  }`) zählt mit.

Nicht gelesen werden:

- `class X~T~`, `class X:::k`, `<<interface>> X`: übergangen.

```
set cl {classDiagram
    class Tier {
        +String name
        +laut()
    }
    Tier <|-- Hund
}
set m [::tclutils::tuflow::parse $cl]
split [dict get [lindex [dict get $m nodes] 0] label] \n
           ;# -> Tier ---------- {+String name} ---------- +laut()
lrange [lindex [dict get $m edges] 0] 0 5
                            ;# -> from Hund to Tier label extends
set k [::tclutils::tuflow::parse \
    "classDiagram\nclass Ort { }\nKunde \"1\" --> \"*\" Ort : hat\n"]
lmap n [dict get $k nodes] {dict get $n label} ;# -> Ort Kunde
dict get [lindex [dict get $k edges] 0] label   ;# -> hat
```

Ein Text ohne Klasse wirft `{TCLUTILS TUCLASS EMPTY}`.

15.6 ER-Diagramm: tuer

tuer 0.4 liest `erDiagram`. Eine Entität wird ein `box`-Knoten mit dem Namen in der ersten Zeile und einem Attribut je weiterer Zeile. Eine Beziehung wird eine Kante ohne Spitze mit Krähenfüßen an beiden Enden. Die Richtung ist `TB`.

Schreibweise	Ergebnis
NAME { … }	Entität; Attributzeile typ name
typ name PK	Attribut typ name (PK); weitere Wörter entfallen
A ||--o{ B : Text	durchgezogene Kante, Beschriftung Text
A }|..|| B : Text	gestrichelte Kante

Schreibweise	Ergebnis
NAME allein	Entität ohne Attribute

Jede Seite der Kardinalität wird eine Marke: `{` oder `}` heißt viele, `o` heißt null. `||` wird `exactlyOne`, `|o` und `o|` `zeroOrOne`, `}|` und `|{` `oneOrMany`, `}o` und `o{` `zeroOrMany`. Anführungszeichen um die Beschriftung fallen weg: `: "gibt auf"` wird die Beschriftung `gibt auf`.

Ein Entitätsname besteht aus Buchstaben jeder Schrift, Ziffern, `_` und `-`, also geht auch `Büro {`. Drei Wörter sind nur dann eine Beziehung, wenn das mittlere `--` oder `..` enthält. Zeilen mit `direction`, `title`, `accTitle` und `accDescr` werden übergangen; jede andere Zeile, die weder Block noch Entität noch Beziehung ist, wirft `{TCLUTILS TUER SYNTAX}`. Bis 0.3 blieb `Büro {` unerkannt, und aus der Attributzeile `string name PK` wurde eine Kante von `string` nach `PK`.

```
set er "erDiagram
    KUNDE ||--o\{ AUFTRAG : gibt
    KUNDE \{
        string name PK
    \}
"
set m [::tclutils::tuflow::parse $er]
set e [lindex [dict get $m edges] 0]
dict get $e startMark                          ;# -> exactlyOne
dict get $e endMark                            ;# -> zeroOrMany
split [dict get [lindex [dict get $m nodes] 0] label] \n
                            ;# -> KUNDE {string name (PK)}
```

Ein Text ohne Entität wirft `{TCLUTILS TUER EMPTY}`.

15.7 Anforderungen: turequirement

turequirement liest `requirementDiagram`. Jede Anforderung und jedes Element wird ein `box`-Knoten. Die erste Zeile nennt die Art in `<<…>>`, die zweite den Namen, danach folgen die Felder `id`, `text`, `type`, `risk`, `verifymethod` und `docref` in dieser Reihenfolge, soweit vorhanden. Die Richtung ist `TB`.

Schreibweise	Ergebnis
requirement n { … }	Knoten <<requirement>>
functionalRequirement, performanceRequirement, interfaceRequirement, physicalRequirement, designConstraint	ebenso, mit dieser Art
element n { … }	Knoten <<element>>
feld: wert im Block	Feldzeile; andere Felder als die sechs entfallen
a - satisfies -> b	Kante a nach b, Beschriftung satisfies

Schreibweise	Ergebnis
b <- derives - a	Kante a nach b

Eine Beziehung zu einem unbekannten Namen legt einen Knoten mit diesem Namen an. Lange Feldwerte werden nicht umbrochen, der Kasten wird breit. Alle Arten zeichnen dieselbe Form.

```
set rq {requirementDiagram
    requirement schnell {
        id: 1
        risk: high
    }
    element server {
        type: simulation
    }
    server - satisfies -> schnell
}
set m [::tclutils::tuflow::parse $rq]
split [dict get [lindex [dict get $m nodes] 0] label] \n
           ;# -> <<requirement>> schnell {id: 1} {risk: high}
dict get [lindex [dict get $m edges] 0] label  ;# -> satisfies
```

Ein Text ohne Anforderung und Element wirft `{TCLUTILS TUREQUIREMENT EMPTY}`.

15.8 Mindmap: tumindmap

tumindmap liest `mindmap`. Die Einrückung bestimmt die Ebene: Eine Zeile, die tiefer steht als die vorige, ist ihr Kind; eine flachere Zeile hängt am nächsten Vorfahren mit kleinerer Einrückung. Ein Tabulator zählt vier Leerzeichen. Die Ids sind erzeugt (`n1`, `n2`, …), die Kanten haben keine Spitze. Das Ergebnis ist ein Baum von oben nach unten (TB), keine Sternform.

Schreibweise	Form
Text	rounded
id((Text)), ((Text))	circle
id{{Text}}	hexagon
id[Text]	box
id(Text)	rounded
)Text(und))Text((	rounded

Eine Klammer zählt nur, wenn sie den ganzen Text umschließt; `API (REST)` bleibt Text. Zeilen mit `::icon(…)` oder `class` entfallen. Eine zweite Zeile auf der Ebene der Wurzel wird eine zweite Wurzel.

```
set mm {mindmap
    root((Lager))
        Eingang
        [Ausgang]
```

```
}
set m [::tclutils::tuflow::parse $mm]
lmap n [dict get $m nodes] {dict get $n shape}
                                  ;# -> circle rounded box
lmap e [dict get $m edges] {dict get $e to}  ;# -> n2 n3
```

Ein Text ohne Zeile nach dem Kopf wirft `{TCLUTILS TUMINDMAP EMPTY}`.

15.9 C4: tuc4

tuc4 liest `C4Context`, `C4Container`, `C4Component`, `C4Dynamic` und `C4Deployment`. Die Richtung ist `TB`. Ein Element `Typ(id, "Name", …)` wird ein Knoten mit der Id und dem ersten Text in Anführungszeichen als Beschriftung; Technik und Beschreibung entfallen. Die Id besteht aus Buchstaben, Ziffern und `_`, Umlaute eingeschlossen: `Person(größe, "Weg")` hat die Id `größe` (bis tclutils 0.68.0 `gr`). Die Füllfarbe kommt aus dem Typ.

Typ	Form	Farbe
Person, Person_Ext	rounded	violett
System, Container, Component	box	blau, grün, orange
Typ mit Db, etwa SystemDb, ContainerDb	cylinder	nach Typ
Typ mit _Ext	wie oben	grau
Rel(a, b, "Text"), BiRel, Rel_U, Rel_D, Rel_L, Rel_R	Kante a nach b mit Spitze	
title Text	Titel, mittig über dem Graphen	

Nicht gelesen werden:

- Grenzen wie `System_Boundary(…) { … }` und `Deployment_Node`: der Rahmen entfällt, die Elemente darin bleiben;
- `BiRel` bekommt nur eine Spitze; die Richtung in `Rel_U` bis `Rel_R` entfällt;
- `UpdateElementStyle`, `UpdateRelStyle` und alle Zeilen mit `Update` am Anfang: übergangen.

Eine Beziehung zu einer unbekannten Id legt einen `box`-Knoten ohne Farbe an.

```
set c4 {C4Context
    title Lager
    Person(kunde, "Kunde")
    System(lag, "Lager")
    Rel(kunde, lag, "holt")
}
set m [::tclutils::tuflow::parse $c4]
dict get [dict get $m meta] title            ;# -> Lager
dict get [lindex [dict get $m nodes] 0] style
                                  ;# -> fill #e8def8 stroke #6a1b9a
```

Ein C4 ohne Elemente wirft `{TCLUTILS TUC4 EMPTY}`.

15.10 Blöcke: tublock

tublock liest `block-beta`. Jedes Wort einer Zeile ist ein Block; Klammern und Anführungszeichen halten Leerzeichen zusammen. Die Formen entsprechen dem Flussdiagramm: `a["Text"] box`, `a("Text") rounded`, `a(("Text")) circle`, `a{"Text"} diamond`, `a{{"Text"}} hexagon`, `a(["Text"]) stadium`, `a[("Text")] cylinder`, `a[["Text"]] box`. Die Blöcke werden als Graph von oben nach unten (TB) gelegt, nicht im Raster. Ids dürfen Umlaute haben: `Büro[Jörg]` ist der Block `Büro` mit der Beschriftung `Jörg` (bis tclutils 0.68.0 ein einziges Token).

Schreibweise	Ergebnis
a --> b, a ==> b, a -.-> b	Kante mit Spitze
a --- b	Kante ohne Spitze
a -->|Text| b, a -- Text --> b	Kante mit Beschriftung
a["Text"]:2	die Spanne :2 entfällt
columns N, space, space:N	übergangen
block:id … end	der Rahmen entfällt, die Blöcke darin bleiben
style, classDef, class, click	übergangen

Pfeilblöcke wie `a<["Text"]>(right)` werden ein Knoten mit genau dieser Zeichenfolge als Id.

```
set bl {block-beta
    columns 3
    a["Eingang"] b(("Prüfung")) c["Lager"]
    a --> b
    b -->|ok| c
}
set m [::tclutils::tuflow::parse $bl]
lmap n [dict get $m nodes] {dict get $n shape}
                                  ;# -> box circle box
dict get [lindex [dict get $m edges] 1] label  ;# -> ok
```

Ein Text ohne Block wirft `{TCLUTILS TUBLOCK EMPTY}`.

15.11 Git-Verlauf: tugit

tugit liest `gitGraph`. Jeder Commit wird ein Knoten, jede Elternbeziehung eine Kante vom Eltern- zum Kind-Commit. Der erste Zweig heißt `main`. Die Richtung ist LR, mit `gitGraph TB:` TB. Jeder Zweig bekommt eine Füllfarbe; Spuren wie bei Mermaid zeichnet das Modul nicht.

Schreibweise	Ergebnis
commit	Knoten c1, c2, … als rounded

Schreibweise	Ergebnis
commit id: "X"	Knoten X
commit tag: "T"	Beschriftung mit [T]
branch name	neuer Zweig ab dem aktuellen Stand, wird aktuell
checkout name	Zweig wechseln
merge name	box "merge name" mit zwei Eltern
cherry-pick id: "X"	rounded "pick X"

`commit type: …` wird übergangen. Eine Commit-Id darf nur einmal vorkommen, wie in Mermaid: eine zweite `commit id: "X"` oder ein `merge … id: "X"` mit vergebener Id wirft `{TCLUTILS TUGIT DUPID}` (bis tclutils 0.68.0 fiel der zweite Commit weg, und eine Kante lief von X auf sich selbst). Erzeugte Ids (`c1`, `c2`, …) überspringen Ids, die der Text schon nutzt.

```
set gg {gitGraph
    commit
    branch dev
    commit
    checkout main
    merge dev
}
set m [::tclutils::tuflow::parse $gg]
lmap n [dict get $m nodes] {dict get $n label}
                                ;# -> c1 c2 {merge dev}
llength [dict get $m edges]                ;# -> 3
```

Ein Text ohne Commit wirft `{TCLUTILS TUGIT EMPTY}`, eine doppelte Commit-Id `{TCLUTILS TUGIT DUPID}`.

15.12 Architektur: tuarchitecture

tuarchitecture liest `architecture-beta` und `architecture`. Dienste werden Knoten, Gruppen Rahmen. Die Richtung ist LR.

Schreibweise	Ergebnis
service id(icon)[Text]	Knoten, Form nach dem Symbol; die Id darf Umlaute haben
service id(icon)[Text] in g	Knoten in der Gruppe g
group id(icon)[Text]	Rahmen um ihre Mitglieder
group id(icon)[Text] in g	verschachtelte Gruppe
junction id	Knoten dot
a:R -- L:b	Kante ohne Spitze
a:R --> L:b, <--, <-->	Spitze am Ziel, am Anfang, an beiden

Schreibweise	Ergebnis
a{group}:R -- L:b	{group} entfällt, die Kante bleibt

Das Symbol bestimmt die Form: `database`, `db`, `disk`, `storage` ergeben `cylinder`, `cloud` und `internet` `rounded`, alle anderen `box`. Die Seiten `L`, `R`, `T`, `B` werden gelesen, aber nicht benutzt; die Lage der Anschlüsse bestimmt `tudiagram`. Ein Ende, das nicht als Dienst steht, wird ein `box`. Die Beschriftung ist der Text in `[…]`, ohne ihn die Id.

```
set ar {architecture-beta
    group api(cloud)[API]
    service db(database)[Datenbank] in api
    service web(server)[Web] in api
    web:R --> L:db
}
set m [::tclutils::tuflow::parse $ar]
lmap n [dict get $m nodes] {dict get $n shape}  ;# -> cylinder box
dict get $m groups      ;# -> {id api label API members {db web}}
```

Ein Text ohne Dienst wirft `{TCLUTILS TUARCHITECTURE EMPTY}`.

15.13 Sequenzdiagramm: tusequence

tusequence liest `sequenceDiagram` und zeichnet selbst, ohne `tudiagram`. Die Fläche ergibt sich aus dem Inhalt (`-width 0 -height 0`); eine Zahl über 0 setzt sie fest.

Schreibweise	Ergebnis
participant A, participant A as Text	Teilnehmer, Reihenfolge der Nennung
actor A as Text	Teilnehmer, als Kasten gezeichnet
A->>B: Text	durchgezogen, gefüllte Spitze
A-->>B: Text	gepunktet, gefüllte Spitze
A->B, A-->B	offene Spitze
A-)B, A--)B	offene Spitze
A-xB, A--xB	Kreuz
A<<->>B, A<<-->>B	gefüllte Spitzen an beiden Enden, durchgezogen oder gepunktet
A->>+B: Text, A->>-B: Text	aktiviert B, deaktiviert A
activate A, deactivate A	Aktivierungsbalken
Note left of A: Text, right of, over A,B	Notiz
loop, opt, alt, par, critical, break, rect … end	Rahmen über die ganze Breite
else, and, option	Trennlinie im Rahmen

Schreibweise	Ergebnis
autonumber	Nachrichten mit 1., 2., …
title Text	Titel, mittig über den Teilnehmern

Ein Teilnehmer entsteht auch aus der ersten Nachricht oder Notiz, die ihn nennt. Eine Nachricht braucht den Doppelpunkt; `A->>B` ohne `:` wird übergangen.

Nicht gelesen werden:

- `create`, `destroy`, `box`, `link`: übergangen;
- Strichmännchen für `actor`.

Ein Titel schiebt die Zeichnung um ein Band von 24 Pixeln nach unten; ist er breiter als die Teilnehmer, wird die Fläche breiter.

`parse` liefert ein Dict mit `type sequence`, `participants` (Ids in Reihenfolge), `labels` (Id auf Beschriftung), `isActor`, `steps`, `autonumber` und `title`. Jeder Schritt ist ein Dict mit `kind message`, `note`, `activate`, `deactivate`, `blockStart`, `blockElse` oder `blockEnd`.

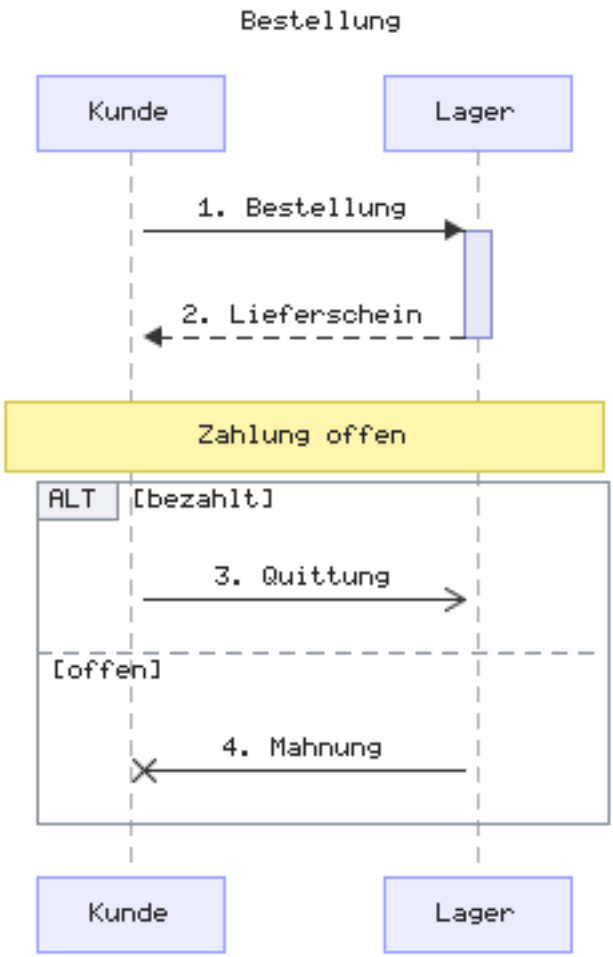
```
package require tclutils::tusequence
set sq "sequenceDiagram\nK->>+L: bestellt\nL-->>-K: liefert\n"
set m [::tclutils::tusequence::parse $sq]
dict get $m participants                    ;# -> K L
lindex [dict get $m steps] 0
        ;# -> kind message from K to L op ->> act + text bestellt
set bi "sequenceDiagram\nK<<->>L: Abgleich\n"
set b [::tclutils::tusequence::parse $bi]
dict get $b participants                    ;# -> K L
```

-errorcode	Ursache
{TCLUTILS TUSEQUENCE EMPTY}	kein Teilnehmer
{TCLUTILS TUSEQUENCE ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUSEQUENCE FONT}	-fontfile nennt keine Datei

Das folgende Skript erzeugt das Bild darunter.

```
set seq {sequenceDiagram
    title Bestellung
    autonumber
    actor K as Kunde
    participant L as Lager
    K->>+L: Bestellung
    L-->>-K: Lieferschein
    Note over K,L: Zahlung offen
    alt bezahlt
        K-)L: Quittung
    else offen
        L-xK: Mahnung
```

```
    end
}
::tclutils::tuflow::writePng $seq 056-sequenz.png
                                                ;# -> 056-sequenz.png
```



Sequenzdiagramm mit Titel, 242 × 374 Pixel

15.14 Gantt: tugantt

tugantt liest `gantt`. Jede Aufgabe ist eine Zeile mit Balken; links steht der Abschnitt, rechts vom Balken der Name. Vorgabe der Breite ist 800. Ohne `-height` ist die Fläche 400 Pixel hoch und wächst mit, wenn die Aufgaben mehr Platz brauchen: Jede Zeile nimmt 26 Pixel, bei 400 Pixeln haben mit Titel 13 Zeilen Platz. Mit `-height` bleibt die Höhe fest.

Schreibweise	Ergebnis
title Text	Titel, mittig oben
dateFormat F	Eingabeformat, Vorgabe YYYY-MM-DD
axisFormat F	Format der Achse, Vorgabe wie dateFormat
section Name	Abschnitt, Hintergrund im Wechsel
Name : done, a1, 2026-03-02, 5d	Aufgabe mit Marke, Id, Start, Dauer

Schreibweise	Ergebnis
Name : 2026-03-20, 2026-03-30	Start und Ende als Datum
Name : after a1, 3d	Start am spätesten Ende der genannten Ids
Name : 3d	Start am Ende der vorigen Aufgabe
Marken active, done, crit	hellblau, grau, rot
Marke milestone	gelbe Raute in der Mitte der Spanne

`dateFormat` und `axisFormat` übersetzen `YYYY`, `YY`, `MM`, `DD`, `HH`, `mm` und `ss` in `clock` -Formate; `%`-Formate wie `%d.%m.` bleiben stehen. `X` heißt Sekunden seit 1970. Dauern sind Zahl und Einheit: `ms`, `s`, `m` oder `min`, `h`, `d`, `w`; eine andere Einheit wirft beim Zeichnen `{TCLUTILS TUGANTT DURATION}` (seit 0.3; bis 0.2 zählte sie 0). Eine Id darf Buchstaben jeder Schrift enthalten (`büro`). Die erste Aufgabe braucht einen Start: Ohne Datum und ohne `after` auf eine frühere Aufgabe wirft das Zeichnen `{TCLUTILS TUGANTT DATE}`, wie Mermaid es ohne Startdatum nicht zeichnet. Die Achse hat bis zu sechs Marken, gerundet auf ganze Einheiten des Achsenformats: Sekunden, wenn es Sekunden zeigt, Minuten bei Minuten, Stunden bei Stunden, sonst Tage. Eine Marke mit derselben Beschriftung wie eine vorige entfällt, so steht jedes Datum nur einmal da.

Nicht gelesen werden: `excludes`, `todayMarker`, `tickInterval`, `weekday`, `vert` (übergangen), `until`, Wochenenden ausnehmen, kompakte Darstellung.

`parse` prüft die Daten noch nicht. Es liefert `type gantt`, `title`, `dateFormat`, `axisFormat`, `haveSection` und `tasks`; jede Aufgabe ist ein Dict mit `title`, `section`, `tags`, `id`, `startTok` und `endTok`. Die Zeiten rechnet erst das Zeichnen aus.

```
package require tclutils::tugantt
set gt "gantt\nsection Bau\nRohbau :crit, b1, 2026-03-02, 2w\n"
set m [::tclutils::tugantt::parse $gt]
set a [lindex [dict get $m tasks] 0]
dict keys $a       ;# -> title section tags id startTok endTok
dict get $a startTok                          ;# -> 2026-03-02
dict get $a endTok                            ;# -> 2w
```

-errorcode	Ursache
{TCLUTILS TUGANTT EMPTY}	keine Zeile mit : nach dem Kopf
{TCLUTILS TUGANTT DURATION}	Dauer mit unbekannter Einheit, etwa 5x; erst beim Zeichnen
{TCLUTILS TUGANTT DATE}	Datum passt nicht zu dateFormat oder gibt es nicht, oder die erste Aufgabe hat keinen Start; erst beim Zeichnen
{TCLUTILS TUGANTT ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUGANTT FONT}	-fontfile nennt keine Datei

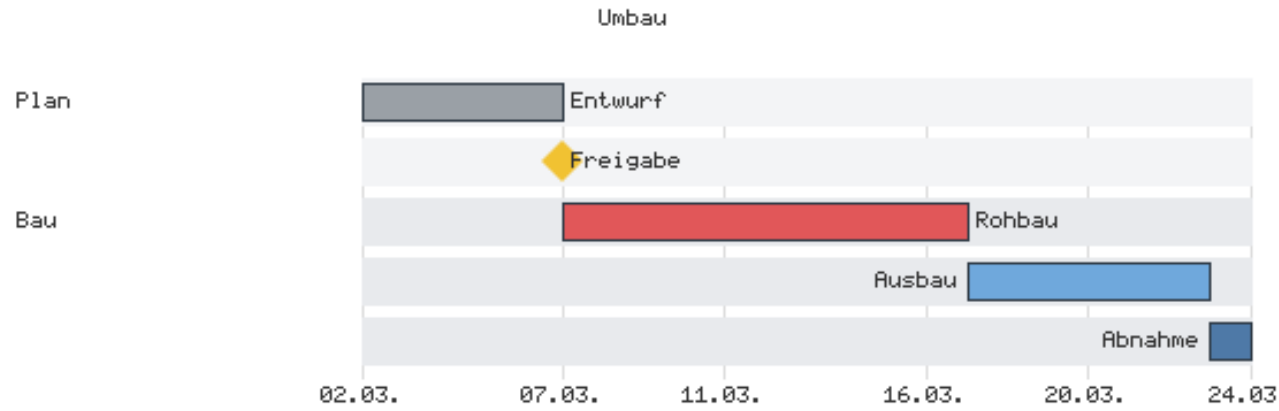
Ein Datum muss es geben: `tugantt` schreibt das gelesene Datum im selben Format zurück und vergleicht es mit der Eingabe, führende Nullen ausgenommen. `2026-13-45` und `2026-02-30`

werfen so unter Tcl 8.6 und 9.0 gleich `DATE, 2026-3-2` gilt.

```
set g1 [::tclutils::tugantt::parse "gantt\nA : 3d\n"]
catch {::tclutils::tugantt::toSvg $g1} m o
dict get $o -errorcode                  ;# -> TCLUTILS TUGANTT DATE
set g2 [::tclutils::tugantt::parse "gantt\nA : 2026-13-45, 1d\n"]
catch {::tclutils::tugantt::toSvg $g2} m o
dict get $o -errorcode                  ;# -> TCLUTILS TUGANTT DATE
```

Das folgende Skript erzeugt das Bild darunter.

```
set gantt {gantt
    title Umbau
    dateFormat YYYY-MM-DD
    axisFormat %d.%m.
    section Plan
    Entwurf  :done, e1, 2026-03-02, 5d
    Freigabe :milestone, m1, after e1, 0d
    section Bau
    Rohbau   :crit, b1, after e1, 10d
    Ausbau   :active, after b1, 6d
    Abnahme  : 1d
}
::tclutils::tuflow::writePng $gantt 056-gantt.png \
    -width 560 -height 200                  ;# -> 056-gantt.png
```



Gantt-Diagramm, 560 × 200 Pixel

15.15 Kreisdiagramm: tupie

tupie liest `pie`. Eine Zeile `"Beschriftung": Zahl` wird ein Sektor; die Anführungszeichen dürfen fehlen, die Beschriftung reicht bis zum letzten Doppelpunkt. Im Sektor steht der Anteil in Prozent, rechts die Legende mit Beschriftung und Anteil.

Schreibweise	Ergebnis
pie, pie title Text	Kopf, Titel optional
pie showData	Legende zeigt auch den Wert

Schreibweise	Ergebnis
title Text	Titel auf eigener Zeile
"Nord" : 42, Sued : 2.5	Sektor

Ein Wert ist eine Dezimalzahl, auch mit Exponent: `42`, `2.5`, `1e3`. Führende Nullen zählen dezimal, `010` ist 10. Ein negativer Wert wirft `{TCLUTILS TUPIE VALUE}` mit der Zeilennummer. Eine Zeile, deren Wert keine Zahl ist, ist kein Sektor. Ein Sektor mit 0 bleibt im Modell. Die Farben wiederholen sich nach zehn Sektoren.

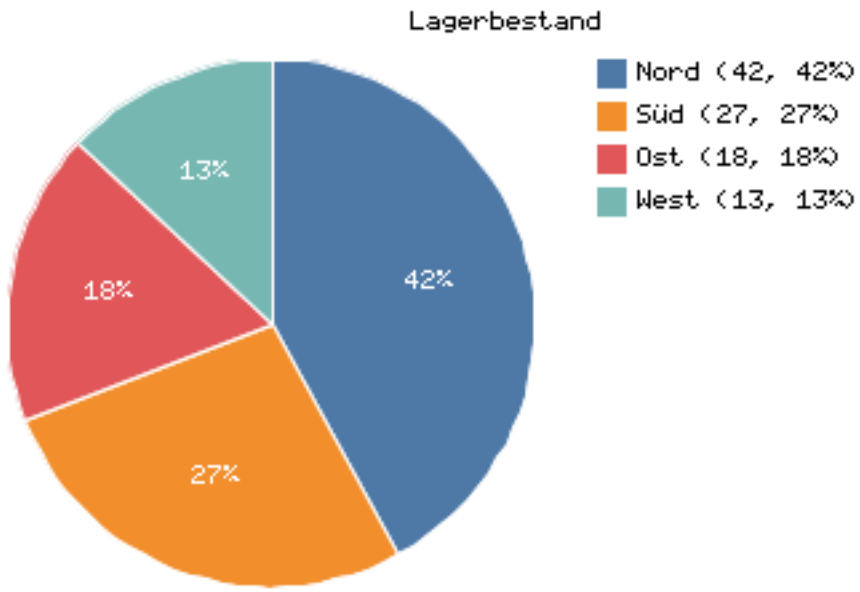
Option	Vorgabe	Wirkung
-width	480	Breite in Pixeln
-height	320	Höhe in Pixeln
-legend	1	0 lässt die Legende weg
-scale	1	Faktor, ganze Zahl ab 1
-fontfile	leer	TTF oder OTF, nur PNG

```
package require tclutils::tupie
set pt "pie title Lager\n\"Nord\" : 1\n\"Sued\" : 3\n"
::tclutils::tupie::parse $pt
             ;# -> title Lager showData 0 slices {{Nord 1} {Sued 3}}
dict get [::tclutils::tupie::parse "pie\nA : 1e3\n"] slices
                                                ;# -> {A 1e3}
catch {::tclutils::tupie::parse "pie\nA : 3\nB : -1\n"} m
set m              ;# -> line 3: negative pie slice value "-1"
```

-errorcode	Ursache
{TCLUTILS TUPIE EMPTY}	kein Sektor
{TCLUTILS TUPIE VALUE}	negativer Wert, oder Summe nicht positiv
{TCLUTILS TUPIE ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUPIE FONT}	-fontfile nennt keine Datei

Das folgende Skript erzeugt das Bild darunter.

```
set pie {pie showData title Lagerbestand
    "Nord" : 42
    "Süd" : 27
    "Ost" : 18
    "West" : 13
}
::tclutils::tuflow::writePng $pie 056-pie.png \
    -width 420 -height 240                  ;# -> 056-pie.png
```



Kreisdiagramm, 420 × 240 Pixel

15.16 XY-Diagramm: tuxychart

tuxychart liest `xychart-beta` mit Balken- und Linienreihen. Vorgabe der Fläche ist 520 × 340.

Schreibweise	Ergebnis
xychart-beta	Kopf; ein angehängtes horizontal steht im Modell, gezeichnet wird senkrecht
title "Text"	Titel
x-axis "Titel" [a, b, c]	Achse mit Kategorien, Titel optional
x-axis "Titel" 1 --> 5	Zahlenachse von 1 bis 5
y-axis "Titel" 0 --> 100	Titel und Bereich, beides optional
bar [1, 2, 3]	Balkenreihe, mehrere erlaubt
line [1, 2, 3]	Linienreihe, mehrere erlaubt

Ohne Bereich wählt `y-axis` die Skala selbst. Der Titel der y-Achse steht waagerecht über der Achse. Mehrere Balkenreihen stehen je Kategorie nebeneinander. `xychart` ohne `-beta` erkennt die Fassade nicht.

```
package require tclutils::tuxychart
set xy {xychart-beta
    x-axis [Jan, Feb, Mar]
    y-axis "EUR" 0 --> 100
    bar [40, 65, 50]
}
set m [::tclutils::tuxychart::parse $xy]
dict get $m categories                      ;# -> Jan Feb Mar
list [dict get $m ymin] [dict get $m ymax]   ;# -> 0 100
dict get $m series             ;# -> {type bar values {40 65 50}}
```

-errorcode	Ursache
{TCLUTILS TUXYCHART EMPTY}	keine Reihe
{TCLUTILS TUXYCHART VALUE}	Wert in einer Reihe keine Zahl
{TCLUTILS TUXYCHART COUNT}	mit Kategorien: eine Reihe hat nicht einen Wert je Kategorie (seit 0.2)
{TCLUTILS TUXYCHART ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUXYCHART FONT}	-fontfile nennt keine Datei

15.17 Quadrant: tuquadrant

tuquadrant liest `quadrantChart`. Beide Achsen laufen von 0 bis 1. Vorgabe der Fläche ist 420 × 420.

Schreibweise	Ergebnis
title Text	Titel
x-axis links --> rechts	Beschriftung der Enden; ohne --> nur links
y-axis unten --> oben	ebenso
quadrant-1 Text bis quadrant-4 Text	1 oben rechts, 2 oben links, 3 unten links, 4 unten rechts
Name: [0.3, 0.6]	Punkt; Werte außerhalb 0..1 werden auf den Rand gesetzt

Angaben hinter dem Punkt wie `radius: 9` oder `color:` entfallen; die Punkte nehmen Farben aus einer festen Reihe. Die Beschriftung der y-Achse steht waagerecht links. Der Kopf ist nicht nötig: Jeder Text mit mindestens einer erkannten Zeile ist gültig.

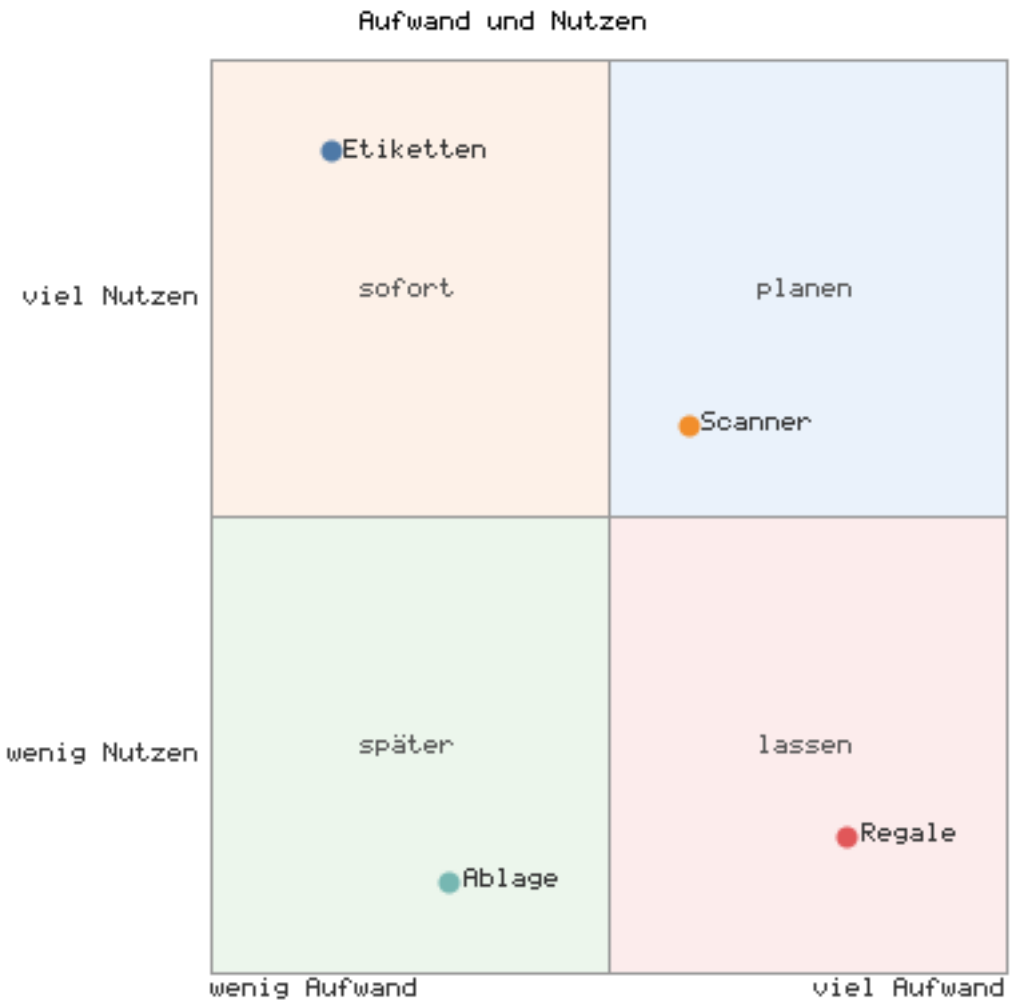
```
package require tclutils::tuquadrant
set q {quadrantChart
    x-axis wenig --> viel
    A: [0.3, 0.6]
    B: [1.4, -0.2]
}
```

```
set m [::tclutils::tuquadrant::parse $q]
list [dict get $m xleft] [dict get $m xright] ;# -> wenig viel
dict get $m points                ;# -> {A 0.3 0.6} {B 1.0 0.0}
```

-errorcode	**Ursache**
{TCLUTILS TUQUADRANT EMPTY}	keine erkannte Zeile
{TCLUTILS TUQUADRANT ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUQUADRANT FONT}	-fontfile nennt keine Datei

Das folgende Skript erzeugt das Bild darunter.

```
set quad {quadrantChart
    title Aufwand und Nutzen
    x-axis wenig Aufwand --> viel Aufwand
    y-axis wenig Nutzen --> viel Nutzen
    quadrant-1 planen
    quadrant-2 sofort
    quadrant-3 später
    quadrant-4 lassen
    Etiketten: [0.15, 0.9]
    Scanner: [0.6, 0.6]
    Regale: [0.8, 0.15]
    Ablage: [0.3, 0.1]
}
::tclutils::tuflow::writePng $quad 056-quadrant.png \
    -width 400 -height 400                  ;# -> 056-quadrant.png
```



Quadrant, 400 × 400 Pixel

15.18 Journey: tujourney

tujourney liest `journey`. Die Aufgaben stehen von links nach rechts in der Reihenfolge des Texts, eine Linie verbindet ihre Bewertungen. Oben stehen die Abschnitte als Bänder, unter jeder Aufgabe die Beteiligten als farbige Punkte, dazu eine Legende. Vorgabe der Fläche ist 600 × 360.

Schreibweise	Ergebnis
title Text	Titel
section Name	Abschnitt für die folgenden Aufgaben
Aufgabe: 5: Ich, Du	Aufgabe, Bewertung, Beteiligte
Aufgabe: 2	Aufgabe ohne Beteiligte

Die Bewertung ist eine Zahl ab 0 aus Ziffern mit optionalem Nachkommateil, `2.5` gilt. Mermaid bewertet von 1 bis 5; Werte über 5 gelten, die Achse reicht dann bis zur nächsten ganzen Zahl über dem größten Wert. Jede andere Bewertung (`gut`, `-1`) wirft `{TCLUTILS TUJOURNEY VALUE}` mit der Zeilennummer. Zeilen mit `accTitle` und `accDescr` entfallen. Lange Namen

stehen waagerecht und können sich überdecken.

```
package require tclutils::tujourney
set jo {journey
    title Tag
    section Morgen
        Kaffee: 5: Ich, Du
        Bus: 2
}
set m [::tclutils::tujourney::parse $jo]
dict get $m tasks
              ;# -> {Kaffee 5 {Ich Du} Morgen} {Bus 2 {} Morgen}
dict get $m actors                          ;# -> Ich Du
set j [::tclutils::tujourney::parse "journey\nTee: 2.5\n"]
lindex [dict get $j tasks] 0 1              ;# -> 2.5
```

-errorcode	**Ursache**
{TCLUTILS TUJOURNEY EMPTY}	keine Aufgabe
{TCLUTILS TUJOURNEY VALUE}	Bewertung keine Zahl ab 0
{TCLUTILS TUJOURNEY ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUJOURNEY FONT}	-fontfile nennt keine Datei

15.19 Zeitleiste: tutimeline

tutimeline liest `timeline`. Die Zeitachse läuft waagerecht, die Zeitpunkte stehen darunter, ihre Ereignisse als Kästen darüber. Abschnitte sind farbige Bänder oben. Vorgabe der Fläche ist 640 × 380.

Schreibweise	**Ergebnis**
title Text	Titel
section Name	Abschnitt für die folgenden Zeitpunkte
2022 : Muster : Alpha	Zeitpunkt mit zwei Ereignissen
: Beta	weiteres Ereignis zum vorigen Zeitpunkt

Getrennt wird an einem Doppelpunkt, dem ein Leerraum oder das Zeilenende folgt. `12:30: Mittag` ist der Zeitpunkt `12:30` mit dem Ereignis `Mittag`; `2020:Start` ohne Leerzeichen bleibt ein Zeitpunkt ohne Ereignis. Lange Texte werden nicht umbrochen.

```
package require tclutils::tutimeline
set tl "timeline\n2021 : Idee\n2022 : Muster : Alpha\n     : Beta\n"
dict get [::tclutils::tutimeline::parse $tl] periods
               ;# -> {2021 Idee {}} {2022 {Muster Alpha Beta} {}}
set uhr "timeline\n12:30 : Mittag\n"
dict get [::tclutils::tutimeline::parse $uhr] periods
                                       ;# -> {12:30 Mittag {}}
```

-errorcode	Ursache
{TCLUTILS TUTIMELINE EMPTY}	kein Zeitpunkt
{TCLUTILS TUTIMELINE ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUTIMELINE FONT}	-fontfile nennt keine Datei

15.20 Sankey: tusankey

tusankey liest `sankey-beta` und `sankey`. Nach dem Kopf folgen CSV-Zeilen `Quelle,Ziel,Wert`. Ein Name mit Komma steht in Anführungszeichen, ein Anführungszeichen darin wird verdoppelt (""). Vorgabe der Fläche ist 700 × 400.

Die Knoten stehen in Spalten nach dem längsten Weg von einer Quelle. Die Höhe eines Knotens folgt dem größeren Wert aus Zu- und Abfluss, die Breite eines Bands dem Wert. Ein Band hat die Farbe seiner Quelle.

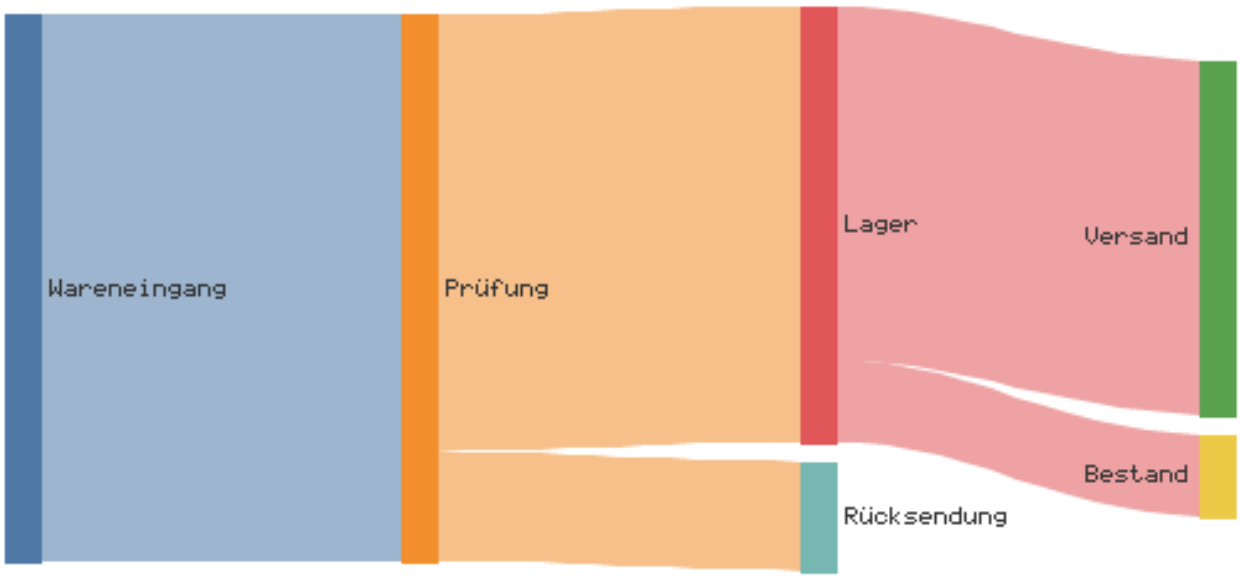
Eine Zeile mit weniger als drei Feldern, leerem Namen oder einem Wert, der keine Zahl oder kleiner 0 ist, wirft `{TCLUTILS TUSANKEY VALUE}` (seit 0.2; bis 0.1 entfiel sie still). Eine Zeile mit dem Wert 0 entfällt: Sie zeichnet nichts, und ein Knoten, der nur in solchen Zeilen vorkommt, erscheint nicht. Felder nach dem dritten werden übergangen. Ein Kreis im Fluss bricht nicht ab, die Spalten sind dann nur ungefähr.

```
package require tclutils::tusankey
set sk "sankey-beta\nQuelle,Werk,10\n\"A,B\",Werk,2\nWerk,X,0\n"
set m [::tclutils::tusankey::parse $sk]
dict get $m nodes                              ;# -> Quelle Werk A,B
dict get $m links          ;# -> {Quelle Werk 10} {A,B Werk 2}
```

-errorcode	Ursache
{TCLUTILS TUSANKEY EMPTY}	keine gültige Zeile
{TCLUTILS TUSANKEY VALUE}	Zeile nicht Quelle,Ziel,Wert mit Zahl ab 0
{TCLUTILS TUSANKEY ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUSANKEY FONT}	-fontfile nennt keine Datei

Das folgende Skript erzeugt das Bild darunter.

```
set sankey {sankey-beta
    Wareneingang,Prüfung,100
    Prüfung,Lager,80
    Prüfung,Rücksendung,20
    Lager,Versand,65
    Lager,Bestand,15
}
::tclutils::tuflow::writePng $sankey 056-sankey.png \
    -width 520 -height 260                    ;# -> 056-sankey.png
```



Sankey-Diagramm, 520 × 260 Pixel

15.21 Radar: turadar

turadar liest `radar-beta` und `radar`. Jede Achse geht vom Mittelpunkt aus, die erste nach oben. Jede Reihe wird ein geschlossenes Vieleck mit halbdurchsichtiger Fläche. Vorgabe der Fläche ist 600 × 600.

Schreibweise	Ergebnis
radar-beta title Text, title Text	Titel
axis a, b, c	Achsen; auch mehrere Zeilen
axis m["Mathe"]	Achse mit Beschriftung
curve id["Name"]{1, 2, 3}	Reihe, Werte in Achsenfolge
curve id{ b: 3, a: 1 }	Reihe, Werte nach Achsen-Id
max N, min N	Skala; Vorgabe min 0, max größter Wert
ticks N	Zahl der Ringe, Vorgabe 5
graticule circle, polygon	Form der Ringe, Vorgabe circle

Schreibweise	Ergebnis
showLegend true, false	Legende

Jede Reihe braucht genau einen Wert je Achse. Stehen die Werte in Achsenfolge und sind es mehr oder weniger als Achsen, wirft `parse COUNT`; fehlt in der Schlüsselform eine Achse oder nennt ein Schlüssel keine Achse, wirft es `AXIS`. Bis tclutils 0.68.0 zählte ein fehlender Wert 0, und überzählige oder vertippte fielen weg. Ohne `["Name"]` nimmt die Reihe ihre Id als Namen. Die Legende erscheint, wenn mindestens eine Reihe einen Namen in `["…"]` hat; `showLegend` legt sie ausdrücklich fest. Unbekannte Zeilen entfallen. Die erste Zeile muss der Kopf sein, sonst `EMPTY`. Die Linien sind gerade, nicht geglättet; `theme` und `config` entfallen.

```
package require tclutils::turadar
set rd {radar-beta
    axis t["Tcl"], c["C"], p
    curve a["Anna"]{4, 3, 2}
    max 5
}
set m [::tclutils::turadar::parse $rd]
dict get $m axes                            ;# -> Tcl C p
dict get $m curves                          ;# -> {Anna {4.0 3.0 2.0}}
dict get $m showLegend                      ;# -> 1
set r2 "radar-beta\naxis a, b, c\ncurve x{1, 2, 3}\n"
dict get [::tclutils::turadar::parse $r2] showLegend  ;# -> 0
```

-errorcode	Ursache
{TCLUTILS TURADAR EMPTY}	Kopf fehlt oder keine Reihe
{TCLUTILS TURADAR AXES}	weniger als drei Achsen
{TCLUTILS TURADAR AXIS}	Schlüsselform ohne Wert für eine Achse, oder Schlüssel ohne Achse
{TCLUTILS TURADAR COUNT}	Zahl der Werte in Achsenfolge passt nicht zu den Achsen
{TCLUTILS TURADAR VALUE}	Wert, min oder max keine Zahl
{TCLUTILS TURADAR ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TURADAR FONT}	-fontfile nennt keine Datei

15.22 Treemap: tutreemap

tutreemap liest `treemap-beta` und `treemap`. Die Einrückung bestimmt die Ebene, ein Tabulator zählt vier Leerzeichen. Jedes Blatt bekommt eine Fläche nach seinem Wert, ein Zweig die Summe seiner Blätter. Die Rechtecke sind nach dem Squarify-Verfahren möglichst quadratisch. Ein Zweig unter der Wurzel bekommt eine Kopfleiste mit dem Namen, ein Blatt Name und Wert, wenn Platz ist. Vorgabe der Fläche ist 640 × 400.

Schreibweise	Ergebnis
"Name"	Zweig
"Name": 30	Blatt mit Wert
Name: 30, Name	ebenso ohne Anführungszeichen
"Name": 5:::klasse, classDef, style	Stil entfällt, auch eine Klasse mit Umlauten (0.3)

Ein Knoten mit Wert und Kindern ist ein Zweig; der Wert entfällt. Texte werden abgeschnitten, nicht umbrochen. Ein Blatt mit einem Wert von 0 oder weniger fällt aus dem Modell, ebenso ein Zweig, der danach kein Blatt mehr hat. Summen und Flächen kommen nur aus den übrigen Blättern: Bei `-5`, `10` und `5` teilen sich `10` und `5` die Fläche im Verhältnis 2 zu 1.

`parse` liefert einen Baum aus Dicts mit `name`, `value` und `children`. Die Wurzel hat den Namen `""`.

```
package require tclutils::tutreemap
set tm {treemap-beta
"Lager"
    "Nord": 30
    "Sued": 15
"Rest": 4
}
set m [::tclutils::tutreemap::parse $tm]
dict get $m value                              ;# -> 49.0
lmap k [dict get $m children] {dict get $k name}  ;# -> Lager Rest
set n [::tclutils::tutreemap::parse "treemap\nN: -5\nA: 10\nB: 5\n"]
dict get $n value                              ;# -> 15.0
```

-errorcode	Ursache
{TCLUTILS TUTREEMAP EMPTY}	erste Zeile kein Kopf, oder kein Knoten
{TCLUTILS TUTREEMAP VALUE}	Zeile nicht lesbar oder Wert keine Zahl
{TCLUTILS TUTREEMAP ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUTREEMAP FONT}	-fontfile nennt keine Datei

15.23 Bitfelder: tupacket

tupacket liest `packet-beta` und `packet`. Jede Zeile nach dem Kopf ist ein Feld aus Bits. Eine Reihe hat 32 Bits; ein Feld über die Reihengrenze wird geteilt und behält seine Farbe. Über jedem Teil stehen die Bitnummern. Bits zwischen den Feldern, die kein Feld abdeckt, erscheinen hellgrau. Die Breite ist 640, die Höhe ergibt sich aus der Zahl der Reihen (`-height 0`).

Schreibweise	Ergebnis
packet-beta title Text, title Text	Titel

Schreibweise	Ergebnis
0-15: "Quellport"	Feld Bit 0 bis 15
64: "F"	Feld aus einem Bit
16-31: Zielport, 'Text'	Beschriftung ohne oder mit einfachen Anführungszeichen

Die Felder werden nach dem ersten Bit sortiert. Die relative Schreibweise `+16: "Text"` aus neueren Mermaid-Fassungen kennt das Modul nicht, sie wirft `VALUE`. Lange Beschriftungen werden abgeschnitten.

```
package require tclutils::tupacket
set pk "packet-beta\n0-15: \"Quellport\"\n16-31: Zielport\n32: F\n"
set m [::tclutils::tupacket::parse $pk]
dict keys $m                        ;# -> title bitsPerRow fields
lrange [dict get $m fields] 1 2     ;# -> {16 31 Zielport} {32 32 F}
```

-errorcode	Ursache
{TCLUTILS TUPACKET EMPTY}	erste Zeile kein Kopf, oder kein Feld
{TCLUTILS TUPACKET VALUE}	Zeile ist kein Feld
{TCLUTILS TUPACKET RANGE}	Ende vor Anfang, oder Felder überlappen
{TCLUTILS TUPACKET ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUPACKET FONT}	-fontfile nennt keine Datei

15.24 Kanban: tukanban

tukanban liest `kanban`. Die erste Zeile nach dem Kopf legt die Einrückung der Spalten fest. Zeilen mit dieser oder kleinerer Einrückung sind Spaltentitel, tiefere Zeilen Karten der aktuellen Spalte. Jede Spalte hat einen farbigen Kopf, die Karten stehen darunter. Die Fläche ergibt sich aus der Zahl der Spalten und dem höchsten Kartenstapel (`-width 0 -height 0`).

Schreibweise	Ergebnis
Offen	Spalte
Spec schreiben	Karte
API@{ ticket: T-1, priority: High }	Karte mit Zusatzzeile

Aus den Angaben in `@{ … }` wird `priority` als farbiges Zeichen gezeichnet (`High` rot, `Medium` orange, `Low` grün, `Critical` und `Blocked` dunkelrot, sonst grau), alle Angaben erscheinen als graue Zeile unter dem Text. Die Mermaid-Form `id[Text]` wird nicht zerlegt: `todo[Erledigt]` erscheint so als Titel. Lange Texte werden abgeschnitten. Bei fester `-height` fallen Karten weg, die nicht mehr in die Spalte passen.

```
package require tclutils::tukanban
set kb {kanban
    Offen
        Spec schreiben
        API@{ ticket: T-1, priority: High }
    Fertig
}
set cols [dict get [::tclutils::tukanban::parse $kb] columns]
lmap c $cols {lindex $c 0}                    ;# -> Offen Fertig
lindex $cols 0 1 1         ;# -> API {ticket: T-1  priority: High}
```

-errorcode	Ursache
{TCLUTILS TUKANBAN EMPTY}	erste Zeile kein Kopf, oder keine Spalte
{TCLUTILS TUKANBAN ARG}	-scale keine ganze Zahl ab 1
{TCLUTILS TUKANBAN FONT}	-fontfile nennt keine Datei

15.25 PNG und Schrift

Ohne `-fontfile` schreibt PNG die Beschriftung mit einer 6×8-Bitmap; sie enthält die deutschen Umlaute. -fontfile nennt eine TTF- oder OTF-Datei; dann zeichnet `tupngdraw` `fillcontours` echte Umrisse. Dafür braucht es das Paket *Glyphs* (A. Buratti), das nicht zu tclutils gehört: das Verzeichnis mit seiner `pkgIndex.tcl` auf `auto_path` legen. Fehlt Glyphs, bleibt es ohne Fehler bei der Bitmap. Eine nicht vorhandene Schriftdatei wirft `{TCLUTILS TUDIAGRAM FONT}`, bei den Arten ohne Graph den Fehler `FONT` des Moduls. SVG übergeht `-fontfile`, dort wählt der Betrachter die Schrift (`monospace`). Die Maße des Layouts bleiben in jedem Fall beim 6×8-Raster.

```
package require tclutils::tuflow
set t "flowchart LR\n  A --> B"
catch {::tclutils::tuflow::toPng $t -fontfile /gibt/es/nicht.ttf} m o
dict get $o -errorcode           ;# -> TCLUTILS TUDIAGRAM FONT
```

Unter Tcl 9.0 und 8.6 entstehen die Bilder dieses Kapitels Byte für Byte gleich. Die Modelle und Fehler stimmen ebenfalls überein, auch die Datumsprüfung in `tugantt`.

15.26 Merkliste

- Ein Eingang: `tuflow::toSvg`, `toPng`, `writeSvg`, `writePng` wählen die Art nach dem ersten Wort.
- `tuflow::parse` liefert nur Graphen, für jede andere Art `UNSUPPORTED`.
- Graph-PNG kennt nur `-scale` und `-fontfile`, Graph-SVG keine Option. Unbekannte Optionen lässt die Fassade fallen.
- Bauen gibt ein neues Dict zurück. `validate` wirft nicht, `addEdge` prüft die Enden nicht.
- Titel im `tudiagram`-Modell und in `sequenceDiagram` stehen mittig über der Zeichnung.
- Nicht erkannte Zeilen entfallen meist ohne Fehler; `sankey` lässt schlechte Werte still fallen, `pie` und `journey` werfen `VALUE`.

- PNG-Schrift: ohne Glyphs bleibt die 6×8-Bitmap, ohne Fehler.

Nächstes Kapitel: Bilder ohne Tk: PNG, SVG, Farben

16 Bilder ohne Tk: PNG, SVG, Farben

Die Module dieses Kapitels schreiben, lesen und untersuchen Bilder in reinem Tcl. Sie brauchen weder Tk noch eine Erweiterung, nur den Befehl `zlib` aus dem Kern und TclOO. Damit laufen sie auch auf einem Server ohne Anzeige. Kästen und Pfeile aus einem Modell zeichnet `tudiagram`, siehe Diagramme; es baut auf den Zeichenflächen `tupngdraw` und `tusvg` aus diesem Kapitel auf.

Modul	Version	Aufgabe
tucolor	0.2	Farbnamen, Hex, HSV, nächster Name
tupng	0.5	PNG kodieren und dekodieren
tupngdraw	0.13	Zeichenfläche, schreibt PNG
tutablepng	0.2	Tabelle als PNG
tucodepng	0.1	Zeichentabelle als PNG
tumonthpng	0.5	Monat, Quartal, Jahr als PNG
tupngpad	0.2	PNGs auf eine Größe bringen
tusvg	0.3	SVG-Zeichenfläche, SVG-Dokument, Symbole
tuimage	0.1	Format und Größe erkennen, data:-URI
tuico	0.2	Windows-.ico mit PNG-Inhalt
tudhash	0.2	Wahrnehmungs-Hash für ähnliche Bilder

Ein Bild kommt in vier Datenformen vor. Welche ein Befehl erwartet, steht bei jedem Modul.

Form	Aufbau	Wo
Zeilenliste	Liste von Zeilen, jede Zeile eine Liste von Pixeln, alle Zeilen gleich lang	tupng::encode*, write*
RGBA-Bytes	Bytefolge der Länge Breite × Höhe × 4, zeilenweise, je Pixel R G B A mit 8 Bit	tupng::decode, encodeRGBARaw, tupngdraw paste, tupngpad
Zahlenliste	flach, zeilenweise: Breite × Höhe Grauwerte oder × 3 Farbwerte	tudhash
Zeichenfläche	TclOO-Objekt, der Befehlsname ist der Griff	tupngdraw::new, tusvg::new

Farben schreiben die Module nicht einheitlich. Die Tabelle zeigt, was jedes annimmt.

Schreibweise	tucolor	tupng	tupngdraw	tusvg (Objekt)
Name	148 CSS/X11-Namen	nein	15 Namen	ungeprüft durchgereicht
#rgb	ja	nein	nein	ungeprüft durchgereicht
#rrggbb	ja	ja	ja	ungeprüft durchgereicht
RRGGBB, RRGGBBAA, #rrggbbaa	nein	ja	ja	ungeprüft durchgereicht
{r g b}	ja	ja	ja	rgb(r,g,b)
{r g b a}	nein	ja	ja	rgba(r,g,b,a/255)

Die 15 Namen von `tupngdraw` sind `black`, `white`, `red`, `green`, `lime`, `blue`, `yellow`, `cyan`, `magenta`, `gray`, `grey`, `silver`, `orange`, `transparent` und `none`; die beiden letzten sind `{0 0 0 0}`. Für alle anderen Namen liefert `tucolor::rgb` das Tripel.

16.1 Farben: tucolor

tucolor löst eine Farbe in ein RGB-Tripel auf und rechnet nach Hex und HSV. Die Namenstabelle ist im Modul eingebaut, sie stimmt mit den Werten von Tk überein, braucht aber kein Tk.

```
::tclutils::tucolor::rgb     farbe      ;# -> {r g b}, je 0..255
::tclutils::tucolor::hex     farbe      ;# -> #rrggbb
::tclutils::tucolor::names              ;# -> 148 Namen, sortiert
::tclutils::tucolor::exists  name       ;# -> 0 | 1
::tclutils::tucolor::nearest farbe      ;# -> nächster Name
::tclutils::tucolor::toHsv   farbe      ;# -> {h s v}
::tclutils::tucolor::fromHsv {h s v}    ;# -> {r g b}
```

`farbe` ist ein Name, `#rgb`, `#rrggbb` oder ein Tripel `{r g b}` aus ganzen Dezimalzahlen 0 bis 255. Namen gelten ohne Rücksicht auf Groß- und Kleinschreibung, Leerzeichen am Rand fallen weg. `#rgb` verdoppelt jede Stelle (`#f80` ist `#ff8800`). `nearest` sucht den kleinsten euklidischen Abstand im RGB-Raum; bei Gleichstand gewinnt der Name, der in der Tabelle zuerst steht (aqua vor `cyan`, `gray` vor `grey`). `toHsv` gibt `h` in Grad 0 bis 360, `s` und `v` in Prozent, alle gerundet. Hin und zurück kann je Kanal eine Einheit abweichen. `fromHsv` nimmt `h` von 0 bis 360 und `s`, `v` von 0 bis 100 als Dezimalzahlen, auch mit Nachkommastellen; 360 ist dasselbe wie 0.

```
package require tclutils::tucolor
::tclutils::tucolor::rgb SteelBlue          ;# -> 70 130 180
::tclutils::tucolor::rgb #f80               ;# -> 255 136 0
::tclutils::tucolor::hex {255 136 0}        ;# -> #ff8800
::tclutils::tucolor::hex navy               ;# -> #000080
::tclutils::tucolor::exists " Navy "        ;# -> 1
::tclutils::tucolor::nearest #fe0000        ;# -> red
```

```
::tclutils::tucolor::nearest {0 255 255}     ;# -> aqua
::tclutils::tucolor::toHsv steelblue         ;# -> 207 61 71
::tclutils::tucolor::fromHsv {207 61 71}     ;# -> 71 131 181
::tclutils::tucolor::fromHsv {300 100 100}   ;# -> 255 0 255
::tclutils::tucolor::fromHsv {120.5 50 50}   ;# -> 64 128 64
::tclutils::tucolor::rgb {010 20 30}         ;# -> 10 20 30
llength [::tclutils::tucolor::names]         ;# -> 148
```

Jede nicht erkannte Farbe wirft `{TCLUTILS TUCOLOR NAME}`, auch `FF0000` ohne `#`, `{256 0 0}`, ein Quadrupel `{255 0 0 255}`, `transparent` und ein Tripel mit Hexzahl, Vorzeichen oder Nachkommastelle wie `{0x10 0 0}`. Eine führende Null im Tripel ist keine Oktalzahl, unter Tcl 8.6 und 9.0 gleich: `{08 0 0}` ergibt `8 0 0`. `fromHsv` wirft `{TCLUTILS TUCOLOR HSV}`, wenn die Liste nicht drei Elemente hat, ein Wert keine Zahl ist oder außerhalb seines Bereichs liegt, etwa `{0 150 100}` oder `{-60 100 100}`.

16.2 PNG kodieren und dekodieren: tupng

tupng schreibt PNG-Dateien mit 8 Bit je Kanal in vier Farbtypen und liest PNG-Dateien zurück in RGBA-Bytes. Grundlage ist allein `zlib` aus dem Kern.

```
::tclutils::tupng::encodeRGB     bild ?option wert ...?
::tclutils::tupng::encodeRGBA    bild ?option wert ...?
::tclutils::tupng::encodeGray    bild ?option wert ...?
::tclutils::tupng::encodeIndexed palette bild ?option wert ...?
::tclutils::tupng::encodeRGBARaw bytes breite hoehe ?option wert ...?
::tclutils::tupng::writeRGB      datei bild ?option wert ...?
::tclutils::tupng::writeRGBA     datei bild ?option wert ...?
::tclutils::tupng::writeGray     datei bild ?option wert ...?
::tclutils::tupng::writeIndexed  datei palette bild ?option wert ...?
::tclutils::tupng::writeRGBARaw  datei bytes breite hoehe ?opt w ...?
::tclutils::tupng::decode        bytes
::tclutils::tupng::readPNG       datei
```

Befehl	Farbtyp	Pixel in der Zeilenliste
encodeRGB	2, Truecolor	RRGGBB, #rrggbb, {r g b}; ein Alphawert fällt weg
encodeRGBA	6, Truecolor mit Alpha	wie oben, dazu RRGGBBAA, {r g b a}; ohne Alpha gilt 255
encodeGray	0, Graustufen	ganze Zahl 0..255
encodeIndexed	3, Palette	Index in die Palette

Option	Vorgabe	Wirkung
-compression	6	zlib-Stufe 0..9
-filter	best	best, none, sub, up, average, paeth

`best` probiert je Zeile alle fünf PNG-Filter und nimmt den mit der kleinsten Summe der Beträge. Das kostet Zeit; `up` ist deutlich schneller (Zahlen am Ende dieses Abschnitts).

`encode*` geben die PNG-Bytes zurück, `write*` den Dateinamen. `encodeRGBARaw` nimmt RGBA-Bytes statt einer Zeilenliste und schreibt immer Farbtyp 6. Die Palette von `encodeIndexed` hat 1 bis 256 Einträge in der Schreibweise von `encodeRGBA`. Hat ein Eintrag Alpha unter 255, schreibt das Modul einen `tRNS`-Block, sonst nicht.

`decode` liefert ein Dict:

Schlüssel	Inhalt
width, height	Größe in Pixeln
colortype	Farbtyp der Datei: 0, 2, 3, 4 oder 6
bitdepth	immer 8
rgba	RGBA-Bytes, Länge Breite × Höhe × 4

Jeder Farbtyp wird nach RGBA umgesetzt. Graustufen werden zu `g g g`. Ein `tRNS`-Farbschlüssel macht bei Graustufen und bei Truecolor (Typ 2) jedes Pixel mit genau diesem Wert durchsichtig (Alpha 0), alle anderen bekommen 255. Bei einer Palette gilt der Alpha-Eintrag aus `tRNS`. `decode` von `encodeRGBARaw` ergibt dieselben Bytes zurück. Vor dem Auswerten prüft `decode` die CRC-Summe jedes Blocks.

```
package require tclutils::tupng
set img {{FF0000 00FF00} {0000FF FFFFFF}}
set png [::tclutils::tupng::encodeRGB $img]
string range $png 1 3                          ;# -> PNG
set d [::tclutils::tupng::decode $png]
list [dict get $d width] [dict get $d colortype]  ;# -> 2 2
binary encode hex [dict get $d rgba]
                               ;# -> ff0000ff00ff00ff0000ffffffffffff
set pal {FF0000 0000FF00}
set png [::tclutils::tupng::encodeIndexed $pal {{0 1} {1 0}}]
binary encode hex [dict get [::tclutils::tupng::decode $png] rgba]
                               ;# -> ff0000ff0000ff000000ff00ff0000ff
set png [::tclutils::tupng::encodeGray {{0 128 255}}]
binary encode hex [dict get [::tclutils::tupng::decode $png] rgba]
                               ;# -> 000000ff808080ffffffffff
set raw [binary format cu* {255 0 0 255 0 0 255 128}]
set png [::tclutils::tupng::encodeRGBARaw $raw 2 1]
expr {[dict get [::tclutils::tupng::decode $png] rgba] eq $raw}
                               ;# -> 1
```

Fehler beim Kodieren:

-errorcode	Ursache
{TCLUTILS TUPNG DIM}	keine Zeile, leere Zeile, Zeilen ungleich lang; bei encodeRGBARaw Breite oder Höhe nicht positiv oder falsche Bytezahl

-errorcode	Ursache
{TCLUTILS TUPNG COLOR}	Pixel nicht lesbar, etwa red oder ein Grauwert 300
{TCLUTILS TUPNG PALETTE}	Palette leer oder länger als 256
{TCLUTILS TUPNG INDEX}	Index außerhalb der Palette
{TCLUTILS TUPNG OPT}	-compression nicht 0..9, -filter unbekannt
{TCLUTILS COMMON OPTION -level}	unbekannte Option, hier -level

Fehler beim Dekodieren tragen `{TCLUTILS TUPNG DECODE grund}`: `SIGNATURE` (keine PNG-Kennung), `IHDR` (Kopf fehlt), `BITDEPTH` (nicht 8 Bit), `INTERLACE`, `COLORTYPE`, `IDAT` (keine Bilddaten), `FILTER` (unbekannter Zeilenfilter). Ein Block, dessen CRC-Summe nicht stimmt oder der vor seiner CRC-Summe abbricht, wirft `{TCLUTILS TUPNG CRC}`. Stimmt die Summe, sind die komprimierten Daten aber ungültig, meldet `zlib` selbst `{TCL ZLIB DATA}`.

Grenzen: Der Kodierer schreibt nur 8 Bit, kein Interlacing, keine Zusatzblöcke wie `gAMA`, `pHYs` oder `tEXt`. Der Dekodierer weist 16-Bit-Bilder und Paletten oder Graustufen mit 1, 2 oder 4 Bit ab (`BITDEPTH`), ebenso Interlacing (`INTERLACE`). Zusatzblöcke außer `tRNS` übergeht er nach der CRC-Prüfung.

Laufzeit, gemessen auf einem Intel Xeon mit 2,8 GHz, Tcl 9.0.4 und 8.6.14 (Werte für 8.6 in Klammern):

Vorgang	Bild	Zeit
readPNG	1024 × 768 RGB, Rauschen	1,48 s (1,45 s)
encodeRGBARaw -filter up	1024 × 768 RGBA	0,75 s (0,75 s)
encodeRGBARaw mit best	1024 × 768 RGBA	4,5 s (4,9 s)
decode	256 × 256 RGBA, Zeichnung	0,04 s (0,04 s)

Für Symbole, Tabellen und Kalenderblätter reicht das; für Fotos in voller Größe ist ein Werkzeug in C die bessere Wahl.

16.3 Zeichnen: tupngdraw

tupngdraw ist eine Zeichenfläche als TclOO-Objekt. Sie hält die Pixel als RGBA-Liste und gibt am Ende über `tupng::encodeRGBARaw` ein PNG aus. Farben werden nach "source over" gemischt: Eine halbdurchsichtige Füllung legt sich über das, was darunter liegt.

```
set p [::tclutils::tupngdraw::new ?option wert ...?]
```

Option	Vorgabe	Wirkung
-width	100	Breite in Pixeln, ganze Zahl ab 1
-height	100	Höhe in Pixeln, ganze Zahl ab 1
-background	white	Farbe der leeren Fläche

Nach new gelten Füllfarbe und Strichfarbe Schwarz, Strichbreite 1 und Kantenglättung an.

Methode	Wirkung
width, height	Größe
pixel x y	{r g b a} an der Stelle; Koordinaten werden gerundet
setfill farbe, setstroke farbe	Füll- und Strichfarbe
setlinewidth n	Strichbreite, ganze Zahl ab 1
setantialias bool	Kantenglättung für Striche und Füllungen
clear ?farbe?	ganze Fläche neu, ohne Farbe mit dem Hintergrund
setpixel x y ?-color f?	ein Pixel, gemischt
line x1 y1 x2 y2 ?...?	Strecke
rect x1 y1 x2 y2 ?...?	Rechteck, beide Ecken eingeschlossen
circle cx cy r ?...?	Kreis
ellipse cx cy rx ry ?...?	Ellipse
polygon {x0 y0 x1 y1 ...} ?...?	Vieleck, mindestens drei Punkte
arc cx cy r a0 a1 ?...?	Bogen, Tortenstück oder Sehne
fillcontours {kontur ...} ?...?	mehrere Konturen als eine Fläche
text x y text ?...?	Text in der eingebauten 6×8-Schrift
textwidth text ?-scale n? ?-spacing s?	Breite in Pixeln
paste x y rgba b h ?-scale n?	RGBA-Bytes einsetzen
data ?-compression n? ?-filter f?	PNG-Bytes
write datei ?-compression n? ?-filter f?	PNG-Datei, gibt den Namen zurück
destroy	Objekt freigeben

Optionen der Figuren:

Option	Vorgabe	Gilt für	Wirkung
-fill	0	rect, circle, ellipse, polygon, arc	Innenfläche mit der Füllfarbe
-outline	1	rect, circle, ellipse, polygon	Umriss mit der Strichfarbe
-color	Strichfarbe	alle Striche, setpixel, text, fillcontours	Farbe nur für diesen Aufruf
-fillcolor	Füllfarbe	Figuren mit -fill	Füllfarbe nur für diesen Aufruf

Option	Vorgabe	Gilt für	Wirkung
-aa	Flächenwert	Striche, Füllungen, fillcontours	Kantenglättung nur für diesen Aufruf
-width	Strichbreite	line, arc	Breite nur für diesen Aufruf
-caps	round	line	round, butt, square
-join	round	rect, polygon, arc	round, bevel, mitre
-rx, -ry	0	rect	Eckenradius; -ry 0 übernimmt -rx
-style	arc	arc	arc, pie, chord
-rule	nonzero	fillcontours	nonzero oder evenodd
-scale	1	text, textwidth, paste	jedes Pixel als n × n
-spacing	0	text, textwidth	zusätzliche Pixel je Zeichen

`arc` misst Winkel in Grad, 0 zeigt nach rechts, wachsende Winkel laufen auf dem Bild im Uhrzeigersinn, weil y nach unten wächst. `text` setzt die linke obere Ecke des ersten Zeichens. Die Schrift kennt ASCII 32 bis 126 und ä ö ü Ä Ö Ü ß; andere Zeichen bleiben leer. `textwidth` ist Länge × (6 + Abstand) × Maßstab, der Abstand zählt auch nach dem letzten Zeichen. `data` und `write` filtern ohne Angabe mit `-filter up`, nicht mit `best`.

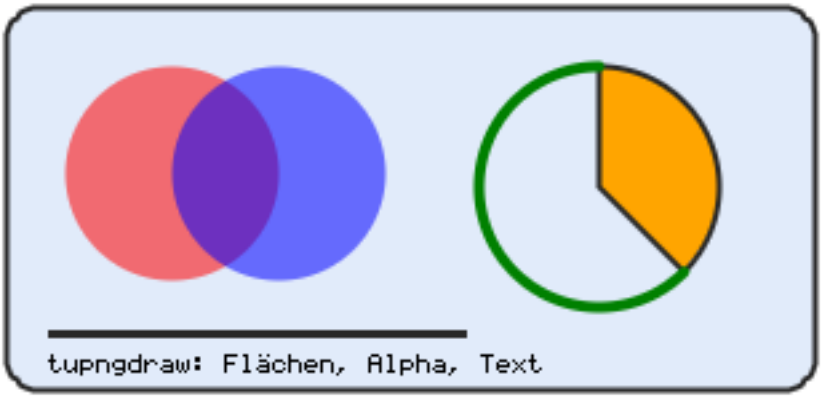
```
package require tclutils::tupngdraw
set p [::tclutils::tupngdraw::new -width 40 -height 20]
$p pixel 0 0                                    ;# -> 255 255 255 255
$p setfill {255 0 0 128}
$p rect 0 0 9 9 -fill 1 -outline 0
$p pixel 5 5                                    ;# -> 255 127 127 255
$p rect 20 2 23 5 -fill 1 -fillcolor black -outline 0
list [$p pixel 23 5] [$p pixel 24 6]
                                ;# -> {0 0 0 255} {255 255 255 255}
$p textwidth Hallo                              ;# -> 30
$p textwidth Hallo -scale 2 -spacing 1          ;# -> 70
string range [$p data] 1 3                      ;# -> PNG
$p destroy
```

`fillcontours` füllt eine Fläche aus mehreren Konturen. Mit `evenodd` wird eine innere Kontur ein Loch; mit `nonzero` entscheidet der Umlaufsinn, zwei gleich laufende Quadrate ergeben eine volle Fläche.

```
set p [::tclutils::tupngdraw::new -width 20 -height 20]
set aussen {2 2 18 2 18 18 2 18}
set innen  {6 6 14 6 14 14 6 14}
$p fillcontours [list $aussen $innen] -color black -rule evenodd
list [$p pixel 10 10] [$p pixel 4 10]
                                ;# -> {255 255 255 255} {0 0 0 255}
$p destroy
```

Das folgende Skript erzeugt das Bild darunter: ein Rechteck mit runden Ecken, zwei halbdurchsichtige Kreise, ein Tortenstück, einen Bogen, eine Linie und Text.

```
package require tclutils::tupngdraw
set p [::tclutils::tupngdraw::new -width 320 -height 160]
$p setstroke {40 40 40}
$p setlinewidth 2
$p setfill {225 235 250}
$p rect 8 8 311 151 -fill 1 -rx 12
$p setfill {255 0 0 140}
$p circle 70 70 40 -fill 1 -outline 0
$p setfill {0 0 255 140}
$p circle 110 70 40 -fill 1 -outline 0
$p arc 230 75 45 -90 45 -style pie -fill 1 -fillcolor orange
$p arc 230 75 45 45 270 -style arc -width 4 -color {0 128 0}
$p line 24 130 180 130 -width 3 -caps butt
$p text 24 138 "tupngdraw: Flächen, Alpha, Text" -color black
$p write 058-zeichnung.png -compression 9
$p destroy
```



Zeichnung mit tupngdraw, 320 × 160 Pixel

Unter Tcl 9.0 und unter Tcl 8.6 entsteht dieselbe Datei Byte für Byte. Tcl 8.6 liest eine Skriptdatei in der Systemkodierung; steht dort nicht UTF-8, wird aus ä ein falsches Zeichen. Abhilfe ist `source -encoding utf-8` oder `\u00e4` im Text.

`paste` mischt nach derselben Regel wie alle anderen Befehle ("source over"): Das Ergebnis-Alpha ist a_q + a_z × (1 − a_q), die Farben werden entsprechend gewichtet (q Quelle, z Ziel, Alpha als Anteil 0 bis 1). Auf einer deckenden Fläche entsteht eine deckende Mischung, auf einer durchsichtigen bleibt das Pixel, wie es ist. Pixel mit Alpha 0 lassen die Fläche unverändert.

```
set p [::tclutils::tupngdraw::new -width 2 -height 1 \
    -background transparent]
$p paste 0 0 [binary format cu* {255 0 0 128}] 1 1
$p setpixel 1 0 -color {255 0 0 128}
list [$p pixel 0 0] [$p pixel 1 0]
                              ;# -> {255 0 0 128} {255 0 0 128}
$p clear white
$p paste 0 0 [binary format cu* {255 0 0 128}] 1 1
$p pixel 0 0                                  ;# -> 255 127 127 255
$p destroy
```

-errorcode	Ursache
{TCLUTILS TUPNGDRAW COLOR}	Farbe nicht lesbar, auch ein Name außerhalb der 15, etwa navy
{TCLUTILS TUPNGDRAW RANGE}	pixel außerhalb der Fläche
{TCLUTILS TUPNGDRAW CAPS}, JOIN, STYLE, RULE	Wert von -caps, -join, -style, -rule unbekannt
{TCLUTILS TUPNGDRAW POLY}	polygon mit weniger als drei Punkten
{TCLUTILS COMMON INTEGER name}	Breite, Höhe, Radius, Strichbreite oder Maßstab nicht ab 1; name ist etwa width, radius, -linewidth
{TCLUTILS COMMON BOOLEAN name}	-fill, -aa oder setantialias kein Wahrheitswert
{TCLUTILS COMMON OPTION name}	unbekannte Option

Jedes Pixel wird in Tcl gesetzt. Ein geglätteter gefüllter Kreis mit Radius 100 auf 256 × 256 Pixeln braucht gemessen 0,14 s unter Tcl 9.0.4 und 0,10 s unter 8.6.14 (Xeon, 2,8 GHz); `data` mit `-filter up` 0,04 s, mit `-filter best` 0,33 s.

16.4 Tabelle als Bild: tutablepng

tutablepng setzt eine Liste von Zeilen als Tabelle in ein PNG: Kopfzeile, Ausrichtung je Spalte, wechselnde Zeilenfarbe, Gitter. Gezeichnet wird mit `tupngdraw` in der 6×8-Schrift.

```
::tclutils::tutablepng::render zeilen ?option wert ...?
::tclutils::tutablepng::write  datei zeilen ?option wert ...?
```

`zeilen` ist eine Liste von Zeilen, jede eine Liste von Zellen. Die breiteste Zeile bestimmt die Spaltenzahl, kürzere Zeilen bekommen leere Zellen. `render` gibt PNG-Bytes zurück (RGBA, Farbtyp 6), `write` den Dateinamen.

Option	Vorgabe	Wirkung
-header	1	erste Zeile als Kopf
-align	l	l, c, r für alle Spalten oder eine Liste je Spalte; fehlende Einträge übernehmen den letzten

Option	Vorgabe	Wirkung
-scale	1	Schriftmaßstab
-padding	6	Rand in der Zelle, Pixel, ganze Zahl ab 0
-spacing	1	Pixel zwischen Zeichen, ganze Zahl ab 0
-background	white	Hintergrund
-gridcolor	{180 180 185}	Gitter
-textcolor	{25 25 25}	Text
-headerbg	{230 230 238}	Kopfzeile, Hintergrund
-headertext	{0 0 0}	Kopfzeile, Text
-zebra	leer	Farbe jeder zweiten Datenzeile; leer heißt aus
-border	1	Gitterlinien

Die Spaltenbreite ist die längste Zelle × (6 + `-spacing`) × `-scale` + 2 × `-padding`, die Zeilenhöhe 8 × `-scale` + 2 × `-padding`. Farben folgen `tupngdraw`. Jede Zelle erscheint genau so, wie sie in der Liste steht; auch `4.80`, `1e3`, `0x1F` oder `010` bleiben unverändert.

```
package require tclutils::tutablepng
package require tclutils::tuimage
set rows {{Artikel Menge Preis} {Schrauben 120 4,80} {Muttern 80}}
set png [::tclutils::tutablepng::render $rows -align {l r r}]
::tclutils::tuimage::dimensions $png         ;# -> 169 60
set png [::tclutils::tutablepng::render {{4.80 010}} -header 0]
::tclutils::tuimage::dimensions $png         ;# -> 73 20
```

Das Bild darunter entsteht mit diesem Skript:

```
package require tclutils::tutablepng
set rows {
    {Artikel     Lager  Menge  Preis}
    {Schrauben   Nord   120    4,80}
    {Muttern     Nord   80     2,10}
    {Scheiben    Süd    450    1,95}
    {Dübel       Süd    60     3,25}
}
::tclutils::tutablepng::write 058-tabelle.png $rows \
    -scale 2 -align {l l r r} -zebra {245 245 248}
```

Artikel	Lager	Menge	Preis
Schrauben	Nord	120	4,80
Muttern	Nord	80	2,10
Scheiben	Süd	450	1,95
Dübel	Süd	60	3,25

Tabelle mit tutablepng, 384 × 140 Pixel

`{}` als Zeilenliste und Zeilen ganz ohne Zellen werfen `{TCLUTILS TUTABLEPNG EMPTY}`. Ein unbekannter `-align`-Wert wirft `{TCLUTILS TUTABLEPNG ALIGN}`. `-header` und `-border` ohne Wahrheitswert werfen `{TCLUTILS COMMON BOOLEAN ...}`, `-scale 0` `{TCLUTILS COMMON INTEGER -scale}`, eine unlesbare Farbe `{TCLUTILS TUPNGDRAW COLOR}`. Ist `-padding` oder `-spacing` keine ganze Zahl ab 0, folgt `{TCLUTILS TUTABLEPNG PADDING}` bzw. `{TCLUTILS TUTABLEPNG SPACING}`.

16.5 Zeichentabelle als Bild: tucodepng

tucodepng zeichnet eine Codetabelle: je Zelle oben der Hexwert, in der Mitte das Zeichen, auf Wunsch unten der Name. Die Daten stammen aus `tucode`, siehe Binärdaten und Dateiinformationen.

```
::tclutils::tucodepng::render von bis ?option wert ...?
::tclutils::tucodepng::write  datei von bis ?option wert ...?
::tclutils::tucodepng::ascii  ?option wert ...?   ;# 0..127
::tclutils::tucodepng::latin1 ?option wert ...?   ;# 128..255
::tclutils::tucodepng::all    ?option wert ...?   ;# 0..255
```

Option	Vorgabe	Wirkung
-columns	16	Zellen je Reihe
-scale	2	Maßstab des Zeichens in der Mitte
-theme	default	default, dark, light
-showcode	1	Hexwert oben
-shownames	0	Name unten, etwa NUL, auml
-title	leer	Überschrift über der Tabelle
-textcmd	leer	eigener Befehl für das Zeichen in der Mitte

Steuerzeichen 0 bis 31 und 127 zeigen ihre Abkürzung, 32 zeigt SP, jeweils in Maßstab 1. Hexwert und Name stehen immer in der 6×8-Schrift. Von 128 bis 255 kennt die Schrift nur ä ö ü Ä Ö Ü ß; die übrigen Zellen bleiben in der Mitte leer, es sei denn, `-textcmd` zeichnet. Der Befehl wird für jedes druckbare Zeichen als `{*}$cmd bild x y b h zeichen farbe` gerufen; `bild` ist die `tupngdraw`-Fläche.

Eine Zelle ist 28 × `-scale` breit, mit Namen mindestens 48 Pixel, und 12 × `-scale` hoch, dazu

10 Pixel für den Hexwert und 10 für den Namen. Die Überschrift ist 12 × `-scale` hoch.

```
package require tclutils::tucodepng
package require tclutils::tuimage
set png [::tclutils::tucodepng::ascii]
::tclutils::tuimage::dimensions $png         ;# -> 896 272
set png [::tclutils::tucodepng::render 64 95 -columns 8 -scale 1]
::tclutils::tuimage::dimensions $png         ;# -> 224 88
```

`von` oder `bis` keine ganze Zahl, `von` größer als `bis` oder ein Bereich außerhalb 0..255 werfen `{TCLUTILS TUCODEPNG RANGE}`. Ein unbekanntes Thema wirft `{TCLUTILS TUCODEPNG THEME}`.

16.6 Monatsblatt: tumonthpng

tumonthpng zeichnet einen Monat als Kalenderblatt: Titel, Wochentagskopf, Spalte mit ISO-Kalenderwochen, sechs Wochen. Hervorgehoben werden heute, Feiertage, Notizen, Wochenende und Tage anderer Monate. Quartal und Jahr setzen mehrere Monate nebeneinander.

```
::tclutils::tumonthpng::render        jahr monat ?opt wert ...?
::tclutils::tumonthpng::write         datei jahr monat ?opt wert ...?
::tclutils::tumonthpng::renderQuarter jahr monat ?opt wert ...?
::tclutils::tumonthpng::writeQuarter  datei jahr monat ?opt wert ...?
::tclutils::tumonthpng::renderYear    jahr ?opt wert ...?
::tclutils::tumonthpng::writeYear     datei jahr ?opt wert ...?
```

`render*` geben PNG-Bytes zurück, `write*` den Dateinamen. Daten sind ISO-Daten `JJJJ-MM-TT`.

Option	Vorgabe	Wirkung
-theme	default	default, dark, light
-scale	2	Maßstab
-today	leer	Tag "heute"; leer heißt Systemdatum
-holidays	leer	Dict Datum → Name
-notes	leer	Dict Datum → Text; der Tag bekommt einen Punkt
-firstweekday	1	erster Tag der Woche, 1 Montag bis 7 Sonntag; verschiebt Raster und Kopf
-showweeks	1	Spalte "KW"
-weekdays	{Mo Di Mi Do Fr Sa So}	Namen für den Kopf, immer Montag bis Sonntag
-monthnames	{Januar ... Dezember}	Monatsnamen im Titel

Option	Vorgabe	Wirkung
-title	leer	render: Titel statt "Monat Jahr"; Quartal und Jahr: Überschrift über dem ganzen Blatt
-select	leer	Liste von Daten und Bereichen von..bis
-selectstyle	outline	outline, fill, both
-selectcolor	leer, dann #1565C0	Farbe der Auswahl
-selectwidth	leer, dann max(2, Maßstab)	Rahmenbreite der Auswahl
-selectalpha	0.3	Anteil der Auswahlfarbe bei fill
-textcmd	leer	eigener Befehl für jede Beschriftung
-cols	3	nur renderYear: Monate je Reihe

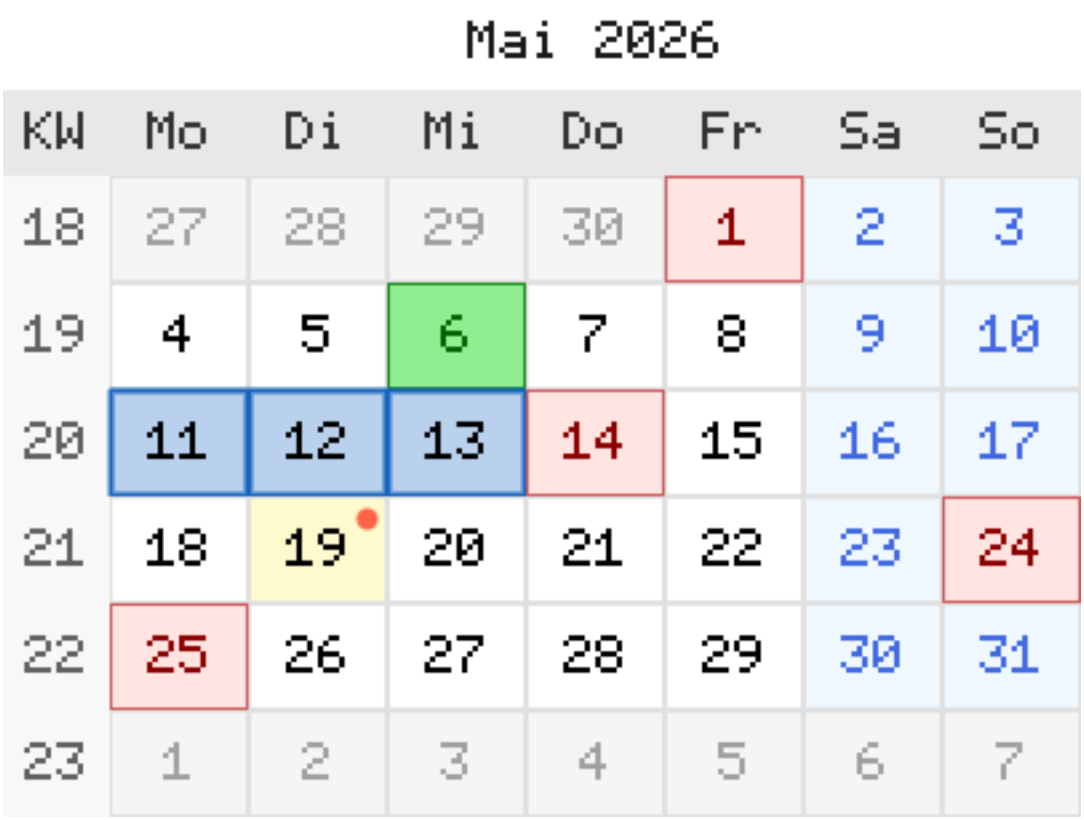
Ein Tag bekommt genau eine Färbung, in dieser Reihenfolge: heute, Feiertag, Notiz, anderer Monat, Wochenende, normal. Den Punkt einer Notiz zeichnet das Modul in jedem Fall. Die Namen in `-holidays` und die Texte in `-notes` erscheinen nicht im Bild, sie markieren nur den Tag. Die Rechnung läuft in UTC über `clock`, das Ergebnis hängt nicht von der Zeitzone ab.

Ein Monatsblock ist 7 × 26 × `-scale` breit, mit KW-Spalte 20 × `-scale` mehr, und 154 × `-scale` hoch. Quartal und Jahr setzen zwischen die Blöcke 10 × `-scale`; ein `-title` kommt dort als Streifen von 18 × `-scale` über alle Blöcke hinzu. Das Jahr ist eine ganze Zahl von 1 bis 9999.

```
package require tclutils::tumonthpng
package require tclutils::tuimage
proc dims {png} { ::tclutils::tuimage::dimensions $png }
dims [::tclutils::tumonthpng::render 2026 6 -today 2026-06-06]
                                                ;# -> 404 308
dims [::tclutils::tumonthpng::render 2026 6 -scale 1 -showweeks 0]
                                                ;# -> 182 154
dims [::tclutils::tumonthpng::renderQuarter 2026 11 -scale 1]
                                                ;# -> 626 154
dims [::tclutils::tumonthpng::renderYear 2026 -scale 1 -cols 4]
                                                ;# -> 838 482
dims [::tclutils::tumonthpng::renderQuarter 2026 11 -scale 1 \
    -title "Q4 2026"]                           ;# -> 626 172
```

Das Quartal ab November 2026 zeigt November, Dezember und Januar 2027. Das Monatsblatt darunter verbindet `tumonthpng` mit den Feiertagen aus `tuholiday` (Notizen und Kalender):

```
package require tclutils::tumonthpng
package require tclutils::tuholiday
::tclutils::tumonthpng::write 058-monat.png 2026 5 \
    -today 2026-05-06 \
    -holidays [::tclutils::tuholiday::holidays 2026] \
    -notes {2026-05-19 Inventur} \
    -select {2026-05-11..2026-05-13} -selectstyle both
```



Mai 2026 mit tumonthpng, 404 × 308 Pixel

`-textcmd` wird als `{*}$cmd bild x y b h text farbe` für Titel, Wochentage, "KW", Wochennummern und Tageszahlen gerufen und zeichnet den Text selbst, etwa mit echten Schriftumrissen über `tupngdraw fillcontours`.

`-firstweekday` verschiebt Raster und Kopf gemeinsam: Mit `-firstweekday 7` beginnt die erste Spalte am Sonntag, und der Kopf zeigt `So Mo Di Mi Do Fr Sa`. `-weekdays` nennt die Namen immer von Montag bis Sonntag; das Modul dreht die Liste selbst.

-errorcode	Ursache
{TCLUTILS TUMONTHPNG MONTH}	Monat nicht 1..12
{TCLUTILS TUMONTHPNG YEAR}	Jahr keine ganze Zahl 1..9999, etwa abc
{TCLUTILS TUMONTHPNG FIRSTWEEKDAY}	-firstweekday nicht 1..7
{TCLUTILS TUMONTHPNG THEME}	Thema unbekannt
{TCLUTILS TUMONTHPNG SELECT}	Eintrag in -select nicht JJJJ-MM-TT oder Bereich
{TCLUTILS TUMONTHPNG SELECTSTYLE}	-selectstyle unbekannt

-errorcode	Ursache
{TCLUTILS COMMON BOOLEAN -showweeks}	-showweeks kein Wahrheitswert
{TCLUTILS COMMON INTEGER -scale}, -cols	Maßstab oder Spalten nicht ab 1

Ein `-today` wie `morgen` ist kein Fehler, es trifft nur keinen Tag.

16.7 Einheitliche Größe: tupngpad

tupngpad bringt mehrere PNG-Dateien auf eine gemeinsame Größe. Jedes Bild wird auf seinen Inhalt zugeschnitten, mit Rand mittig gesetzt und auf eine Hintergrundfarbe gelegt. Typisch sind freigestellte Bilder mit durchsichtigem Hintergrund.

```
::tclutils::tupngpad::bbox  rgba b h                ;# -> {x0 y0 x1 y1}
::tclutils::tupngpad::trim  rgba b h                ;# -> {rgba b h}
::tclutils::tupngpad::padTo rgba b h zb zh ?option wert ...?
::tclutils::tupngpad::batch dateien zielordner ?option wert ...?
```

`bbox` sucht alle Pixel mit Alpha über 0 und gibt die Ecken einschließlich zurück, bei einem ganz durchsichtigen Bild die leere Liste. `trim` schneidet auf diesen Kasten; ist nichts sichtbar, kommt die Eingabe unverändert zurück. `padTo` setzt den Inhalt auf eine Fläche `zb × zh` und gibt PNG-Bytes zurück.

Option von padTo	Vorgabe	Wirkung
-background	white	Farbe der Fläche, Schreibweise wie tupngdraw
-align	center	center, nw, n, ne, w, e, sw, s, se
-filter	leer	PNG-Filter; leer heißt up

Option von batch	Vorgabe	Wirkung
-margin	4	Rand in Pixeln, ganze Zahl ab 0
-background	white	wie oben
-trim	1	erst auf den Inhalt schneiden
-align	center	wie oben
-square	0	quadratische Ausgabe
-size	leer	{B H} für den Innenbereich, ganze Zahlen ab 1; leer heißt größtes Eingabebild

`batch` liest jede Datei mit `tupng::readPNG`, schreibt sie unter gleichem Namen in den Zielordner (der bei Bedarf angelegt wird) und gibt ein Dict zurück: `size {b h}` und `files` mit abwechselnd Eingabe- und Ausgabepfad. Alle Ausgaben haben Farbtyp 6. Ein zu kleines `-size` schneidet den Inhalt ab. Halbdurchsichtige Pixel werden mit dem Hintergrund gemischt, siehe `paste` bei `tupngdraw`: Auf einem deckenden Hintergrund werden sie deckend, mit `-background transparent` bleiben sie halbdurchsichtig.

```
package require tclutils::tupngpad
set px {}
for {set y 0} {$y < 3} {incr y} {
    for {set x 0} {$x < 4} {incr x} {
        if {$y == 1 && $x in {1 2}} {
            lappend px 255 0 0 255
        } else {
            lappend px 0 0 0 0
        }
    }
}
set rgba [binary format cu* $px]
::tclutils::tupngpad::bbox $rgba 4 3          ;# -> 1 1 2 1
lassign [::tclutils::tupngpad::trim $rgba 4 3] r w h
list $w $h                                    ;# -> 2 1
set png [::tclutils::tupngpad::padTo $r $w $h 6 4 -align nw]
set d [::tclutils::tupng::decode $png]
binary encode hex [string range [dict get $d rgba] 0 11]
                              ;# -> ff0000ffff0000ffffffffff
::tclutils::tupng::writeRGBARaw in1.png $rgba 4 3
set img2 {{00000000 00FF00} {00FF00 00FF00}}
::tclutils::tupng::writeRGBA in2.png $img2
set res [::tclutils::tupngpad::batch {in1.png in2.png} aus -margin 2]
dict get $res size                            ;# -> 6 6
```

`batch` prüft alle Optionen, bevor es eine Datei liest.

-errorcode	Ursache
{TCLUTILS TUPNGPAD MARGIN}	-margin unter 0 oder keine ganze Zahl
{TCLUTILS TUPNGPAD TRIM}	-trim kein Wahrheitswert
{TCLUTILS TUPNGPAD SQUARE}	-square kein Wahrheitswert
{TCLUTILS TUPNGPAD SIZE}	-size nicht leer und nicht {B H} mit ganzen Zahlen ab 1
{TCLUTILS COMMON VALUE -align}	-align unbekannt; die Meldung nennt die erlaubten Werte

Fehlende Dateien und kaputte PNGs melden `open` (`{POSIX ENOENT ...}`) und `tupng::decode` mit ihren Codes.

16.8 SVG: tusvg

tusvg erzeugt SVG-Text. Es hat zwei Zugänge: eine Zeichenfläche `new`, deren Methoden denen von `tupngdraw` gleichen, und eine ältere Prozedur-Schnittstelle mit einem Dokument als Dict, Verläufen, Gruppen und 149 Symbolnamen. Die Bilder dieses Buchs sind PNG; SVG steht hier nur als Text.

16.8.1 Zeichenfläche

```
set c [::tclutils::tusvg::new ?-width n? ?-height n? ?-background f?]
```

Vorgaben wie bei `tupngdraw`: 100 × 100, Hintergrund `white`. Mit `-background none` oder leer entfällt das Hintergrundrechteck. Die Methoden `width`, `height`, `setfill`, `setstroke`, `setlinewidth`, `rect`, `line`, `polygon`, `circle`, `ellipse`, `text`, `textwidth`, `data`, `write` und `destroy` nehmen dieselben Argumente wie bei `tupngdraw` (`-fill`, `-outline`, `-color`, `-fillcolor`, `-rx`, `-ry`, `-width`, `-caps`, `-scale`, `-spacing`). Dazu kommen:

Methode	Wirkung
text ... -anchor a	start (Vorgabe), middle, end
polyline punkte ?-color f? ?-width n?	offener Linienzug
path d ?-fill? ?-outline? ?-color? ?-fillcolor?	SVG-Pfad
addDef text	Eintrag in <defs>
icon name x y groesse ?...?	Symbol einsetzen, siehe unten

`data` gibt den SVG-Text zurück, `write` schreibt ihn in UTF-8 und gibt den Dateinamen zurück. `textwidth` rechnet wie `tupngdraw`; `text` setzt dazu `textLength`, damit der Text im Betrachter genau diese Breite einnimmt. So bleibt ein Kasten, der nach `textwidth` bemessen ist, in SVG und PNG gleich groß. Unterschiede zu `tupngdraw`: Farben werden nicht geprüft (`nichtsda` und `{999 0 0}` gehen durch); `-aa` und `-join` sind unbekannte Optionen; `arc`, `pixel`, `setpixel`, `clear`, `setantialias`, `paste` und `fillcontours` fehlen; `rect` hat die Breite `x2 − x1`, während `tupngdraw` beide Eckpixel mitzählt, also ein Pixel mehr.

```
package require tclutils::tusvg
set c [::tclutils::tusvg::new -width 120 -height 40]
$c setfill {200 220 255}
$c rect 4 4 115 35 -fill 1 -rx 6
$c text 10 16 "A<B" -color navy
$c textwidth "A<B"                          ;# -> 18
puts [$c data]
$c destroy
```

Ausgabe:

```
<svg xmlns="http://www.w3.org/2000/svg" width="120" height="40"
 viewBox="0 0 120 40">
  <rect x="0" y="0" width="120" height="40" fill="white"/>
  <rect x="4" y="4" width="111" height="31" rx="6"
 fill="rgb(200,220,255)" stroke="black" stroke-width="1"/>
  <text x="10" y="23" font-family="monospace" font-size="8"
 textLength="18" lengthAdjust="spacingAndGlyphs"
 text-anchor="start" fill="navy">A&lt;B</text>
</svg>
```

Die langen Zeilen sind hier umbrochen; im Original steht jedes Element auf einer Zeile.

-errorcode	Ursache
{TCLUTILS TUSVG COLOR}	Farbe als Liste mit 2 oder mehr als 4 Elementen
{TCLUTILS TUSVG CAPS}	-caps unbekannt
{TCLUTILS TUSVG ANCHOR}	-anchor unbekannt
{TCLUTILS TUSVG POLY}	polygon mit weniger als drei Punkten
{TCLUTILS COMMON ...}	Optionen, Wahrheitswerte, Ganzzahlen wie bei tupngdraw

Die Methode `icon` nimmt die Optionen der Prozedur `icon` (`-color`, `-strokeWidth`). Sie setzt das Symbol als verschachteltes `<svg>`-Element mit `x`, `y`, Breite und Höhe `groesse` und `viewBox="0 0 24 24"` ein, ohne XML-Deklaration.

```
set c [::tclutils::tusvg::new -width 40 -height 40]
$c icon save 4 4 16 -color navy
set d [$c data]
regexp -all {<svg} $d                       ;# -> 2
regexp {<svg x="4" y="4" width="16"} $d     ;# -> 1
$c destroy
```

16.8.2 Dokument als Dict

```
set svg [::tclutils::tusvg::create breite hoehe ?-viewBox v? ?-id i?]
::tclutils::tusvg::rect        svg x y b h ?option wert ...?
::tclutils::tusvg::circle      svg cx cy r ?...?
::tclutils::tusvg::ellipse     svg cx cy rx ry ?...?
::tclutils::tusvg::line        svg x1 y1 x2 y2 ?...?
::tclutils::tusvg::polyline    svg punkte ?...?
::tclutils::tusvg::polygon     svg punkte ?...?
::tclutils::tusvg::path        svg d ?...?
::tclutils::tusvg::textElement svg x y text ?...?
::tclutils::tusvg::linearGradient svg id x1 y1 x2 y2 stops
::tclutils::tusvg::radialGradient svg id cx cy r stops
set g [::tclutils::tusvg::group svg ?-transform t? ?-opacity o?]
::tclutils::tusvg::addToGroup g element
::tclutils::tusvg::addGroup   svg $g
::tclutils::tusvg::toString   $svg                ;# -> SVG-Text
::tclutils::tusvg::write      $svg datei          ;# -> datei
```

Hier ist `rect` anders als bei der Zeichenfläche: Ecke, Breite, Höhe. Die Element-Prozeduren nehmen den Namen der Dict-Variablen und hängen an. Ihre Optionen heißen wie SVG-Attribute in Kamelschreibweise: `-fill`, `-stroke`, `-strokeWidth`, `-opacity`, `-id`, bei Linien `-strokeLinecap` und `-strokeLinejoin`, bei Text `-fontSize`, `-fontFamily`, `-fontWeight`, `-textAnchor`. Die Vorgabe für `-fill` ist `#000000`, bei `line`, `polyline` und `path` `none` mit Strich `#000000`. Unbekannte Optionen werden still übergangen, Werte nicht geprüft. `punkte` steht als Text `"x,y x,y ..."` und geht unverändert in die Ausgabe. Die Verläufe kommen in `<defs>` und geben `url(#id)` zurück, passend für `-fill`. Ein Stopp

ist `{offset farbe ?deckkraft?}`, Offset und Koordinaten in Prozent. In eine Gruppe kommen Elemente, indem die Element-Prozedur die Gruppenvariable bekommt. `toString` beginnt mit der XML-Deklaration.

```
set svg [::tclutils::tusvg::create 40 20 -viewBox "0 0 40 20"]
::tclutils::tusvg::rect svg 2 2 36 16 -fill "#eef" -stroke navy -rx 3
::tclutils::tusvg::textElement svg 20 14 "Lager & Co" \
    -fontSize 10 -textAnchor middle
set g [::tclutils::tusvg::group svg -transform "translate(5,5)"]
::tclutils::tusvg::rect g 0 0 4 4
::tclutils::tusvg::addGroup svg $g
puts [::tclutils::tusvg::toString $svg]
```

Ausgabe:

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" width="40" height="20"
 viewBox="0 0 40 20">
  <rect x="2" y="2" width="36" height="16" rx="3" fill="#eef"
 stroke="navy" stroke-width="1"/>
  <text x="20" y="14" fill="#000000" font-size="10"
 font-family="sans-serif" text-anchor="middle">Lager &amp;amp; Co</text>
  <g transform="translate(5,5)">
    <rect x="0" y="0" width="4" height="4" fill="#000000"/>
  </g>
</svg>
```

16.8.3 Symbole

```
::tclutils::tusvg::icons                        ;# -> Liste der Namen
::tclutils::tusvg::icon     name groesse ?-color f? ?-strokeWidth w?
::tclutils::tusvg::saveIcon name groesse datei ?-color f?
```

`icons` nennt alle 149 Namen, die `icon` kennt, darunter Doppelnamen wie `trash` und `delete` oder `rotate-left` und `rotate_left`. `icon` gibt ein Dokument-Dict mit `viewBox "0 0 24 24"` zurück; `toString` macht daraus Text. Die Vorgabe für `-color` ist `#333333`, die Strichbreite wächst ohne Angabe mit der Größe (1.5 bis 16, 1.75 bis 24, 2.0 bis 32, sonst 2.5). Ein unbekannter Name ergibt ohne Fehler ein Ersatzsymbol mit Fragezeichen. Die Symbole bestehen nur aus Linien und Flächen, ohne `<text>`.

```
llength [::tclutils::tusvg::icons]            ;# -> 149
set ic [::tclutils::tusvg::icon save 16]
dict get $ic width                            ;# -> 16
dict get $ic viewBox                          ;# -> 0 0 24 24
```

16.9 Bildformat erkennen: tuimage

tuimage liest Format, MIME-Typ und Größe aus den ersten Bytes eines Bildes. Es dekodiert keine Pixel. Dazu kommen `data:`-URIs.

```
::tclutils::tuimage::type       bytes   ;# -> png|jpeg|gif|bmp|webp|""
::tclutils::tuimage::mime       bytes   ;# -> image/png ... oder ""
::tclutils::tuimage::dimensions bytes   ;# -> {breite hoehe} oder {}
::tclutils::tuimage::inspect    bytes   ;# -> Dict
::tclutils::tuimage::dataUri    mime bytes
::tclutils::tuimage::fromDataUri uri    ;# -> {mime m bytes b}
```

Format	Kennung	Größe aus
PNG	\x89PNG\r\n\x1a\n	IHDR
JPEG	\xff\xd8\xff	erster SOF-Marker
GIF	GIF87a, GIF89a	Bildschirmbeschreibung
BMP	BM	Info-Kopf; negative Höhe wird positiv
WebP	RIFF … WEBP	VP8 , VP8L oder VP8X

`inspect` gibt `type`, `mime`, `width` und `height`; bei unbekanntem Format sind alle vier leer. Die Prüfung ist knapp: Jede Zeichenkette, die mit `BM` beginnt, gilt als BMP. Ist die Datei zu kurz für den Kopf, bleibt `dimensions` leer. Die Bytes müssen binär gelesen sein.

`dataUri` setzt `data:mime; base64,` vor die Base64-Form, ohne den MIME-Typ zu prüfen. `fromDataUri` dekodiert Base64 nur, wenn `base64` im Kopf steht, nicht in einem Parameter in Anführungszeichen wie `charset="utf-8;base64"` (0.2); sonst kommt der Inhalt unverändert zurück, ohne Prozent-Dekodierung. Ungültiges Base64 wie `!!!` ergibt leere Bytes, keinen Fehler. Base64 übernimmt `tubase64`, siehe Kodierung, Hash, Vergleich.

```
package require tclutils::tuimage
package require tclutils::tupng
set png [::tclutils::tupng::encodeGray {{0 255}}]
::tclutils::tuimage::inspect $png
                        ;# -> type png mime image/png width 2 height 1
::tclutils::tuimage::type "BM irgendwas"     ;# -> bmp
::tclutils::tuimage::dimensions Hallo        ;# ->
set uri [::tclutils::tuimage::dataUri image/png $png]
string range $uri 0 21          ;# -> 
set r [::tclutils::tuimage::fromDataUri $uri]
expr {[dict get $r bytes] eq $png}           ;# -> 1
::tclutils::tuimage::fromDataUri "data:,Hallo%20Welt"
                               ;# -> mime {} bytes Hallo%20Welt
```

Eine Zeichenkette ohne `data:` am Anfang wirft `{TCLUTILS TUIMAGE DATAURI}`. Die übrigen Befehle werfen nicht.

16.10 Windows-Symbole: tuico

tuico schreibt und liest `.ico`-Dateien. Die Einträge sind PNG-Bilder, die Windows seit Vista in Symbolen annimmt und die einen Alphakanal tragen. Das Modul rechnet keine Pixel um; die PNGs kommen etwa aus `tupngdraw`. Alle Befehle stehen auch als Ensemble `tclutils::tuico write|info|extract` bereit.

```
::tclutils::tuico::write   datei eintraege         ;# -> Anzahl Bytes
::tclutils::tuico::info    datei                   ;# -> Dicts
::tclutils::tuico::extract datei groesse ?ziel?    ;# -> PNG-Bytes
```

`eintraege` ist eine Liste von Paaren `{groesse png}`. Die Größe ist die Kantenlänge 1 bis 256, das PNG muss genau so groß und quadratisch sein. `write` prüft alle Einträge, bevor es schreibt; ein Fehler lässt keine halbe Datei zurück. Die Einträge stehen in der gegebenen Reihenfolge, je mit 1 Ebene und 32 Bit; 256 steht im Verzeichnis als

1. `info` gibt je Eintrag `width`, `height`, `bpp`, `format` (`png`

oder `bmp`), `offset` und `length`, mit 256 statt 0. `extract` sucht den ersten Eintrag mit dieser Breite und schreibt ihn auf Wunsch zusätzlich nach `ziel`. BMP-Einträge erkennt `info`, schreiben kann das Modul sie nicht.

```
package require tclutils::tuico
package require tclutils::tupngdraw
set entries {}
foreach s {32 16} {
    set p [::tclutils::tupngdraw::new -width $s -height $s \
        -background transparent]
    $p setfill {30 144 255}
    $p circle [expr {$s/2}] [expr {$s/2}] [expr {$s/2 - 1}] \
        -fill 1 -outline 0
    lappend entries [list $s [$p data]]
    $p destroy
}
::tclutils::tuico::write app.ico $entries    ;# -> 752
lmap e [::tclutils::tuico::info app.ico] {dict get $e width}
                                             ;# -> 32 16
dict get [lindex [::tclutils::tuico::info app.ico] 1] format
                                             ;# -> png
string length [::tclutils::tuico::extract app.ico 16 x16.png]
                                             ;# -> 247
```

-errorcode	**Ursache**
{TCLUTILS TUICO NOENTRIES}	leere Liste
{TCLUTILS TUICO TOOMANY}	mehr als 65535 Einträge
{TCLUTILS TUICO BADENTRY}	Eintrag ist kein Paar
{TCLUTILS TUICO BADSIZE}	Größe nicht 1..256
{TCLUTILS TUICO BADPNG}	Inhalt ist kein PNG
{TCLUTILS TUICO SIZEMISMATCH}	PNG-Größe passt nicht zur Angabe
{TCLUTILS TUICO BADFILE}	Datei zu kurz oder kein .ico
{TCLUTILS TUICO TRUNCATED}	Verzeichniseintrag abgeschnitten

-errorcode	Ursache
{TCLUTILS TUICO NOSUCHSIZE}	extract findet die Größe nicht
{TCLUTILS TUICO READFAILED}, WRITEFAILED	Datei nicht zu öffnen

`info` liest alle Felder des Verzeichnisses ohne Vorzeichen, wie das Format sie festlegt. Kündigt der Kopf mehr Einträge an, als die Datei enthält, wirft `info TRUNCATED`, auch bei mehr als 32767.

16.11 Ähnliche Bilder: tudhash

tudhash rechnet einen Differenz-Hash (dHash) mit 64 Bit. Ähnliche Bilder bekommen ähnliche Hashes; der Abstand ist die Zahl der verschiedenen Bits. So fallen Bilder auf, die verkleinert, neu gespeichert oder leicht verändert sind und deren Bytes sich deshalb unterscheiden.

```
::tclutils::tudhash::fromGray b h grauwerte    ;# -> 16 Hexzeichen
::tclutils::tudhash::fromRGB  b h rgbwerte     ;# -> 16 Hexzeichen
::tclutils::tudhash::distance hexA hexB        ;# -> 0..64
::tclutils::tudhash::similar  hexA hexB ?max?  ;# -> 0 | 1
```

`grauwerte` sind b × h Zahlen 0..255, `rgbwerte` b × h × 3, jeweils zeilenweise. `fromRGB` rechnet Grau als `(77 R + 150 G + 29 B) >> 8`. Das Bild wird auf 9 × 8 Felder gemittelt (Mittelwert über jedes Feld, ganzzahlig); dann vergleicht jedes Feld seinen rechten Nachbarn, Bit 1 heißt "rechts heller". `distance` nimmt Groß- und Kleinbuchstaben. `similar` ist `distance <= max`, Vorgabe 10. Die Verkleinerung ist ein einfacher Kastenfilter; Hashes anderer Werkzeuge mit derselben Bitordnung, aber anderem Filter, können um einige Bits abweichen.

```
package require tclutils::tudhash
set g {}
for {set y 0} {$y < 8} {incr y} {
    for {set x 0} {$x < 9} {incr x} { lappend g [expr {$x * 20}] }
}
::tclutils::tudhash::fromGray 9 8 $g         ;# -> ffffffffffffffff
set g2 [lmap v $g {expr {255 - $v}}]
::tclutils::tudhash::fromGray 9 8 $g2        ;# -> 0000000000000000
::tclutils::tudhash::distance FFFF000000000000 ffff000000000001
                                             ;# -> 1
::tclutils::tudhash::similar ffffffffffffffff fffffffffffffff0
                                             ;# -> 1
::tclutils::tudhash::similar ffffffffffffffff fffffffffffffff0 3
                                             ;# -> 0
```

Für PNG-Dateien liefert `tupng` die Pixel ohne Tk. Das folgende Beispiel zeichnet dasselbe Motiv in zwei Größen und einmal verschoben und vergleicht die Hashes.

```
package require tclutils::tudhash
package require tclutils::tupngdraw
proc pngHash {png} {
    set d [::tclutils::tupng::decode $png]
```

```
    binary scan [dict get $d rgba] cu* px
    set rgb {}
    foreach {r g b a} $px { lappend rgb $r $g $b }
    ::tclutils::tudhash::fromRGB \
        [dict get $d width] [dict get $d height] $rgb
}
proc bild {w h dx} {
    set p [::tclutils::tupngdraw::new -width $w -height $h]
    $p setfill {0 0 160}
    $p circle [expr {$w/3 + $dx}] [expr {$h/2}] [expr {$h/3}] \
        -fill 1 -outline 0
    $p rect [expr {$w*2/3}] [expr {$h/4}] [expr {$w-4}] \
        [expr {$h*3/4}] -fill 1 -fillcolor orange -outline 0
    set png [$p data]
    $p destroy
    return $png
}
set a [pngHash [bild 90 60 0]]              ;# -> 0018191919191900
set b [pngHash [bild 45 30 0]]              ;# -> 0018191909191900
set c [pngHash [bild 90 60 25]]             ;# -> 0006050505050700
::tclutils::tudhash::distance $a $b         ;# -> 1
::tclutils::tudhash::distance $a $c         ;# -> 20
```

Die halbe Größe liegt einen Bit-Abstand entfernt, die Verschiebung zwanzig. Ein Bild mit 256 × 256 Pixeln braucht für `fromRGB` gemessen 0,018 s unter Tcl 9.0.4 und 0,017 s unter 8.6.14 (Xeon, 2,8 GHz).

-errorcode	Ursache
{TCLUTILS TUDHASH DIM}	Breite oder Höhe keine ganze Zahl ab 1
{TCLUTILS TUDHASH DATA}	Liste hat die falsche Länge
{TCLUTILS TUDHASH HASH}	Hash nicht 16 Hexzeichen

Beide Befehle prüfen Breite und Höhe zuerst, `a` als Breite wirft also `DIM`. Die Werte selbst prüft keiner der beiden Befehle auf 0..255.

Nächstes Kapitel: Provider

17 Provider

17.1 Eine API, mehrere Backends

tuprovider beschreibt Dateien und Ordner einheitlich: `list`, `stat`, `get`, `put`, `mkdir`, `move`, `delete`, … — je nach Capabilities (`caps`).

Backend	Paket	typisch
lokal	tclutils::tuprovider	Dateisystem

Backend	Paket	typisch
ZIP	tuprovider::zip	Archiv nur lesen
DAV	tuprovider::dav	WebDAV
FTP	tuprovider::ftp	braucht tcllib-ftp beim open
SFTP	tuprovider::sftp	eigenes require, OpenSSH-sftp

17.2 Caps pro Schema

`caps` liefert eine Liste der Operationen, die das Backend wirklich kann. Die Tabelle gilt für 0.69.0 (gleich seit 0.64.0):

Schema	caps
local	copy delete get list mkdir move put stat
zip	get list stat
dav	delete get list put stat
ftp	delete get list mkdir move put stat
sftp	delete get list mkdir move put stat

Verbindlich ist die Tabelle in `docs/tuprovider.md`; der Test `tests/tuprovider-caps.test` liest genau diese Zeilen. ZIP ist also schreibgeschützt. WebDAV kennt weder `mkdir` noch `copy`/`move`: der DAV-Provider leitet nur an `tudav` weiter, und `tudav` hat kein öffentliches MKCOL/MOVE/COPY. Kopieren über Provider-Grenzen geht in der App mit `get` + `put`.

```
package require tclutils::tuprovider
set p [::tclutils::tuprovider::open local]
lsort [$p caps]      ;# -> copy delete get list mkdir move put stat
```

Ein unbekanntes Schema bricht ohne `-errorcode` ab. Die Meldung nennt die Schemas, die geladen sind: nur mit `tuprovider` heißt sie `unknown scheme 'unbekannt' (have: local)`. ZIP und DAV erscheinen darin, sobald ihr Paket geladen ist.

17.2.1 Eintrag

`list` und `stat` liefern Dicts. Darin stehen mindestens `name`, `path`, `type` (`file` oder `dir`), `size` und `mtime`. `path` geht unverändert wieder in `list`, `stat` und `get`. `get` liefert die Bytes, ohne Kodierung. Wer Text will, wandelt selbst.

```
$p put [file join $dir a.txt] eins
dict get [$p stat [file join $dir a.txt]] size    ;# -> 4
dict get [$p stat [file join $dir a.txt]] type    ;# -> file
$p get [file join $dir a.txt]                      ;# -> eins
```

`put` des lokalen Providers schreibt über eine temporäre Datei; scheitert es, bleibt die alte Datei (0.3). `mtime` kann leer sein. Beim ZIP-Provider ist der Mitgliedspfad `/notiz.txt`, die Zeit bleibt leer, `size` ist die Bytezahl.

17.2.2 Schreiben, das die caps verbieten

ZIP hat `get`, `list` und `stat`. `put` darauf antwortet ohne `-errorcode`: `provider ::tclutils::tuprovider::Zip does not support 'put'`. Kopieren von einem Provider zum anderen ist `get` und danach `put` auf dem Ziel, nicht `copy`. `copy` bleibt im selben Provider.

`head` ist keine Cap. Die Basisklasse liest per `get` und kürzt. SFTP überträgt dabei die ganze Datei. Der lokale Provider liest nur die verlangten Bytes.

17.3 Apps darüber

tkutils-Widgets (`tkufilelist`, `tkufiletree`) und ctrlutils-Controller (`cufileops`) sprechen nur die Provider-API. Rezepte und Dateiblick: Buch *Utils-Stack*. HOWTOs im Lib-Baum: `docs/guide/howto-*.md`.

17.4 Zusammenfassung

- Provider = einheitliche Speicher-API.
- Caps steuern, was die UI anbieten darf. ZIP ist nur lesend, und `put` darauf hat keinen `-errorcode`. DAV hat kein `mkdir`, `copy`, `move`.

Nächstes Kapitel: Bausteine für Anwendungen

18 Bausteine für Anwendungen

Siebzehn kleine Module, die eine Anwendung um ihre eigentliche Aufgabe herum braucht: Protokoll und Ereignisse, Prüfen und Rechnen, Umgebung und Auslieferung, Ausgabe im Terminal und auf der Seite. Alle sind reines Tcl ohne Tk und liegen im Umbrella `tclutils`. Plattformabhängig sind `tuappinfo`, `tusysinfo`, `tudeploy`, `tuexe`, `tuopen` und `tuterm`; `tuzipfs` braucht für die meisten Befehle Tcl 9.

Gruppe	Module
Protokoll und Ereignisse	tulog, tuevent, turegistry
Prüfen und Rechnen	tuvalidate, tulist, tumath, tunumany
Umgebung und Auslieferung	tuappinfo, tusysinfo, tuedid, tudeploy, tuexe, tuopen, tuzipfs
Ausgabe	tuterm, tupagespec, tulayout

Unbekannte Optionen und Optionen ohne Wert melden die Module unterschiedlich: über `::tclutils::common::parseOptions` mit `{TCLUTILS COMMON OPTION <opt>}` oder mit einem eigenen Code. Die Abschnitte nennen den Code jeweils.

18.1 tulog

tulog ist ein Protokoll mit vier Stufen. Ein Logger ist ein Befehl; die Stufen `info` und `error` sind Unterbefehle und verdecken daher keine Tcl-Befehle.

```
::tclutils::tulog::new ?option wert ...?        ;# -> Logger-Befehl
$log debug|info|warn|error nachricht
$log log stufe nachricht
$log setLevel stufe
$log level
$log destroy
::tclutils::tulog::assert ausdruck ?nachricht?
```

Option	Vorgabe	Wirkung
-channel	stderr	Kanal für die Ausgabe
-level	info	niedrigste Stufe, die ausgegeben wird
-name	""	Name hinter der Stufe
-timestamp	1	Zeitstempel %Y-%m-%dT%H:%M:%S, lokale Zeit

Die Stufen steigen in der Reihenfolge `debug`, `info`, `warn`, `error`. Eine Nachricht erscheint, wenn ihre Stufe mindestens die des Loggers ist. Jede Zeile hat die Form `ZEIT [STUFE] name: nachricht`; ohne Zeitstempel fällt `ZEIT` weg, ohne Namen steht `[STUFE]: nachricht`. Nach jeder Zeile wird der Kanal geleert.

new liefert den Befehlsnamen `::tclutils::tulog::logger<n>`. Die Unterbefehle zum Schreiben, `setLevel` und `destroy` liefern `""`, `level` die aktuelle Stufe. Nach `destroy` gibt es den Befehl nicht mehr.

```
package require tclutils::tulog
set log [::tclutils::tulog::new -name app -channel stdout \
    -timestamp 0]
$log info "Start"
$log debug "unterdrückt"
$log warn "Platte fast voll"
$log setLevel debug
$log level                                  ;# -> debug
$log destroy
```

Ausgabe auf `stdout`:

```
[INFO] app: Start
[WARN] app: Platte fast voll
```

`assert` wertet den Ausdruck mit `expr` im Kontext des Aufrufers aus. Ist er falsch, bricht es mit `{TCLUTILS TULOG ASSERT}` ab; die Meldung ist die übergebene Nachricht oder `assertion failed: <ausdruck>`.

```
set n 3
::tclutils::tulog::assert {$n > 2}          ;# -> (leer)
catch {::tclutils::tulog::assert {$n > 5}} m
set m                          ;# -> assertion failed: $n > 5
```

Fall	Code
unbekannte Stufe bei -level, log, setLevel	{TCLUTILS TULOG LEVEL}
-timestamp nicht boolesch	{TCLUTILS COMMON BOOLEAN -timestamp}
unbekannte Option bei new	{TCLUTILS COMMON OPTION <opt>}
unbekannter Unterbefehl, falsche Zahl Argumente	ohne -errorcode
Assertion falsch	{TCLUTILS TULOG ASSERT}

new prüft alle Optionen, bevor es den Logger anlegt; nach einem Fehler bleibt kein Logger zurück. `-channel` prüft `new` nicht. Ein Kanal, den es nicht gibt, fällt erst bei der ersten Ausgabe auf: `can not find channel named "…"` mit `{TCL LOOKUP CHANNEL …}`.

18.2 tuevent

tuevent ist ein Ereignisbus nach dem Muster Veröffentlichen und Abonnieren. Jeder Bus ist ein Token; Abonnenten sind Befehlspräfixe.

Befehl	Wirkung	Rückgabe
create	neuer Bus	Token tuevent<n>
subscribe bus ereignis handler	Handler anmelden; derselbe Handler nur einmal	""
unsubscribe bus ereignis handler	Handler abmelden; fehlt er, geschieht nichts	""
emit bus ereignis ?arg ...?	Handler aufrufen	Zahl der aufgerufenen Handler
handlers bus ereignis	angemeldete Handler	Liste in Anmeldereihenfolge
events bus	Ereignisse mit Handlern	sortierte Liste
clear bus ?ereignis?	Handler eines Ereignisses oder aller löschen	""
destroy bus	Bus mit allen Anmeldungen löschen	""

`emit` ruft jeden Handler auf globaler Ebene auf, die Argumente hinten angehängt, in der Reihenfolge der Anmeldung. Ein Fehler in einem Handler bricht `emit` ab und geht an den Aufrufer; die folgenden Handler laufen dann nicht. Ein Token, das `create` nicht geliefert hat oder das `destroy` gelöscht hat, ergibt `{TCLUTILS TUEVENT BUS}` mit `unknown event bus: "<token>"`.

```
package require tclutils::tuevent
set bus [::tclutils::tuevent::create]
set log {}
proc merke {id} { lappend ::log $id }
```

```
::tclutils::tuevent::subscribe $bus saved merke
::tclutils::tuevent::subscribe $bus saved merke
::tclutils::tuevent::emit $bus saved 42      ;# -> 1
set log                                      ;# -> 42
::tclutils::tuevent::emit $bus closed        ;# -> 0
::tclutils::tuevent::events $bus             ;# -> saved
::tclutils::tuevent::destroy $bus
```

18.3 turegistry

turegistry legt Werte unter Schlüsseln ab, etwa Datenbankverbindungen oder Dienste, die mehrere Teile einer Anwendung brauchen. Jede Registry ist ein Token.

Befehl	Wirkung	Rückgabe
create	neue Registry	Token turegistry<n>
put reg schlüssel wert	Wert ablegen, alten ersetzen	der Wert
get reg schlüssel ?vorgabe?	Wert lesen	Wert oder Vorgabe
has reg schlüssel	Schlüssel vorhanden?	1 oder 0
keys reg ?muster?	Schlüssel, mit string match gefiltert	sortierte Liste
remove reg schlüssel	Schlüssel löschen	1, wenn er da war, sonst 0
destroy reg	Registry löschen	""

`get` ohne Vorgabe bricht bei fehlendem Schlüssel mit `{TCLUTILS TUREGISTRY KEY}` ab, Meldung `registry key not set: "<schlüssel>"`. Eine leere Vorgabe `{}` zählt als Vorgabe. Ein unbekanntes Token ergibt `{TCLUTILS TUREGISTRY REG}`.

```
package require tclutils::turegistry
set reg [::tclutils::turegistry::create]
::tclutils::turegistry::put $reg db conn1    ;# -> conn1
::tclutils::turegistry::put $reg db.timeout 30
::tclutils::turegistry::get $reg db          ;# -> conn1
::tclutils::turegistry::get $reg cache leer  ;# -> leer
::tclutils::turegistry::keys $reg db.*       ;# -> db.timeout
::tclutils::turegistry::remove $reg db       ;# -> 1
::tclutils::turegistry::has $reg db          ;# -> 0
::tclutils::turegistry::destroy $reg
```

Mehr als ein Vorgabewert bei `get` ist ein Aufruffehler ohne `-errorcode`, aber nur, wenn der Schlüssel fehlt. Ist er gesetzt, liefert `get` den Wert und übergeht die überzähligen Argumente.

18.4 tuvalidate

tuvalidate enthält Prüfungen, die `1` oder `0` liefern. Am geprüften Wert scheitern sie nicht; Fehler gibt es nur bei falschen Argumenten. Die Prüfungen für E-Mail und URL erkennen die übliche

Form, sie sind keine vollständigen RFC-Prüfungen.

Befehl	1, wenn
email s	lokal@domain.tld; lokal aus A-Z a-z 0-9 . _ % + -, Domain aus A-Z a-z 0-9 . -, Endung mindestens zwei ASCII-Buchstaben
url s	beginnt mit http://oder https://, ohne Leerraum, das erste Zeichen danach ist nicht /$. ? #
ipv4 s	vier Gruppen aus 1–3 Ziffern, jede höchstens 255, ohne führende Null
port s	nur Dezimalziffern, Wert 1 bis 65535
alpha s	nur A-Z a-z, nicht leer
alnum s	nur A-Z a-z 0-9, nicht leer
numeric s	string is double -strict
integer s	string is integer -strict
length s min max	min <= [string length s] <= max
pattern s re	regexp re s trifft
inList s liste	s ist Element der Liste (genauer Vergleich)

```
package require tclutils::tuvalidate
::tclutils::tuvalidate::email a@b.com             ;# -> 1
::tclutils::tuvalidate::email "jörg@b.de"    ;# -> 0
::tclutils::tuvalidate::url ftp://tcl.tk/         ;# -> 0
::tclutils::tuvalidate::ipv4 10.0.0.256           ;# -> 0
::tclutils::tuvalidate::ipv4 010.0.0.1            ;# -> 0
::tclutils::tuvalidate::ipv4 0.0.0.0              ;# -> 1
::tclutils::tuvalidate::port 0x50                 ;# -> 0
::tclutils::tuvalidate::port 080                  ;# -> 1
::tclutils::tuvalidate::numeric 1,5               ;# -> 0
::tclutils::tuvalidate::length abc 3 20           ;# -> 1
::tclutils::tuvalidate::pattern 2024-03-15 {^\d{4}-\d\d-\d\d$}
                                                  ;# -> 1
```

`alpha` und `alnum` kennen nur ASCII; für Unicode-Buchstaben gilt `string is alpha`. `ipv4` prüft streng wie `inet_pton`: Eine Gruppe mit führender Null (`010`) ergibt 0, die Gruppe `0` allein ist gültig. `port` nimmt nur die ASCII-Ziffern `0` bis `9`, ohne Vorzeichen, ohne Leerraum und ohne `0x`; `0x50` und `" 80"` ergeben 0. Führende Nullen sind erlaubt und zählen dezimal: `080` ist Port 80, unter Tcl 8.6 und 9 gleich.

Fall	Code
length: min oder max keine ganze Zahl	{TCLUTILS TUVALIDATE ARG}
pattern: ungültiger regulärer Ausdruck	{TCLUTILS TUVALIDATE REGEX}

18.5 tulist

tulist ergänzt die Listenbefehle des Kerns. `lreverse`, `lsort -unique`, `lsearch` und `lmap` bleiben beim Kern. `map`, `filter`, `reduce`, `all` und `any` nehmen einen Befehlspräfix, keinen Skriptrumpf; er läuft auf globaler Ebene mit dem Element hinten angehängt. Lokale Variablen des Aufrufers sind darin nicht sichtbar; `apply` mit Parametern löst das.

Befehl	Wirkung
unique liste	Doppelte entfernen, Reihenfolge des ersten Auftretens bleibt
flatten liste ?tiefe?	Unterlisten auflösen, Vorgabe für tiefe ist 1; tiefe <= 0 liefert die Liste unverändert
chunk liste größe	Stücke zu höchstens größe Elementen
zip liste ...	Tupel, endet mit der kürzesten Liste; ohne Liste {}
sum liste	Summe, leere Liste 0
avg liste	Mittelwert als Gleitkommazahl
min liste, max liste	kleinstes, größtes Element nach < und >
reduce liste start cmd	Linksfaltung: acc = cmd acc element
map liste cmd	Ergebnisse von cmd element
filter liste cmd	Elemente, für die cmd element wahr ist
all liste cmd, any liste cmd	1/0; leere Liste: all 1, any 0
take liste n, drop liste n	erste n Elemente bzw. der Rest; n <= 0 ergibt {} bzw. die ganze Liste

```
package require tclutils::tulist
::tclutils::tulist::unique {a b a c b}            ;# -> a b c
::tclutils::tulist::flatten {{1 2} {3 {4 5}}}     ;# -> 1 2 3 {4 5}
::tclutils::tulist::flatten {{1 2} {3 {4 5}}} 2   ;# -> 1 2 3 4 5
::tclutils::tulist::chunk {1 2 3 4 5} 2           ;# -> {1 2} {3 4} 5
::tclutils::tulist::zip {a b c} {1 2}             ;# -> {a 1} {b 2}
::tclutils::tulist::avg {1 2 3 4}                 ;# -> 2.5
::tclutils::tulist::max {b a c}                   ;# -> c
::tclutils::tulist::reduce {1 2 3 4} 0 ::tcl::mathop::+
                                                  ;# -> 10
::tclutils::tulist::map {1 2 3} {apply {{x} {expr {$x*$x}}}}
                                                  ;# -> 1 4 9
::tclutils::tulist::filter {1 2 3 4} {apply {{x} {expr {$x%2==0}}}}
                                                  ;# -> 2 4
::tclutils::tulist::drop {a b c d} 2              ;# -> c d
```

`min` und `max` vergleichen Zahlen numerisch, sonst als Text. `sum` und `avg` rechnen mit `expr`;

ein nicht numerisches Element bricht mit `{ARITH DOMAIN {non-numeric string}}` ab. Ebenso ein nicht numerisches `n` bei `take` und `drop`.

Fall	Code
chunk: Größe keine positive ganze Zahl	{TCLUTILS TULIST SIZE}
avg, min, max einer leeren Liste	{TCLUTILS TULIST EMPTY}

18.6 tumath

tumath ergänzt `expr` um Rechnungen, die dort fehlen. `round`, `ceil`, `floor` und `abs` bleiben bei `expr`.

Befehl	Rückgabe	Fehler
clamp x lo hi	x, auf lo bis hi begrenzt	lo > hi: ARG
inRange x lo hi	1, wenn lo <= x <= hi	–
percent teil ganzes	teil /ganzes * 100 als Gleitkommazahl	ganzes == 0: DIVZERO
sign x	-1, 0 oder 1	–
gcd a b	größter gemeinsamer Teiler, nicht negativ; gcd 0 0 ist 0	keine ganzen Zahlen: ARG
lcm a b	kleinstes gemeinsames Vielfaches; 0, wenn ein Argument 0 ist	über gcd
factorial n	n!, mit Tcls großen Ganzzahlen	n negativ oder keine ganze Zahl: ARG
roundTo x stellen	round(x * 10**stellen) /10**stellen, Gleitkommazahl	stellen negativ oder keine ganze Zahl: ARG

Die Codes lauten `{TCLUTILS TUMATH ARG}` und `{TCLUTILS TUMATH DIVZERO}`.

```
package require tclutils::tumath
::tclutils::tumath::clamp 12 0 10            ;# -> 10
::tclutils::tumath::percent 30 200           ;# -> 15.0
::tclutils::tumath::gcd -24 36               ;# -> 12
::tclutils::tumath::lcm 4 6                  ;# -> 12
::tclutils::tumath::factorial 25   ;# -> 15511210043330985984000000
::tclutils::tumath::roundTo 3.14159 2        ;# -> 3.14
::tclutils::tumath::roundTo 2.5 0            ;# -> 3.0
::tclutils::tumath::roundTo 1.005 2          ;# -> 1.0
```

`roundTo` rechnet mit Gleitkommazahlen. `1.005 * 100` ergibt `100.49999…`, daher `1.0` statt `1.01`. Für Beträge mit fester Stellenzahl rundet `tunum::format` auf den Dezimalziffern, siehe Zeichen, Zahlen, Datum und Pfade.

18.7 tunumany

tunumany ist ein Eingang für Zahlen in zwei Schreibweisen. Beträge mit Tausendertrennung und Währung gehen an `tunum::parse`, Größen mit SI- oder IEC-Einheit (`1.5K`, `2Mi`) an `tunumfmt::fromHuman`. Die beiden Parser selbst bleiben getrennt; `tunum` beschreibt das Kapitel Zeichen, Zahlen, Datum und Pfade.

```
::tclutils::tunumany::parse text ?-default wert? ?-prefer weg?
```

Option	Vorgabe	Wirkung
-default	""	Rückgabe, wenn nichts lesbar ist
-prefer	auto	auto, si oder locale: welcher Parser zuerst gefragt wird

Mit `auto` wählt `parse` den SI-Weg, wenn der getrimmte Text eine Zahl mit Punkt oder Komma als Dezimalzeichen, optional Leerzeichen und genau einen der Buchstaben `K M G T P E Z Y` (auch klein) mit optionalem `i` ist. Alles andere geht an `tunum::parse` mit Locale `auto`. Liefert der gewählte Parser nichts, fragt `parse` den anderen. Beide Module lädt `parse` erst beim Aufruf; fehlt eines, fällt dieser Weg aus.

```
package require tclutils::tunumany
::tclutils::tunumany::parse "1.234,56"       ;# -> 1234.56
::tclutils::tunumany::parse "1,234.56"       ;# -> 1234.56
::tclutils::tunumany::parse "12 €"      ;# -> 12.0
::tclutils::tunumany::parse 42               ;# -> 42.0
::tclutils::tunumany::parse 1.5K             ;# -> 1500
::tclutils::tunumany::parse 2Mi              ;# -> 2097152
::tclutils::tunumany::parse 2KiB             ;# -> 2048
::tclutils::tunumany::parse "3 G"            ;# -> 3000000000
::tclutils::tunumany::parse 1.5m             ;# -> 1500000
::tclutils::tunumany::parse 1,5K             ;# -> 1500
::tclutils::tunumany::parse 1,500K -default x ;# -> x
::tclutils::tunumany::parse x -default 0     ;# -> 0
```

Kleinbuchstaben stehen für dieselben Einheiten: `m` ist Mega, nicht Milli. Ein Dezimalkomma vor der Einheit zählt: `1,5K` ist 1500 (bis tclutils 0.68.0 der leere String). Stehen genau drei Ziffern hinter dem Komma (`1,500K`), könnte es auch Tausender trennen; diese Form liest `parse` nicht und gibt die Vorgabe zurück. Weil die Vorgabe `""` auch für Unlesbares steht, hilft ein `-default` , das keine Zahl sein kann, die beiden Fälle zu trennen. Ganze Zahlen ohne Einheit laufen über `tunum` und kommen als Gleitkommazahl zurück. Ein `i` direkt nach dem Einheitenbuchstaben macht die Einheit zur IEC-Einheit, auch mit nachgestelltem `B`: `2KiB` ergibt 2048, `2KB` ergibt 2000.

Fall	Code
unbekannte Option, Option ohne Wert (parse 1.5 -default)	{TCLUTILS TUNUMANY OPTION}
-prefer nicht auto, si, locale	{TCLUTILS TUNUMANY PREFER}

18.8 tuappinfo

tuappinfo sammelt Angaben über Tcl, Programm, Plattform, Umgebung, Suchpfade und geladene Pakete in einem Textbericht, etwa für einen Info-Dialog oder eine Fehlermeldung an den Entwickler. Die Anzeige übernimmt der Aufrufer.

```
::tclutils::tuappinfo::trackTm datei
::tclutils::tuappinfo::buildReport ?-title text? ?-anonymize 0|1?
::tclutils::tuappinfo::writeLog datei ?-title text? ?-anonymize 0|1?
```

Option	Vorgabe	Wirkung
-title	Application Information	Überschrift ==== <titel> ====
-anonymize	0	Benutzer-, Rechner-und Pfadangaben verdecken

`trackTm` merkt eine Moduldatei für den letzten Abschnitt vor und liefert die Liste aller gemerkten Dateien. `buildReport` liefert den Bericht als einen String, `writeLog` schreibt ihn als UTF-8 über eine temporäre Datei (`common::replaceFile`) und liefert den Dateinamen; scheitert das Schreiben, bleibt ein altes Protokoll stehen. Bis 0.2 schrieb es in der Systemkodierung.

Der Bericht hat diese Abschnitte in dieser Reihenfolge:

Abschnitt	Inhalt
Kopf	Titel, Timestamp:, PID:
--- Tcl / Tk ---	info patchlevel, tk_patchLevel (nur mit Tk; bis 0.2 tk_version), info nameofexecutable, Betriebssystem und Version, Maschine, Host:
--- Environment ---	USER, USERNAME, HOME, TEMP, TMP, PATH, soweit gesetzt
--- auto_path ---	ein Eintrag pro Zeile
--- Loaded Packages ---	jedes Paket mit package provide-Version
--- tm::path ---	tcl::tm::path list
--- Loaded TM Modules ---	die mit trackTm gemerkten Dateien, sonst (none tracked)

Plattformabhängig laut Code: `Host:` steht nur da, wenn die Umgebungsvariable `COMPUTERNAME` gesetzt ist; Windows setzt sie, unter Linux und macOS fehlt die Zeile in der Regel. `USERNAME`, `TEMP` und `TMP` sind typische Windows-Variablen, `USER` und `HOME` typische unter Linux und macOS.

`-anonymize 1` ersetzt:

Angabe	Ersatz
USER, USERNAME	<user-xxxxxxxx>
Host	<host-xxxxxxxx>

Angabe	Ersatz
HOME	<HOME>
TEMP, TMP	<TEMP>
PATH	<anonymized>
Pfade unter dem Heimatverzeichnis	<HOME>/…
alle anderen Pfade	<path>/<letzter Teil>
gemerkte Moduldateien	<tm-xxxxxxxx>.tm

xxxxxxxx ist ein gleichbleibender, nicht kryptografischer Hash aus acht Hexziffern; gleiche Namen ergeben gleiche Werte. Die Pfadregel gilt für `Executable` und für jeden Eintrag in `auto_path` und `tm::path`. Heimatverzeichnis ist `$HOME`, ohne `HOME` `$USERPROFILE`; ein Pfad gilt als darunter, wenn er gleich ist oder mit `<heim>/` beginnt. Aus `/usr/share/tcltk` wird `<path>/tcltk`, aus `$HOME/lib/tm` wird `<HOME>/lib/tm`. Trennzeichen erscheinen als `/`; unter Windows vergleicht `tuappinfo` das Heimatverzeichnis ohne Unterschied von Groß/Klein. Unverändert bleiben `Platform`, `Machine`, die Versionen und die Paketliste.

```
package require tclutils::tuappinfo
::tclutils::tuappinfo::trackTm /opt/app/lib/tm/foo-1.0.tm
set r [::tclutils::tuappinfo::buildReport -title "MeineApp 1.0"]
lindex [split $r \n] 0                ;# -> ==== MeineApp 1.0 ====
string match "*(none tracked)*" $r    ;# -> 0
```

Die übrigen Zeilen hängen vom Rechner ab: unter Linux etwa `Tcl version: 9.0.4`, `Platform: Linux <kernel>`, `HOME: /home/<name>`.

Fall	Code
unbekannte Option, Option ohne Wert	{TCLUTILS TUAPPINFO OPTION}
-anonymize nicht boolesch	{TCLUTILS TUAPPINFO VALUE}

`writeLog` prüft die Optionen über `buildReport`, bevor es die Datei anlegt.

18.9 tusysinfo

tusysinfo beschreibt den Rechner: Betriebssystem, Prozessor, Arbeitsspeicher, Modell, Festplatten und Monitore, unter Linux, macOS und Windows. `tuappinfo` beschreibt die Tcl-Laufzeit, `tusysinfo` die Maschine darunter.

```
::tclutils::tusysinfo::collect ?option wert ...?        ;# -> Dict
::tclutils::tusysinfo::report  info ?-title text? ?-anonymize 0|1?
::tclutils::tusysinfo::toJson  info ?-indent n?
```

Das Modul liest zuerst Dateien und die Registry und ruft ein Programm nur dort auf, wo nichts anderes die Angaben hat. Fehlt ein Programm, bleiben seine Felder leer; `collect` scheitert daran nicht.

System	Quellen
Linux	/etc/os-release, /proc/cpuinfo, /proc/meminfo, /sys/class/dmi/id, /sys/block, /sys/class/drm/*/edid; kein Programm, kein root
macOS	sw_vers, uname, sysctl, system_profiler -json (einige Sekunden)
Windows	Registry (Paket registry) für System, CPU, Modell und die EDID der Monitore; ein PowerShell-Aufruf (CIM) für Speicher, Festplatten und die aktiven Monitore

Option	Vorgabe	Wirkung
-sections	alle	Auswahl aus os cpu memory machine disks displays
-serials	0	1: Seriennummern von Rechner, Festplatten, Monitoren
-commands	1	0: kein Programm aufrufen
-timeout	10000	Grenze in ms je Programmaufruf
-platform	dieser Rechner	linux, macos, windows
-root	leer	Linux: /etc, /proc, /sys unter diesem Ordner lesen (Tests)

`collect` liefert ein Dict mit `platform`, `host` und je Abschnitt einem Dict (`os`, `cpu`, `memory`, `machine`) oder einer Liste von Dicts (`disks`, `displays`). Größen stehen in Byte. Ein Monitor trägt die Felder aus seiner EDID (Abschnitt `tuedid`), den Anschluss und `present` (1 angeschlossen, 0 nicht angeschlossen). Unter Windows gilt ein Monitor ohne Status als nicht angeschlossen. Kommt von PowerShell keine oder eine unlesbare Antwort, steht eine Liste `warnings` im Dict und `report` zeigt einen Abschnitt "Warnings"; Speicher, Festplatten und Monitorstatus fehlen dann, statt leer zu erscheinen.

```
package require tclutils::tusysinfo
set i [::tclutils::tusysinfo::collect -sections {os memory}]
lsort [dict keys $i]                  ;# -> host memory os platform
dict get $i os id                     ;# -> debian (je nach Rechner)
puts [::tclutils::tusysinfo::report $i -anonymize 1]
```

`report` zeigt Festplatten in GB wie auf dem Etikett und den Speicher in GiB, `toJson` schreibt Zahlen als Zahlen. Die Kommandozeile `bin/tusysinfo.tcl` gibt dasselbe aus, mit `--json`, `--serials`, `--anonymize` und `--sections`.

Seriennummern, Rechnernamen und MAC-Adressen machen einen Rechner und an einem Büroarbeitsplatz seinen Benutzer erkennbar. Seriennummern kommen nur mit `-serials 1`, MAC-Adressen gar nicht; `report -anonymize 1` verdeckt Rechnername und Seriennummern.

Weitere Eigenheiten:

- Linux: die Seriennummer des Rechners (`product_serial`) darf nur root lesen. Virtuelle Maschinen haben oft keine DMI-Daten und keinen Monitor.
- Windows: Die Registry behält jeden Monitor, der einmal angeschlossen war, oft denselben

unter mehreren Einträgen (anderer Anschluss, anderer Treiber). `tusysinfo` führt jede EDID einmal und setzt `present` nach `Get-PnpDevice -Class Monitor`: 1 bei Status OK, sonst 0, leer ohne diese Liste. In einer Remotedesktop-Sitzung gelten die echten Monitore als nicht angeschlossen; `report` schreibt dann "(not connected)". `WmiMonitorID` meldet auf manchen Rechnern "Nicht unterstützt" und wird deshalb nicht benutzt. Windows 11 meldet als `ProductName` weiter "Windows 10 …"; ab Build 22000 steht Windows 11 da. SSD oder HDD kommt aus `Get-PhysicalDisk`; eine NVMe-Platte gilt als SSD.
- Platzhalter der Firmware wie "Default string" oder "To be filled by O.E.M." bleiben leer.
- macOS: `system_profiler` braucht einige Sekunden; `-sections {os cpu memory}` lässt ihn weg. Er nennt beim Monitor Hersteller, Modell, Produktcode, Woche und Jahr, aber keine Größe; `widthCm`, `heightCm` und `diagonal` bleiben 0. Je APFS-Container (`disk3`) gibt es einen Eintrag, seine Volumes stehen in `volumes`; die Größe ist die des Containers, etwas weniger als die der Platte.
- Linux: `disks` zeigt die Geräte, nicht Loop-Geräte, RAM-Disks und Device-Mapper-Volumes (LVM, LUKS); RAID (`md*`) bleibt. Debian testing/sid hat keine `VERSION_ID`; `version` kommt dann aus `/etc/debian_version`, etwa `forky/sid`.
- Geprüft auf einem Linux-PC mit zwei Monitoren (Debian sid), einem Windows-11-PC und einem Mac mini (Apple Silicon, macOS 27). Die Tests spielen aufgezeichnete Ausgaben dieser Rechner nach.

Fall	Code
unbekannte Option, Option ohne Wert	{TCLUTILS TUSYSINFO OPTION}
unbekannter Abschnitt	{TCLUTILS TUSYSINFO SECTION}
unbekannte Plattform	{TCLUTILS TUSYSINFO PLATFORM}

18.10 tuedid

tuedid liest den EDID-Block eines Monitors. Darin beschreibt sich der Monitor in 128 Byte: Hersteller, Produktcode, Seriennummer, Woche und Jahr der Herstellung, Größe in Zentimetern und die bevorzugte Auflösung; Textfelder tragen Modellnamen und eine Seriennummer als Text.

```
::tclutils::tuedid::parse    bytes       ;# -> Dict
::tclutils::tuedid::parseHex text        ;# Hexziffern, Leerzeichen
    und ":" egal
::tclutils::tuedid::vendor   kürzel      ;# DEL -> Dell
```

Die Bytes liegen unter Linux in `/sys/class/drm/card*-*/edid`, unter Windows in der Registry (`Enum\DISPLAY\…\Device Parameters`, Wert `EDID`); `xrandr --verbose` und `ioreg` unter macOS geben sie als Hex aus. `tusysinfo` liest sie selbst.

```
package require tclutils::tuedid
set fh [open /sys/class/drm/card0-DP-1/edid rb]
set d [::tclutils::tuedid::parse [read $fh]]
close $fh
dict get $d vendor                    ;# -> Dell
dict get $d diagonal                  ;# -> 27.2 (Zoll)
dict get $d preferred                 ;# -> width 1920 height 1080
    refresh 60.0
```

Schlüssel	Inhalt
valid	1 bei richtiger Prüfsumme; ein Block mit falscher Summe wird trotzdem gelesen
manufacturer, vendor	PNP-Kürzel (DEL) und Name für bekannte Kürzel
productCode, serial, serialText, name	Produktcode (Hex), Seriennummer als Zahl und als Text, Modellname
week, year	Herstellung, 0 wenn unbekannt
widthCm, heightCm, diagonal	Größe; 0 bei Beamern ohne Angabe
preferred	erste detaillierte Auflösung: width, height, refresh

Erweiterungsblöcke (CEA-861, DisplayID) zählt `extensions`; gelesen werden sie nicht. Weniger als 128 Byte werfen `{TCLUTILS TUEDID SHORT}`, ein fehlender EDID-Kopf `{TCLUTILS TUEDID HEADER}`, Text, der kein Hex ist, `{TCLUTILS TUEDID HEX}`.

18.11 tudeploy

tudeploy findet die Modulverzeichnisse und Ressourcen, die eine Anwendung neben sich mitbringt, trägt sie in `tcl::tm::path` ein und lädt Pakete daraus. Es kennt keine bestimmte Anwendung; Verzeichnisse und Pakete kommen als Argumente. Das Modul ist zugleich ein Ensemble: `::tclutils::tudeploy roots` und `::tclutils::tudeploy::roots` sind derselbe Aufruf.

```
::tclutils::tudeploy::baseDirs
::tclutils::tudeploy::platformTag
::tclutils::tudeploy::roots ?option wert ...?
::tclutils::tudeploy::addModulePaths ?option wert ...?
::tclutils::tudeploy::require pakete ?option wert ...?
::tclutils::tudeploy::resourceDirs name ?option wert ...?
::tclutils::tudeploy::sourceModule paket ?option wert ...?
```

`baseDirs` liefert die Verzeichnisse des Hauptskripts (`argv0`) und des laufenden Programms (`info nameofexecutable`), normalisiert, ohne Doppelte, nur vorhandene. `platformTag` liefert `<os><bits>`:

tcl_platform	Ergebnis
platform = windows	windows32 oder windows64
platform = unix, os = Darwin	macos32 oder macos64
platform = unix, jedes andere os	linux32 oder linux64

Die Bits kommen aus `tcl_platform(pointerSize)`. Auch BSD-Systeme erhalten damit `linux…`.

18.11.1 roots und addModulePaths

Option	Vorgabe	Wirkung
-base	[baseDirs]	Basisverzeichnisse
-env	{}	Namen von Umgebungsvariablen; deren Werte kommen unverändert an den Anfang
-roots	vendor {libs common} libs {lib tm}	Unterpfade je Basis, jeder als Liste von Pfadteilen
-parents	1	auch die Elternverzeichnisse der Basen durchsuchen

`roots` liefert die vorhandenen Verzeichnisse in dieser Reihenfolge: zuerst die Werte der `-env`-Variablen, die gesetzt und nicht leer sind; dann für jede Basis jeder Unterpfad aus `-roots`; mit `-parents 1` danach dasselbe für jedes Elternverzeichnis. Ein Wurzelverzeichnis enthält Paketordner: `<wurzel>/<paket>/<modul>-<version>.tm` ergibt `package require <paket>::<modul>`.

`addModulePaths` nimmt dieselben Optionen, ruft für jede Wurzel `tcl::tm::path add` auf und liefert die Wurzeln, die danach im Modulpfad stehen, neu eingetragen oder schon vorhanden. Lehnt `tcl::tm::path` ein Verzeichnis ab, weil es in einem schon eingetragenen liegt oder eines enthält, übergeht `addModulePaths` das ohne Fehler und lässt es in der Rückgabe weg.

```
package require tclutils::tudeploy
::tclutils::tudeploy::platformTag     ;# linux64, windows64, macos64 …
::tclutils::tudeploy::roots -base [list $appdir] -parents 0
```

Mit `$appdir/vendor` und `$appdir/lib/tm` vorhanden liefert `roots` die Liste `$appdir/vendor $appdir/lib/tm`.

18.11.2 require

`require pakete ?option wert ...?` trägt die Wurzeln ein und führt für jedes Paket `package require` aus.

Option	Vorgabe	Wirkung
-tmadd	1	vorher addModulePaths mit den übrigen Optionen aufrufen

Option	Vorgabe	Wirkung
-fail	0	bei fehlenden Paketen abbrechen statt 0 liefern
-base, -env, -roots, -parents	wie roots	gehen an addModulePaths

Die Rückgabe ist `1`, wenn alle Pakete geladen sind, sonst `0`. Mit `-fail 1` bricht `require` mit `{TCLUTILS TUDEPLOY REQUIRE}` ab, die Meldung nennt die fehlenden Pakete: `package(s) not found: foo::nix`. Auch mit `-tmadd 0` prüft `require` die Optionen von `roots`; `-tmadd` und `-fail` müssen boolesch sein.

```
if {![::tclutils::tudeploy::require {meinlib::kern meinlib::ui}]} {
    error "Module fehlen"
}
```

18.11.3 resourceDirs

`resourceDirs name ?option wert ...?` liefert die vorhandenen Verzeichnisse, in denen eine mitgelieferte Ressource liegen kann, etwa ein Decoder. Je Basis, mit `-parents 1` auch je Elternverzeichnis, in dieser Reihenfolge:

1. `vendor/<name>/<tag>`
2. `vendor/<name>`
3. `vendor/<tag>`
4. `vendor`
5. `bin`
6. die Basis selbst

Option	Vorgabe	Wirkung
-base, -env, -parents	wie roots	
-tag	[platformTag]	Plattformordner

`-roots` kennt `resourceDirs` nicht. Ein leerer Name ergibt `{TCLUTILS TUDEPLOY USAGE}`. Die Liste passt direkt zu `tuexe::find -dirs`:

```
set dirs [::tclutils::tudeploy::resourceDirs webp -env MYAPP_WEBP]
set exe [::tclutils::tuexe::find dwebp -dirs $dirs]
```

18.11.4 sourceModule

`sourceModule paket ?option wert ...?` lädt ein Paket, dessen `.tm` nicht an der Stelle liegt, die sein Name verlangt, zum Beispiel ein Paket `qpdf` als `vendor/qpdf/lib/qpdf-0.2.tm`. Ein eigener Eintrag `vendor/qpdf/lib` im Modulpfad scheitert, weil er unter `vendor` liegt. `sourceModule` sucht deshalb in den Verzeichnissen nach `<letzter Namensteil>-<version>.tm`, sortiert nach Version absteigend, und führt `source` auf globaler Ebene aus, bis das Paket vorhanden ist.

Option	Vorgabe	Wirkung
-dirs	{}	Verzeichnisse direkt; sonst roots mit den übrigen Optionen

Option	Vorgabe	Wirkung
-fail	0	bei Misserfolg abbrechen statt 0 liefern
-base, -env, -roots, -parents	wie roots	nur ohne -dirs

Rückgabe `1` oder `0`; mit `-fail 1` `{TCLUTILS TUDEPLOY REQUIRE}` mit `could not source module for package: <paket>`. `sourceModule` prüft zuerst die Optionen; ist das Paket danach schon geladen, liefert es sofort `1`. Dateien, deren Versionsteil keine gültige Version ist, und Dateien, deren `source` fehlschlägt, übergeht es.

```
::tclutils::tudeploy::sourceModule qpdf -env QPDF_TM \
    -roots {{vendor qpdf lib}}
```

Fall	Code
unbekannte Option, Option ohne Wert, Argument ohne -	{TCLUTILS TUDEPLOY OPTION}
-tmadd, -fail nicht boolesch	{TCLUTILS TUDEPLOY VALUE}
resourceDirs ohne Namen	{TCLUTILS TUDEPLOY USAGE}
Paket fehlt, mit -fail 1	{TCLUTILS TUDEPLOY REQUIRE}

Eine Anwendung muss `tclutils` selbst erst ladbar machen, bevor sie `tudeploy` nutzen kann. Wie `tclutils` in den Modulpfad kommt, steht im Kapitel Installation.

18.12 tuexe

tuexe sucht externe Programme in mitgelieferten Verzeichnissen und im `PATH` und probiert dabei mehrere Namen.

```
::tclutils::tuexe::find namen ?-dirs liste? ?-pathfirst 0|1? ?-all 0|1?
::tclutils::tuexe::all namen ?-dirs liste? ?-pathfirst 0|1?
::tclutils::tuexe::exists namen ?-dirs liste? ?-pathfirst 0|1?
```

Option	Vorgabe	Wirkung
-dirs	{}	zusätzliche Verzeichnisse
-pathfirst	0	1: erst PATH, dann -dirs
-all	0	1: alle Treffer statt des ersten

`namen` ist ein Name oder eine Liste. Für jeden Namen nacheinander durchsucht `find` erst alle `-dirs`, dann den `PATH` (mit `-pathfirst 1` umgekehrt), und liefert den ersten Treffer als normalisierten Pfad, sonst `""`. Ein späterer Name kommt erst dran, wenn der frühere nirgends gefunden ist: bei `{gs gswin64c}` gewinnt ein `gs` im `PATH` gegen ein `gswin64c` in `-dirs`. `all` liefert alle Treffer ohne Doppelte in Suchreihenfolge, `exists` `1` oder `0`.

Im `PATH` sucht `tuexe` mit `auto_execok` und nimmt dessen erstes Wort. In den `-dirs` gilt laut Code:

Plattform	geprüft wird
Linux, macOS	<dir>/<name>; reguläre Datei und ausführbar
Windows	<dir>/<name>, dann <dir>/<name><ext> für jede Endung aus PATHEXT (klein geschrieben), ohne PATHEXT .exe .com .bat .cmd; reguläre Datei, keine Prüfung auf Ausführbarkeit

```
package require tclutils::tuexe
::tclutils::tuexe::find {gs gswin64c gswin32c}   ;# Pfad oder ""
::tclutils::tuexe::find qpdf -dirs [list $vendorBin]
::tclutils::tuexe::exists gibtsnicht             ;# -> 0
```

Unter Linux liefert `find sh` etwa `/usr/bin/sh`; mit einem ausführbaren `sh` in `$vendorBin` und `-dirs [list $vendorBin]` dessen Pfad, mit zusätzlich `-pathfirst 1` wieder `/usr/bin/sh`.

Eine unbekannte Option ergibt `{TCLUTILS TUEXE OPTION}` mit `unknown option '<opt>'`, eine Option ohne Wert (`find sh -dirs`) denselben Code mit `option '<opt>' requires a value`.

18.13 tuopen

tuopen öffnet eine URL oder Datei mit dem Standardprogramm des Systems, eine Datei im Editor oder einen Ordner im Dateimanager, und nennt das Konfigurationsverzeichnis einer Anwendung.

Befehl	Wirkung	Rückgabe
command ziel ?-platform p? ?-os o?	Befehl zum Öffnen bilden, nicht ausführen	Befehlsliste
launch ziel ?-platform p? ?-os o?	Befehl im Hintergrund starten	Befehlsliste
editCommand ziel ?-editor cmd? ?-platform p? ?-os o?	Editorbefehl bilden	Befehlsliste
edit ziel ?-editor cmd? ?-platform p? ?-os o?	Editor starten	Befehlsliste
openDir ziel ?-platform p? ?-os o?	Ordner öffnen; ist ziel eine Datei, deren Ordner	Befehlsliste
configDir app ?-platform p? ?-os o?	Konfigurationsverzeichnis	Pfad
configFile app datei ?-platform p? ?-os o?	Datei darin	Pfad

Option	Vorgabe	Wirkung
-platform	tcl_platform(platform)	windows oder anderes
-os	tcl_platform(os)	enthält es darwin (ohne Groß/Klein), gilt macOS

Option	Vorgabe	Wirkung
-editor	""	eigener Editorbefehl, auch mehrere Wörter; geht allem anderen vor

Was laut Code je Plattform entsteht:

	Windows	macOS	Linux und andere Unix
command, URL	rundll32 url.dll,FileProtocolHandler <ziel>	open <ziel>	xdg-open <ziel>
command, sonst	cmd.exe /c start "" <ziel>	open <ziel>	xdg-open <ziel>
editCommand	notepad <ziel>	open -e <ziel>	$EDITOR <ziel>, sonst xdg-open <ziel>
configDir	%APPDATA%\<app>, sonst %USERPROFILE%\AppData\Roaming\<app>	~/Library/Application Support/<app>	$XDG_CONFIG_HOME/<app>, sonst ~/.config/<app>

Als URL gilt unter Windows ein Ziel, das mit einem Schema aus mindestens zwei Zeichen und `:` beginnt (`https:`, `mailto:`); `C:/…` ist ein Laufwerk. `rundll32` reicht die URL ohne `cmd.exe` weiter, ein `&` in der Abfrage bleibt so Teil der URL. `cmd.exe` würde dort den Befehl trennen.

`$EDITOR` wird als Liste gelesen, `vim -n` ergibt `vim -n <ziel>`. Das Heimatverzeichnis ist `$HOME`, ohne `HOME file normalize ~`. `configDir` legt nichts an. tusettings nennt unter Linux und macOS dasselbe Verzeichnis und schreibt INI-Dateien hinein; unter Windows weicht nur der Ersatz ohne `APPDATA` ab.

`command`, `editCommand` und `configDir` starten nichts; mit `-platform` und `-os` zeigen sie das Ergebnis für jede Plattform:

```
package require tclutils::tuopen
::tclutils::tuopen::command https://tcl.tk/ -platform windows
            ;# -> rundll32 url.dll,FileProtocolHandler https://tcl.tk/
::tclutils::tuopen::command C:/doc/a.pdf -platform windows
                           ;# -> cmd.exe /c start {} C:/doc/a.pdf
::tclutils::tuopen::command a.pdf -platform unix -os Darwin
                           ;# -> open a.pdf
::tclutils::tuopen::editCommand notiz.txt -editor {code -w}
                           ;# -> code -w notiz.txt
::tclutils::tuopen::configDir meineapp -platform unix -os Linux
                           ;# ~/.config/meineapp, mit $HOME aufgelöst
```

`launch`, `edit` und `openDir` führen den Befehl mit `exec … &` aus und liefern ihn zurück. `launch` kehrt zurück, sobald der Prozess gestartet ist; ob das Programm die Datei danach öffnen kann, meldet es nicht.

```
::tclutils::tuopen::launch https://tcl.tk/
::tclutils::tuopen::openDir [file join $dir bericht.pdf]
```

Fall	Code
ziel leer oder nur Leerraum	{TCLUTILS TUOPEN TARGET}
app leer oder nur Leerraum	{TCLUTILS TUOPEN APP}
exec scheitert, etwa ohne xdg-open	{TCLUTILS TUOPEN LAUNCH}, cannot open "<ziel>": … bzw. cannot edit …
unbekannte Option	{TCLUTILS COMMON OPTION <opt>}

`namespace export` nennt alle sieben Befehle; nach `namespace import ::tclutils::tuopen::*` stehen sie ohne Präfix bereit.

18.14 tuzipfs

tuzipfs legt sich um `zipfs` aus Tcl 9: ZIP-Archive einhängen und lesen und eigenständige Programme (Zipkits) bauen. Das Paket lädt auch unter Tcl 8.6 und steht im Umbrella. Befehle, die `zipfs` brauchen, brechen dort mit `{TCLUTILS TUZIPFS UNAVAILABLE}` ab, Meldung `zipfs is not available in this Tcl interpreter; Tcl 9 is required`. ZIP-Dateien unter Tcl 8.6 liest und schreibt `tuzip`, siehe Text und Dateien.

Befehl	Wirkung	Tcl 8.6
available	1, wenn der Befehl zipfs existiert	0
requireAvailable	1, sonst Fehler	UNAVAILABLE
root	zipfs root, also //zipfs:/	UNAVAILABLE
mounts	zipfs mount: Paare aus Einhängepunkt und Archiv	UNAVAILABLE
mount archiv ?name?	einhängen, liefert //zipfs:/<name>	UNAVAILABLE
unmount punkt	aushängen, liefert punkt	UNAVAILABLE
listFiles punkt ?-glob m? ?-recursive b?	Einträge, sortiert	UNAVAILABLE
find punkt ?…?	dasselbe wie listFiles	UNAVAILABLE
exists pfad	zipfs exists	UNAVAILABLE
readFile pfad	Inhalt als Bytes	UNAVAILABLE
withMounted archiv var rumpf ?-mount name?	einhängen, Rumpf ausführen, aushängen	UNAVAILABLE
rcopy quelle ziel	rekursive Bytekopie, liefert ziel	läuft

Befehl	Wirkung	Tcl 8.6
copyStdlib ziel ?-tk b? ?-from q?	tcl_library (und tk_library) kopieren	läuft
mkimg datei vfs ?…?	zipfs mkimg mit Prüfungen	UNAVAILABLE
buildImage -out datei -vfs dir ?…?	copyStdlib, dann mkimg	Optionsprüfung und copyStdlib laufen, mkimg bricht ab

18.14.1 Archiv lesen

`mount` ohne Namen erzeugt `tclutils_<pid>_<clicks>`. Ein fehlendes Archiv meldet `zipfs` selbst mit `{POSIX ENOENT …}`.

Option von listFiles	Vorgabe	Wirkung
-glob	*	Muster gegen den letzten Namensteil oder den ganzen Pfad
-recursive	1	1: alle Einträge über zipfs find; 0: nur die oberste Ebene über glob

Die Liste enthält auch Verzeichnisse. Mit `-recursive 0` gilt `-glob` nur für die oberste Ebene. `readFile` liest binär; Text entsteht mit `encoding convertfrom`.

`withMounted` hängt ein, setzt die Variable im Kontext des Aufrufers auf den Einhängepunkt, führt den Rumpf dort aus und hängt danach immer aus, auch nach einem Fehler. Die Rückgabe ist das Ergebnis des Rumpfs.

```
package require tclutils::tuzipfs
if {[::tclutils::tuzipfs::available]} {
    ::tclutils::tuzipfs::withMounted $zip mp {
        set xml [::tclutils::tuzipfs::listFiles $mp -glob *.xml]
        set txt [::tclutils::tuzipfs::readFile $mp/notiz.txt]
    }
}
```

Für ein Archiv mit `notiz.txt` (Inhalt `Hallo`) und `sub/a.xml` ergibt das unter Tcl 9 `xml = //zipfs:/tclutils_<pid>_<clicks>/sub/a.xml` und `txt = Hallo`; ohne `-glob` enthielte die Liste zusätzlich `…/notiz.txt` und das Verzeichnis `…/sub`.

18.14.2 Programm bauen

`zipfs mkimg` ersetzt das ZIP-Archiv, das am Basekit hängt. Ein Basekit von BAWT oder Magicsplat trägt seine Standardbibliothek in diesem Archiv (`//zipfs:/app/tcl_library`). Ein Abbild nur mit der Anwendung verliert sie und startet nicht. `copyStdlib` legt sie deshalb vorher in den VFS-Baum.

`rcopy` kopiert Byte für Byte und liest dabei auch aus einem eingehängten `zipfs`. Dateien und Ordner, deren Name mit einem Punkt beginnt, kopiert es mit; `.` und `..` übergeht es.

Option von copyStdlib	Vorgabe	Wirkung
-tk	0	auch tk_library kopieren
-from	running	running: info library und $::tk_library des laufenden Interpreters; sonst ein Verzeichnis mit tcl_library/(und tk_library/), etwa der Einhängepunkt eines fremden Basekits

Option von mkimg	Vorgabe	Wirkung
-strip	das VFS-Verzeichnis	Pfadanfang, der im Archiv wegfällt
-basekit	laufendes Programm	Vorlage; ohne Angabe muss das laufende Programm selbst ein Zipkit sein
-password	""	Kennwort für das Archiv

Option von buildImage	Vorgabe	Wirkung
-out	–	Ausgabedatei, Pflicht
-vfs	–	fertiger VFS-Baum, Pflicht
-basekit	""	an mkimg
-stdlib	none	none, cli (nur tcl_library) oder gui (mit tk_library)
-stdlibfrom	running	an copyStdlib -from

```
file mkdir app.vfs
file copy meineapp.tcl [file join app.vfs main.tcl]
::tclutils::tuzipfs::buildImage -out meineapp -vfs app.vfs \
    -basekit basekit-tcl -stdlib cli
```

`buildImage` liefert die Ausgabedatei. `main.tcl` an der Wurzel des Archivs startet beim Aufruf des Programms. Ob das Ergebnis ohne installiertes Tcl läuft, hängt vom Basekit ab: Nur ein statisch gebautes ist eigenständig.

Fall	Code
zipfs fehlt	{TCLUTILS TUZIPFS UNAVAILABLE}
-tk 1 ohne tk_library	{TCLUTILS TUZIPFS NOTK}
-from ohne tcl_library	{TCLUTILS TUZIPFS NOSTDLIB}
mkimg: VFS kein Verzeichnis	{TCLUTILS TUZIPFS NODIR}
buildImage ohne -out/-vfs, -stdlib falsch	{TCLUTILS TUZIPFS OPTION}
-recursive, -tk nicht boolesch	{TCLUTILS COMMON BOOLEAN <opt>}
unbekannte Option	{TCLUTILS COMMON OPTION <opt>}

18.15 tuterm

tuterm erzeugt ANSI-Steuerfolgen (SGR) für Schriftattribute und Farben im Terminal und entfernt sie wieder.

Befehl	Rückgabe
style ?spec ...?	Steuerfolge; "", wenn ausgeschaltet
wrap text ?spec ...?	Steuerfolge, Text, Reset; nur der Text, wenn ausgeschaltet
off attr	Folge zum Abschalten eines Attributs
names ?was?	all (Vorgabe): Kategorien; fg/bg: Farbnamen; ein Attribut: on off; reset und Unbekanntes: {}
strip text	Text ohne Folgen der Form ESC [… m
enable ?bool?	Schalter lesen oder setzen; liefert den Stand
auto	Schalter aus NO_COLOR setzen; liefert den Stand
enableVT, disableVT	VT-Modus der Windows-Konsole; 1 bei Erfolg

Spec	Wirkung
reset	alles zurücksetzen
bold dim italic underline blink reverse invisible strike	Attribut ein
<attr>:on, <attr>:off	Attribut ein oder aus
fg:<farbe>, bg:<farbe>	Vorder-und Hintergrund; fgcolor: und bgcolor: gleichwertig

Eine Farbe ist ein Name (`black` … `white`, `gray`, `bright_black` … `bright_white`), ein Index `0` bis `255` (256 Farben) oder `#rrggbb` (24 Bit). `gray` und `bright_black` sind dieselbe Farbe. Jede Spec ist ein eigenes Argument; `"fg:red bold"` als ein Argument ist eine unbekannte Farbe.

```
package require tclutils::tuterm
proc zeige {s} { string map [list \x1b {\e}] $s }
zeige [::tclutils::tuterm::style bold fg:red]   ;# -> \e[1m\e[31m
zeige [::tclutils::tuterm::wrap OK fg:green]    ;# -> \e[32mOK\e[0m
zeige [::tclutils::tuterm::style fg:208 bg:#00aa00]
                          ;# -> \e[38;5;208m\e[48;2;0;170;0m
zeige [::tclutils::tuterm::off underline]       ;# -> \e[24m
::tclutils::tuterm::strip [::tclutils::tuterm::wrap OK bold]
                                                ;# -> OK
::tclutils::tuterm::enable 0                    ;# -> 0
::tclutils::tuterm::wrap OK bold                ;# -> OK
::tclutils::tuterm::enable 1                    ;# -> 1
```

Nach dem Laden ist `tuterm` eingeschaltet, auch wenn `NO_COLOR` gesetzt ist oder die Ausgabe

in eine Datei geht. Erst `auto` wertet `NO_COLOR` aus: Ist die Variable vorhanden, auch leer, schaltet es aus, sonst ein. Ob `stdout` ein Terminal ist, prüft `tuterm` nicht. Ausgeschaltet prüft `style` die Specs nicht; Tippfehler fallen dann nicht auf.

Plattform laut Code: `enableVT` und `disableVT` liefern außerhalb von Windows sofort `1`. Unter Windows setzen bzw. löschen sie das Bit `0x0004` im Konsolenmodus von `stdout` über das Paket `twapi`; fehlt `twapi` oder scheitert der Aufruf, liefern sie `0`.

Fall	Code
unbekannte Spec oder attr: mit anderem Wert als on/off	{TCLUTILS TUTERM STYLE}
unbekannte Farbe, Index außerhalb 0–255	{TCLUTILS TUTERM VALUE}
off für reset oder eine Farbkategorie	{TCLUTILS TUTERM CATEGORY}
enable mit nicht booleschem Wert	{TCLUTILS COMMON BOOLEAN bool}

`enable` nimmt jeden booleschen Wert von Tcl (`0`/`1`, `yes`/`no`, `true`/`false`, `on`/`off`) und liefert `0` oder `1`; nach einem Fehler bleibt der Schalter unverändert.

18.16 tupagespec

tupagespec übersetzt Seitenangaben wie `1-3,5,7-` in eine Liste von Seitennummern und eine Liste zurück in die kurze Form.

```
::tclutils::tupagespec::parse angabe gesamt ?-base 1?
::tclutils::tupagespec::count angabe gesamt
::tclutils::tupagespec::compact seiten
```

Teil	Bedeutung
leer, all, *	alle Seiten
N	eine Seite
A-B	Bereich; A > B wird getauscht
A-	A bis gesamt
-B	1 bis B
end, last	steht für gesamt

Teile stehen durch Komma getrennt; Leerraum um ein Komma und leere Teile übergeht `parse`, Groß/Klein spielt keine Rolle. Leerraum innerhalb eines Teils (`1 - 3`) ist ein Syntaxfehler.

Option	Vorgabe	Wirkung
-base	1	1: Seiten ab 1; 0: jede Nummer um 1 kleiner

`parse` liefert die Seiten sortiert und ohne Doppelte, `count` deren Anzahl (ohne `-base`). `compact` sortiert, entfernt Doppelte und fasst Folgen zu `A-B` zusammen; eine leere Liste ergibt `""`.

```
package require tclutils::tupagespec
::tclutils::tupagespec::parse "1-3,5" 10        ;# -> 1 2 3 5
::tclutils::tupagespec::parse "5-" 8            ;# -> 5 6 7 8
::tclutils::tupagespec::parse "-3" 8            ;# -> 1 2 3
::tclutils::tupagespec::parse "4-last" 6        ;# -> 4 5 6
::tclutils::tupagespec::parse "5-3,1,1" 10      ;# -> 1 3 4 5
::tclutils::tupagespec::parse "2-4" 10 -base 0  ;# -> 1 2 3
::tclutils::tupagespec::count "1-3,5" 10        ;# -> 4
::tclutils::tupagespec::compact {8 7 1 2 2 3}   ;# -> 1-3,7-8
```

Fall	Code
Teil nicht lesbar	{TCLUTILS TUPAGESPEC SYNTAX}, invalid page-spec part: '<teil>'
Seite außerhalb 1..gesamt	{TCLUTILS TUPAGESPEC RANGE}, page <n> out of range (1..<gesamt>)
unbekannte Option	{TCLUTILS TUPAGESPEC OPTION}
-base nicht 0 oder 1; gesamt keine nicht negative ganze Zahl	{TCLUTILS TUPAGESPEC VALUE}

Seite 0 und Bereiche über das Ende (`8-12` bei 10 Seiten) sind RANGE-Fehler, keine stillen Kürzungen.

Zahlen sind Folgen der ASCII-Ziffern `0` bis `9` und immer dezimal, auch mit führender Null; unter Tcl 8.6 und 9 gilt dasselbe. Das gilt auch für ein `gesamt` nur aus Ziffern:

```
::tclutils::tupagespec::parse 08 10             ;# -> 8
::tclutils::tupagespec::parse 07-09 10          ;# -> 7 8 9
::tclutils::tupagespec::parse 9-end 010         ;# -> 9 10
```

18.17 tulayout

tulayout rechnet für Seitenlayouts in Millimetern, Ursprung oben links: Papierformate, Umrechnung in Pixel, Raster und Blöcke. Ein Block ist ein Dict; ein Layout hat ein Unter-Dict `blocks` von Block-ID zu Block. Den Tk-Editor dazu beschreibt das Buch *tkutils* (`tkulayoutcanvas`).

Schlüssel	Bedeutung
label	Bezeichnung
x, y, w, h	Position und Größe in mm
show	1 sichtbar, 0 verborgen
lockedY	1: senkrechte Lage automatisch

18.17.1 Seite und Maßstab

Befehl	Rückgabe
pageSize papier ?ausrichtung?	{breite höhe} in mm
mmToPx mm maßstab	mm * maßstab
pxToMm px maßstab	px / maßstab
snap mm ?-grid 5.0?	auf das Raster gerundet
fitScale seiteB seiteH sichtB sichtH ?-marginPx 20?	größter Maßstab, mit dem die Seite samt Rand in die Sicht passt

`pageSize` kennt `a4` (210 × 297), `a5` (148 × 210) und `letter` (216 × 279), ohne Unterschied von Groß/Klein. Ein freies Maß in mm steht als Liste `{breite höhe}` oder als `BxH` bzw. `BXH`, etwa `54x56` oder `80.5X200`; beide Teile müssen positive Zahlen sein. `1.2.3x5` ist ein `PAPER`-Fehler. Nur `landscape` tauscht Breite und Höhe; jede andere Angabe gilt als Hochformat. `fitScale` rechnet `min((sichtB - 2*rand) / seiteB, (sichtH - 2*rand) / seiteH)`.

```
package require tclutils::tulayout
::tclutils::tulayout::pageSize A4 landscape     ;# -> 297 210
::tclutils::tulayout::pageSize 54x56            ;# -> 54 56
::tclutils::tulayout::mmToPx 210 2              ;# -> 420.0
::tclutils::tulayout::snap 12.6                 ;# -> 15.0
::tclutils::tulayout::snap 7 -grid 2            ;# -> 8
::tclutils::tulayout::fitScale 210 297 800 600 -marginPx 0
                                        ;# -> 2.0202020202020203
```

18.17.2 Blöcke

Befehl	Wirkung
defaultBlockKeys	x y w h show label lockedY
isAutoY block	1, wenn lockedY wahr oder y == 0 ist
normalizeBlock block ?-defaults d?	nur label x y w h show lockedY, fehlende aus d; x y w h als Gleitkommazahl, show als 0/1
validateBlocks blocks	jeden Block normalisieren und prüfen; liefert das neue Dict
ensureBlocks blocks definitionen	Blöcke mit ihren Definitionen ergänzen, fehlende anlegen, alles prüfen
blockOrder blocks ?definitionen?	IDs: ohne Definitionen sortiert; mit erst deren Reihenfolge, dann die übrigen
blockRect block ?-height h?	{x1 y1 x2 y2}
mergeBlocks basis vorlage ?-keys k?	Schlüssel k aus der Vorlage in gleichnamige Blöcke der Basis übernehmen
mergeLayout ziel vorlage ?-keys k?	mergeBlocks auf blocks zweier Layouts

Die Vorgabe für `-defaults` ist `x 0 y 0 w 40 h 8 show 1`, für `-keys x y w show`. `mergeBlocks` übergeht Blöcke der Vorlage, die in der Basis fehlen, und übernimmt Werte unverändert, ohne sie zu normalisieren. `blockRect` nimmt die Höhe aus `-height`, sonst aus dem Block, sonst 8.

```
set blocks {
    kopf    {label Kopf x 20 y 30 w 170 h 15 show 1}
    tabelle {label Tabelle x 20 y 0 w 170 h 40 show 1}
}
::tclutils::tulayout::isAutoY [dict get $blocks tabelle]  ;# -> 1
::tclutils::tulayout::blockRect [dict get $blocks kopf]
                                    ;# -> 20.0 30.0 190.0 45.0
set m [::tclutils::tulayout::mergeBlocks $blocks {kopf {x 25 h 99}}]
dict get $m kopf x                                          ;# -> 25
dict get $m kopf h                                          ;# -> 15
::tclutils::tulayout::blockOrder $blocks     ;# -> kopf tabelle
```

`ensureBlocks` ergänzt jeden vorhandenen Block um die Schlüssel, die ihm fehlen: zuerst aus seiner Definition, dann aus den Standardwerten von `-defaults`. Ein Wert im Block geht der Definition vor. Eine ID, die nur in den Definitionen steht, entsteht allein aus ihrer Definition. `blocks` darf leer sein; dann kommt alles aus den Definitionen. Das Ergebnis prüft `ensureBlocks` wie `validateBlocks`.

```
set e [::tclutils::tulayout::ensureBlocks {kopf {x 20}} \
    {kopf {w 170 h 15} fuss {x 20 y 280 w 170 h 10}}]
dict get $e kopf          ;# -> x 20.0 y 0.0 w 170.0 h 15.0 show 1
dict get $e fuss                ;# -> x 20.0 y 280.0 w 170.0 h 10.0
::tclutils::tulayout::ensureBlocks {} {kopf {w 170 h 15}}
                                       ;# -> kopf {w 170.0 h 15.0}
```

Fall	Code
unbekanntes Papier, Maß nicht lesbar oder nicht positiv	{TCLUTILS TULAYOUT PAPER}
Maßstab nicht positiv	{TCLUTILS TULAYOUT SCALE}
Raster nicht positiv	{TCLUTILS TULAYOUT GRID}
unbekannte Option, Option ohne Wert	{TCLUTILS TULAYOUT OPTION}
validateBlocks: blocks leer oder kein Dict; ensureBlocks: blocks, Definitionen oder ein Block kein Dict	{TCLUTILS TULAYOUT BLOCKS}
leere Block-ID	{TCLUTILS TULAYOUT BLOCKID}
x, y, w, h keine Zahl oder negativ, show nicht boolesch	{TCLUTILS TULAYOUT VALUE}
Layout ohne blocks	{TCLUTILS TULAYOUT LAYOUT}
Sicht abzüglich zweimal Rand nicht positiv	{TCLUTILS TULAYOUT VIEWPORT}

Die Meldung bei VALUE nennt Block und Schlüssel, etwa `block 'kopf': bad x 'abc'` oder `block 'kopf': height must be >= 0`. Optionen prüfen `snap`, `normalizeBlock`, `blockRect`, `mergeBlocks`, `mergeLayout` und `fitScale` gleich: Erlaubt ist nur die jeweils genannte Option.

Nächstes Kapitel: Netz und HTTPS

19 Netz und HTTPS

19.1 tuhttps

tuhttps ist die gemeinsame HTTPS-Politik für Clients, die `::http::register` nutzen. Zertifikate werden geprüft; Abschalten nur über `-insecure 1`. Eine interne CA gibt man mit `-cafile` an. `tuhttps::with prefix script` registriert `https` nur für die Dauer eines Skripts und stellt danach die vorige Registrierung wieder her, auch bei Fehlern. Im Skript kein `return` verwenden (Kommentar im Code). Eine Anwendung, die selbst danach `https` registriert, oder asynchrone `::http::geturl -command`-Aufrufe während `with` sieht `tuhttps` nicht: die Registrierung ist befristet, nicht gesperrt.

Clients: `tufetch`, `tudav`, `tupostgrest`. Roh-REST ohne tclutils-Politik: Buch *Tcl REST*.

19.2 tls 1.x und tls 2.0

tls	CA-Speicher
2.0 (Tcl 9)	OpenSSL-Standardpfade, keine Datei nötig
1.x (Tcl 8.6, auch Windows)	findet den System-Speicher nicht selbst

Nur unter tls < 2.0 sucht `tuhttps` ein CA-Bundle, und zwar in dieser Reihenfolge: `$env(SSL_CERT_FILE)`, dann feste Pfade (`/etc/ssl/certs/ca-certificates.crt`, `/etc/pki/tls/certs/ca-bundle.crt`, `/etc/ssl/ca-bundle.pem`, `/etc/ssl/cert.pem`, `/usr/local/etc/openssl/cert.pem`). Findet es keines, wirft es `{TCLUTILS TUHTTPS NOCA}` und nennt den Ausweg (`-cafile`, `SSL_CERT_FILE`). Es fällt nie auf "ungeprüft" zurück. Unter Windows mit tls 1.x: `SSL_CERT_FILE` setzen oder `-cafile` übergeben.

Unter Windows mit tls 2.0 entscheidet OpenSSL. Findet es keinen Speicher, scheitert der Handshake. `-cafile` bleibt der Ausweg.

19.3 Was socketCmd zurückgibt

`socketCmd` liefert das Befehlspräfix für `::http::register`. Es öffnet keinen Socket. Unter Tcl 8.6.18 mit tls 2.0.1:

Aufruf	Präfix
-host example.org	::tls::socket -autoservername 1 -require 1
-insecure 1 -host example.org	::tls::socket -autoservername 1 -request 0 -require 0

Aufruf	Präfix
-host 127.0.0.1	::tls::socket -autoservername 0 -require 1

Eine IP bekommt kein SNI (`-autoservername 0`). tls ab 2.0 hängt kein `-cafile` an. Die Suche von `caFile` läuft trotzdem und liefert die erste lesbare Datei. Auf diesem Linux ist das `/etc/ssl/certs/ca-certificates.crt`. Gebraucht wird sie erst, wenn `socketCmd` ein tls älter als 2.0 sieht. Eine nicht lesbare `-cafile` wirft auch unter tls 2 `{TCLUTILS TUHTTPS CAFILE}`.

19.4 tufetch

tufetch 0.4 holt per nativem `http + tls`, sonst per `curl` oder `wget`. Alle drei Wege prüfen das Zertifikat. `-cafile` und `-insecure` gelten für alle drei.

Befehl	Wirkung
get url ?optionen?	Rumpf als UTF-8-Text
download url pfad ?optionen?	Rumpf als Bytes in die Datei, Rückgabe native, curl oder wget

Option	Vorgabe	Wirkung
-timeout	30000	Millisekunden
-redirects	5	danach {TCLUTILS TUFETCH REDIRECT}
-method	get	get oder post
-headers		flache Liste Name Wert
-data		Rumpf, zusammen mit -method post
-type		Content-Type dieses Rumpfs
-cafile		CA-Datei für die Prüfung
-insecure	0	jedes Zertifikat gilt

Fehlt unter tls 1.x jedes CA-Bundle, tritt der native Weg zur Seite und `curl`/`wget` übernehmen (sie bringen eigene CA-Speicher mit). Gibt es keines der beiden, kommt `{TCLUTILS TUFETCH NOMETHOD}` mit dem Ausweg in der Meldung (`no HTTP method available: …` plus Text von `NOCA`). Ganz ohne tls und ohne curl/wget lautet sie `no HTTP method available -- install the Tcl tls package or curl/wget`.

`{TCLUTILS TUFETCH CAFILE}` heißt dagegen: die mit `-cafile` genannte Datei ist nicht lesbar. Diesen Code gibt es nur auf dem nativen Weg, also mit `tls`. Ohne `tls` reicht `tufetch` die Datei an curl (`--cacert`) bzw. wget (`--ca-certificate=`) weiter. Ein Fehler von curl kommt dann als `CONNECT`, einer von wget als `CHILDSTATUS` (laut Code). `tuhttps` meldet den Fall als `{TCLUTILS TUHTTPS CAFILE}`. Ohne `tls` kommt dort `{TCLUTILS TUHTTPS NOTLS}` — die tls-Prüfung läuft vor der `-cafile`-Prüfung.

Weitere Codes von `tufetch`: `{TCLUTILS TUFETCH HTTP <Klasse> <Status>}` (etwa `HTTP 4XX 404`), `URL` (nur `http://` und `https://`), `TIMEOUT`, `CONNECT` und `REDIRECT`.

Präfix-Traps wie `trap {TCLUTILS TUFETCH HTTP}` fangen eine ganze Klasse. Über wget wird ein HTTP-Status nicht übersetzt; er erscheint als `CHILDSTATUS` von `exec`.

Ohne Server bleiben diese beiden Fehler:

```
package require tclutils::tufetch
::tclutils::tufetch::get ftp://example.com/a
    ;# -> {TCLUTILS TUFETCH URL}  unsupported or invalid URL
::tclutils::tufetch::get https://example.com/ -cafile
    /gibt/es/nicht.pem
    ;# -> {TCLUTILS TUFETCH CAFILE}  cannot read CA file

package require tclutils::tuhttps
try {
    ::tclutils::tuhttps::socketCmd -cafile /gibt/es/nicht.pem
} trap {TCLUTILS TUHTTPS CAFILE} msg {
    puts "CA-Datei: $msg"
} trap {TCLUTILS TUHTTPS NOTLS} msg {
    puts "kein tls: $msg"
}
```

Mit tls lautet die Meldung `cannot read CA file: /gibt/es/nicht.pem`, ohne tls `https needs the tls package: can't find package tls`. Fehlt unter tls 1.x jedes Bundle, kommt `NOCA` mit dem Text, `-cafile` oder `SSL_CERT_FILE` zu setzen.

19.5 tusparql

tusparql ist ein schlanker SPARQL-Client. Den Transport übernimmt `tufetch`, die Query-Strings baut `tuurl`, das Ergebnis liest `tujson` (SPARQL 1.1 Query Results JSON). Damit gelten dieselben Voraussetzungen wie bei `tufetch`: `http` mit `tls`, sonst `curl` oder `wget`. XML-Ergebnisse liest das Modul nicht.

Befehl	Wirkung
query endpunkt sparql ?optionen?	SELECT; Liste von Dicts, je Zeile Variable auf Wert
ask endpunkt sparql ?optionen?	ASK; 1 oder 0

Option	Vorgabe	Wirkung
-method	get	post schickt die Abfrage als Rumpf; jeder andere Wert gilt als get
-timeout	30000	Millisekunden, an tufetch weitergereicht
-headers		weitere Kopfzeilen, flache Liste Name Wert; ein Accept darin ersetzt -accept
-accept	application/sparql-results+json	Wert der Kopfzeile Accept

Laut Code hängt `get` an den Endpunkt`?query=…&format=json` an, mit `+` für das Leerzeichen. `post` schickt die Abfrage roh mit dem Content-Type

`application/sparql-query`. `Accept` geht genau einmal hinaus: steht in `-headers` schon ein `Accept` (gleich in welcher Schreibung), gilt dieses, sonst der Wert von `-accept`. Die Antwort liest `tusparql` immer als JSON. Ein Endpunkt, der schon einen Query-String hat, etwa `…/sparql?key=1`, bekommt die Parameter mit `&` angehängt.

Eine Ergebniszeile enthält nur `value` je Variable; `type`, `datatype` und `xml:lang` fallen weg. Eine Variable ohne Bindung, etwa hinter `OPTIONAL`, fehlt im Dict der Zeile. Die Reihenfolge der Variablen kommt aus `head vars`. Fehlt diese Liste, gelten die Variablen aller Zeilen in der Reihenfolge ihres ersten Auftretens, nicht nur die der ersten Zeile.

Das Beispiel braucht einen erreichbaren Endpunkt; das Ergebnis hängt von dessen Daten ab und steht deshalb nur als Form da.

```
package require tclutils::tusparql
set ep https://query.wikidata.org/sparql
set q {SELECT ?ort WHERE { ?ort wdt:P31 wd:Q515 } LIMIT 2}
::tclutils::tusparql::query $ep $q -timeout 10000
    ;# -> {ort http://www.wikidata.org/entity/Q…} {ort …}
::tclutils::tusparql::ask $ep {ASK { wd:Q64 wdt:P31 wd:Q515 }}
    ;# -> 1
```

Die URL eines `get`-Aufrufs für `ASK {}` an `https://example.org/sparql` lautet `https://example.org/sparql?query=ASK+%7B%7D&format=json`. Eine Antwort mit zwei Zeilen, deren zweite `gang` nicht bindet, ergibt `{ort Nord gang 3} {ort Süd}`.

Eine Antwort ohne `boolean` wirft bei `ask` `{TCLUTILS TUSPARQL NOTASK}`. Fehler beim Transport und HTTP-Status kommen unverändert aus `tufetch` (`{TCLUTILS TUFETCH …}`), ein Antworttext, der kein JSON ist, als `{TCLUTILS TUJSON PARSE offset}`. Eine unbekannte Option wirft `{TCLUTILS COMMON OPTION -name}`.

19.6 Zusammenfassung

- Eine Stelle für HTTPS: `tuhttps`.
- tls 1.x braucht ein CA-Bundle; sonst `NOCA` bzw. `NOMETHOD`.
- `CAFILE` = `-cafile` nicht lesbar, nicht "CA fehlt".
- `-insecure` bewusst und lokal halten.
- SPARQL: `tusparql` über `tufetch`, nur JSON-Ergebnisse, Zeilen als Dicts ohne Typ.
- Query-Strings: Kapitel Kodierung, Hash und Vergleich (`tuurl`).

Nächstes Kapitel: tusettings

20 tusettings

20.1 Zweck

tusettings **0.1** legt pro App Einstellungen als INI in das Plattform-Config-Verzeichnis, plus Verzeichnisse für cache/data/temp und eine Recent-Liste. Pure Tcl, kein Tk. Neu in 0.64.0, seither im Umbrella. Die INI schreibt `tuini`; sie lässt sich von Hand lesen und ändern. Dateien schreibt und liest das Modul als UTF-8.

```
package require tclutils::tusettings
set s [::tclutils::tusettings::load meineapp -defaults {
    window {width 800 height 600}
}]
dict set s window width 1024
::tclutils::tusettings::save meineapp $s     ;# -> Pfad der
    settings.ini
::tclutils::tusettings::recentAdd meineapp /pfad/zur/datei
::tclutils::tusettings::value $s window width ;# -> 1024
```

`save` liefert den Pfad, unter Linux `$HOME/.config/meineapp/settings.ini`.

Die Befehle liegen im Namensraum `::tclutils::tusettings`. Die Kurzform `tusettings::load` löst Tcl nur innerhalb von `namespace eval ::tclutils` auf; auf globaler Ebene bricht sie mit `invalid command name` ab.

20.2 API (Kurz)

Befehl	Rolle
dir kind app ?-create bool?	config / cache / data / temp
file app ?name?	<config>/<name>.ini, Default-Name settings
load app ?-defaults d? ?-name n?	INI zu Dict, über die Defaults gelegt
save app settings ?-name n?	Dict zu INI (Temp-Datei, dann Rename); liefert den Pfad
value settings section key ?default?	Section/Key lesen
recentAdd app path ?-list l? ?-max n?	Pfad nach oben, liefert die neue Liste
recentList app ?-list l? ?-existing bool?	Liste, neuester zuerst
recentRemove app path ?-list l? /recentClear app ?-list l?	Eintrag bzw. Liste löschen

`load` legt die Datei über `-defaults`; neue Schlüssel einer späteren App-Version bekommen so ihren Default. Fehlt die Datei, liefert `load` die Defaults. `dir` legt nur mit `-create 1` etwas an; `save` und `recentAdd` erzeugen das Config-Verzeichnis selbst. `-list` (Default `recent`) wählt die Recent-Datei.

20.3 Verzeichnisse pro Plattform

kind	Linux / BSD	Windows	macOS
config	$XDG_CONFIG_HOME oder ~/.config	%APPDATA%	~/Library/Application Support
cache	$XDG_CACHE_HOME oder ~/.cache	%LOCALAPPDATA%	~/Library/Caches

kind	Linux / BSD	Windows	macOS
data	$XDG_DATA_HOME oder ~/.local/share	%APPDATA%	~/Library/Application Support
temp	$TMPDIR oder /tmp	%TEMP% oder %TMP%	$TMPDIR oder /tmp

An jedes Verzeichnis wird der App-Name angehängt. Unter Windows greifen ohne die Umgebungsvariablen `<home>/AppData/Roaming` bzw. `<home>/AppData/Local` (temp: `…/Local/Temp`). macOS wird an `tcl_platform(os) = Darwin` erkannt.

Ohne `XDG_*` und `TMPDIR` sind das unter Linux `~/.config/meineapp`, `~/.cache/meineapp`, `~/.local/share/meineapp` und `/tmp/meineapp`. Die Spalten für Windows und macOS stehen so im Code.

20.4 Recent-Liste

Eine Textdatei pro Liste (`<config>/<list>.txt`, ein Pfad pro Zeile, UTF-8). `recentAdd` normalisiert den Pfad, setzt ihn nach oben, wirft einen älteren Eintrag derselben Datei hinaus (unter Windows ohne Groß/Klein-Unterschied) und behält `-max` Einträge (Default 10). `recentList -existing 1` lässt verschwundene Dateien weg.

20.5 Grenzen und Fehler

INI kann nicht alles. `save` lehnt ab: Werte mit Zeilenumbruch oder Leerzeichen am Rand; leere Schlüssel, Schlüssel mit `=`, Zeilenumbruch, Leerzeichen am Rand oder mit `;`, `#`, `[` am Anfang; Sektionsnamen mit `]` oder Zeilenumbruch. Geprüft wird vor dem Schreiben, die Datei bleibt dann unverändert. Scheitert das Schreiben selbst, bleibt die alte Datei ebenfalls, die Temp-Datei wird gelöscht, und kein Kanal bleibt offen (0.2). Kommentare in der Datei überleben `save` nicht.

```
try {
    ::tclutils::tusettings::save meineapp {window {title " x"}}
} trap {TCLUTILS TUSETTINGS VALUE} msg {
    puts "abgelehnt: $msg"
}
```

Die Meldung lautet `value of window.title: no line breaks or outer blanks in INI`.

Code	Fall
{TCLUTILS TUSETTINGS APP}	App-Name leer oder mit Pfadzeichen
{TCLUTILS TUSETTINGS KIND}	dir mit unbekanntem kind
{TCLUTILS TUSETTINGS SYNTAX}	INI-Datei nicht parsebar
{TCLUTILS TUSETTINGS VALUE}	Wert, Schlüssel oder Sektion nicht INI-tauglich; -max keine positive Ganzzahl; Pfad mit Zeilenumbruch

Unbekannte Optionen: `{TCLUTILS COMMON OPTION <opt>}`. Bei `SYNTAX` entscheidet der

Aufrufer, ob er auf die Defaults zurückfällt; die Meldung nennt die Datei und die fehlerhafte Zeile, etwa `…/settings.ini: line without '=': garbage line`.

`tkutils::tkugeometry` speichert Fenstergeometrie darüber; Details im Buch *tkutils*.

20.6 Zusammenfassung

- INI-Settings + Recent, plattformübliche Pfade.
- Voller Namensraum `::tclutils::tusettings::…`.
- GUI-Geometrie: Buch *tkutils*.

Nächstes Kapitel: Tasks mit tutask

21 Tasks mit tutask

21.1 Zweck

tutask **0.2** (seit tclutils 0.66.0) führt Aufgaben als Tasks auf Koroutinen aus. Ein Task wartet mit `await` auf eine Operation; die Ereignisschleife läuft währenddessen weiter. Dazu kommen Timeout, Abbruch mit Aufräumen, Gruppen, ein Thread-Pool und eine Sandbox.

Pure Tcl ab 8.6 (`package require Tcl 8.6-`). Nur der Pool braucht das Paket `Thread` und lädt es erst beim ersten `pool`. Der Umbrella lädt `tutask` mit.

`tutask` 0.2 verlangt `tclutils::common 0.2`. Wer einen älteren Baum überkopiert, löscht `tutask-0.1.tm`, sonst liegen beide Versionen nebeneinander.

```
package require tclutils::tutask
namespace import ::tclutils::tutask::*
```

Exportiert werden `spawn`, `run`, `current`, `await`, `sleep`, `timer`, `onwrite`, `pipe`, `group`, `pool`, `sandbox`, `tasks` und `stats`. Die Beispiele dieses Kapitels setzen den Import voraus.

21.2 Der Vertrag: Starter

Jede Operation ist ein Befehlspräfix, ein Starter. `await` hängt einen Callback `cb` an und ruft den Starter auf. Der Starter gibt einen Abbruchbefehl zurück (ein Präfix, darf leer sein) und meldet später genau einmal:

```
starter cb                    ;# -> Abbruchbefehl
{*}$cb ok wert
{*}$cb error meldung ?optionen?
```

Nur die erste Meldung zählt. Eine späte Meldung nach Timeout oder Abbruch verfällt. Eigene Operationen folgen denselben Zeilen:

```
proc meinOp {cb} {
    set id [after 100 [list {*}$cb ok fertig]]
    return [list after cancel $id]
}
run apply {{} { await meinOp }}      ;# -> fertig
```

Mitgeliefert: `timer`, `pipe`, `onwrite`, Tasks und Gruppen (über `wait`), `$pool op` und

`$sandbox op`.

21.3 Tasks

```
set t [spawn apply {{n} {
    sleep 50
    return [expr {$n * 2}]
}} 21]
$t state         ;# -> running
$t wait          ;# -> 42
$t state         ;# -> ok
$t destroy
```

Befehl	Wirkung
spawn ?-detach bool? cmd ?arg ...?	Task-Objekt; läuft sofort bis zum ersten await
run ?-timeout ms? cmd ?arg ...?	spawn, wait, destroy; liefert den Wert
current	laufender Task oder leer
$t state	running, ok, error oder cancelled
$t result	Wert; sonst der Fehler des Tasks, CANCELLED oder NOTDONE
$t wait ?-timeout ms?	im Task: await; außerhalb: blockiert in vwait
$t cancel	Abbruch; 1, wenn der Task noch lief
$t ondone cmd	ruft cmd mit angehängtem Task-Objekt, sobald er fertig ist
$t destroy	bricht einen laufenden Task vorher ab

`run` ist für Skripte und Tests gedacht. Ein Timeout bei `wait` bricht den Task ab; `state` ist danach `cancelled`. `-timeout 0` (Default) heißt: kein Timeout. Ein Task mit `-detach 1` zerstört sich nach dem Ende selbst.

21.4 Warten: await und sleep

`await starter ?-timeout ms?` liefert den Wert der Operation oder wirft deren Fehler mit dem eigenen `-errorcode`. Läuft der Timeout ab, ruft `await` den Abbruchbefehl der Operation und wirft `{TCLUTILS TUTASK TIMEOUT}`. `sleep ms` pausiert den Task, ohne die Ereignisschleife anzuhalten.

```
try {
    run apply {{} { await [timer 500 x] -timeout 50 }}
} trap {TCLUTILS TUTASK TIMEOUT} msg {
    puts $msg                        ;# -> timeout after 50 ms
}
```

`await` funktioniert in jeder Koroutine; `cancel` braucht einen Task. Außerhalb jeder Koroutine

wirft `await {TCLUTILS TUTASK NOTASK}`.

`-timeout` muss eine ganze Zahl ab 0 sein. Ab 0.66.0 prüfen das `await`, `run`, `$t wait` und `$g wait` gleich: ein Wert wie `abc` oder `-5` wirft `{TCLUTILS TUTASK OPTION}` mit `-timeout must be a non-negative integer (ms)`. `run` prüft vor `spawn`; ein falscher Wert hinterlässt keinen Task. Bis 0.65.0 prüfte nur `await`. `run`, `$t wait` und `$g wait` außerhalb eines Tasks nahmen den Wert stillschweigend an und warteten ohne Timeout.

```
catch {run -timeout abc {after 10}} msg opts
dict get $opts -errorcode    ;# -> TCLUTILS TUTASK OPTION
tasks                        ;# -> (leer)
```

21.5 Abbrechen mit Aufräumen

`$t cancel` setzt den Task mit einer Abbruchanforderung fort. Sein `await` wirft dort `{TCLUTILS TUTASK CANCELLED}`. Die `finally`-Zweige sind gelaufen, wenn `cancel` zurückkehrt. Die Operation, auf die der Task wartete, wird mit abgebrochen.

```
set t [spawn apply {{} {
    try { sleep 10000 } finally { puts aufgeraeumt }
}}]
$t cancel        ;# gibt "aufgeraeumt" aus, -> 1
$t state         ;# -> cancelled
$t destroy
```

- Aus dem Task selbst heraus wirkt `cancel` erst beim nächsten `await`.
- Ein Task, der `CANCELLED` fängt und wieder wartet, wird erneut abgebrochen.
- `destroy` aus dem eigenen Task wirft `{TCLUTILS TUTASK INSIDE}`; dort `cancel` verwenden.
- Wartet ein Task per `wait` auf einen anderen Task und wird abgebrochen, bricht das auch den anderen ab: beide enden `cancelled`, beide `finally`-Zweige laufen.

21.6 Operationen

Operation	Wert
timer ms ?wert?	nach ms Millisekunden wert (Default leer)
onwrite varName	der neue Inhalt, sobald die Variable geschrieben wird
pipe cmdlist ?-encoding enc? ?-profile p?	Ausgabe der Pipeline, stdout und stderr zusammen

`onwrite` nimmt eine globale oder voll qualifizierte Variable; ohne führendes `::` setzt es `::` davor. Typisch: ein Knopf setzt die Variable.

`pipe` startet die Pipeline nicht blockierend (`open |… 2>@1`). Ein Exit-Code ungleich 0 wirft Tcls `{CHILDSTATUS pid code}`. Abbruch und Timeout beenden die Prozesse: `kill` unter Unix, `taskkill /T /F` unter Windows. `-profile` gibt es nur unter Tcl 9; unter 8.6 wirft die Option `{TCLUTILS TUTASK OPTION}` mit `pipe: -profile needs Tcl 9`.

```
run apply {{} { await [pipe {echo hallo}] }}     ;# -> "hallo\n"
try {
    run apply {{} { await [pipe {sh -c {exit 3}}] }}
} trap CHILDSTATUS {msg opts} {
    lindex [dict get $opts -errorcode] 2        ;# -> 3
}
```

Unter Unix trifft `kill` nur die Prozesse, die `pid` für den Kanal nennt. Startet die Pipeline ihrerseits Kinder (etwa `sh -c "sleep 30 & …"`), laufen diese nach Abbruch oder Timeout weiter. Die Roadmap im tclutils-Baum plant dafür einen Prozessgruppen-Kill (`setsid`). Windows beendet die Pipeline laut CHANGELOG mit `taskkill /T /F`.

21.7 Gruppen

```
set g [group]
$g spawn apply {{} { sleep 30; return a }}
$g spawn apply {{} { return b }}
$g wait                       ;# -> a b
$g destroy
```

- `-mode all` (Default): alle Ergebnisse in `spawn`-Reihenfolge. Der erste Fehler bricht die übrigen Tasks ab und wird geworfen.
- `-mode first`: der erste fertige Task gewinnt, die übrigen werden abgebrochen. `$g winner` nennt den Gewinner.
- `$g wait` nimmt auch `-timeout ms`. `$g tasks` listet die Tasks; `$g destroy` zerstört sie mit.
- Leere Gruppe: `all` liefert eine leere Liste, `first` wirft `{TCLUTILS TUTASK EMPTY}`. Ein anderer Modus wirft `{TCLUTILS COMMON VALUE -mode}`.
- Wird ein Task abgebrochen, der auf eine Gruppe wartet, bricht das die Tasks der Gruppe ab.

`$g destroy` direkt nach `$g wait` ist unbedenklich, auch wenn `wait` außerhalb eines Tasks mit `-mode first` lief oder `-mode all` mit einem Fehler endete: Späte Rückmeldungen abgebrochener Tasks verfallen. Bis 0.65.0 meldete `bgerror` dann `invalid command name "::oo::Obj…::my"`.

21.8 Thread-Pool

```
set p [pool -size 2 -init {proc rechne {n} { expr {$n * 2} }}]
run apply {{p} { await [$p op {rechne 21}] -timeout 10000 }} $p
# -> 42
$p stats          ;# -> workers 1 idle 1 queued 0 jobs 0
$p destroy
```

- `-size` (Default 2) ist die Höchstzahl der Worker; sie entstehen erst bei Bedarf. `-init` läuft in jedem neuen Worker.
- Jeder Job läuft in einem Worker-Thread mit eigenem Interpreter. Es gibt keine gemeinsamen Variablen.
- Fehler behalten `-errorcode` und `-errorinfo` des Workers.

- Ein Job in der Warteschlange wird beim Abbruch entfernt. Ein laufender Job bekommt `thread::cancel`; der Worker lebt weiter (nach einem Timeout auf `while 1 {}` nimmt derselbe Pool den nächsten Job an).
- `$p destroy` gibt die Worker frei; wartende Tasks erhalten `CANCELLED`.
- Ohne Paket `Thread` wirft `pool {TCLUTILS TUTASK NOTHREAD}`. Scheitert `-init`, wirft der erste Job `{TCLUTILS TUTASK INIT}`.

Gemessen: zwei Jobs mit je `after 300` in einer Gruppe brauchten zusammen rund 300 ms, liefen also parallel.

`pool -size 0` wirft `-size must be a positive integer`. Bei `-init {error boom}` bekommt der Wartende `{TCLUTILS TUTASK INIT}` mit `pool -init failed: boom`; `$p stats` zeigt danach `workers 0 idle 0 queued 0 jobs 0`. Bis 0.65.0 meldete `bgerror` dabei zusätzlich einen Fehler, und der Job blieb in der Warteschlange.

21.9 Sandbox

```
set s [sandbox -commands 1000 -time 500]
$s eval {expr {6*7}}                  ;# -> 42
catch {$s eval {while 1 {info level}}} msg opts
dict get $opts -errorcode
# -> TCLUTILS TUTASK LIMIT COMMANDS
$s eval {set x 1}                     ;# -> 1 (Grenzen wieder frisch)
run apply {{s} { await [$s op {string repeat ab 2}] }} $s   ;# -> abab
$s destroy
```

Die Sandbox ist ein sicherer Interpreter (`-safe 1`, Default; `-safe 0` für einen normalen). Optionen: `-commands n` und `-time ms` (je 0 = keine Grenze), `-aliases {name präfix …}` für Befehle, die das Kind aufrufen darf. `$s interp` liefert den Interpreter.

- Die Grenzen gelten pro Auswertung. Überschreitung wirft `{TCLUTILS TUTASK LIMIT COMMANDS}` bzw. `{TCLUTILS TUTASK LIMIT TIME}`.
- Andere Fehler behalten den Code des Kinds, etwa `{TCL LOOKUP COMMAND open}` für `open` im sicheren Interpreter.
- Das Skript läuft im aufrufenden Thread. `op` lässt sich abbrechen, bis es startet (nächster Durchlauf der Ereignisschleife); danach begrenzen es nur die Limits. Für echte Parallelität den Pool nehmen.
- Eine leere Schleife `while 1 {}` zählt keine Befehle. `-commands` allein hält sie nicht an; der Lauf hing bis zum äußeren Abbruch. `-time 500` beendete sie nach 500 ms mit `LIMIT TIME`. Deshalb `-time` immer mit setzen.

21.10 Lecks und vergessene Fehler

`tasks` liefert die lebenden Task-Objekte, `stats` ein Dict mit `tasks running awaits pipes pools sandboxes`. Nach einem Testlauf sollten alle Werte 0 sein:

```
stats
# -> tasks 0 running 0 awaits 0 pipes 0 pools 0 sandboxes 0
```

Einen Fehler, den weder `result`, `wait`, `ondone` noch eine Gruppe abgeholt hat, meldet das

Task-Objekt beim `destroy` über `bgerror`, mit dem `-errorcode` des Tasks:

```
tutask: error in task <obj> was never collected: <Meldung>
```

Das gilt auch für Tasks mit `-detach 1`.

21.11 In Tk-Programmen

Im Ereignis-Handler nur `spawn` aufrufen und zurückkehren; das Warten erledigt der Task. Kein `$t wait` außerhalb eines Tasks in einem Handler: das startet ein verschachteltes `vwait`, dieselbe Falle wie `vwait` im Callback. Im Task sind `wait` und `await` unbedenklich.

`examples/demo-task.tcl` im tclutils-Baum zeigt beides: unter `tclsh` einen Konsolenteil, unter `wish` zusätzlich einen Fortschrittsbalken mit Abbrechen-Knopf. Der Handler startet nur den Task; `ondone` zerstört ihn. Nach dem Abbruch setzt der `finally`-Zweig die Knöpfe zurück, `stats` ist danach überall 0. Widgets selbst: Buch *tkutils*.

21.12 Fehlercodes

Alle Codes beginnen mit `TCLUTILS TUTASK`:

Grund	Fall	Meldung
CANCELLED	Task abgebrochen, Pool zerstört	task cancelled
TIMEOUT	-timeout abgelaufen	timeout after <ms> ms
NOTASK	await außerhalb einer Koroutine	await outside a task or coroutine
NOTDONE	result eines laufenden Tasks	task <obj> is still running
OPTION	falsches Argument (spawn ohne Befehl, timer -1, leere pipe, -timeout x)	je Fall, bei -timeout: -timeout must be a non-negative integer (ms)
NOTHREAD	pool ohne Paket Thread	the Thread package is not available
EMPTY	wait -mode first auf leerer Gruppe	group is empty
INIT	-init des Pools gescheitert	pool -init failed: <Fehler>
INSIDE	destroy aus dem eigenen Task	a task cannot destroy itself while running; use cancel
PROTOCOL	Koroutine unerwartet fortgesetzt (intern)	je Fall
LIMIT COMMANDS	Befehlsgrenze der Sandbox	Meldung von Tcl
LIMIT TIME	Zeitgrenze der Sandbox	Meldung von Tcl

Unbekannte Optionen laufen über `common::parseOptions` und werfen `{TCLUTILS COMMON OPTION <opt>}`.

21.13 Zusammenfassung

- `tutask` 0.2: Tasks auf Koroutinen, im Umbrella.
- Eine Operation ist ein Starter: Callback rein, Abbruchbefehl raus.
- `cancel` lässt `finally` laufen und bricht die erwartete Operation ab.
- Gruppen (`all`, `first`), Pool (Paket `Thread`), Sandbox (Limits).
- Unter Unix tötet `pipe` beim Abbruch keine Enkelprozesse.
- `-commands` allein stoppt keine leere Schleife; `-time` mit setzen.
- Ab 0.66.0 prüfen `await`, `run`, `$t wait` und `$g wait -timeout` gleich; ein falscher Wert wirft `{TCLUTILS TUTASK OPTION}`.

Nächstes Kapitel: Tests und tutestrun

22 Tests und tutestrun

22.1 Suite

```
tclsh8.6 tests/all.tcl
tclsh8.6 tests/all.tcl -match 'tusettings-*'
```

Jede `*.test`-Datei läuft in eigenem Prozess; tcltest-Optionen wie `-match` reicht der Runner durch. Das Banner zeigt, wohin `tclutils` aufgelöst wird. Zeitgrenze pro Datei: Umgebung TCLUTILS_TESTGRENZE (Default 120 s). Am Ende listet der Runner "Files with problems" oder meldet `All test files passed`; Exit-Code 1 bei Problemen. Findet `all.tcl` den Runner nicht, bricht es mit Exit-Code 2 und `all.tcl: tclutils::tutestrun not found (set TCLUTILS_TM)` ab.

Ergebnis für tclutils 0.69.0 unter Linux (Xvfb), macOS und Windows:

Lauf	Dateien	Tests	bestanden	übersprungen	fehlgeschlagen
Tcl 8.6.14 mit tls 1.7.22, Thread 2.8.9, tDOM 0.9.3	154	2620	2595	25	0
Tcl 9.0.4 (aus den Quellen gebaut) mit tls 2.0 (OpenSSL 3.0.13), Thread 3.0.6, tDOM 0.9.7	154	2620	2603	17	0
macOS (Darwin 27, arm64), Tcl/Tk 9.0.4 (Aqua), Stand vor den Schreibfehler-Tests	154	2582	2567	15	0

Lauf	Dateien	Tests	bestanden	übersprungen	fehlgeschlagen
Windows, Tcl 8.6.17 (Systemkodierung cp1252), Stand vor den Schreibfehler-Tests	154	2582	2545	37	0
Windows, Tcl 9.0.4, Stand vor den Schreibfehler-Tests	154	2582	2549	33	0

Kein Test schlägt fehl. Übersprungene Tests hängen an fehlenden Paketen oder an der Plattform (Constraints wie `tdbc`, `haveTk`, `net`, `windows`).

22.2 tutestrun

tutestrun **0.2** ist der gemeinsame Runner für tclutils, tkutils und ctrlutils. **Nicht** im Umbrella — `tests/all.tcl` sourct die `.tm` aus dem Baum unter Test (eigenes `lib/tm`, `$env(TCLUTILS_TM)` oder der Geschwister-Checkout), sonst gewinnt ein altes Install.

```
exit [::tclutils::tutestrun::run $here {tclutils} \
    -limitvar TCLUTILS_TESTGRENZE -argv $argv]
```

Was `run` tut:

1. **Eigene Bäume zuerst.** Nur die eigenen Bäume (`lib/tm` des Repos und die Geschwister-Checkouts der genannten Libs) werden aus `TCL8_6_TM_PATH` / `TCL9_0_TM_PATH` entfernt und im Runner vorn eingesetzt. Fremde Einträge bleiben stehen. Grund: steht der Baum schon hinter einem `site-tcl` mit derselben Versionsnummer, bewirkt `tcl::tm::path add` nichts, und getestet würde die installierte Kopie.
2. **Banner.** Wohin jede Lib aufgelöst wird — gesucht, ohne etwas zu laden —, andere Versionen daneben als `(also: <version>)`, gesetzte Variablen (`TCL8_6_TM_PATH`, `TCL9_0_TM_PATH`, `TCLLIBPATH`, `<LIB>_TM`) als `note: … is set: …`.
3. **Grenze pro Datei.** `-limit` Sekunden (Default 120) oder der Wert der Umgebungsvariable aus `-limitvar`, wenn er eine positive Ganzzahl ist; eine hängende Datei wird beendet (Unix: `kill -TERM`, dann `-KILL`; Windows: `taskkill /T /F`).
4. **Ungültige Bytes.** Tcl 9 liest Kanäle strikt: ein Byte, das in der Systemkodierung ungültig ist, ließ den Lauf bis zur Grenze stehen. tutestrun ersetzt solche Bytes (`-profile replace`); Tcl 8.6 braucht das nicht.
5. **Testdateien in UTF-8.** Jede Datei startet als `tclsh -encoding utf-8 datei.test`. Tcl 8.6 läse sie sonst in der Systemkodierung, unter Windows cp1252, und jeder erwartete Wert mit Umlaut würde falsch; mit tutestrun 0.1 schlugen so 19 Tests fehl.
6. **Zusammenfassung.** Fehlgeschlagene Tests, fehlende tcltest-Summe, Exit ungleich 0 oder Hänger landen unter "Files with problems". tcltest beendet eine Datei auch bei Fehlern mit 0 — die Summenzeile zählt.

Optionen von `run`: `-argv`, `-limit`, `-limitvar`. Unbekannte Optionen: `{TCLUTILS TUTESTRUN OPTION <opt>}`. Eine direkt gestartete Testdatei ohne Runner ist dem `site-tcl`-Problem weiter ausgesetzt.

22.3 Herkunft prüfen: tupkgfinder 0.2

Die Paketdatenbank kennt pro Version nur ein `ifneeded`. Liegt dieselbe Version zweimal auf den Pfaden, sieht man die zweite Kopie dort nicht. tupkgfinder 0.2 sucht deshalb selbst:

- `candidates` — jede Kopie eines Pakets auf Modulpfaden und `auto_path`, als Dict `{package version kind path}`.
- `shadows` — pro Paket mit mehr als einer Kopie: welche aktiv ist, und für jede Kopie ein `status` (`active`, `same version`, `other version`). `duplicate` ist 1, wenn eine Version doppelt liegt.

```
package require tclutils::tupkgfinder
foreach d [::tclutils::tupkgfinder::candidates -pattern
    tclutils::tunum] {
    puts "[dict get $d version] [dict get $d kind] [dict get $d path]"
}
foreach d [::tclutils::tupkgfinder::shadows -pattern tclutils::*] {
    if {[dict get $d duplicate]} { puts "doppelt: [dict get $d
    package]" }
}
```

Geladen wird dabei nichts. Die Ausgabe ist `0.4 tm …/lib/tm/tclutils/tunum-0.4.tm`; `shadows` fand keine doppelte Kopie, und `tclutils::tunum` war danach nicht geladen. `shadows` lässt für Pakete mit nicht registrierten Kopien ein unerfüllbares `package require` laufen; dabei liest Tcl die `pkgIndex.tcl`-Dateien, lädt aber nichts.

`kind` ist `tm` (Pfad der `.tm`-Datei) oder `pkgIndex` (Pfad der `pkgIndex.tcl`; nur wörtliche `package ifneeded NAME VERSION`-Zeilen werden erkannt). Falsche Optionen werfen `{TCLUTILS TUPKGFINDER OPTION}`, etwa `unknown option "-x" (-pattern, -tmpaths, -autopath)`.

22.4 Prüfregeln: tulint

tulint prüft Tcl-Quelltexte und liefert die Befunde als Daten: eine Liste von Dicts mit `file`, `line`, `rule`, `severity` und `message`. Das Modul gibt nichts aus und beendet nichts; die Kommandozeile ist `bin/tulint.tcl`. Die Regelnamen sind die Kennungen aus dem Buch *Tcl/Tk Qualität*:

Regel	Stufe	Findet
ENC-1	warning	Nicht-ASCII-Zeichen in einer *.test-Datei
ENC-2	error	source ohne -encoding in pkgIndex.tcl
ENC-3	warning	Nicht-ASCII-Zeichen außerhalb von Kommentaren in einem *.tcl-Skript
TST-3	error	derselbe Testname zweimal in einer Datei

Regel	Stufe	Findet
DAT-2	info	open mit w, wb, w+ oder TRUNC außerhalb von Testdateien

```
package require tclutils::tulint
set f [::tclutils::tulint::checkText \
    "test a {} -result {}\ntest a {} -result {}\n" -file x.test]
::tclutils::tulint::format $f
;# -> x.test:2: error TST-3: test name "a" already used in line 1
::tclutils::tulint::summary $f        ;# -> error 1 warning 0 info 0
```

TST-3 liest ganze Befehle (`info complete`), nicht Zeilen, und steigt in geklammerte Rümpfe ab. Zwei Zweige desselben `if` oder `switch` gelten als Alternativen. `check pfade ?-rules namen? ?-exclude muster?` läuft über Verzeichnisse. Eigene Regeln kommen mit `register name stufe beschreibung cmd` dazu; `cmd` bekommt Dateiname und Text und liefert Paare `{zeile meldung}`. Ein Kommentar `# tulint: ignore` REGEL gilt für die nächste Zeile, `;# tulint: ignore` REGEL am Zeilenende für diese Zeile.

`bin/tulint.tcl` endet mit 1, sobald ein Befund der Stufe `error` dabei ist. `--severity` filtert nur die Ausgabe, nicht den Exit-Code. Eine unbekannte Option oder Regel ist ein Fehler `{TCLUTILS TULINT OPTION}` bzw. `RULE`.

22.5 Windows

Laut CHANGELOG läuft die Suite seit 0.64.0 auch unter Windows: keine `/dev/null`- oder Unix-Wurzeln in Fixtures; Tests, die ein CA-Bundle brauchen, werden ohne eines übersprungen (Constraint `ca0K`). In 0.65.0 kamen die Windows-Tests `tutask` 2.8w/4.4w für `pipe` hinzu.

22.6 Zusammenfassung

- `tests/all.tcl` → tutestrun: eigene Bäume zuerst, Banner, Limit.
- Runner aus dem Baum sourcen, nicht blind require.
- Doppelte Kopien findet `tupkgfinder::shadows`.
- Fehlerklassen aus dem Qualitätsbuch prüft `tulint`.

Nächstes Kapitel: Grenzen

23 Grenzen

- Kein Tk — Widgets und Dialoge gehören nach tkutils.
- Kein vollständiges GNU-coreutils; nur die abgebildeten Filter.
- SFTP und TDBC: eigenes `package require`, ggf. externe Pakete.
- `tuflow` lädt `tudiagram` sofort. Die übrigen Diagramm-Module erst beim ersten Abruf der betreffenden Art. `parse` liest Flowcharts; `pie` und die weiteren Arten zeichnet `toSvg`. `validate` wirft nicht. Siehe Diagramme.
- `tustr::graphemes` ist vereinfachtes UAX #29 (keine Hangul-Silben, keine indischen Konjunkte). `slugify` liefert ASCII; Schriften ohne lateinische Buchstaben fallen weg.
- `tupdf` zählt Seiten an `/Type /Page`. `zugferd` sucht Marker, es prüft die Rechnung nicht.

`object` wirft `NOTFOUND`, wenn die Nummer nach dem Aufblasen fehlt. Siehe ODF und PDF-Struktur.
- `tuodf` liest Absätze aus `text:p`. Tabellen übergeht es. `text` verbindet mit `\n`, ein leerer Schlussabsatz lässt den Text damit enden.
- `tupdftext` liest keine Verschlüsselung und keinen Text aus Bildern, hält Spalten nicht auseinander und setzt am Zeilenende getrennte Wörter nicht zusammen.
- `tuuniq` entfernt nur benachbarte Doppelte. `tucut` ist kein CSV.
- `tujson::parse` macht `true` zu `1` und `null` zu leer.
- `tuhash` ist reines Tcl. Text wird vor dem Digest nach UTF-8 gewandelt.
- `tumd` kennt keine verschachtelten Listen, Tabellen und Setext-Überschriften. Der Anker behält Buchstaben jeder Schrift, Ziffern, `_` und `-`.
- `tuical` und `tuvcard` lassen Eigenschaftswerte roh. `toIcs` und `toVcf` schreiben CR/LF.
- `tuholiday` kennt nur `-region de` und keine landesspezifischen Tage.
- `tudict::getOr` stellt die Vorgabe vor den Pfad. `flatten` nimmt eine gerade Liste als Dict.
- `tudiff` baut die Vergleichstabelle im Speicher, Grenze `-maxcells`.
- `tusqlite` lädt `sqlite3` nicht. Eine als NULL geschriebene Zelle kommt aus `rows` leer zurück.
- `tutdbc` lädt keinen TDBC-Treiber und steht nicht im Umbrella. SQL-NULL bleibt eine Spalte mit leerem Wert. Siehe TDBC und MaxMind.
- `tummdb` hält die Datei im Speicher. `open` und `close` sind nicht exportiert. Eine Adresse ohne Satz ergibt die leere Zeichenkette.
- `tudate::diff -unit days` teilt Sekunden durch 86400. `relative` antwortet englisch.
- `tupath::relative` verlangt zwei absolute oder zwei relative Pfade.
- Ein ZIP-Provider kennt kein `put`. Die Meldung hat keinen `-errorcode`.
- `tufetch` ohne Server liefert `URL` und `CAFILE`. `socketCmd` öffnet keinen Socket. Unter tls 2 hängt es kein CA-Bundle an.
- `tutask`: Die Sandbox läuft im aufrufenden Thread, parallel ist nur der Pool (Paket `Thread`). `pipe` beendet unter Unix beim Abbruch nur die direkten Prozesse, keine Enkel. `-commands` allein stoppt keine leere Schleife. Siehe Tasks mit tutask.
- Modul-Docs und man sind die API-Wahrheit; dieses Buch ist der Pfad.

Stack, Dateiblick, Explorer: Buch *Utils-Stack*.

23.1 Windows-Hinweise

Die Tabelle gibt Code und CHANGELOG wieder, nicht eigene Tests unter Windows.

Thema	Verhalten unter Windows
tunum::format	rundet wie unter Linux (seit 0.4), nicht nach C-Laufzeit
tupath::clean	.. endet an der Laufwerkswurzel C:/
tusettings::dir	%APPDATA%, %LOCALAPPDATA%, %TEMP%
Recent-Liste	gleiche Datei ohne Groß/Klein-Unterschied erkannt
HTTPS mit tls 1.x	kein System-CA-Speicher: SSL_CERT_FILE oder -cafile
tufetch ohne CA-Bundle	weicht auf curl/wget aus, sonst NOMETHOD

Thema	Verhalten unter Windows
Testsuite	läuft laut CHANGELOG; CA-Tests ohne Bundle übersprungen
tutask::pipe	Abbruch per taskkill /T /F, auch Kindprozesse

23.2 Zusammenfassung

- Engines ja, GUI und App-Glue nein.
- Optionale Backends explizit laden.
- Unter Windows: CA-Bundle für tls 1.x bereitstellen.
- `tutask` hat dokumentierte Grenzen bei Prozessen und Sandbox-Limits.

Nächstes Kapitel: Fallen

24 Fallen

- `lappend auto_path` statt `tcl::tm::path add` für `.tm`-Bäume.
- Suite ohne Runner starten und dabei `site-tcl` mit gleicher Version vor dem Workspace laden.
- `package require tclutils::tutestrun` gegen ein altes Install — Runner aus dem Baum `source`-n.
- HTTPS-Politik pro Modul selbst registrieren und nie zurücknehmen — `tuhttps` nutzen.
- `tusettings`-Werte mit Zeilenumbruch oder führenden Leerzeichen — INI-Grenze, Fehler `TUSETTINGS VALUE`.
- `tusettings::load …` ohne vollen Namensraum aufrufen — `invalid command name`; richtig `::tclutils::tusettings::load`.
- `TUFETCH CAFILE` als "keine CA" lesen — es heißt: `-cafile` nicht lesbar. Fehlt jede CA unter tls 1.x: `TUHTTPS NOCA` bzw. `TUFETCH NOMETHOD`.
- In einen ZIP-Provider schreiben wollen — caps sind `get list stat`.
- Beträge mit `format %.2f` runden — Ergebnis hängt von der C-Laufzeit ab; `tunum::format` nutzen.
- `tuuniq` auf einer unsortierten Liste — es entfernt nur Nachbarn, `a b b a` bleibt `a b a`.
- `tucut` für CSV mit Anführungszeichen — das schneidet am Trennzeichen; Felder mit Komma gehören zu `tucsv`.
- `tujson::parse` und danach `null` oder `false` prüfen — `null` ist leer, `true` ist `1`, `false` ist `0`. Die Art behält `parseTyped`.
- In einen ZIP-Provider schreiben und auf einen `-errorcode` warten — die Meldung ist `does not support 'put'`, ohne Code.
- `string length` als sichtbare Zeichenzahl nehmen — `tustr::graphemes`. `truncate` und `padLeft` rechnen weiter mit `string length`.
- `tustr::slugify` und den Anker von `tumd` für dasselbe halten — `slugify` macht aus "Größe" `groesse` (ASCII), `tumd` den Anker `größe`.
- Eine eingerückte Markdown-Liste als verschachtelt lesen — `tumd` macht daraus eine flache Liste.
- `tunotes::create` den Speicherwert übergeben — die Änderung verlangt den

Variablennamen. `get` und `search` bekommen den Wert.
- `tunotes::search` als regulären Ausdruck lesen — es ist `string match`, ein Stern in der Frage ist ein Musterzeichen.
- Landesspezifische Feiertage bei `tuholiday` erwarten — nur die bundesweite Menge `de`, dazu Oster- und Pfingstsonntag (ohne sie: `-sundays 0`).
- `tuflow::parse` auf ein `pie` — `{TCLUTILS TUFLOW UNSUPPORTED}`. Zeichnen geht über `toSvg`.
- `tudiagram::addEdge` auf unbekannte Ids und einen Fehler erwarten — die Kante hängt am Modell. `validate` meldet `NONODE` und wirft nicht.
- `flowchart TD` als Richtung `TD` im Modell lesen — sie wird `TB`.
- `tunum::parse -locale de-strict` und ein Punkt als Dezimalzeichen — `956.06` wird `95606.0`.
- `tudate::diff -unit days` über die Sommerzeit als Kalendertage lesen — Sekunden durch 86400; 15. März bis 15. April 2024 ergibt 30.
- `02/03/2026` als 3. Februar lesen — `tudate::parse` nimmt `d/m/Y` vor `m/d/Y`, das ist der 2. März. `m/d/Y` greift erst, wenn `d/m/Y` kein Datum ergibt (`10/29/2026`).
- Eine Zeichenkette mit Umlaut an `tubase64::encode` oder `tucrc::crc32` ohne `-encoding utf-8` — ö wird das eine Byte `f6`, € ist `BYTES`. `tuhash` nimmt dagegen UTF-8.
- `tudate::relative` deutsch erwarten — die Texte sind `today`, `tomorrow`, `yesterday`.
- `tudict::getOr` wie `dict get` lesen — die Vorgabe steht vor dem Pfad: `getOr $d vorgabe schlüssel …`.
- `tudict::flatten` auf einem Wert wie `a b` — gerade Länge gilt als Dict, `{name {a b}}` wird `name.a b`.
- `tuhash` auf Dateibytes mit der Textvorgabe — `sha256` kodiert UTF-8; die Bytes der Datei hasht `sha256File`.
- Ungültiges Base64 als Fehler erwarten — `decode` von `!!!` ist leer, ohne `-errorcode`.
- SQL-NULL aus `tusqlite::rows` am Sentinel erkennen — die Zelle kommt leer zurück. NULL setzt `insert` mit `null`.
- In `tusqlite::rows` einen Platzhalter `:x` aus einer Variablen `x` erwarten — seit 0.2 kommt der Wert nur aus dem Parameter-Dict; ohne Eintrag ist er NULL.
- Bei TDBC `allrows -as dicts` eine fehlende Spalte als leer lesen — NULL-Spalten fallen aus dem Dict. `tutdbc::rows` setzt sie leer wieder ein. `0` bleibt `0`.
- `tummdb::close` auf einem unbekannten Handle als Fehler erwarten — der Aufruf gibt die leere Zeichenkette zurück. `metadata` wirft `HANDLE`.
- `tupdftext` auf einem Scan — `{TCLUTILS TUPDFTEXT NOTEXT}`, nicht ein stiller Leerstring. `-notext empty` liefert die leere Seite.
- `tupdf::object` auf eine Nummer, die nach dem Aufblasen fehlt — `{TCLUTILS TUPDF NOTFOUND}`. Die Meldung nennt einen Objektstrom.
- `tuodf::text` nach einem leeren Schlussabsatz ohne Zeilenumbruch erwarten — die Absätze hängen an `\n`, der Text endet damit.
- Eine zweite Kopie derselben Version im Modulpfad übersehen — `tupkgfinder::shadows`.
- `$t wait` außerhalb eines Tasks in einem Tk-Handler aufrufen — verschachteltes `vwait`; im Handler nur `spawn`.
- `await` außerhalb einer Koroutine — `TUTASK NOTASK`; in `spawn` oder `run` einpacken.
- Sandbox nur mit `-commands` begrenzen — `while 1 {}` läuft weiter; `-time` mit setzen.
- Unter Unix erwarten, dass `tutask::pipe` beim Abbruch auch Enkelprozesse beendet — nur

die direkten Prozesse der Pipeline.
- Einen Task-Fehler nie abholen — beim `destroy` kommt er über `bgerror`.
- tkutils-Code in tclutils erwarten.

24.1 Behoben in 0.66.0

Diese Fallen gelten bis 0.65.0. In 0.66.0 sind sie behoben:

- `$g wait -mode first` außerhalb eines Tasks und die Gruppe sofort zerstören — in 0.65.0 später `bgerror` "invalid command name ...::my"; bis 0.65.0 nötig: `after idle [list $g destroy]`.
- `pool` scheitert im Konstruktor (`NOTHREAD`, `-size 0`) — in 0.65.0 zusätzlich `bgerror can't read "jobs": no such variable`.
- `-init` des Pools scheitert — in 0.65.0 bleibt der Job in der Warteschlange (`queued 1 jobs 1`).
- Älteres `tclutils::common` als 0.2 auf dem Pfad — `tutask` 0.1 lädt trotzdem, `group` und `pool` scheitern erst beim Aufruf. `tutask` 0.2 verlangt `common` 0.2.
- `run -timeout`, `$t wait -timeout` oder `$g wait -timeout` außerhalb eines Tasks mit `abc` oder `-5` — bis 0.65.0 still angenommen, es gab keinen Timeout. Ab 0.66.0 `TUTASK OPTION` wie bei `await`.

24.2 Zusammenfassung

- Die meisten Fallen betreffen Modulpfade, HTTPS-Politik und Rundung.
- Neu in 0.65.0: Fallen rund um `tutask` (Tk-Handler, Gruppen, Sandbox, Prozesse). Vier davon und die stille Annahme falscher `-timeout`-Werte sind in 0.66.0 behoben.

Nächstes Kapitel: Module

25 Module (Orientierung)

Keine vollständige Liste — Stand und Status: `docs/guide/module-status.md` und `README.md` im tclutils-Baum. Hygiene-Zählung mit `tools/check-modules.tcl -require test,doc,man`: **146** Module im Baum 0.69.0, keines ohne Test, Doc oder man-Seite; 25 davon nicht direkt im Umbrella.

25.1 Kategorien

Kategorie	Beispiele
Textfilter	tucat, tuhead, tugrep, tusort, tucut, tused (Kapitel Text und Dateien); tunl, tufold, tufmt, tuexpand, tucolumn, tutable, tutac, turev, tushuf, tuseq, tutr, tuwc, tupr (Kapitel Weitere Zeilenfilter)
Zusammenführen / Abläufe	tupaste, tujoin, tucomm, tucsplit, tusplit, tutee, tuxargs, tutsort, tuawk, tufuzzy, tuagrep (Kapitel Zusammenführen, Aufteilen, Abläufe)

Kategorie	Beispiele
Daten	tucsv, tujson, tuini, tuxml, tusqlite, tuldif, tubookmark (Kapitel Datenformate); tutdbc, tummdb (Kapitel TDBC und MaxMind)
Kodierung	tubase64, tubase32, tucrc, tuhash, tuxxhash, tuiconv, tuuuid, tuurl, tudict, tudiff, tupatch (Kapitel Kodierung, Hash und Vergleich)
Binärdaten	tuhexdump, tuod, tustrings, tuhexedit, tucmp, tusize, tustat, tucode (Kapitel Binärdaten und Dateiinformationen)
Notizen	tumd, tuical, turrule, tuvcard, tunotes, tunotesdb, tucal, tuholiday (Kapitel Notizen und Kalender)
Dokumente	tupdftext, tupdf, tuodf (Kapitel Text aus PDF-Dateien, ODF und PDF-Struktur)
Diagramme	tudiagram, tuflow und die Arten aus toSvg (Kapitel Diagramme)
Bilder	tucolor, tupng, tupngdraw, tutablepng, tucodepng, tumonthpng, tupngpad, tusvg, tuimage, tuico, tudhash (Kapitel Bilder ohne Tk)
FS / Archiv	tufile, tufind, tuzip
Provider	tuprovider, ...::zip, ...::dav
Anwendung	tulog, tuevent, turegistry, tuvalidate, tulist, tumath, tunumany, tuappinfo, tudeploy, tuexe, tuopen, tuzipfs, tuterm, tupagespec, tulayout (Kapitel Bausteine für Anwendungen)
Netz	tufetch, tudav, tuhttps, tusparql
Settings / Test	tusettings, tutestrun, tulint
Tasks / async	tutask
Strings / Datum / Zahlen	tustr, tudate, tunum, tupath (Kapitel Zeichen, Zahlen, Datum und Pfade)
Diagnose	tupkgfinder, tuappinfo, tusysinfo, tuedid

25.2 Neu in 0.69.0

Modul	Version	Änderung
tucsv	0.2	-profile excel-de oder rfc4180, text -bom 1, -encoding für file und writeFile (Vorgabe UTF-8); writeFile behält die Zeilenenden von -newline

Modul	Version	Änderung
tusqlite	0.2	rows und value mit Parameter-Dict, Variablen des Aufrufers erreichen die Abfrage nicht mehr (Verhalten); neu update, delete
tusysinfo	0.1	neu: Betriebssystem, CPU, Speicher, Modell, Festplatten, Monitore; im Umbrella; bin/tusysinfo.tcl
tuedid	0.1	neu: EDID-Block eines Monitors lesen; im Umbrella
tuholiday	0.2	-sundays 0 für die neun bundesweiten Tage; Vorgabe 1 wie bisher
tuzip	0.2	Namen in UTF-8 mit UTF-8-Flag (Verhalten); Lesen mit -encoding auto
tudiff	0.2	-eol ignore|strict, -encoding für Dateien; text übergeht CR auch in Zeichenketten (Verhalten)
tubookmark	0.3	Reihenfolge bleibt, Schlüssel folders, HREF in einfachen Anführungszeichen (Verhalten); <DD>-Beschreibung und Lesezeichenleiste bleiben erhalten
tuical, tuvcard	0.2	toIcs und toVcf schließen auch die letzte Zeile mit CR/LF ab (Verhalten)
tuini	0.2	setValue, addSection, toIni lehnen ab, was INI nicht fassen kann, etwa einen Zeilenumbruch im Wert (Verhalten)
tupatch	0.3	das letzte \n bleibt; \ No newline at end of file wie bei diff -u (Verhalten)
tusplit	0.3	Zeilenteile behalten ihre Zeilenenden; file -lines teilt Bytes (Verhalten)
tutee	0.3	writeFile kopiert Bytes, gibt die Bytezahl zurück (Verhalten)
tumd	0.2	Anker wie bei GitHub, mit Umlauten und -1, -2 (Verhalten)
tuiconv	0.2	streng unter 8.6 und 9.0, -replace 1 (Verhalten)
tucat	0.2	keine Nummer nach dem letzten \n, files zählt durch (Verhalten)

Modul	Version	Änderung
tuldif	0.3	Binärattribute behalten ihre Bytes, -binary (Verhalten)
tucsplit	0.2	Teile behalten ihre Zeilenenden, -encoding (Verhalten)
tuhead, tutail	0.2	Zeilenenden wie head/tail, text = file, COUNT (Verhalten)
tugrep	0.2	keine leere Zeile hinter dem letzten \n (Verhalten)
tustr	0.3	slugify schreibt Umlaute um, toCamel/ toSnake mit Unicode (Verhalten)
tuer	0.4	Entitätsnamen mit Umlauten; SYNTAX (Verhalten)
tugantt	0.3	unbekannte Einheit DURATION; Ids mit Umlauten; writeSvg/writePng über temporäre Datei (Verhalten)
tuxychart	0.2	COUNT, wenn Werte und Kategorien nicht passen; Schreiben über temporäre Datei (Verhalten)
tusankey	0.2	schlechte Zeilen VALUE statt still weg; Schreiben über temporäre Datei (Verhalten)
tudate	0.3	Tage und Monate, die es nicht gibt, sind PARSE, auch unter 8.6; m/d/Y nach d/m/ Y; mehr Formate (Verhalten)
tunumany	0.3	Dezimalkomma vor der Einheit, 1,5K ist 1500 (Verhalten)
tubase64, tucrc	0.2	-encoding binary|utf-8; ein Zeichen über 255 ist BYTES (Verhalten)
tudav	0.3	Basic-Anmeldung in UTF-8; . und .. in einer href aufgelöst (Verhalten)
tuflow	0.4	Knoten-Ids mit Umlauten (Verhalten)
tuc4	0.2	Ids mit Umlauten (Verhalten)
tugit	0.3	doppelte Commit-Id ist DUPID (Verhalten)
turadar	0.3	genau ein Wert je Achse, sonst COUNT oder AXIS; Schreiben über temporäre Datei (Verhalten)

Modul	Version	Änderung
tusort, tuuniq, tucut, tujoin, tucolumn, tufmt	0.2	abschließendes \n wie in der Eingabe (Verhalten)
tupaste, tushuf, tufold	0.3	abschließendes \n wie in der Eingabe (Verhalten)
tutestrun	0.2	Testdateien mit -encoding utf-8 (Verhalten)
tuarchitecture	0.3	Ids mit Umlauten (Verhalten)
tustate, tublock	0.2	Ids mit Umlauten (Verhalten)
tunotes	0.2	Datei immer UTF-8; ältere Dateien in der Systemkodierung werden noch gelesen (Verhalten)
tumindmap	0.2	Ids mit Umlauten vor einer Form (Verhalten)
tulint	0.1	neu: Prüfregeln für Tcl-Quelltexte, nicht im Umbrella
common	0.3	neu replaceFile: Schreiben über temporäre Datei
tused, tusettings	0.2	Schreiben über replaceFile; scheitert es, bleibt die alte Datei und kein Kanal offen (Verhalten)
tuprovider	0.3	lokales put über replaceFile (Verhalten)
tutreemap	0.3	:::Klasse mit Umlauten ganz entfernt; Schreiben über temporäre Datei (Verhalten)
tuclass	0.3	; in Anführungszeichen trennt keine Mitglieder (Verhalten)
tuimage	0.2	;base64 in einem Parameter in Anführungszeichen schaltet Base64 nicht ein (Verhalten)
tupacket, tujourney, tutimeline, tupie, tusequence	0.3	writeSvg/writePng über replaceFile (Verhalten)
tukanban, tuquadrant	0.2	writeSvg/writePng über replaceFile (Verhalten)
tudiagram	0.6	writePng über replaceFile (Verhalten)
tusvg	0.4	write über replaceFile (Verhalten)
tupngdraw	0.14	write über replaceFile (Verhalten)

Modul	Version	Änderung
tupng	0.6	write* über replaceFile (Verhalten)
tuappinfo	0.3	Tk-Zeile mit Patchlevel; writeLog UTF-8 über replaceFile (Verhalten)

Beim Überkopieren eines älteren Baums `tucsv-0.1.tm`, `tusqlite-0.1.tm`, `tuholiday-0.1.tm`, `tuzip-0.1.tm`, `tudiff-0.1.tm`, `tubookmark-0.2.tm`, `tuical-0.1.tm`, `tuvcard-0.1.tm`, `tuini-0.1.tm`, `tupatch-0.2.tm`, `tusplit-0.2.tm`, `tutee-0.2.tm`, `tumd-0.1.tm`, `tuiconv-0.1.tm`, `tucat-0.1.tm`, `tuldif-0.2.tm`, `tucsplit-0.1.tm`, `tuhead-0.1.tm`, `tutail-0.1.tm`, `tugrep-0.1.3.tm`, `tustr-0.2.tm`, `tuer-0.3.tm`, `tugantt-0.2.tm`, `tuxychart-0.1.tm`, `tusankey-0.1.tm`, `tudate-0.2.tm`, `tunumany-0.2.tm`, `tubase64-0.1.tm`, `tucrc-0.1.tm`, `tudav-0.2.tm`, `tuflow-0.3.tm`, `tuc4-0.1.tm`, `tugit-0.2.tm`, `turadar-0.2.tm`, `tusort-0.1.tm`, `tuuniq-0.1.tm`, `tucut-0.1.tm`, `tupaste-0.2.tm`, `tujoin-0.1.2.tm`, `tushuf-0.2.tm`, `tucolumn-0.1.tm`, `tufold-0.2.tm`, `tufmt-0.1.tm`, `tutestrun-0.1.tm`, `tuarchitecture-0.2.tm`, `tustate-0.1.tm`, `tublock-0.1.tm`, `tunotes-0.1.tm`, `tumindmap-0.1.tm`, `common-0.2.tm`, `tused-0.1.3.tm`, `tusettings-0.1.tm`, `tuimage-0.1.tm`, `tuprovider-0.2.tm`, `tutreemap-0.2.tm`, `tuclass-0.2.tm`, `tupacket-0.2.tm`, `tujourney-0.2.tm`, `tukanban-0.1.tm`, `tutimeline-0.2.tm`, `tupie-0.2.tm`, `tuquadrant-0.1.tm`, `tusequence-0.2.tm`, `tudiagram-0.5.tm`, `tusvg-0.3.tm`, `tupngdraw-0.13.tm`, `tupng-0.5.tm`, `tuappinfo-0.2.tm` und `tclutils-0.68.0.tm` löschen. Der Umbrella steht auf 0.69.0.

25.3 Neu in 0.68.0

Korrekturen aus der Prüfung dieses Buchs; jede mit Regressionstest. Mit "Verhalten" markierte Zeilen ändern bisherige Ergebnisse.

Modul	Version	Änderung
tuagrep	0.2	-maxdist geprüft; search ohne leere Schlusszeile; Fehlercodes (Verhalten); zahlähnlicher Text bleibt
tuappinfo	0.2	-anonymize verdeckt alle Pfade; unbekannte Optionen sind Fehler
tuarchitecture	0.2	Beschriftung, die auf die Id zurückfällt
tuawk	0.2	Regeln in eigenem Rahmen (8.6 = 9.0); NR/NF vorab 0; Option ohne Wert ist Fehler
tubase32	0.2	decode prüft Länge und Füllbits (LENGTH, BITS) (Verhalten)
tubookmark	0.2	Maskierung von "; jedes <A> ein Eintrag
tucal	0.2	year zeilenweise wie cal; Titel mittig; keine Leerzeichen am Zeilenende (Verhalten)

Modul	Version	Änderung
tuclass	0.2	einzeiliger Klassenblock; Kanten mit Kardinalitäten bleiben
tucmp	0.2	vergleicht Bytes; Zeichen über 255 sind BYTES (Verhalten)
tucode	0.2	signs mit Optionen; lookup U+00E4; Matrix bündig
tucolor	0.2	Tripel dezimal (8.6 = 9.0); fromHsv prüft Bereiche (HSV) (Verhalten)
tucomm	0.2	-sort 1 sortiert wie der Vergleich (Verhalten)
tudeploy	0.2	addModulePaths ohne abgelehnte Wurzeln; Optionen zuerst geprüft
tudhash	0.2	Breite und Höhe geprüft
tudiagram	0.5	Titel wird gezeichnet; unbekanntes Theme ist THEME; zahlähnlicher Text bleibt
tuer	0.3	Anführungszeichen um Beschriftungen entfernt
tuexe	0.2	Option ohne Wert ist Fehler
tuexpand	0.2	unexpand wie GNU; expand lehnt -all ab (Verhalten)
tufile	0.2	type, mime unverändert
tufind	0.2	-tails-Name unverändert
tuflow	0.3	Kopfzeile ohne Groß/Klein; A & B --> C; sankey erkannt
tufold	0.2	-words 1 ohne Leerzeichen am Zeilenende
tufuzzy	0.2	Fehlercodes; -nocase geprüft
tugantt	0.2	ohne Startdatum DATE; Achse ohne doppelte Daten; Höhe automatisch
tugit	0.2	Commit-Id als Beschriftung
tugrep	0.1.3	Dateiname bei -fileswithmatches
tuhexdump	0.2	Optionen geprüft; Fehlercodes; BYTES
tuhexedit	0.2	OFFSET, RANGE (kein Schreiben über das Ende), BYTES, ENCODING (Verhalten)
tuico	0.2	Verzeichnis vorzeichenlos gelesen
tujoin	0.1.2	Optionen über parseOptions; Fehlercodes
tujourney	0.2	Bewertungen mit Nachkommastelle; sonst VALUE

Modul	Version	Änderung
tulayout	0.2	ensureBlocks nimmt Definitionen; Optionen geprüft (OPTION)
tuldif	0.2	Base64 über UTF-8; CHANGETYPE (Verhalten)
tulog	0.2	Optionen vor dem Anlegen geprüft
tummdb	0.2	IPv6 an :: richtig aufgefüllt
tumonthpng	0.5	Kopf folgt -firstweekday; -title bei Quartal und Jahr (Verhalten); zahlähnlicher Text bleibt
tunl	0.2	Start, Schritt, Breite geprüft
tunotesdb	0.2	fromStore behält Kinder; Kind vor Elter erlaubt
tunumany	0.2	Option ohne Wert ist Fehler
tunumfmt	0.2	KiB usw. als IEC (2KiB = 2048)
tuod	0.2	-format/-address vorab geprüft; BYTES
tuopen	0.2	alle Befehle exportiert; URLs unter Windows über rundll32
tupacket	0.2	Feldbeschriftung
tupagespec	0.2	Zahlen immer dezimal; zahlähnlicher Text bleibt
tupaste	0.2	Optionen über parseOptions
tupatch	0.2	-reverse/-strict geprüft
tupie	0.2	1e3 gültig; negative Werte VALUE
tupng	0.5	tRNS bei Farbtyp 2; CRC geprüft (CRC); zahlähnlicher Text bleibt
tupngdraw	0.13	paste mischt Alpha richtig
tupngpad	0.2	Optionen geprüft; unbekanntes -align ist Fehler (Verhalten)
tuprovider	0.2	head gibt Inhalt unverändert
tuprovider::sftp	0.2	Host im sftp-Ziel
turadar	0.2	Legende nur bei benannten Reihen; zahlähnlicher Text bleibt
turrule	0.2	YEARLY mit BYMONTH/BYDAY/BYMONTHDAY; UNTIL als ganzer Tag; UNSUPPORTED (Verhalten)
tuseq	0.2	Optionen an jeder Stelle; große Zahlen unter 8.6

Modul	Version	Änderung
tusequence	0.2	<<->>; Titel wird gezeichnet; zahlähnlicher Text bleibt
tushuf	0.2	große -seed unter 8.6
tusize	0.2	Links mit eigener Größe; human bis YiB (Verhalten)
tusparql	0.2	Endpunkt mit ?; Accept nur einmal; alle Variablen; zahlähnlicher Text bleibt
tusplit	0.2	splitBytes mit BYTES; fehlender Wert ist Fehler
tustat	0.2	Links mit lstat und target; modeOctal (Verhalten)
tustrings	0.2	-minlength geprüft; -encoding entfällt; BYTES (Verhalten)
tusvg	0.3	Methode icon gültig; 149 Symbole
tutable	0.2	COLUMNS, HEADERS; Mindestbreite 3
tutablepng	0.2	Zelltext unverändert; -padding/-spacing geprüft
tutdbc	0.2	Spaltenwerte in rows unverändert (01234)
tutee	0.2	erst alle Ziele öffnen, bei Fehler aufräumen
tuterm	0.2	names reset; enable geprüft (Verhalten)
tutimeline	0.2	12:30 bleibt ganz
tutr	0.2	RANGE, SET2; letzte Zuordnung gilt; nur verdichten möglich
tutreemap	0.2	Werte ≤ 0 fallen weg
tutsort	0.2	Zyklusmeldung nur mit Zyklusknoten
tuvalidate	0.2	port nur Dezimalziffern; ipv4 ohne führende Nullen
tuwc	0.2	file: bytes ist die Dateigröße (Verhalten)
tuxxhash	0.2	Text als UTF-8, Option -encoding (Verhalten)
tuzipfs	0.3	rcopy mit Punktdateien

Beim Überkopieren eines älteren Baums die alte Datei des Moduls löschen (z. B. `tuawk-0.1.tm` neben `tuawk-0.2.tm`). Der Umbrella stand auf 0.68.0.

25.4 Neu in 0.67.0

Modul	Version	Änderung
tupdftext	0.1	neu: Text einer PDF je Seite, reines Tcl; im Umbrella; tupdf.tcl text

Sonst blieben die Modulversionen gleich; der Umbrella stand auf 0.67.0.

25.5 Neu in 0.66.0

Modul	Version	Änderung
tutask	0.2	verlangt common 0.2; Pool-und Gruppen-Fehler behoben; -timeout überall geprüft

Sonst blieben die Modulversionen gleich; der Umbrella stand auf 0.66.0. Beim Überkopieren eines älteren Baums `tutask-0.1.tm` löschen.

25.6 Neu in 0.65.0

Modul	Version	Änderung
tutask	0.1	neu: Tasks auf Koroutinen, await, cancel, Gruppen, Pool, Sandbox; im Umbrella

Sonst blieben die Modulversionen gleich; der Umbrella stand auf 0.65.0.

25.7 Neu und geändert in 0.64.0

Modul	Version	Änderung
tustr	0.2	graphemes, isPrintable (Unicode)
tudate	0.2	zones
tunum	0.4	format rundet halb von null weg, plattformgleich
tupath	0.1	clean behält die Wurzel (C:/ unter Windows)
tusettings	0.1	neu: INI-Settings, dir, Recent-Liste
tutestrun	0.1	neu: gemeinsamer Runner, nicht im Umbrella
tupkgfinder	0.2	candidates, shadows

Aus 0.63.0 weiter relevant: `tuhttps` 0.1 (neu), `tufetch` 0.4, `tudav` 0.2, `tupostgrest` 0.2, `tuprovider::ftp` 0.2 (tcllib erst beim `open`), `common` 0.2 (`ensureOneOf`).

25.8 Zusammenfassung

- 146 Module im Baum 0.69.0, alle mit Test, Doc und man-Seite.
- Neu in 0.69.0: `tucsv`, `tusqlite`, `tuholiday`, `tuzip`, `tudiff`, `tuical`, `tuvcard`, `tuini`, `tumd`, `tuiconv`, `tucat`, `tucsplit`, `tuhead`, `tutail`, `tugrep`, `tuxychart`, `tusankey` 0.2, `tupatch`, `tusplit`, `tutee`, `tuldif`, `tustr`, `tugantt` 0.3, `tuer` 0.4, `tubookmark` 0.3, `tudate`, `tunumany`, `tubase64`, `tucrc`, `tudav`, `tuflow`, `tuc4`, `tugit`, `turadar` und neun Zeilenfilter, neu `tusysinfo` und `tuedid` 0.1.
- In 0.68.0 haben 76 Module Korrekturen und eine neue Version bekommen.
- Neu in 0.67.0: `tupdftext` 0.1.
- In 0.66.0 geändert ist nur `tutask` (0.2); neu in 0.65.0 war `tutask` 0.1.

Nächstes Kapitel: Glossar

26 Glossar

Umbrella
Paket `tclutils`, lädt die Kernmodule.

Modulpfad
`tcl::tm::path` — wo `.tm`-Dateien gefunden werden.

Provider
Einheitliche Speicher-API (`tuprovider` + Backends).

caps
Liste der Operationen, die ein Provider wirklich kann (z. B. ZIP: `get list stat`).

tusettings
Per-User-INI und Recent-Listen.

tutestrun
Gemeinsamer `tests/all.tcl`-Runner der drei Libs.

tuhttps
Gemeinsame HTTPS-Politik: prüfende `tls`-Sockets, `https` nur befristet registriert.

tutask
Tasks auf Koroutinen mit `await`, Timeout, `cancel`, Gruppen, Pool und Sandbox (neu in 0.65.0, in 0.66.0 Version 0.2).

Task
Koroutine im Besitz eines Objekts (`spawn`); Zustand `running`, `ok`, `error` oder `cancelled`.

Starter
Befehlspräfix einer Operation: bekommt einen Callback, gibt einen Abbruchbefehl zurück, meldet einmal `ok` oder `error`.

Sandbox
Sicherer Interpreter mit Befehls- und Zeitgrenze (`tutask::sandbox`).

NOCA
tls < 2.0 und kein CA-Bundle gefunden.

Graphem
Sichtbares Zeichen, ggf. aus mehreren Codepoints (`tustr::graphemes`).

tudiagram
Knoten-Kanten-Modell, Layout in Pixeln, SVG und PNG in reinem Tcl.

tuflow
Flowchart-Text nach `tudiagram`. `toSvg` zeichnet auch die anderen Arten.

tumd
Kleine CommonMark-Teilmenge nach HTML, ohne verschachtelte Listen.

tuholiday
Ostern und die bundesweiten Feiertage, Region `de`.

tutdbc
NULL-feste Helfer auf einer bestehenden TDBC-Verbindung. Kein Treiber, nicht im Umbrella.

tummdb
Leser für MaxMind-DB 2.0. `open` und `close` nur qualifiziert.

tuhash
SHA-256, SHA-1 und MD5 in reinem Tcl. Textvorgabe ist UTF-8.

tuurl

Prozentkodierung und Query-Strings. An der Query ist `-plus` an.

tupdftext
Text einer PDF Seite für Seite, reines Tcl, ohne poppler (0.1, neu in 0.67.0).

tupdf
PDF-Struktur: Version, Objekte, Infos, ZUGFeRD-Hinweis. Der Seitentext liegt bei `tupdftext`.

tuodf
Schlichtes OpenDocument aus Absätzen. `mimetype` ist der erste Eintrag.

Schatten-Kopie
Zweite Kopie eines Pakets auf den Suchpfaden, die `package require` nicht nimmt (`tupkgfinder::shadows`).

26.1 Zusammenfassung

- Begriffe aus Pfaden, Providern, HTTPS, Tests, `tutask`, Notizen, PDF, ODF und TDBC.

Nächstes Kapitel: Fehlermeldungen

27 Fehlermeldungen

Typische `-errorcode`-Muster (Auszug). Vollständig: jeweiliges `docs/<modul>.md`.

Code	Bedeutung
{TCLUTILS COMMON OPTION ...}	unbekannte Option (parseOptions)
{TCLUTILS COMMON VALUE ...}	Wert nicht in erlaubter Menge
{TCLUTILS COMMON BOOLEAN ...}	Option verlangt einen Wahrheitswert
{TCLUTILS COMMON INTEGER ...}	Option verlangt eine positive Ganzzahl
{TCLUTILS TUSETTINGS APP}	App-Name leer oder mit Pfadzeichen
{TCLUTILS TUSETTINGS KIND}	unbekanntes kind bei dir
{TCLUTILS TUSETTINGS SYNTAX}	INI nicht parsebar
{TCLUTILS TUSETTINGS VALUE}	Wert oder Schlüssel nicht INI-tauglich
{TCLUTILS TUHTTPS NOTLS}	Paket tls fehlt
{TCLUTILS TUHTTPS CAFILE}	-cafile keine lesbare Datei
{TCLUTILS TUHTTPS NOCA}	tls < 2.0 und kein CA-Bundle gefunden
{TCLUTILS TUFETCH CAFILE}	-cafile keine lesbare Datei (nur auf dem nativen Weg mit tls)
{TCLUTILS TUFETCH NOMETHOD}	kein Transport (auch: tls 1.x ohne CA-Bundle, kein curl/wget)
{TCLUTILS TUFETCH CONNECT}	Verbindung/DNS/Zertifikat gescheitert
{TCLUTILS TUFETCH HTTP <Klasse> <Status>}	HTTP-Status außerhalb 2xx, z. B. HTTP 4XX 404

Code	Bedeutung
{TCLUTILS TUFETCH URL}	URL weder http:// noch https://
{TCLUTILS TUFETCH TIMEOUT}	Zeitüberschreitung
{TCLUTILS TUFETCH REDIRECT}	zu viele Weiterleitungen
{TCLUTILS TUCSV QUOTE}	CSV-Feld mit offenem Anführungszeichen (-strict 1)
{TCLUTILS TUCSV PROFILE <name>}	-profile weder excel-de noch rfc4180
{TCLUTILS TUJSON PARSE offset}	JSON an dieser Stelle ungültig
{TCLUTILS TUJSON TYPE}	toJson mit unbekannter Art
{TCLUTILS TUINI SYNTAX}	INI nicht parsebar, auch ein offener Abschnitt
{TCLUTILS TUBASE64 MAXLEN}	-maxlen keine ganze Zahl ab 0
{TCLUTILS TUBASE64 BYTES}	encode ohne -encoding utf-8 mit einem Zeichen über 255
{TCLUTILS TUCRC BYTES}	crc32, adler32 ohne -encoding utf-8 mit einem Zeichen über 255
{TCLUTILS TUHASH ENCODING}	-encoding weder utf-8 noch binary
{TCLUTILS TUUUID INVALID}	version ohne UUID-Form
{TCLUTILS TUUUID VERSION}	generate -version weder 4 noch 7
{TCLUTILS TUDICT ARG}	getOr ohne Schlüssel
{TCLUTILS TUDIFF OPTION MAXCELLS}	-maxcells keine positive ganze Zahl
{TCLUTILS TUDIFF TOO_LARGE}	Zeilen alt mal Zeilen neu über -maxcells
{TCLUTILS TUXML NAME <name>}	Elementname ungültig
{TCLUTILS TUXML ATTR <name>}	Attributname ungültig
{TCLUTILS TUSQLITE EMPTY}	insert ohne Spalte, update ohne neuen Wert
{TCLUTILS TUSQLITE WHERE}	update oder delete mit leerem Bedingungs-Dict
{TCLUTILS TUSQLITE PARAM <name>}	Platzhaltername in rows/value ungültig
{TCLUTILS TUSYSINFO OPTION}, SECTION, PLATFORM	collect: unbekannte Option, unbekannter Abschnitt, unbekannte Plattform
{TCLUTILS TUEDID SHORT}, HEADER, HEX	EDID kürzer als 128 Byte, ohne EDID-Kopf, kein Hex
{TCLUTILS TUTDBC EMPTY}	tutdbc::insert ohne Spalte
{TCLUTILS TUMMDB FORMAT}	Datei ist keine MaxMind-DB

Code	Bedeutung
{TCLUTILS TUMMDB HANDLE}	unbekanntes Handle bei metadata oder lookup
{TCLUTILS TUICAL SYNTAX}	iCalendar-Zeile ohne :, oder END ohne BEGIN
{TCLUTILS TUICAL ARG}	newComponent mit ungerader Eigenschaftsliste
{TCLUTILS TUVCARD SYNTAX}	vCard-Zeile ohne :
{TCLUTILS TUNOTES NOTFOUND}	Notiz oder Elter fehlt
{TCLUTILS TUHOLIDAY YEAR}	easter ohne Jahreszahl
{TCLUTILS TUHOLIDAY REGION}	-region ist nicht de
{TCLUTILS TUDIAGRAM DUPID}	Knoten-Id schon vorhanden
{TCLUTILS TUDIAGRAM DIR}	Richtung weder LR, TB, RL noch BT
{TCLUTILS TUDIAGRAM ARG}	Knotenform außerhalb der acht Namen
{TCLUTILS TUFLOW BADID}	ungültige Knoten-Id, etwa mit angehängtem -…
{TCLUTILS TUFLOW EMPTY}	Flowchart ohne Knoten
{TCLUTILS TUFLOW UNSUPPORTED}	parse auf einer Art, die nur toSvg zeichnet
{TCLUTILS TUGIT DUPID}	Commit-Id kommt zweimal vor
{TCLUTILS TUICAL VALUE}, {TCLUTILS TUVCARD VALUE}	Wert mit Zeilenumbruch an setProperty oder addProperty
{TCLUTILS TUSQLITE OPTION}	value mit unbekannter Option oder Option ohne Wert
{TCLUTILS TURADAR COUNT}	Zahl der Werte passt nicht zu den Achsen
{TCLUTILS TURADAR AXIS}	Reihe ohne Wert für eine Achse, oder Schlüssel ohne Achse
{TCLUTILS TUPIE EMPTY}	Kreisdiagramm ohne Sektor
{TCLUTILS TUPIE VALUE}	negativer Wert, oder Sektoren summieren sich nicht positiv
{TCLUTILS TUC4 EMPTY}	C4 ohne Elemente
{TCLUTILS TUSTR ARG}	isPrintable nicht mit genau einem Zeichen; truncate mit negativer Länge
{TCLUTILS TUPATH RELATIVE}	relative: ein Pfad absolut, der andere relativ
{TCLUTILS TUPATH ARGS}	commonPath ohne Pfad

Code	Bedeutung
{TCLUTILS TUPATH NOTLINK}	readlink auf keiner symbolischen Verknüpfung
{TCLUTILS TUDATE PARSE}	tudate::parse liest das Datum nicht, auch einen Tag, den es nicht gibt (2026-02-29)
{TCLUTILS TUDATE UNIT}	tudate::add mit unbekannter Einheit
{TCLUTILS TUNUM DECIMALS}	format -decimals keine ganze Zahl ab 0
{TCLUTILS TUNUM LOCALE}	parse-Locale nicht auto/de-strict, oder format-Locale nicht de
{TCLUTILS TUPDFTEXT FILE}	PDF-Datei nicht lesbar
{TCLUTILS TUPDFTEXT FORMAT}	kein %PDF--Kopf, keine Objekte oder keine Seiten
{TCLUTILS TUPDFTEXT ENCRYPTED}	PDF verschlüsselt
{TCLUTILS TUPDFTEXT NOTEXT}	keine Seite hat Text
{TCLUTILS TUPDFTEXT OPTION}	unbekannte Option oder fehlender Wert
{TCLUTILS TUTESTRUN OPTION ...}	unbekannte Runner-Option
{TCLUTILS TUTASK CANCELLED}	Task abgebrochen oder Pool zerstört
{TCLUTILS TUTASK TIMEOUT}	-timeout von await/wait abgelaufen
{TCLUTILS TUTASK NOTASK}	await außerhalb einer Koroutine
{TCLUTILS TUTASK NOTDONE}	result eines laufenden Tasks
{TCLUTILS TUTASK OPTION}	falsches Argument (spawn, timer, pipe, -timeout)
{TCLUTILS TUTASK NOTHREAD}	pool ohne Paket Thread
{TCLUTILS TUTASK EMPTY}	wait -mode first auf leerer Gruppe
{TCLUTILS TUTASK INIT}	-init des Pools gescheitert
{TCLUTILS TUTASK INSIDE}	destroy aus dem eigenen Task
{TCLUTILS TUTASK PROTOCOL}	Koroutine unerwartet fortgesetzt
{TCLUTILS TUTASK LIMIT COMMANDS}	Befehlsgrenze der Sandbox
{TCLUTILS TUTASK LIMIT TIME}	Zeitgrenze der Sandbox
{TCLUTILS TUPKGFINDER OPTION}	unbekannte Option / falsche Anzahl
{TCLUTILS TUPDF FORMAT}	kein %PDF--Kopf, oder endobj fehlt
{TCLUTILS TUPDF ARG}	Objektnummer ist keine ganze Zahl

Code	Bedeutung
{TCLUTILS TUPDF NOTFOUND}	Objektnummer nach dem Aufblasen nicht im Text
{TCLUTILS TUNL START|INCREMENT|WIDTH}	tunl: Start oder Schritt keine Ganzzahl, Breite keine Ganzzahl ab 1
{TCLUTILS TUTR RANGE}	tutr: absteigender Bereich wie c-a
{TCLUTILS TUTR SET2}	tutr: leere zweite Menge ohne -delete
{TCLUTILS TUTABLE COLUMNS}	tutable: Zeile mit mehr Zellen als Köpfe
{TCLUTILS TUTABLE HEADERS}	tutable: leere Kopfliste
{TCLUTILS TUAWK OPTION}	tuawk: unbekannte Option oder Option ohne Wert
{TCLUTILS TUAGREP MAXDIST}	tuagrep: -maxdist keine Ganzzahl ab 0
{TCLUTILS TUSPLIT BYTES}	tusplit::splitBytes: Zeichen über \xFF
{TCLUTILS TUSPLIT ENCODING name}	tusplit::splitLines: unbekannte -encoding
{TCLUTILS TUICONV UNMAPPABLE enc}	tuiconv: Zeichen in enc nicht darstellbar (ohne -replace 1)
{TCLUTILS TUICONV INVALID enc}	tuiconv: Bytes in enc ungültig (ohne -replace 1)
{TCLUTILS TULDIF BINARY}	tuldif::toLdif: Binärwert mit Zeichen über \xFF
{TCLUTILS TULDIF OPTION}	tuldif: Option außer -binary
{TCLUTILS TUINI VALUE}	tuini: Wert, Schlüssel oder Abschnitt, den INI nicht fassen kann
{TCLUTILS TUCSPLIT ENCODING name}	tucsplit: unbekannte -encoding
{TCLUTILS TUHEAD COUNT}, {TCLUTILS TUTAIL COUNT}	Anzahl keine ganze Zahl ab 0
{TCLUTILS TUER SYNTAX}	tuer: Zeile weder Block noch Entität noch Beziehung
{TCLUTILS TUGANTT DURATION}	tugantt: Dauer mit unbekannter Einheit
{TCLUTILS TUXYCHART COUNT}	tuxychart: Zahl der Werte passt nicht zu den Kategorien
{TCLUTILS TUSANKEY VALUE}	tusankey: Zeile nicht Quelle,Ziel,Wert mit Zahl ab 0
{TCLUTILS <MODUL> BYTES}	Bytes erwartet, Zeichen über 255 gefunden (tuhexdump, tuod, tustrings, tucmp, tuhexedit, tuxxhash)

Code	Bedeutung
{TCLUTILS TUHEXEDIT OFFSET|RANGE|ENCODING}	ungültiger Offset; Schreiben über das Dateiende; Zeichen nicht kodierbar
{TCLUTILS TUHEXDUMP OFFSET|LENGTH}	-offset bzw. -length ungültig
{TCLUTILS TUCODE LOOKUP}	tucode::lookup: leer oder außerhalb der Tabelle
{TCLUTILS TUPNG CRC}	PNG-Block mit falscher oder fehlender CRC
{TCLUTILS TUCOLOR HSV}	fromHsv: h nicht 0..360 oder s/v nicht 0..100
{TCLUTILS TUTABLEPNG PADDING|SPACING}	keine Ganzzahl ab 0
{TCLUTILS TUMONTHPNG YEAR|FIRSTWEEKDAY}	Jahr nicht 1..9999; erster Wochentag nicht 1..7
{TCLUTILS TUPNGPAD TRIM|SQUARE|SIZE}	ungültiger Wert der Option
{TCLUTILS TULAYOUT OPTION}	tulayout: unbekannte Option oder Option ohne Wert
{TCLUTILS TUAPPINFO OPTION|VALUE}	tuappinfo: unbekannte Option; ungültiger Wert
{TCLUTILS TUDEPLOY VALUE}	-tmadd oder -fail kein Wahrheitswert
{TCLUTILS TUXXHASH OPTION|ENCODING}	tuxxhash: unbekannte Option; -encoding nicht utf-8/binary
{TCLUTILS TULDIF CHANGETYPE}	LDIF-Änderungssatz (changetype:) wird nicht gelesen
{TCLUTILS TURRULE UNSUPPORTED}	RRULE-Teil, den turrule nicht auswertet
{TCLUTILS TURRULE BYDAY|VALUE}	unbekanntes Wochentagskürzel; ungültiger Wert
{TCLUTILS TUBASE32 LENGTH|BITS}	Base32: unmögliche Länge; Füllbits nicht 0
{TCLUTILS TUDIAGRAM THEME}	unbekanntes Diagramm-Theme
{TCLUTILS TUJOURNEY VALUE}	Bewertung keine Zahl 1..5
{TCLUTILS TUPROVIDER FTP NOPKG}	FTP-Backend ohne tcllib-ftp

`can't find package tclutils::…` — Modulpfad / `TCLUTILS_TM` prüfen (Kapitel Installation). Wortlaut der `tutask`-Meldungen: Kapitel Tasks mit tutask.

27.1 Zusammenfassung

- Fehlercodes haben die Form `{TCLUTILS <MODUL> <GRUND>}`.
- Präfix-Traps (`trap {TCLUTILS TUFETCH}`) fangen ganze Gruppen.

28 API-Katalog: Tclutils-Module

Dieser Katalog wurde aus den mitgelieferten `.tm`-Quelldateien abgeleitet. Er zeigt

Prozedursignaturen, nicht automatisch vollständige Verträge. Für Semantik, Optionen, Fehlercodes und Nebenwirkungen ist die jeweilige Implementierung maßgeblich.

Die Befehle sind mit vollständigem Namensraum notiert. Eine `args`-Signatur bedeutet variable Argumente; die erlaubten Optionen müssen in der jeweiligen Modulimplementierung geprüft werden.

Erfasste Module mit Prozeduren: **143**.

28.1 common

Quelle: `lib/tm/tclutils/common-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::common::readFile {path args}
::tclutils::common::readBinaryFile {path}
::tclutils::common::writeFile {path data args}
::tclutils::common::replaceFile {path data args}
::tclutils::common::splitLines {text}
::tclutils::common::splitDelimited {line delimiter}
::tclutils::common::parseOptions {defaults args}
::tclutils::common::ensureBoolean {value optionName}
::tclutils::common::ensurePositiveInteger {value what}
::tclutils::common::ensureOneOf {value allowed what}
```

28.2 tuagrep

Quelle: `lib/tm/tclutils/tuagrep-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuagrep::match {line pattern args}
::tclutils::tuagrep::search {text pattern args}
::tclutils::tuagrep::file {filename pattern args}
::tclutils::tuagrep::files {fileList pattern args}
```

28.3 tuappinfo

Quelle: `lib/tm/tclutils/tuappinfo-0.3.tm`. Keine Exportliste im einfachen Quelltextmuster ermittelt; sichtbare Prozeduren ohne führenden Unterstrich aufgeführt.

28.4 tuarchitecture

Quelle: `lib/tm/tclutils/tuarchitecture-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuarchitecture::parse {text}
```

28.5 tuawk

Quelle: `lib/tm/tclutils/tuawk-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuawk::run {text rules args}
::tclutils::tuawk::file {path rules args}
```

28.6 tubase32

Quelle: `lib/tm/tclutils/tubase32-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tubase32::encode {data args}
::tclutils::tubase32::decode {text args}
::tclutils::tubase32::encodeFile {path args}
::tclutils::tubase32::decodeFile {path args}
```

28.7 tubase64

Quelle: `lib/tm/tclutils/tubase64-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tubase64::encode {data args}
::tclutils::tubase64::decode {text args}
::tclutils::tubase64::encodeFile {path args}
::tclutils::tubase64::decodeFile {path}
```

28.8 tubin

Quelle: `lib/tm/tclutils/tubin-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tubin::bytesToHex {bytes args}
::tclutils::tubin::hexToBytes {hex}
::tclutils::tubin::bytesToUnsignedList {bytes {offset 0} {length -1}}
::tclutils::tubin::u8 {bytes {offset 0}}
::tclutils::tubin::u16le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 2 s 0xffff 0 }
::tclutils::tubin::u32le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 4 i 0xffffffff 0 }
::tclutils::tubin::u64le {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 8 w "" [expr {1 << 64}] }
::tclutils::tubin::u16be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 2 S 0xffff 0 }
::tclutils::tubin::u32be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 4 I 0xffffffff 0 }
::tclutils::tubin::u64be {bytes {offset 0}} { ScanUnsigned $bytes
    $offset 8 W "" [expr {1 << 64}] }
::tclutils::tubin::uintle {bytes {offset 0} {length 0}}
::tclutils::tubin::i8 {bytes {offset 0}} { Signed [u8    $bytes
    $offset]  8 }
::tclutils::tubin::i16le {bytes {offset 0}} { Signed [u16le $bytes
    $offset] 16 }
::tclutils::tubin::i16be {bytes {offset 0}} { Signed [u16be $bytes
    $offset] 16 }
::tclutils::tubin::i32le {bytes {offset 0}} { Signed [u32le $bytes
    $offset] 32 }
::tclutils::tubin::i32be {bytes {offset 0}} { Signed [u32be $bytes
    $offset] 32 }
::tclutils::tubin::i64le {bytes {offset 0}} { Signed [u64le $bytes
    $offset] 64 }
::tclutils::tubin::i64be {bytes {offset 0}} { Signed [u64be $bytes
    $offset] 64 }
```

```
::tclutils::tubin::f32le {bytes {offset 0}}
::tclutils::tubin::f64le {bytes {offset 0}}
::tclutils::tubin::uintbe {bytes {offset 0} {length 0}}
::tclutils::tubin::f32be {bytes {offset 0}}
::tclutils::tubin::f64be {bytes {offset 0}}
::tclutils::tubin::packU8 {value} { binary format c [CheckRange $value
    0xff u8] }
::tclutils::tubin::packU16LE {value} { binary format s [CheckRange
    $value 0xffff u16] }
::tclutils::tubin::packU32LE {value} { binary format i [CheckRange
    $value 0xffffffff u32] }
::tclutils::tubin::packU64LE {value} { binary format w [CheckRange
    $value [expr {(1 << 64) - 1}] u64] }
::tclutils::tubin::packU16BE {value} { binary format S [CheckRange
    $value 0xffff u16] }
::tclutils::tubin::packU32BE {value} { binary format I [CheckRange
    $value 0xffffffff u32] }
::tclutils::tubin::packU64BE {value} { binary format W [CheckRange
    $value [expr {(1 << 64) - 1}] u64] }
::tclutils::tubin::ascii {bytes}
::tclutils::tubin::reader {subcmd args}
```

28.9 tublock

Quelle: `lib/tm/tclutils/tublock-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tublock::parse {text}
```

28.10 tubookmark

Quelle: `lib/tm/tclutils/tubookmark-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tubookmark::parse {html}
::tclutils::tubookmark::serialize {bookmarks args}
```

28.11 tuc4

Quelle: `lib/tm/tclutils/tuc4-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuc4::parse {text}
```

28.12 tucal

Quelle: `lib/tm/tclutils/tucal-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucal::render {year month args}
::tclutils::tucal::month {args}
::tclutils::tucal::now {args}
::tclutils::tucal::year {year args}
::tclutils::tucal::three {args}
```

28.13 tucat

Quelle: `lib/tm/tclutils/tucat-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucat::text {text args}
::tclutils::tucat::file {path args}
::tclutils::tucat::files {paths args}
```

28.14 tuclass

Quelle: `lib/tm/tclutils/tuclass-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuclass::parse {text}
```

28.15 tucmp

Quelle: `lib/tm/tclutils/tucmp-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucmp::data {a b}
::tclutils::tucmp::files {file1 file2}
::tclutils::tucmp::equal {file1 file2}
::tclutils::tucmp::firstDifference {file1 file2}
```

28.16 tucode

Quelle: `lib/tm/tclutils/tucode-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucode::lookup {value}
::tclutils::tucode::lookupChar {char}
::tclutils::tucode::render {from to args}
::tclutils::tucode::table {from to args}
::tclutils::tucode::ascii {args}
::tclutils::tucode::latin1 {args}
::tclutils::tucode::ansi {args}
::tclutils::tucode::groups {}
::tclutils::tucode::signs {args}
```

28.17 tucodepng

Quelle: `lib/tm/tclutils/tucodepng-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucodepng::render {from to args}
::tclutils::tucodepng::write {file from to args}
::tclutils::tucodepng::ascii {args} { return [render 0 127 {*}$args] }
::tclutils::tucodepng::latin1 {args} { return [render 128 255
    {*}$args] }
::tclutils::tucodepng::all {args} { return [render 0 255 {*}$args] }
```

28.18 tucolor

Quelle: `lib/tm/tclutils/tucolor-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucolor::rgb {color}
::tclutils::tucolor::hex {color}
::tclutils::tucolor::names {}
::tclutils::tucolor::exists {name}
::tclutils::tucolor::nearest {color}
::tclutils::tucolor::toHsv {color}
::tclutils::tucolor::fromHsv {hsv}
```

28.19 tucolumn

Quelle: `lib/tm/tclutils/tucolumn-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucolumn::table {text args}
::tclutils::tucolumn::fill {items args}
::tclutils::tucolumn::text {text args}
::tclutils::tucolumn::file {path args}
```

28.20 tucomm

Quelle: `lib/tm/tclutils/tucomm-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucomm::compareLines {aLines bLines args}
::tclutils::tucomm::text {aText bText args}
::tclutils::tucomm::files {aFile bFile args}
::tclutils::tucomm::formatColumns {comparison args}
```

28.21 tucrc

Quelle: `lib/tm/tclutils/tucrc-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucrc::hex32 {value}
::tclutils::tucrc::crc32 {data args}
::tclutils::tucrc::adler32 {data args}
::tclutils::tucrc::crc32File {path}
::tclutils::tucrc::adler32File {path}
```

28.22 tucsplit

Quelle: `lib/tm/tclutils/tucsplit-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucsplit::splitText {text pattern args}
::tclutils::tucsplit::file {path pattern args}
```

28.23 tucsv

Quelle: `lib/tm/tclutils/tucsv-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucsv::parseLine {line args}
::tclutils::tucsv::joinLine {fields args}
::tclutils::tucsv::parse {data args}
::tclutils::tucsv::text {rows args}
::tclutils::tucsv::file {path args}
::tclutils::tucsv::dicts {rows args}
::tclutils::tucsv::writeFile {path rows args}
```

28.24 tucut

Quelle: `lib/tm/tclutils/tucut-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tucut::field {line delimiter index}
::tclutils::tucut::fields {line delimiter fieldSpec {joiner {}}}
::tclutils::tucut::chars {line charSpec}
::tclutils::tucut::line {line args}
::tclutils::tucut::text {text args}
::tclutils::tucut::file {path args}
```

28.25 tudate

Quelle: `lib/tm/tclutils/tudate-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudate::parse {str args}
::tclutils::tudate::iso {seconds args}
::tclutils::tudate::add {seconds count unit}
::tclutils::tudate::diff {a b args}
::tclutils::tudate::relative {seconds args}
::tclutils::tudate::today {}
::tclutils::tudate::zones {}
```

28.26 tudav

Quelle: `lib/tm/tclutils/tudav-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudav::client {url args}
::tclutils::tudav::configure {c args}
::tclutils::tudav::lastStatus {c}
::tclutils::tudav::destroy {c}
::tclutils::tudav::propfind {c args}
::tclutils::tudav::report {c path body args}
::tclutils::tudav::listResources {c args}
::tclutils::tudav::get {c path}
::tclutils::tudav::put {c path data args}
::tclutils::tudav::delete {c path args}
::tclutils::tudav::calendarQuery {c path fromIso toIso}
::tclutils::tudav::addressbookMultiget {c path hrefs}
```

28.27 tudeploy

Quelle: `lib/tm/tclutils/tudeploy-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudeploy::baseDirs {}
::tclutils::tudeploy::platformTag {}
::tclutils::tudeploy::roots {args}
::tclutils::tudeploy::addModulePaths {args}
::tclutils::tudeploy::require {packages args}
::tclutils::tudeploy::resourceDirs {name args}
::tclutils::tudeploy::sourceModule {pkg args}
```

28.28 tudhash

Quelle: `lib/tm/tclutils/tudhash-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudhash::fromGray {w h grayList}
::tclutils::tudhash::fromRGB {w h rgbList}
::tclutils::tudhash::distance {a b}
::tclutils::tudhash::similar {a b {maxDist 10}}
```

28.29 tudiagram

Quelle: `lib/tm/tclutils/tudiagram-0.6.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudiagram::create {args}
::tclutils::tudiagram::setMeta {d args}
::tclutils::tudiagram::addNode {d id args}
::tclutils::tudiagram::addEdge {d from to args}
::tclutils::tudiagram::addGroup {d id args}
::tclutils::tudiagram::validate {d}
::tclutils::tudiagram::layout {d}
```

28.30 tudict

Quelle: `lib/tm/tclutils/tudict-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudict::getOr {dict default args}
::tclutils::tudict::paths {dict {prefix {}}}
::tclutils::tudict::flatten {dict {sep .}}
::tclutils::tudict::mergeDeep {args}
::tclutils::tudict::invert {dict}
```

28.31 tudiff

Quelle: `lib/tm/tclutils/tudiff-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tudiff::text {oldText newText args}
::tclutils::tudiff::files {oldFile newFile args}
::tclutils::tudiff::unifiedText {oldText newText args}
::tclutils::tudiff::unified {oldFile newFile args}
::tclutils::tudiff::contextText {oldText newText args}
::tclutils::tudiff::context {oldFile newFile args}
::tclutils::tudiff::directory {oldDir newDir args}
```

28.32 tuedid

Quelle: `lib/tm/tclutils/tuedid-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuedid::vendor {code}
::tclutils::tuedid::parseHex {text}
::tclutils::tuedid::parse {bytes}
```

28.33 tuer

Quelle: `lib/tm/tclutils/tuer-0.4.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuer::parse {text}
```

28.34 tuevent

Quelle: `lib/tm/tclutils/tuevent-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuevent::create {}
::tclutils::tuevent::subscribe {bus event handler}
::tclutils::tuevent::unsubscribe {bus event handler}
::tclutils::tuevent::emit {bus event args}
::tclutils::tuevent::handlers {bus event}
::tclutils::tuevent::events {bus}
::tclutils::tuevent::clear {bus {event ""}}
::tclutils::tuevent::destroy {bus}
```

28.35 tuexe

Quelle: `lib/tm/tclutils/tuexe-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuexe::find {names args}
::tclutils::tuexe::all {names args}
::tclutils::tuexe::exists {names args}
```

28.36 tuexpand

Quelle: `lib/tm/tclutils/tuexpand-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuexpand::expand {text args}
::tclutils::tuexpand::unexpand {text args}
::tclutils::tuexpand::file {path args}
```

28.37 tufetch

Quelle: `lib/tm/tclutils/tufetch-0.4.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufetch::get {url args}
::tclutils::tufetch::download {url path args}
```

28.38 tufile

Quelle: `lib/tm/tclutils/tufile-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufile::detect {path}
::tclutils::tufile::type {path}
::tclutils::tufile::mime {path}
::tclutils::tufile::describe {path}
::tclutils::tufile::checkExtension {path}
```

28.39 tufind

Quelle: `lib/tm/tclutils/tufind-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufind::all {args}
::tclutils::tufind::files {args}
::tclutils::tufind::directories {args}
::tclutils::tufind::walk {root patterns callback args}
```

28.40 tuflow

Quelle: `lib/tm/tclutils/tuflow-0.4.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuflow::parse {text args}
::tclutils::tuflow::toSvg {text args}
::tclutils::tuflow::toPng {text args}
::tclutils::tuflow::writeSvg {text file args}
::tclutils::tuflow::writePng {text file args}
```

28.41 tufmt

Quelle: `lib/tm/tclutils/tufmt-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufmt::reflow {text args}
::tclutils::tufmt::reflowFile {path args}
```

28.42 tufold

Quelle: `lib/tm/tclutils/tufold-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufold::foldLine {line args}
::tclutils::tufold::text {input args}
::tclutils::tufold::file {path args}
```

28.43 tufuzzy

Quelle: `lib/tm/tclutils/tufuzzy-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tufuzzy::distance {a b args}
::tclutils::tufuzzy::searchDistance {pattern text args}
::tclutils::tufuzzy::similarity {a b args}
::tclutils::tufuzzy::subsequence {pattern text args}
::tclutils::tufuzzy::bestMatch {pattern candidates args}
```

28.44 tugantt

Quelle: `lib/tm/tclutils/tugantt-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tugantt::parse {text}
::tclutils::tugantt::toSvg {model args}
::tclutils::tugantt::toPng {model args}
::tclutils::tugantt::writeSvg {model file args}
::tclutils::tugantt::writePng {model file args}
```

28.45 tugit

Quelle: `lib/tm/tclutils/tugit-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tugit::parse {text}
```

28.46 tugrep

Quelle: `lib/tm/tclutils/tugrep-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tugrep::match {line pattern args}
::tclutils::tugrep::search {text pattern args}
::tclutils::tugrep::file {filename pattern args}
::tclutils::tugrep::files {fileList pattern args}
```

28.47 tuhash

Quelle: `lib/tm/tclutils/tuhash-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuhash::sha256 {data args}
::tclutils::tuhash::sha1 {data args}
::tclutils::tuhash::md5 {data args}
::tclutils::tuhash::sha256File {path} { return [_sha256
    [::tclutils::common::readBinaryFile $path]] }
::tclutils::tuhash::sha1File {path} { return [_sha1
    [::tclutils::common::readBinaryFile $path]] }
::tclutils::tuhash::md5File {path} { return [_md5
    [::tclutils::common::readBinaryFile $path]] }
```

28.48 tuhead

Quelle: `lib/tm/tclutils/tuhead-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuhead::text {text {count 10}}
::tclutils::tuhead::file {path {count 10} args}
```

28.49 tuhexdump

Quelle: `lib/tm/tclutils/tuhexdump-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuhexdump::data {bytesData args}
::tclutils::tuhexdump::file {filename args}
```

28.50 tuhexedit

Quelle: `lib/tm/tclutils/tuhexedit-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuhexedit::readBytes {filename offset length}
::tclutils::tuhexedit::writeBytes {filename offset bytes {createBackup
    1}}
::tclutils::tuhexedit::backup {filename {backupName ""}}
::tclutils::tuhexedit::findHex {filename hex}
::tclutils::tuhexedit::findString {filename text {encoding utf-8}}
::tclutils::tuhexedit::patch {filename offset hex args}
::tclutils::tuhexedit::dump {filename args}
```

28.51 tuholiday

Quelle: `lib/tm/tclutils/tuholiday-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuholiday::easter {year}
::tclutils::tuholiday::holidays {year args}
::tclutils::tuholiday::isHoliday {iso args}
```

28.52 tuhttps

Quelle: `lib/tm/tclutils/tuhttps-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuhttps::caFile {}
::tclutils::tuhttps::socketCmd {args}
::tclutils::tuhttps::with {prefix script}
```

28.53 tuical

Quelle: `lib/tm/tclutils/tuical-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuical::parse {ics}
::tclutils::tuical::toIcs {comps}
::tclutils::tuical::components {comp {type ""}}
::tclutils::tuical::properties {comp {name ""}}
::tclutils::tuical::property {comp name}
::tclutils::tuical::find {comps type}
::tclutils::tuical::events {comps}
::tclutils::tuical::escapeText {s}
::tclutils::tuical::unescapeText {s}
::tclutils::tuical::setProperty {comp name value {params {}}}
::tclutils::tuical::addProperty {comp name value {params {}}}
::tclutils::tuical::removeProperty {comp name}
::tclutils::tuical::todos {comps}
::tclutils::tuical::journals {comps}
::tclutils::tuical::eventInfo {comp}
::tclutils::tuical::todoInfo {comp}
::tclutils::tuical::journalInfo {comp}
::tclutils::tuical::newComponent {type {props {}}}
```

28.54 tuico

Quelle: `lib/tm/tclutils/tuico-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuico::write {outFile entries}
::tclutils::tuico::info {icoFile}
::tclutils::tuico::extract {icoFile size {outFile ""}}
```

28.55 tuiconv

Quelle: `lib/tm/tclutils/tuiconv-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuiconv::encodings {}
::tclutils::tuiconv::convert {data from to args}
::tclutils::tuiconv::readFile {filename encodingName args}
::tclutils::tuiconv::writeFile {filename text encodingName args}
::tclutils::tuiconv::convertFile {infile outfile from to args}
```

28.56 tuimage

Quelle: `lib/tm/tclutils/tuimage-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuimage::type {bytes}
::tclutils::tuimage::mime {bytes}
::tclutils::tuimage::dimensions {bytes}
::tclutils::tuimage::inspect {bytes}
::tclutils::tuimage::dataUri {mimetype bytes}
::tclutils::tuimage::fromDataUri {uri}
```

28.57 tuini

Quelle: `lib/tm/tclutils/tuini-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuini::parse {ini}
::tclutils::tuini::toIni {data}
::tclutils::tuini::sections {data}
::tclutils::tuini::keys {data {section ""}}
::tclutils::tuini::get {data section key {default ""}}
::tclutils::tuini::has {data section key}
::tclutils::tuini::setValue {data section key value}
```

28.58 tujoin

Quelle: `lib/tm/tclutils/tujoin-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tujoin::lines {leftLines rightLines args}
::tclutils::tujoin::texts {leftText rightText args}
::tclutils::tujoin::files {leftPath rightPath args}
```

28.59 tujourney

Quelle: `lib/tm/tclutils/tujourney-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tujourney::parse {text}
::tclutils::tujourney::toSvg {model args}
::tclutils::tujourney::toPng {model args}
::tclutils::tujourney::writeSvg {model file args}
::tclutils::tujourney::writePng {model file args}
```

28.60 tujson

Quelle: `lib/tm/tclutils/tujson-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tujson::escape {text}
::tclutils::tujson::quote {text}
::tclutils::tujson::minify {json}
::tclutils::tujson::validate {json}
::tclutils::tujson::pretty {json args}
::tclutils::tujson::parse {json}
::tclutils::tujson::fromJson {json}
::tclutils::tujson::parseTyped {json}
::tclutils::tujson::filePretty {infile outfile args}
::tclutils::tujson::fileMinify {infile outfile}
::tclutils::tujson::toJson {typed args}
::tclutils::tujson::str {s} { return [list string $s] }
::tclutils::tujson::num {n}
::tclutils::tujson::bool {b}
::tclutils::tujson::null {} { return [list null {}] }
::tclutils::tujson::obj {pairs} { return [list object $pairs] }
::tclutils::tujson::arr {elems} { return [list array $elems] }
```

28.61 tukanban

Quelle: `lib/tm/tclutils/tukanban-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tukanban::parse {text}
::tclutils::tukanban::toSvg {model args}
::tclutils::tukanban::toPng {model args}
::tclutils::tukanban::writeSvg {model file args}
::tclutils::tukanban::writePng {model file args}
```

28.62 tulayout

Quelle: `lib/tm/tclutils/tulayout-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tulayout::pageSize {paper {orientation portrait}}
::tclutils::tulayout::mmToPx {mm scale}
::tclutils::tulayout::pxToMm {px scale}
::tclutils::tulayout::snap {mm args}
::tclutils::tulayout::mergeBlocks {base preset args}
::tclutils::tulayout::mergeLayout {target preset args}
```

28.63 tuldif

Quelle: `lib/tm/tclutils/tuldif-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuldif::parse {ldif args}
::tclutils::tuldif::toLdif {entries args}
::tclutils::tuldif::dn {entry}
::tclutils::tuldif::get {entry attr}
::tclutils::tuldif::attributes {entry}
::tclutils::tuldif::toDict {entry}
```

28.64 tulint

Quelle: `lib/tm/tclutils/tulint-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tulint::register {name severity description cmd}
::tclutils::tulint::unregister {name}
::tclutils::tulint::rules {}
::tclutils::tulint::checkText {text args}
::tclutils::tulint::checkFile {path args}
::tclutils::tulint::check {paths args}
::tclutils::tulint::summary {findings}
::tclutils::tulint::format {findings args}
```

28.65 tulist

Quelle: `lib/tm/tclutils/tulist-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tulist::unique {list}
::tclutils::tulist::flatten {list {depth 1}}
::tclutils::tulist::chunk {list size}
::tclutils::tulist::zip {args}
::tclutils::tulist::sum {list}
::tclutils::tulist::avg {list}
::tclutils::tulist::min {list}
::tclutils::tulist::max {list}
```

28.66 tulog

Quelle: `lib/tm/tclutils/tulog-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tulog::new {args}
::tclutils::tulog::assert {exprStr {msg ""}}
```

28.67 tumath

Quelle: `lib/tm/tclutils/tumath-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tumath::clamp {x lo hi}
::tclutils::tumath::inRange {x lo hi}
::tclutils::tumath::percent {part whole}
::tclutils::tumath::sign {x}
::tclutils::tumath::gcd {a b}
::tclutils::tumath::lcm {a b}
::tclutils::tumath::factorial {n}
::tclutils::tumath::roundTo {x ndigits}
```

28.68 tumd

Quelle: `lib/tm/tclutils/tumd-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tumd::frontmatter {markdown}
::tclutils::tumd::headings {markdown}
::tclutils::tumd::toc {markdown}
::tclutils::tumd::toHtml {markdown}
```

28.69 tumindmap

Quelle: `lib/tm/tclutils/tumindmap-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tumindmap::parse {text}
```

28.70 tummdb

Quelle: `lib/tm/tclutils/tummdb-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tummdb::metadata {h}
::tclutils::tummdb::lookup {h ip}
```

28.71 tumonthpng

Quelle: `lib/tm/tclutils/tumonthpng-0.5.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tumonthpng::render {year month args}
::tclutils::tumonthpng::write {file year month args}
::tclutils::tumonthpng::renderQuarter {year month args}
::tclutils::tumonthpng::renderYear {year args}
::tclutils::tumonthpng::writeQuarter {file year month args}
::tclutils::tumonthpng::writeYear {file year args}
```

28.72 tunl

Quelle: `lib/tm/tclutils/tunl-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunl::text {text args}
::tclutils::tunl::file {path args}
```

28.73 tunotes

Quelle: `lib/tm/tclutils/tunotes-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunotes::create {storeVar title content {tags {}}
    {parentId ""}}
::tclutils::tunotes::update {storeVar id title content {tags {}}
    {parentId KEEP}}
::tclutils::tunotes::delete {storeVar id {cascade 1}}
::tclutils::tunotes::move {storeVar id newParentId}
::tclutils::tunotes::get {store id}
::tclutils::tunotes::exists {store id}
::tclutils::tunotes::roots {store}
::tclutils::tunotes::children {store parentId}
```

28.74 tunotesdb

Quelle: `lib/tm/tclutils/tunotesdb-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunotesdb::init {db}
::tclutils::tunotesdb::create {db title content {tagList {}} {parentId
    ""}}
::tclutils::tunotesdb::update {db id title content {tagList {}}
    {parentId KEEP}}
::tclutils::tunotesdb::get {db id}
::tclutils::tunotesdb::exists {db id}
::tclutils::tunotesdb::delete {db id {cascade 1}}
::tclutils::tunotesdb::move {db id newParentId}
```

28.75 tunum

Quelle: `lib/tm/tclutils/tunum-0.4.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunum::parse {s args}
::tclutils::tunum::isNumber {s}
::tclutils::tunum::format {x args}
::tclutils::tunum::sum {values args}
```

28.76 tunumany

Quelle: `lib/tm/tclutils/tunumany-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunumany::parse {str args}
```

28.77 tunumfmt

Quelle: `lib/tm/tclutils/tunumfmt-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tunumfmt::toHuman {number args}
::tclutils::tunumfmt::fromHuman {str args}
::tclutils::tunumfmt::text {text args}
::tclutils::tunumfmt::file {path args}
```

28.78 tuod

Quelle: `lib/tm/tclutils/tuod-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuod::bytes {bytesData args}
::tclutils::tuod::data {bytesData args}
::tclutils::tuod::file {filename args}
```

28.79 tuodf

Quelle: `lib/tm/tclutils/tuodf-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuodf::createText {odtFile paragraphs args}
::tclutils::tuodf::part {odtFile name}
::tclutils::tuodf::parts {odtFile}
::tclutils::tuodf::paragraphs {odtFile}
::tclutils::tuodf::text {odtFile}
```

28.80 tuopen

Quelle: `lib/tm/tclutils/tuopen-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuopen::command {target args}
::tclutils::tuopen::launch {target args}
::tclutils::tuopen::editCommand {target args}
::tclutils::tuopen::edit {target args}
::tclutils::tuopen::openDir {target args}
```

28.81 tupacket

Quelle: `lib/tm/tclutils/tupacket-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupacket::parse {text}
::tclutils::tupacket::toSvg {model args}
::tclutils::tupacket::toPng {model args}
::tclutils::tupacket::writeSvg {model file args}
::tclutils::tupacket::writePng {model file args}
```

28.82 tupagespec

Quelle: `lib/tm/tclutils/tupagespec-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupagespec::parse {spec total args}
::tclutils::tupagespec::count {spec total}
::tclutils::tupagespec::compact {pages}
```

28.83 tupaste

Quelle: `lib/tm/tclutils/tupaste-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupaste::lines {lineLists args}
::tclutils::tupaste::texts {textList args}
::tclutils::tupaste::files {paths args}
```

28.84 tupatch

Quelle: `lib/tm/tclutils/tupatch-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupatch::parse {patchText}
::tclutils::tupatch::text {sourceText patchText args}
::tclutils::tupatch::fromFile {sourceFile patchText args}
```

28.85 tupath

Quelle: `lib/tm/tclutils/tupath-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupath::normalize {path}
::tclutils::tupath::clean {path}
::tclutils::tupath::isAbsolute {path}
::tclutils::tupath::components {path}
::tclutils::tupath::join {args}
```

28.86 tupdf

Quelle: `lib/tm/tclutils/tupdf-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupdf::version {pdfFile}
::tclutils::tupdf::objects {pdfFile}
::tclutils::tupdf::object {pdfFile id}
::tclutils::tupdf::trailer {pdfFile}
::tclutils::tupdf::zugferd {pdfFile}
::tclutils::tupdf::metadata {pdfFile}
::tclutils::tupdf::summary {pdfFile}
```

28.87 tupdftext

Quelle: `lib/tm/tclutils/tupdftext-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupdftext::pages {file args}
::tclutils::tupdftext::text {file args}
::tclutils::tupdftext::summary {file}
```

28.88 tupie

Quelle: `lib/tm/tclutils/tupie-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupie::parse {text}
::tclutils::tupie::toSvg {model args}
::tclutils::tupie::toPng {model args}
::tclutils::tupie::writeSvg {model file args}
::tclutils::tupie::writePng {model file args}
```

28.89 tupkgfinder

Quelle: `lib/tm/tclutils/tupkgfinder-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupkgfinder::paths {}
::tclutils::tupkgfinder::versions {packageName}
::tclutils::tupkgfinder::pkgInfo {packageName}
::tclutils::tupkgfinder::which {packageName}
::tclutils::tupkgfinder::findFileSystem {pattern args}
::tclutils::tupkgfinder::report {packageName}
```

28.90 tupng

Quelle: `lib/tm/tclutils/tupng-0.6.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupng::encodeRGB {image args}
::tclutils::tupng::encodeRGBA {image args}
::tclutils::tupng::encodeGray {image args}
::tclutils::tupng::encodeIndexed {palette image args}
```

28.91 tupngdraw

Quelle: `lib/tm/tclutils/tupngdraw-0.14.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupngdraw::new {args}
```

28.92 tupngpad

Quelle: `lib/tm/tclutils/tupngpad-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupngpad::bbox {rgba w h}
::tclutils::tupngpad::trim {rgba w h}
::tclutils::tupngpad::padTo {rgba w h tw th args}
::tclutils::tupngpad::batch {files outdir args}
```

28.93 tupostgrest

Quelle: `lib/tm/tclutils/tupostgrest-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupostgrest::new {baseUrl args}
::tclutils::tupostgrest::token {client jwt}
::tclutils::tupostgrest::request {client method path args}
::tclutils::tupostgrest::get {client table args}
::tclutils::tupostgrest::insert {client table row args}
::tclutils::tupostgrest::update {client table patch args}
::tclutils::tupostgrest::delete {client table args}
::tclutils::tupostgrest::rpc {client fn {args_ {}}}
```

28.94 tupr

Quelle: `lib/tm/tclutils/tupr-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tupr::text {text args}
::tclutils::tupr::file {path args}
```

28.95 tuprovider

Quelle: `lib/tm/tclutils/tuprovider-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuprovider::register {scheme class}
::tclutils::tuprovider::schemes {}
::tclutils::tuprovider::open {scheme args}
```

28.96 tuquadrant

Quelle: `lib/tm/tclutils/tuquadrant-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuquadrant::parse {text}
::tclutils::tuquadrant::toSvg {model args}
::tclutils::tuquadrant::toPng {model args}
::tclutils::tuquadrant::writeSvg {model file args}
::tclutils::tuquadrant::writePng {model file args}
```

28.97 turadar

Quelle: `lib/tm/tclutils/turadar-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::turadar::parse {text}
::tclutils::turadar::toSvg {model args}
::tclutils::turadar::toPng {model args}
::tclutils::turadar::writeSvg {model file args}
::tclutils::turadar::writePng {model file args}
```

28.98 turegistry

Quelle: `lib/tm/tclutils/turegistry-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::turegistry::create {}
::tclutils::turegistry::put {reg key value}
::tclutils::turegistry::get {reg key args}
::tclutils::turegistry::has {reg key}
::tclutils::turegistry::keys {reg {pattern *}}
::tclutils::turegistry::remove {reg key}
::tclutils::turegistry::destroy {reg}
```

28.99 turequirement

Quelle: `lib/tm/tclutils/turequirement-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::turequirement::parse {text}
```

28.100 turev

Quelle: `lib/tm/tclutils/turev-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::turev::line {str}
::tclutils::turev::text {text}
::tclutils::turev::file {path}
```

28.101 turrule

Quelle: `lib/tm/tclutils/turrule-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::turrule::parse {rr}
::tclutils::turrule::occurrences {args}
::tclutils::turrule::eventsInRange {ics fromIso toIso}
```

28.102 tusankey

Quelle: `lib/tm/tclutils/tusankey-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusankey::parse {text}
::tclutils::tusankey::toSvg {model args}
::tclutils::tusankey::toPng {model args}
::tclutils::tusankey::writeSvg {model file args}
::tclutils::tusankey::writePng {model file args}
```

28.103 tused

Quelle: `lib/tm/tclutils/tused-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tused::replace {text pattern replacement args}
::tclutils::tused::substitute {line pattern replacement args}
::tclutils::tused::delete {text pattern args}
::tclutils::tused::process {text rules}
::tclutils::tused::processFile {infile outfile rules args}
::tclutils::tused::script {text scriptText}
```

28.104 tuseq

Quelle: `lib/tm/tclutils/tuseq-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuseq::range {args}
::tclutils::tuseq::text {args}
```

28.105 tusequence

Quelle: `lib/tm/tclutils/tusequence-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusequence::parse {text}
::tclutils::tusequence::toSvg {model args}
::tclutils::tusequence::toPng {model args}
::tclutils::tusequence::writeSvg {model file args}
::tclutils::tusequence::writePng {model file args}
```

28.106 tusettings

Quelle: `lib/tm/tclutils/tusettings-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusettings::dir {kind app args}
::tclutils::tusettings::file {app {name settings}}
::tclutils::tusettings::load {app args}
::tclutils::tusettings::save {app settings args}
::tclutils::tusettings::value {settings section key {default ""}}
```

28.107 tushuf

Quelle: `lib/tm/tclutils/tushuf-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tushuf::lines {lineList args}
::tclutils::tushuf::text {text args}
::tclutils::tushuf::file {path args}
```

28.108 tusize

Quelle: `lib/tm/tclutils/tusize-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusize::bytes {path}
::tclutils::tusize::human {n args}
::tclutils::tusize::entries {dir}
```

28.109 tusort

Quelle: `lib/tm/tclutils/tusort-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusort::sortList {items args}
::tclutils::tusort::text {text args}
::tclutils::tusort::file {path args}
```

28.110 tusparql

Quelle: `lib/tm/tclutils/tusparql-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusparql::query {endpoint sparql args}
::tclutils::tusparql::ask {endpoint sparql args}
```

28.111 tusplit

Quelle: `lib/tm/tclutils/tusplit-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusplit::splitLines {text args}
::tclutils::tusplit::splitBytes {data args}
::tclutils::tusplit::file {path args}
::tclutils::tusplit::lines {path n args}
::tclutils::tusplit::bytes {path n args}
```

28.112 tusqlite

Quelle: `lib/tm/tclutils/tusqlite-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusqlite::null {}
::tclutils::tusqlite::quoteId {name}
::tclutils::tusqlite::insert {db table data}
::tclutils::tusqlite::rows {db sql {params {}}}
::tclutils::tusqlite::value {db sql args}
::tclutils::tusqlite::update {db table set where}
::tclutils::tusqlite::delete {db table where}
```

28.113 tustat

Quelle: `lib/tm/tclutils/tustat-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tustat::file {path}
::tclutils::tustat::files {paths}
::tclutils::tustat::render {statDict args}
```

28.114 tustate

Quelle: `lib/tm/tclutils/tustate-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tustate::parse {text}
```

28.115 tustr

Quelle: `lib/tm/tclutils/tustr-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tustr::isEmpty {s} { return [expr {$s eq ""}] }
::tclutils::tustr::truncate {s maxlen {ellipsis "..."}}
::tclutils::tustr::padLeft {s width {char " "}}
::tclutils::tustr::padRight {s width {char " "}}
::tclutils::tustr::center {s width {char " "}}
```

28.116 tustrings

Quelle: `lib/tm/tclutils/tustrings-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tustrings::extract {data args}
::tclutils::tustrings::file {filename args}
```

28.117 tusvg

Quelle: `lib/tm/tclutils/tusvg-0.4.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusvg::new {args}
```

28.118 tusysinfo

Quelle: `lib/tm/tclutils/tusysinfo-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tusysinfo::collect {args}
::tclutils::tusysinfo::report {info args}
::tclutils::tusysinfo::toJson {info args}
```

28.119 tutable

Quelle: `lib/tm/tclutils/tutable-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutable::render {headers rows args}
```

28.120 tutablepng

Quelle: `lib/tm/tclutils/tutablepng-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutablepng::render {rows args}
::tclutils::tutablepng::write {file rows args}
```

28.121 tutac

Quelle: `lib/tm/tclutils/tutac-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutac::lines {lineList}
::tclutils::tutac::text {text}
::tclutils::tutac::file {path}
```

28.122 tutail

Quelle: `lib/tm/tclutils/tutail-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutail::text {text {count 10}}
::tclutils::tutail::file {path {count 10} args}
```

28.123 tutask

Quelle: `lib/tm/tclutils/tutask-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutask::await {starter args}
::tclutils::tutask::timer {ms {value ""}}
::tclutils::tutask::sleep {ms}
::tclutils::tutask::onwrite {varName}
::tclutils::tutask::pipe {cmdlist args}
::tclutils::tutask::spawn {args}
::tclutils::tutask::current {}
::tclutils::tutask::run {args}
```

28.124 tutdbc

Quelle: `lib/tm/tclutils/tutdbc-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutdbc::null {}
::tclutils::tutdbc::quoteId {name}
::tclutils::tutdbc::rows {conn sql {binds {}}}
::tclutils::tutdbc::value {conn sql {binds {}} {default ""}}
::tclutils::tutdbc::execute {conn sql {binds {}}}
::tclutils::tutdbc::insert {conn table data}
::tclutils::tutdbc::transaction {conn script}
```

28.125 tutee

Quelle: `lib/tm/tclutils/tutee-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutee::write {data paths args}
::tclutils::tutee::writeFile {inputFile paths args}
::tclutils::tutee::copyChannel {inChan outChans args}
```

28.126 tuterm

Quelle: `lib/tm/tclutils/tuterm-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuterm::style {args}
::tclutils::tuterm::wrap {text args}
::tclutils::tuterm::off {attr}
::tclutils::tuterm::names {{what all}}
::tclutils::tuterm::strip {text}
::tclutils::tuterm::enable {args}
::tclutils::tuterm::auto {}
::tclutils::tuterm::enableVT {}
::tclutils::tuterm::disableVT {}
```

28.127 tutestrun

Quelle: `lib/tm/tclutils/tutestrun-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutestrun::run {dir libs args}
```

28.128 tutimeline

Quelle: `lib/tm/tclutils/tutimeline-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutimeline::parse {text}
::tclutils::tutimeline::toSvg {model args}
::tclutils::tutimeline::toPng {model args}
::tclutils::tutimeline::writeSvg {model file args}
::tclutils::tutimeline::writePng {model file args}
```

28.129 tutr

Quelle: `lib/tm/tclutils/tutr-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutr::translate {text set1 set2}
::tclutils::tutr::delete {text set1}
::tclutils::tutr::squeeze {text set1}
::tclutils::tutr::file {path set1 set2 args}
```

28.130 tutreemap

Quelle: `lib/tm/tclutils/tutreemap-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutreemap::parse {text}
::tclutils::tutreemap::toSvg {model args}
::tclutils::tutreemap::toPng {model args}
::tclutils::tutreemap::writeSvg {model file args}
::tclutils::tutreemap::writePng {model file args}
```

28.131 tutsort

Quelle: `lib/tm/tclutils/tutsort-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tutsort::pairs {tokens}
::tclutils::tutsort::text {text}
::tclutils::tutsort::file {path}
```

28.132 tuuniq

Quelle: `lib/tm/tclutils/tuuniq-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuuniq::uniqList {items args}
::tclutils::tuuniq::adjacentCount {items args}
::tclutils::tuuniq::count {items args}
::tclutils::tuuniq::text {text args}
::tclutils::tuuniq::file {path args}
::tclutils::tuuniq::countText {text args}
::tclutils::tuuniq::countFile {path args}
```

28.133 tuurl

Quelle: `lib/tm/tclutils/tuurl-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuurl::encode {str args}
::tclutils::tuurl::decode {str args}
::tclutils::tuurl::buildQuery {d args}
::tclutils::tuurl::parseQuery {q args}
```

28.134 tuuuid

Quelle: `lib/tm/tclutils/tuuuid-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuuuid::v4 {}
::tclutils::tuuuid::v7 {}
::tclutils::tuuuid::nil {}
::tclutils::tuuuid::generate {args}
::tclutils::tuuuid::validate {uuid}
::tclutils::tuuuid::version {uuid}
```

28.135 tuvalidate

Quelle: `lib/tm/tclutils/tuvalidate-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuvalidate::email {s}
::tclutils::tuvalidate::url {s}
::tclutils::tuvalidate::ipv4 {s}
::tclutils::tuvalidate::port {s}
::tclutils::tuvalidate::alpha {s} { return [regexp {^[A-Za-z]+$} $s] }
::tclutils::tuvalidate::alnum {s} { return [regexp {^[A-Za-z0-9]+$}
    $s] }
::tclutils::tuvalidate::numeric {s} { return [string is double -strict
    $s] }
::tclutils::tuvalidate::integer {s} { return [string is integer
    -strict $s] }
```

28.136 tuvcard

Quelle: `lib/tm/tclutils/tuvcard-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuvcard::parse {vcf}
::tclutils::tuvcard::toVcf {cards}
::tclutils::tuvcard::properties {card {name ""}}
::tclutils::tuvcard::property {card name}
::tclutils::tuvcard::get {card name}
::tclutils::tuvcard::names {card}
::tclutils::tuvcard::fullName {card}
::tclutils::tuvcard::addProperty {card name value {params {}}}
::tclutils::tuvcard::removeProperty {card index}
::tclutils::tuvcard::setProperty {card index name value {params {}}}
```

28.137 tuwc

Quelle: `lib/tm/tclutils/tuwc-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuwc::text {data}
::tclutils::tuwc::file {filename}
```

28.138 tuxargs

Quelle: `lib/tm/tclutils/tuxargs-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuxargs::batches {items args}
::tclutils::tuxargs::apply {items commandPrefix args}
::tclutils::tuxargs::command {items commandPrefix args}
```

28.139 tuxml

Quelle: `lib/tm/tclutils/tuxml-0.1.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuxml::escape {text args}
::tclutils::tuxml::unescape {text}
::tclutils::tuxml::declaration {{encoding UTF-8}}
::tclutils::tuxml::attrs {attributes}
::tclutils::tuxml::tag {name {attributes {}}}
::tclutils::tuxml::element {name attributes content}
::tclutils::tuxml::textElement {name attributes text}
```

28.140 tuxxhash

Quelle: `lib/tm/tclutils/tuxxhash-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuxxhash::xxh32 {data args}
::tclutils::tuxxhash::xxh32file {path {seed 0}}
```

28.141 tuxychart

Quelle: `lib/tm/tclutils/tuxychart-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuxychart::parse {text}
::tclutils::tuxychart::toSvg {model args}
::tclutils::tuxychart::toPng {model args}
::tclutils::tuxychart::writeSvg {model file args}
::tclutils::tuxychart::writePng {model file args}
```

28.142 tuzip

Quelle: `lib/tm/tclutils/tuzip-0.2.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuzip::entries {zipfile args}
::tclutils::tuzip::names {zipfile args}
::tclutils::tuzip::readMember {zipfile member args}
::tclutils::tuzip::extract {zipfile member outfile args}
::tclutils::tuzip::create {zipfile files args}
::tclutils::tuzip::createMembers {zipfile members args}
```

28.143 tuzipfs

Quelle: `lib/tm/tclutils/tuzipfs-0.3.tm`. Auf `namespace export` begrenzt.

```
::tclutils::tuzipfs::available {}
::tclutils::tuzipfs::requireAvailable {}
::tclutils::tuzipfs::root {}
::tclutils::tuzipfs::mounts {}
::tclutils::tuzipfs::mount {archive {mountName ""}}
::tclutils::tuzipfs::unmount {mountpoint}
```

29 Praxis: CSV, Datenstrukturen und sichere Fehlerbehandlung

29.1 CSV-Zeilen verarbeiten

`tucsv` stellt die Funktionen `parseLine`, `joinLine`, `parse`, `text`, `file`, `dicts` und `writeFile` bereit. Die Parseroptionen `-delimiter`, `-quote`, `-trim` und `-strict` sind im Quelltext von `tucsv-0.2.tm` definiert; `text`, `file` und `writeFile` kennen dazu `-profile`, `-bom` und `-encoding`. Die Ausgabe von `joinLine` kann über `-alwaysQuote` gesteuert werden.

```
package require tclutils::tucsv

set record [list {Müller, GmbH} {Er sagte "Hallo"} 48683]
set csvLine [::tclutils::tucsv::joinLine $record]
set parsed [::tclutils::tucsv::parseLine $csvLine]
if {$parsed ne $record} {
    error "CSV roundtrip failed"
}
puts $csvLine
```

Die Werte werden als Tcl-Liste aufgebaut. Dadurch bleiben eingebettete Kommas und

Anführungszeichen in einem Feld erhalten. Der Test prüft die Rückübersetzung, nicht die Bytegleichheit zu einer fremden CSV-Datei.

29.2 Fehlerfälle gezielt behandeln

Ein nicht geschlossenes Anführungszeichen erzeugt bei aktiviertem `-strict` den Fehlercode `{TCLUTILS TUCSV QUOTE}`. Damit kann der Aufrufer Parserfehler getrennt von Datei- oder Laufzeitfehlern behandeln.

```
package require tclutils::tucsv

try {
    ::tclutils::tucsv::parseLine {"nicht geschlossen}
} trap {TCLUTILS TUCSV QUOTE} {message options} {
    puts "CSV input rejected: $message"
}
```

Die `trap`-Klausel reagiert nur auf den angegebenen Fehlercode. Für andere Fehler bleibt eine eigene Behandlung oder Weitergabe erforderlich.

29.3 Dictionaries mit `tudict`

Die Implementierung von `tudict-0.1.tm` exportiert `getOr`, `paths`, `flatten`, `mergeDeep` und `invert`. Für verschachtelte Daten sind insbesondere die Argumentreihenfolge und der Umgang mit fehlenden Schlüsseln zu beachten.

```
package require tclutils::tudict

set defaults [dict create ui [dict create language de theme light]]
set overrides [dict create ui [dict create theme dark]]
set settings [::tclutils::tudict::mergeDeep $defaults $overrides]
puts [dict get $settings ui language]
puts [dict get $settings ui theme]
```

`mergeDeep` ist für verschachtelte Dictionaries gedacht. Prüfen Sie vor produktivem Einsatz, ob einzelne Werte selbst gültige Dictionaries sein können und wie die Anwendung diese interpretiert.

29.4 Dateipfade und Portabilität

`tupath` exportiert `normalize`, `clean`, `isAbsolute`, `components`, `join`, `relative`, `commonPath` und `readlink`. `file normalize` und `tupath::clean` sind nicht gleichbedeutend: Normalisierung kann vom Dateisystem und der Plattform abhängen; rein textuelle Bereinigung ist davon zu unterscheiden.

```
package require tclutils::tupath
set relativePath [::tclutils::tupath::join docs manual index.md]
puts $relativePath
```

Grenzen: Der API-Katalog nennt Signaturen; die Beispiele oben sind aus den Quelltexten abgeleitet. Eine vollständige plattformübergreifende Ausführung der Beispiele ist damit nicht behauptet.

30 Lizenz

Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

Text

Dieses Buch ist unter der Creative-Commons-Lizenz "Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen und weiterverbreiten sowie es remixen, verändern und darauf aufbauen, auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen Sie Ihre Beiträge nur unter derselben Lizenz wie das Original verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk, pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist jeweils angegeben.

MIT License

Copyright (c) 2026 Gregor Ebbing

```
Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

-

-fontfile 123

C

Capabilities 146
caps 203

G

Graphem 203

K

Koroutinen 180

M

Modulpfad 12, 203

N

NOCA 203

P

Provider 203

S

Sandbox 184, 203
Schatten-Kopie 204
Starter 180, 203

T

Task 203
tclutils 11
tical 86
tls 174
tuagrep 42
tuappinfo 156
tuarchitecture 106
tuawk 38
tubase32 59
tublock 105
tubookmark 55
tuc4 104
tucal 84
tuclass 100
tucmp 72
tucode 75
tucodepng 134
tucolor 125
tucolumn 24
tucomm 32
tucsplit 34

tucsv 49
tudate 44
tudeploy 160
tudhash 145
tudiagram 92, 203
tudict 64
tuedid 159
tuer 101
tuevent 150
tuexe 163
tuexpand 23
tufetch 175
tuflow 92, 203
tufmt 22
tufold 22
tufuzzy 41
tugantt 109
tugit 105
tuhash 203
tuhexdump 67
tuhexedit 70
tuholiday 203
tuhttps 174, 203
tuico 143
tuiconv 62
tuimage 142
tuini 51
tujoin 31
tujourney 116
tujson 50
tukanban 122
tulayout 171
tuldif 54
tulint 188
tulist 153
tulog 148
tumath 154
tumd 78, 203
tumindmap 103
tummdb 57, 203
tumonthpng 135
tunl 21
tunotesdb 82
tunum 44
tunumany 155
tuod 68
tuodf 90, 204

tuopen 164
tupacket 121
tupagespec 170
tupaste 30
tupatch 65
tupath 44
tupdf 90, 204
tupdftext 87, 204
tupie 111
tupkgfinder 188
tupng 126
tupngdraw 128
tupngpad 138
tupr 29
tuprovider 146
tuquadrant 114
turadar 119
turegistry 151
turequirement 102
turev 26
turrule 79
tusankey 118
tuseq 27
tusequence 107
tusettings 177, 203
tushuf 26
tusize 73
tusparql 176
tusplit 35
tusqlite 53
tustat 74
tustate 99
tustr 44
tustrings 69
tusvg 139
tusysinfo 157
tutable 25
tutablepng 132
tutac 26
tutask 180, 203
tutdbc 57, 203
tutee 36
tuterm 169
tutestrun 187, 203
tutimeline 117
tutr 28
tutreemap 120

tutsort 38
tuurl 203
tuvalidate 151
tuwc 28
tuxargs 37
tuxml 52
tuxxhash 61
tuxychart 113
tuzipfs 166

U

Umbrella 203